

CSCI89: Introduction to Artificial Intelligence

통합 강의 노트

Contents

Chapter 1

용어 정리

강의명: CSCI E-89: 인공지능 개론

주차: Lecture 1

교수명: UIUC Student

목적: 본 문서는 CSCI E-89 '인공지능 개론' 1강의 핵심 내용을 통합 정리한 LaTeX 노트입니다. 강의 운영 방식, 평가 기준, 필수 선수 지식부터 신경망의 기본 원리, 활성화 함수, 손실 함수, 최적화 알고리즘까지 다룹니다. 특히, 비전공자도 쉽게 이해할 수 있도록 개념 설명에 중점을 두었으며, 실제 과제 수행에 필요한 코드 예시와 주의사항을 포함하여 시험 대비 및 실습에 즉시 활용 가능하도록 구성했습니다. 모든 내용은 강의 자료를 기반으로 하며, 추가 설명과 비유를 통해 이해도를 높였습니다.

1.1 용어 정리

Table 1.1: 주요 용어 및 설명

용어	쉬운 설명	원어	비고
신경망	사람 뇌의 뉴런 연결 방식을 모방한 컴퓨터 모델	Neural Network	복잡한 패턴 학습에 사용
활성화 함수	신경망 뉴런의 출력을 결정하는 비선형 함수	Activation Function	비선형성 도입으로 복잡한 문제 해결
손실 함수	모델의 예측이 실제 값과 얼마나 다른지 측정하는 함수	Loss Function	개별 데이터 포인트의 오차
비용 함수	전체 데이터셋에 대한 손실 함수의 평균	Cost Function	모델 학습의 최종 목표
경사 하강법	비용 함수를 최소화하기 위해 기울기를 따라 내려가는 최적화 방법	Gradient Descent	모델 파라미터 업데이트 방식
확률적 경사 하강법	한 번에 하나의 데이터 포인트로 경사를 계산하여 업데이트	Stochastic Gradient Descent (SGD)	계산 효율적, 지역 최적점 탈출 용이
미니 배치 경사 하강법	작은 데이터 묶음(미니 배치)으로 경사를 계산하여 업데이트	Mini-Batch Gradient Descent	GD와 SGD의 장점 결합
하이퍼파라미터	모델 학습 전에 사용자가 직접 설정해야 하는 값	Hyperparameter	학습률, 배치 크기 등
학습률	경사 하강법에서 한 번에 이동하는 스텝의 크기	Learning Rate	너무 크면 발산, 너무 작으면 느림
원-핫 인코딩	범주형 데이터를 이진 벡터로 표현하는 방식	One-Hot Encoding	분류 문제의 출력 레이블 표현
소프트맥스	여러 클래스에 대한 확률 분포를 출력하는 활성화 함수	Softmax Function	분류 문제의 최종 출력층에 사용
교차 엔트로피	분류 문제에서 주로 사용되는 손실 함수	Cross-Entropy Loss	예측 확률과 실제 레이블 간의 차이 측정
구조적 토픽 모델링	텍스트에서 주제를 자동으로 추출하는 통계 모델	Structural Topic Modeling (STM)	R 언어를 사용하여 구현

강의 개요 및 운영

강의 목표 및 내용

이 강의는 신경망(Neural Networks)에 대한 소개를 목표로 합니다. 수강생들이 신경망에 대한 사전 지식이 없다고 가정하고, 기본적인 개념부터 심층적으로 다룰 예정입니다. 특히 자연어 처리(Natural Language Processing, NLP) 분야에 신경망이 어떻게 적용되는지에 초점을 맞춥니다.

강의 자료 및 접근

강의 자료는 Canvas를 통해 제공되며, 다음을 포함합니다:

- **강의 슬라이드:** 강의에서 사용되는 모든 슬라이드.
- **예시 코드:** 강의 내용과 관련된 코드 예시.
- **조교(TA) 자료:** TA 세션에서 사용되는 추가 파일.

모든 자료는 강의 후 Canvas의 'Files' 탭에서 다운로드할 수 있습니다. 제공된 코드는 자유롭게 활용할 수 있으며, 필요시 출처를 명시하면 됩니다.

핵심 정보

코드 활용 팁: 강의에서 제공된 코드를 수정하여 과제를 제출하는 것은 허용됩니다. 이는 정책 위반이 아니며, 수업에서 제시된 예시나 TA가 제공한 자료를 기반으로 작업하는 것을 권장합니다.

소통 채널

강의 관련 소통은 다음 채널을 통해 이루어집니다:

- **공지 (Announcements):** Canvas의 'Announcements' 탭을 통해 중요한 정보가 게시됩니다. 이메일로도 발송되므로 주기적으로 확인하거나 이메일을 주의 깊게 살펴보세요.
- **WhatsApp 그룹:** 수강생 간의 자유로운 소통을 위해 개설됩니다. 교수진이 상시 모니터링하지는 않지만, 필요시 참고 자료를 공유하거나 답변을 제공할 수 있습니다.
- **Piazza:** 공식적인 질문 및 답변 포럼입니다. 교수진과 TA가 질문에 답변할 의무를 느끼는 채널이므로, 강의 내용에 대한 질문은 Piazza를 통해 하는 것이 가장 좋습니다. 다른 학생들도 답변을 볼 수 있어 효율적입니다.
- **이메일:** 교수진이나 TA에게 직접 문의할 때 사용합니다.
- **Canvas Inbox:** Canvas 내에서 교수진이나 TA에게 직접 메시지를 보낼 수 있습니다.

주의

학생 간 연락: Canvas에서는 학생 이름이 서로에게 숨겨져 있어 직접 연락할 수 없습니다. 다른 학생과 소통하고 싶다면 WhatsApp 그룹을 활용하세요. 연락처 공유는 자율에 맡겨집니다.

조교(TA) 및 교수 세션

- **조교 (TA):** 총 5명의 TA가 있으며, Andrea가 수석 TA(Head TA)입니다. Andrea는 딥러닝 및 자연어 처리 분야에서 5년 이상 TA 경험이 있는 베테랑이므로, 행정적 질문이나 누구에게 물어야 할지 모르는 질문은 Andrea에게 먼저 이메일로 문의하세요.
- **TA 세션:** 주 2회 진행됩니다 (보통 수/목요일, 토/일요일). 강의 내용을 보충하고 구현 예시를 다룹니다. TA는 강의 계획을 따르지만, 세션 내용은 TA의 재량에 따라 유연하게 진행됩니다. TA들은 박사 학위 소지자도 있을 정도로 전문성이 높습니다.
- **교수 세션:** 매주 금요일 1시간 동안 진행됩니다. 주로 파이썬(Python) 구현 예시를 시연하고 토론하며, 이론은 주로 화요일 강의에서 다룹니다.

핵심 정보

세션 활용: 모든 세션은 녹화되어 제공되므로, 실시간 참석이 어렵더라도 나중에 시청할 수 있습니다. 질문이 있다면 실시간 세션에 참여하는 것이 가장 좋습니다.

수업 참여 방식

이 강의는 온라인 옵션을 제공하므로, 모든 세션(강의, TA 세션, 교수 세션)에 직접 참석할 필요가 없습니다. 모든 세션은 녹화되어 Canvas의 'Class Recordings' 링크를 통해 다시 볼 수 있습니다. 질문이 있을 경우, 다른 학생의 발언을 존중하며 문장 중간에 끊지 않는 선에서 언제든지 질문할 수 있습니다.

평가 및 마감

성적 배점

강의 성적은 다음 요소들로 구성됩니다:

- 과제 (Homework): 65%
- 퀴즈 (Quizzes): 20%
- 최종 프로젝트 (Final Project): 15%

과제 (Assignments)

- **마감:** 매주 일요일 (보스턴 시간 기준)에 마감됩니다. 첫 과제는 9월 14일입니다.
- **제출:**
 - **보고서:** Microsoft Word 또는 PDF 형식으로 제출합니다. 코드의 핵심 부분, 결과(플롯, 표), 간략한 설명을 포함해야 합니다. 주피터 노트북(Jupyter Notebook)을 PDF로 변환하여 제출하는 것도 허용됩니다.
 - **원본 코드:** 주피터 노트북 파일(.ipynb) 또는 여러 개의 파이썬(.py) 파일인 경우 압축 파일(.zip) 형태로 제출해야 합니다. TA가 코드를 직접 실행하여 검토할 수 있도록 합니다.
- **지연 페널티:** 과제 제출이 늦어지면 페널티가 부과됩니다 (예: 3일 지연 시 30% 감점).
- **모범 답안 공유:** 매주 5개 정도의 모범 답안이 Canvas에 공유됩니다. 이는 신경망 최적화의 비볼록성 (non-convexity)으로 인해 학생마다 다른 결과가 나올 수 있기 때문에, 서로의 접근 방식을 학습하고

비교하는 데 도움이 됩니다. 자신의 답안 공유를 원치 않으면 TA에게 문의하세요.

- **성능 목표:** 일부 과제에서는 특정 성능(예: 85% 정확도)을 달성하도록 요구할 수 있습니다. 이는 가장 어려운 부분이 될 수 있으며, 다양한 아키텍처나 파라미터 초기화 방법을 실험해야 할 수도 있습니다.

예제

과제 제출 예시:

- **보고서:** 문제 해결 과정, 주요 코드 스냅샷, 결과 플롯, 표, 그리고 결과에 대한 간략한 논의를 포함합니다.
- **코드:** 보고서에 포함된 코드의 원본 파일(예: Jupyter Notebook)을 제출합니다. 여러 파일인 경우 하나의 ZIP 파일로 묶습니다.

퀴즈 (Quizzes)

- **마감:** 다음 강의(화요일) 전까지 마감됩니다. 첫 퀴즈는 9월 9일 20시 10분(보스턴 시간)까지입니다.
- **엄격한 마감:** 퀴즈 마감은 매우 엄격합니다. 마감 후에는 재응시나 대체 퀴즈가 제공되지 않습니다. 질문이 다음 강의에서 논의되기 때문입니다.
- **응시 조건:** 퀴즈는 7일 동안 언제든지 열어 풀 수 있으며, 시간 제한이나 감독이 없습니다. 15분 정도의 시간을 할애하여 마감 기한 내에 제출해야 합니다.

주의

퀴즈 마감 엄수: 퀴즈는 마감 기한을 놓치면 재응시 기회가 없습니다. 이는 강의 정책이며, 질문이 공개적으로 논의되기 때문에 대체 퀴즈를 만드는 것은 공정성 문제와 막대한 작업량을 야기합니다.

최종 프로젝트 (Final Project)

- **마감:** 강의 마지막 날인 12월 16일(월요일)에 마감됩니다. 이는 매우 엄격한 마감 기한입니다.
- **주제 선정:** 11월 25일(추수감사절 직전)까지 프로젝트 주제를 선정해야 합니다. 11월 9일에 제안된 주제 목록이 게시될 예정이며, 자신만의 주제를 제안하는 것도 적극 권장됩니다.
- **내용:** 자연어 처리(NLP)와 명시적으로 관련된 프로젝트여야 합니다. 신경망 외의 다른 언어 처리 기법(예: 구조적 토픽 모델링)도 허용됩니다.
- **제출물:**
 - **보고서:** 7-15페이지 분량.
 - **슬라이드:** 발표에 사용될 슬라이드.
 - **영상 발표:** 10-15분 분량의 발표 영상.
 - **작동하는 데모 코드:** 실행 가능한 코드.

주의

최종 프로젝트 마감: 최종 프로젝트 마감은 학교 정책상 매우 엄격합니다. 마감 기한을 놓칠 경우, 학교의 승인을 받아야만 연장이 가능합니다.

AI 기술 사용 정책

- **사용 권장:** 이 강의에서는 AI 도구(예: 생성형 AI) 사용을 적극 권장합니다. 이는 직장에서 이러한 도구를 사용하는 것이 불가피해질 것이기 때문입니다.
- **인용 필수:** AI 도구를 사용하여 생성된 코드, 텍스트 등을 제출할 때는 반드시 사용한 AI 도구를 명시하고 인용해야 합니다.
- **최종 책임:** AI가 생성한 결과물에 대한 최종 책임은 학생 본인에게 있습니다. "AI가 그랬다"고 변명할 수 없습니다.
- **기술 학습:** AI 도구를 효과적으로 사용하는 것은 중요한 기술입니다. 원하는 바를 명확하게 설명하고, 예시를 제공하며, 반복적인 수정을 통해 AI의 도움을 받는 방법을 배우는 것이 중요합니다.

선수 지식 및 교재

필수 선수 지식

- **미적분학 (Calculus):** 신경망의 역전파(backpropagation) 원리를 이해하는 데 도움이 됩니다. 이 강의에서는 역전파를 직접 구현하도록 요구하지 않으므로, 미적분학 지식이 '매우' 중요하지는 않습니다. 하지만 책에 제시된 공식을 이해하고 기본적인 용어(예: 미분)를 아는 수준이면 충분합니다.
- **확률론 (Probability Theory):** 생성 모델(generative models)을 다룰 때 필요합니다. 무작위 변수(random variable), 정규 분포(normal distribution)와 같은 기본 개념에 대한 직관적인 이해가 필요합니다. 엄밀한 정의를 알 필요는 없습니다.
- **파이썬 (Python):** 가장 중요한 선수 지식입니다. 이 강의에서는 파이썬 기초를 다루지 않으므로, 코딩 능력이 있다고 가정합니다. CSCI E-7과 같은 파이썬 입문 과정을 수강한 수준을 기대합니다.

권장 교재

이 강의에서는 특정 교재를 필수로 지정하지 않습니다. 하나의 교재가 모든 내용을 특정 순서나 방식으로 다루기 어렵기 때문입니다. 다음은 인기 있는 교재 목록이며, 수강생들은 다양한 자료를 활용하여 주제별로 학습하는 것을 권장합니다.

- **Bird 저서:** 매우 인기 있는 책입니다.
- **Transformers를 이용한 자연어 처리 (Natural Exposing with Transformers):** 트랜스포머(Transformers) 모델을 간략하게 다룰 예정입니다.
- **Text as Data:** 사회 과학자를 위해 쓰인 책이지만, 구조적 토픽 모델링(Structural Topic Modeling, STM)에 대한 중요한 챕터(13장)가 포함되어 있습니다. 이 책을 구매할 필요는 없으며, 해당 주제에 대한 원본 연구 논문을 통해 동일한 정보를 얻을 수 있습니다.
- **선택 교재:** 수학적 내용이 더 많아 읽기 어려울 수 있는 책입니다. 선택 사항입니다.

핵심 정보

학습 전략: 특정 책의 챕터를 따라가기보다는, 강의에서 다루는 '주제'를 중심으로 학습하는 것이 효과적입니다. 예를 들어, 순환 신경망(Recurrent Neural Network)을 배운다면, 여러 책이나 온라인 자료, 연구 논문 등 다양한 출처를 통해 해당 주제를 깊이 이해하는 데 집중하세요. 이는 다양한 관점과 설명 방식을 접하며 자신에게 맞는 학습 방법을 찾는 데 도움이 됩니다.

R 언어 사용

이 강의에서 R 언어는 구조적 토픽 모델링(STM)을 다룰 때만 사용됩니다. STM은 R에 구현된 패키지를 사용하므로, 이 특정 모델을 다룰 때만 R을 사용하게 됩니다. R에 대한 사전 지식이 없어도 괜찮으며, 필요한 부분만 간략하게 소개될 예정입니다. 만약 최종 프로젝트 주제로 STM을 선택한다면 R을 사용해야 합니다.

신경망(Neural Networks) 소개

수동 특징 설계의 한계

선형 회귀 예시

신경망의 필요성을 이해하기 위해, 먼저 선형 회귀(Linear Regression) 모델을 살펴봅시다. 우리가 예측하려는 출력($\hat{y}$)이 입력(x)의 다항식 형태로 주어지는 경우를 가정해봅시다:

$$\hat{y} = w_0 + w_1x + w_2x^2 + w_3x^3$$

여기서 x 는 입력 데이터이고, w_0, w_1, w_2, w_3 는 모델이 학습해야 할 파라미터(가중치)입니다. 이 모델은 x, x^2, x^3 을 각각 특징(feature) u_1, u_2, u_3 으로 간주하여 선형 모델을 피팅하는 것과 같습니다. 즉, $u_1 = x, u_2 = x^2, u_3 = x^3$ 으로 정의하면, $\hat{y} = w_0 + w_1u_1 + w_2u_2 + w_3u_3$ 가 됩니다. 이러한 모델은 파라미터 w 에 대해 선형이므로 '선형 회귀'라고 불립니다.

예제

수동 특징 설계의 문제점:

- **수동 설계:** x^2, x^3 과 같은 특징을 사람이 직접 설계해야 합니다. 이는 데이터를 시각적으로 분석하여 적절한 형태를 추측해야 함을 의미합니다.
- **고차원 데이터의 어려움:** 이미지(수많은 픽셀), 언어(단어 벡터 시퀀스)와 같은 고차원 데이터에서는 시각적 분석이나 수동 특징 설계가 거의 불가능합니다.
- **비선형 파라미터의 복잡성:** 만약 x 의 거듭제곱($x^{p_1}, x^{p_2}, x^{p_3}$)에서 p_1, p_2, p_3 까지 파라미터로 학습해야 한다면, 문제는 매우 비선형적이고 복잡해져 수치적으로 해결하기 어려워집니다.

신경망의 기본 아이디어: 자동 특징 학습

신경망은 이러한 수동 특징 설계의 한계를 극복하기 위해 고안되었습니다. 핵심 아이디어는 모델이 특징을 자동으로 학습하도록 하는 것입니다. 이를 위해 신경망은 '비선형성'을 도입하면서도 '중간 단계'에서는 선형 변환'을 사용하는 방식을 채택합니다.

- **비선형성 도입:** 신경망은 입력에 비선형 함수(f)를 적용하여 새로운 특징을 생성합니다.
- **중간 선형 변환:** 이 비선형 함수는 입력의 '선형 결합'에 적용됩니다.

$$u_1 = f(w_{10} + w_{11}x_1 + w_{12}x_2 + \dots)$$

$$u_2 = f(w_{20} + w_{21}x_1 + w_{22}x_2 + \dots)$$

그리고 최종 출력 $\hat{y}$ 또한 이러한 중간 특징($u_1, u_2, \dots$)의 선형 결합에 비선형 함수를 적용하여 얻습니다.

$$\hat{y} = f(w_0 + w_1 u_1 + w_2 u_2 + \dots)$$

여기서 w 값들은 모델이 학습해야 할 파라미터입니다.

핵심 정보

왜 중간에 선형 변환을 사용할까? 이러한 구조는 미분(differentiation)을 쉽게 하기 위함입니다. 미적분학의 연쇄 법칙(chain rule)을 적용하면, 비선형 함수 f 가 선형 결합에 적용된 형태는 쉽게 미분할 수 있습니다. 이는 신경망의 파라미터를 효율적으로 최적화하는 데 결정적인 역할을 합니다.

신경망의 구조 (시각적 표현)

신경망은 여러 층(layer)으로 구성되며, 각 층은 여러 뉴런(neuron)을 포함합니다.

- **입력층 (Input Layer):** 외부 데이터를 받아들이는 부분입니다 (예: x_1, x_2).
- **은닉층 (Hidden Layer):** 입력층과 출력층 사이에 위치하며, 자동으로 특징을 학습하는 역할을 합니다 (예: $u_1, u_2, \dots$).
- **출력층 (Output Layer):** 최종 예측 값을 생성합니다 (예: $\hat{y}$).

각 뉴런은 이전 층의 뉴런들로부터 입력을 받아 선형 결합을 수행한 후, 활성화 함수(f)를 적용하여 다음 층으로 출력을 전달합니다. 각 연결선은 가중치(weight, w)를 가지며, 각 뉴런은 편향(bias, w_0)을 가집니다.

Figure 1.1: 신경망의 시각적 구조 (예시)

(이미지 설명: 입력 x_1, x_2 가 여러 은닉 뉴런 $u_1, u_2, \dots$ 으로 연결되고, 이 은닉 뉴런들이 다시 출력 $\hat{y}$ 로 연결되는 다층 구조를 보여주는 다이어그램. 각 연결선에는 가중치가, 각 뉴런에는 활성화 함수가 적용됨.)

신경망은 본질적으로 '함수의 중첩(nested functions)' 형태를 가집니다. 즉, 함수가 선형 결합에 적용되고, 그 결과가 다시 다른 함수의 선형 결합에 적용되는 방식입니다. 충분히 많은 뉴런과 층을 사용하면, 신경망은 원하는 어떤 비선형 함수라도 원하는 정확도로 근사할 수 있다는 '범용 근사 정리(Universal Approximation Theorem)'가 존재합니다.

활성화 함수 (Activation Functions)

개념 및 역할

활성화 함수(f)는 신경망의 각 뉴런에서 선형 결합된 입력 값에 적용되는 비선형 함수입니다.

- **비선형성 도입:** 활성화 함수가 없다면, 아무리 층을 깊게 쌓아도 결국 선형 모델과 동일한 결과를 내게 됩니다. 활성화 함수는 신경망이 복잡한 비선형 관계를 학습할 수 있도록 해줍니다.

- **생물학적 뉴런의 비유:** 역사적으로 활성화 함수는 생물학적 뉴런의 '활성화' 개념에서 유래했습니다. 뉴런은 특정 임계값 이상의 전기적 신호가 들어오면 활성화되어 다음 뉴런으로 신호를 전달합니다.
- **편향 (Bias):** 편향(w_0)은 뉴런의 활성화 임계값을 조절하는 역할을 합니다. 이는 선형 결합의 결과가 0이 아닌 다른 지점에서 활성화되도록 기준점을 이동시킵니다.

활성화 함수의 종류 및 특징

다양한 활성화 함수가 있으며, 각각의 특성에 따라 신경망의 학습 방식과 성능에 영향을 미칩니다.

1. 헤비사이드(Heaviside) 함수 (계단 함수)

- **정의:** 입력 Z 가 0보다 작으면 0, 0보다 크거나 같으면 1을 출력합니다.

$$f(Z) = \begin{cases} 0 & \text{if } Z < 0 \\ 1 & \text{if } Z \geq 0 \end{cases}$$

- **문제점:** 이 함수는 0에서 미분 불가능하며, 다른 모든 지점에서는 미분값이 0입니다. 이는 경사 하강법 (Gradient Descent)과 같은 최적화 알고리즘이 기울기를 통해 학습할 수 없게 만듭니다 (기울기가 0이므로 파라미터가 업데이트되지 않음). 따라서 실제 신경망에서는 사용되지 않습니다.

2. 렐루(ReLU: Rectified Linear Unit)

- **정의:** 입력 Z 가 0보다 작으면 0, 0보다 크거나 같으면 Z 를 출력합니다.

$$f(Z) = \max(0, Z)$$

- **특징:** 계산이 매우 효율적이고, 양수 영역에서는 기울기가 1로 일정하여 기울기 소실 (vanishing gradient) 문제를 완화합니다.
- **문제점:** 음수 영역에서는 기울기가 0이므로, 한 번 음수 값이 들어오면 해당 뉴런이 더 이상 활성화되지 않는 '죽은 ReLU(Dying ReLU)' 문제가 발생할 수 있습니다. 하지만 확률적 최적화 기법 덕분에 실제로는 잘 작동합니다.

3. 리키 렐루(Leaky ReLU)

- **정의:** ReLU의 '죽은 ReLU' 문제를 해결하기 위해, 음수 영역에서도 작은 기울기(α)를 가집니다.

$$f(Z) = \begin{cases} \alpha Z & \text{if } Z < 0 \\ Z & \text{if } Z \geq 0 \end{cases}$$

여기서 α 는 0과 1 사이의 작은 양수 값(예: 0.01)입니다.

- **특징:** 음수 영역에서도 기울기가 존재하여 뉴런이 죽는 것을 방지합니다.
- **하이퍼파라미터:** α 는 사용자가 직접 설정해야 하는 하이퍼파라미터입니다.

4. 맥스아웃(Maxout)

- 정의: 여러 선형 함수의 최댓값을 출력합니다.

$$f(Z_1, Z_2, \dots, Z_k) = \max(Z_1, Z_2, \dots, Z_k)$$

여기서 Z_i 는 각각 다른 가중치와 편향을 가진 선형 결합입니다.

- 특징: ReLU와 Leaky ReLU를 일반화한 형태로, 더 많은 유연성을 제공하지만 파라미터 수가 증가합니다.

5. 시그모이드(Sigmoid)

- 정의: 입력 Z 를 0과 1 사이의 값으로 압축합니다.

$$f(Z) = \frac{1}{1 + e^{-Z}}$$

- 특징: 출력이 확률로 해석될 수 있어 이진 분류(binary classification) 문제의 출력층에 주로 사용됩니다.
- 문제점: 입력값이 매우 크거나 작으면 기울기가 0에 가까워져 기울기 소실 문제가 발생할 수 있습니다.

6. 하이퍼볼릭 탄젠트(Tanh)

- 정의: 입력 Z 를 -1과 1 사이의 값으로 압축합니다.

$$f(Z) = \frac{e^Z - e^{-Z}}{e^Z + e^{-Z}}$$

- 특징: 시그모이드와 유사하지만, 출력이 0을 중심으로 대칭이므로 학습에 더 유리할 수 있습니다. 시그모이드와 마찬가지로 기울기 소실 문제가 있습니다.

7. 지수 선형 단위(ELU: Exponential Linear Unit)

- 정의: 음수 영역에서 지수 함수 형태를 가집니다.

$$f(Z) = \begin{cases} \alpha(e^Z - 1) & \text{if } Z < 0 \\ Z & \text{if } Z \geq 0 \end{cases}$$

- 특징: ReLU의 장점을 가지면서도 음수 영역에서 부드러운 변화를 보여 '죽은 ReLU' 문제를 완화합니다.

핵심 정보

활성화 함수 선택의 중요성: 이론적으로는 비선형 활성화 함수를 충분히 많이 사용하면 어떤 함수라도 근사할 수 있으므로, 어떤 활성화 함수를 선택하든 큰 차이가 없을 수 있습니다. 하지만 실제 구현 및 최적화 관점에서는 활성화 함수의 선택이 학습 속도, 수렴 여부, 지역 최저점(local minimum)에 빠지는 경향 등 성능에 큰 영향을 미칠 수 있습니다. 따라서 실제 문제에서는 다양한 활성화 함수를 실험해보고 최적의 것을 찾는 것이 중요합니다.

신경망 유형: 회귀와 분류

신경망은 출력층의 활성화 함수와 뉴런 수에 따라 다양한 문제를 해결할 수 있습니다.

1. 회귀 (Regression)

- 목표: 연속적인 숫자 값을 예측합니다 (예: 주택 가격, 온도).
- 출력층:
 - 단일 스칼라 출력: 출력 뉴런 1개, 활성화 함수 없음(선형 출력) 또는 ReLU (0 이상의 값 예측).
 - 다차원 벡터 출력: 출력 뉴런 M 개, 활성화 함수 없음(선형 출력).
- Keras 예시:

```

1 from tensorflow import keras
2 from tensorflow.keras import layers
3
4 model = keras.Sequential([
5     layers.Dense(16, activation='relu', input_shape=(n,)), # 은닉층: 개 16 뉴
      런, ReLU 활성화
6     layers.Dense(1) # 출력층: 개 1 뉴런 스칼라(출력), 활성화 함수 없음 선형()
7 ])
8

```

Listing 1.1: Keras 회귀 모델 예시 (ReLU 활성화 함수 사용)

이 모델은 0 이상의 값을 출력할 수 있습니다. 만약 음수 값도 예측해야 한다면 출력층에 활성화 함수를 지정하지 않거나 `linear`를 사용합니다.

2. 분류 (Classification)

- 목표: 입력 데이터가 어떤 범주(클래스)에 속하는지 예측합니다 (예: 고양이/개, 긍정/부정).
- 출력층:
 - 이진 분류 (2개 클래스): 출력 뉴런 1개, 시그모이드(sigmoid) 활성화 함수. 출력은 0과 1 사이의 확률입니다.
 - 다중 분류 (M 개 클래스): 출력 뉴런 M 개, 소프트맥스(softmax) 활성화 함수. 출력은 각 클래스에 속할 확률 분포(총합이 1)입니다.
- Keras 예시:

```

1 from tensorflow import keras
2 from tensorflow.keras import layers
3
4 model = keras.Sequential([
5     layers.Dense(16, activation='relu', input_shape=(n,)), # 은닉층: 개 16 뉴
      런, ReLU 활성화
6     layers.Dense(M, activation='softmax') # 출력층: 개 M 뉴런 (개 M 클래
      스), Softmax 활성화
7 ])
8

```

Listing 1.2: Keras 분류 모델 예시 (Softmax 활성화 함수 사용)

손실 함수 (Loss Functions)

개념 및 역할

손실 함수(Loss Function)는 모델의 예측($\hat{y}$)이 실제 관측 값(y)과 얼마나 다른지 측정하는 함수입니다.

- **개별 손실 (L_i):** 하나의 데이터 포인트(x_i, y_i)에 대한 예측($\hat{y}_i$)과 실제 값(y_i) 사이의 차이를 측정합니다.
- **비용 함수 (J):** 전체 데이터셋에 대한 개별 손실들의 평균입니다. 신경망 학습의 목표는 이 비용 함수를 최소화하는 것입니다.

$$J(W) = \frac{1}{M} \sum_{i=1}^M L_i(W, x_i, y_i)$$

여기서 M 은 데이터 포인트의 총 개수, W 는 신경망의 모든 파라미터(가중치와 편향)를 나타냅니다.

손실 함수의 선택은 해결하려는 문제의 유형(회귀 또는 분류)에 따라 달라집니다.

회귀 문제용 손실 함수

회귀 문제는 연속적인 숫자 값을 예측하는 문제이므로, 예측 값과 실제 값 사이의 '거리'를 측정하는 손실 함수를 사용합니다.

1. 제곱 오차 (Squared Error)

- **정의:** 예측 값과 실제 값의 차이를 제곱한 것입니다.
 - 1차원 출력: $L_i = (\hat{y}_i - y_i)^2$
 - 다차원 출력 (M 차원): $L_i = \sum_{j=1}^M (\hat{y}_{ij} - y_{ij})^2$ (유클리드 거리의 제곱)
- **비용 함수:** 평균 제곱 오차(Mean Squared Error, MSE)라고 불립니다.

$$\text{MSE} = \frac{1}{M} \sum_{i=1}^M (\hat{y}_i - y_i)^2$$

- **특징:** 오차가 클수록 더 큰 페널티를 부여합니다. 미분 가능하여 최적화에 용이합니다.

2. 절대 오차 (Absolute Error)

- **정의:** 예측 값과 실제 값의 차이에 절댓값을 취한 것입니다.
 - 1차원 출력: $L_i = |\hat{y}_i - y_i|$
 - 다차원 출력 (M 차원): $L_i = \sqrt{\sum_{j=1}^M (\hat{y}_{ij} - y_{ij})^2}$ (유클리드 거리)
- **비용 함수:** 평균 절대 오차(Mean Absolute Error, MAE)라고 불립니다.

$$\text{MAE} = \frac{1}{M} \sum_{i=1}^M |\hat{y}_i - y_i|$$

- **특징:** 이상치(outlier)에 덜 민감합니다. 0에서 미분 불가능하지만, 실제 최적화에서는 큰 문제가 되지 않습니다.

두 함수의 비교

- **최저점 위치:** MSE와 MAE 모두 최적의 파라미터 W 를 찾으면 동일한 전역 최저점(global minimum)을 가리킵니다. 즉, 한 함수를 최소화하는 W 는 다른 함수도 최소화합니다.
- **수렴 특성:** 두 함수의 '모양(shape)'이 다르기 때문에, 최적화 알고리즘의 수렴 속도와 지역 최저점에 빠지는 경향이 달라집니다.
 - **MSE:** 오차가 클 때 기울기가 커서 빠르게 움직이고, 최저점에 가까워질수록 기울기가 작아져 정밀하게 수렴합니다.
 - **MAE:** 기울기가 일정하여 최저점에 가까워져도 스텝 크기가 크게 줄어들지 않아 최저점 주변에서 진동할 수 있습니다.
- **실제 적용:** 실제로는 전역 최저점을 찾기 어렵고 지역 최저점에 빠질 가능성이 높기 때문에, 두 손실 함수 중 어떤 것이 더 나은 수렴 경로를 제공하는지 실험해보고 선택해야 합니다.

분류 문제용 손실 함수

분류 문제는 데이터가 특정 범주에 속할 '확률'을 예측하는 문제이므로, 확률 분포 간의 차이를 측정하는 손실 함수를 사용합니다.

1. 원-핫 인코딩 (One-Hot Encoding)

- **개념:** 범주형 데이터를 컴퓨터가 이해할 수 있는 이진 벡터 형태로 변환하는 방법입니다.
- **예시:** '고양이', '개' 두 클래스가 있다면,
 - 고양이: $[1, 0]$
 - 개: $[0, 1]$
 세 클래스('고양이', '개', '소')라면:
 - 고양이: $[1, 0, 0]$
 - 개: $[0, 1, 0]$
 - 소: $[0, 0, 1]$
- **역할:** 실제 관측 값(y)을 이러한 원-핫 벡터로 표현하여 모델의 예측($\hat{y}$, 확률 벡터)과 비교합니다.

2. 소프트맥스 (Softmax) 함수

- **개념:** 다중 분류 문제의 출력층에 사용되는 활성화 함수로, 신경망의 최종 출력을 각 클래스에 속할 확률 분포로 변환합니다.
- **정의:** M 개의 클래스가 있을 때, 각 클래스 k 에 대한 확률 $\hat{y}_k$ 는 다음과 같이 계산됩니다.

$$\hat{y}_k = \frac{e^{Z_k}}{\sum_{j=1}^M e^{Z_j}}$$

여기서 Z_k 는 신경망의 이전 층에서 나온 선형 결합 값(로짓)입니다.

- **특징:**
 - 모든 $\hat{y}_k$ 는 0보다 크고 1보다 작습니다.
 - 모든 $\hat{y}_k$ 의 합은 1입니다 ($\sum_{k=1}^M \hat{y}_k = 1$).

- **역할:** 신경망이 '고양이일 확률 0.9, 개일 확률 0.1'과 같은 형태로 예측을 출력할 수 있게 합니다.

3. 분류 문제에서 MSE가 부적합한 이유

분류 문제에서 MSE를 손실 함수로 사용하는 것은 일반적으로 좋지 않은 결과를 초래합니다.

- **이질적인 값 비교:** MSE는 연속적인 확률 값(예측 $\hat{y}$)과 이산적인 이진 값(원-핫 인코딩 y) 사이의 차이를 측정합니다. 이는 본질적으로 다른 유형의 값을 비교하는 것입니다.
- **비선형성 증가:** 이러한 이질적인 값의 차이를 제공하면, 파라미터 W 에 대한 비용 함수가 매우 비선형적이고 복잡한 형태를 띠게 됩니다.
- **최적화 어려움:** 결과적으로 MSE를 사용하여 분류 문제를 최적화하려고 하면, 알고리즘이 쉽게 지역 최저점에 빠져 전역 최저점을 찾기 매우 어려워집니다.

4. 교차 엔트로피 (Cross-Entropy Loss)

분류 문제에서 가장 널리 사용되는 손실 함수입니다.

- **정의:** 예측 확률 분포($\hat{y}$)가 실제 레이블 분포(y)와 얼마나 다른지 측정합니다.
 - **이진 교차 엔트로피 (Binary Cross-Entropy):** 2개 클래스 분류에 사용됩니다.

$$L_i = -[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

여기서 y_i 는 0 또는 1, $\hat{y}_i$ 는 0과 1 사이의 확률입니다.

- **범주형 교차 엔트로피 (Categorical Cross-Entropy):** M 개 이상의 클래스 분류에 사용됩니다.

$$L_i = - \sum_{k=1}^M y_{ik} \log(\hat{y}_{ik})$$

여기서 y_{ik} 는 i 번째 데이터의 실제 레이블(원-핫 벡터), $\hat{y}_{ik}$ 는 i 번째 데이터의 k 번째 클래스 예측 확률입니다.

- **예시 (범주형 교차 엔트로피):**
 - 실제 레이블 $y = [1, 0]$ (고양이)
 - 예측 $\hat{y} = [0.99, 0.01]$ (고양이일 확률 99%)

$$L = -[1 \cdot \log(0.99) + 0 \cdot \log(0.01)] \approx -[1 \cdot (-0.01)] = 0.01$$

손실이 매우 작습니다.

- 실제 레이블 $y = [1, 0]$ (고양이)
- 예측 $\hat{y} = [0.01, 0.99]$ (개일 확률 99%)

$$L = -[1 \cdot \log(0.01) + 0 \cdot \log(0.99)] \approx -[1 \cdot (-4.6)] = 4.6$$

손실이 매우 큼니다.

- **장점:**
 - **강력한 페널티:** 모델이 잘못된 클래스를 높은 확신으로 예측할 때 매우 큰 손실을 부여합니다.

- **최적화 용이성**: MSE보다 파라미터에 대한 비용 함수의 형태가 더 부드럽고 볼록하여 최적화가 훨씬 용이합니다.
- **수치적 안정성**: 소프트맥스 함수는 0을 출력하지 않으므로, $\log(0)$ 으로 인한 무한대 값은 실제로는 발생하지 않습니다.

핵심 정보

손실 함수와 메트릭(Metric)의 차이:

- **손실 (Loss)**: 신경망이 '최소화'해야 할 대상입니다. 학습 과정에서 파라미터를 업데이트하는 데 사용됩니다.
- **메트릭 (Metric)**: 모델의 '성능을 평가'하는 데 사용됩니다. 학습 과정 중 주기적으로 계산되어 사용자에게 모델의 상태를 보여주지만, 직접적으로 최소화되는 대상은 아닙니다 (예: 정확도 (accuracy)).

최적화 알고리즘 (Optimizers)

개념 및 역할

최적화 알고리즘(Optimizer)은 신경망의 파라미터(가중치 W)를 조정하여 비용 함수($J(W)$)를 최소화하는 방법을 정의합니다. 신경망의 비용 함수는 일반적으로 매우 복잡하고 비볼록(non-convex)한 형태를 가지며, 수많은 지역 최저점(local minimum)을 포함합니다. 따라서 최적화 알고리즘은 전역 최저점(global minimum)에 최대한 가깝게 도달하는 효율적인 방법을 찾아야 합니다.

Figure 1.2: 비용 함수의 복잡한 지형 (예시)

(이미지 설명: 파라미터 공간(W)에서 비용 함수(J)가 여러 개의 봉우리와 골짜기(지역 최저점)를 가진 복잡한 3D 지형처럼 그려진 그림. 전역 최저점을 찾는 것이 목표임을 시각적으로 보여줌.)

경사 하강법 (Gradient Descent, GD)

원리

경사 하강법은 비용 함수의 기울기(gradient)를 계산하여, 기울기가 가리키는 반대 방향(가장 가파르게 내려가는 방향)으로 파라미터를 업데이트하는 방법입니다.

- **기울기 (Gradient)**: 각 파라미터에 대한 비용 함수의 편미분 값들을 모아놓은 벡터입니다. 기울기는 함수가 가장 빠르게 증가하는 방향을 가리킵니다.

$$\nabla J(W) = \left[\frac{\partial J}{\partial W_1}, \frac{\partial J}{\partial W_2}, \dots, \frac{\partial J}{\partial W_D} \right]$$

- **파라미터 업데이트**:

$$W_{\text{새로운}} = W_{\text{이전}} - \alpha \cdot \nabla J(W_{\text{이전}})$$

여기서 α 는 학습률(learning rate)로, 한 번에 얼마나 크게 이동할지 결정하는 하이퍼파라미터입니다.

문제점

- **높은 계산 비용:** 한 번의 파라미터 업데이트를 위해 전체 데이터셋(M 개의 모든 관측치)에 대한 손실 함수의 기울기를 계산해야 합니다. 데이터셋이 클수록 한 스텝에 드는 계산 비용이 매우 커집니다.
- **지역 최저점:** GD는 결정론적(deterministic) 알고리즘이므로, 시작점에 따라 가장 가까운 지역 최저점에 갇힐 수 있습니다.
- **학습률의 중요성:** α 가 너무 크면 최저점을 지나쳐 발산할 수 있고, 너무 작으면 수렴하는 데 매우 오랜 시간이 걸립니다.

확률적 경사 하강법 (Stochastic Gradient Descent, SGD)

원리

SGD는 GD의 높은 계산 비용 문제를 해결하기 위해 고안되었습니다. 한 번의 파라미터 업데이트를 위해 전체 데이터셋이 아닌, '단 하나의 무작위로 선택된 데이터 포인트'에 대한 손실 함수의 기울기만 계산하여 사용합니다.

- **파라미터 업데이트:**

$$W_{\text{새로운}} = W_{\text{이전}} - \alpha \cdot \nabla L_i(W_{\text{이전}})$$

여기서 L_i 는 i 번째 데이터 포인트에 대한 손실 함수입니다.

장점

- **계산 효율성:** 각 업데이트 스텝의 계산 비용이 매우 저렴합니다.
- **지역 최저점 탈출:** 단일 데이터 포인트의 기울기를 사용하므로 업데이트 방향에 무작위성(stochasticity)이 생깁니다. 이 무작위성은 알고리즘이 얽은 지역 최저점에서 벗어나는 데 도움을 줍니다.

문제점

- **높은 변동성:** 업데이트 방향이 매우 불규칙하고 노이즈가 심하여, 수렴 경로가 불안정하고 최저점 주변에서 크게 진동할 수 있습니다. 이는 수렴하는 데 오랜 시간이 걸리거나 최저점에 정확히 도달하기 어렵게 만듭니다.
- **병렬 처리의 어려움:** 각 스텝마다 파라미터가 업데이트되므로, 다음 스텝의 기울기 계산은 이전 스텝의 업데이트된 파라미터에 의존합니다. 이로 인해 병렬 계산이 어렵습니다.

미니 배치 경사 하강법 (Mini-Batch Gradient Descent)

미니 배치 경사 하강법은 GD와 SGD의 장점을 결합한 가장 널리 사용되는 최적화 알고리즘입니다.

원리

한 번의 파라미터 업데이트를 위해 전체 데이터셋이나 단일 데이터 포인트가 아닌, '작은 묶음(미니 배치)'의 데이터 포인트에 대한 손실 함수의 평균 기울기를 계산하여 사용합니다.

- **파라미터 업데이트:**

$$W_{\text{새로운}} = W_{\text{이전}} - \alpha \cdot \frac{1}{B} \sum_{j \in \text{batch}} \nabla L_j(W_{\text{이전}})$$

여기서 B 는 미니 배치 크기(mini-batch size)입니다.

장점

- **계산 효율성:** SGD보다는 스텝당 계산 비용이 높지만, GD보다는 훨씬 효율적입니다.
- **변동성 제어:** SGD보다 업데이트 방향의 노이즈가 적어 수렴이 더 안정적입니다.
- **병렬 처리:** 미니 배치 내의 모든 데이터 포인트에 대한 기울기 계산은 동일한 파라미터 W 를 기준으로 이루어지므로 병렬 처리가 가능합니다. 이는 GPU와 같은 하드웨어 가속을 효과적으로 활용할 수 있게 합니다.
- **지역 최저점 탈출:** 여전히 어느 정도의 무작위성을 유지하여 얇은 지역 최저점에서 벗어나는 데 도움을 줍니다.

미니 배치 크기의 중요성

미니 배치 크기는 최적화 과정의 핵심 하이퍼파라미터입니다.

- **작은 배치 크기 (예: 32, 64):**
 - 더 많은 무작위성: SGD에 가까워져 지역 최저점 탈출에 유리합니다.
 - 스텝당 계산은 빠르지만, 전체 데이터셋을 한 번 처리하는 데 더 많은 스텝이 필요합니다.
- **큰 배치 크기 (예: 1024 이상):**
 - 더 적은 무작위성: GD에 가까워져 수렴 경로가 안정적이지만, 지역 최저점에 갇힐 가능성이 높습니다.
 - 스텝당 계산은 느리지만, 전체 데이터셋을 한 번 처리하는 데 필요한 스텝 수가 적습니다.
- **수프 비유:** 수프 맛을 보기 위해 스푼으로 떠먹는 것에 비유할 수 있습니다. 냄비 크기(데이터셋 크기)와 상관없이, 스푼 크기(미니 배치 크기)가 적절해야 정확하게 맛을 볼 수 있습니다. 즉, 최적화의 무작위성은 데이터셋 크기가 아닌 미니 배치 크기에 의해 주로 결정됩니다.
- **일반적인 선택:** 32, 64, 128과 같은 2의 거듭제곱이 일반적으로 사용됩니다. 이는 GPU 메모리 구조와 관련이 있습니다.

다른 최적화 기법

미니 배치 경사 하강법을 기반으로 다양한 고급 최적화 기법들이 개발되었습니다. 이들은 학습률을 동적으로 조절하거나, 과거 기울기 정보를 활용하여 수렴 속도를 높이고 안정성을 개선합니다.

- **모멘텀 (Momentum):** 과거 업데이트 방향에 관성(momentum)을 부여하여 진동을 줄이고 수렴을 가속화합니다.
- **네스테로프 가속 경사 (Nesterov Accelerated Gradient, NAG):** 모멘텀과 유사하지만, 미리 한 발짝 내딛은 후의 기울기를 계산하여 더 정확한 업데이트를 수행합니다.
- **적응형 학습률 기법 (Adaptive Learning Rate Methods):**

- **Adagrad, RMSprop, Adadelta:** 각 파라미터마다 고유한 학습률을 적용하여, 자주 업데이트되는 파라미터는 학습률을 줄이고, 드물게 업데이트되는 파라미터는 학습률을 늘립니다.
- **Adam (Adaptive Moment Estimation):** RMSprop과 Momentum의 장점을 결합한 가장 인기 있고 강력한 최적화 알고리즘 중 하나입니다. 대부분의 경우 좋은 기본 성능을 제공합니다.

핵심 정보

최적화 알고리즘 선택:

- **기본 선택:** Adam 옵티마이저는 대부분의 문제에서 좋은 성능을 보여주므로, 기본적으로 사용하기 좋습니다.
- **실험:** SGD, Adam 등 다양한 옵티마이저와 그들의 하이퍼파라미터(학습률, 배치 크기 등)를 변경하며 실험하여 문제에 가장 적합한 것을 찾아야 합니다.
- **문서 참고:** 각 옵티마이저의 파라미터에 대한 자세한 정보는 Keras와 같은 라이브러리 문서를 참고하세요.

실습/코드/오류와 해결

과제 1 예시

첫 번째 과제는 신경망의 기본 개념을 이해하고 파이썬으로 구현하는 능력을 평가합니다.

예제

퀴즈 예시:

- 주어진 신경망 구조에서 순전파(forward propagation)를 통해 출력을 계산합니다.
- 주어진 파이썬 구현 코드에서 신경망의 파라미터(가중치 및 편향) 수를 계산합니다.

예제

과제 예시:

- 특정 신경망을 사용하여 검증 정확도(validation accuracy)를 계산하고 보고합니다.
- 다양한 최적화 알고리즘(optimizers)과 하이퍼파라미터(hyperparameters)를 실험하여 모델 성능을 비교합니다.

과제 제출 형식

과제는 다음 두 가지 형태로 제출해야 합니다.

- **보고서 (PDF/Word):** 문제 해결 과정, 주요 코드 스냅샷, 결과(플롯, 표), 그리고 결과에 대한 간략한 논의를 포함합니다. 주피터 노트북을 PDF로 변환하여 제출하는 것도 허용됩니다.
- **원본 코드 (Jupyter Notebook 또는 ZIP 파일):** TA가 코드를 직접 실행하여 검토할 수 있도록 합니다. 단일 주피터 노트북 파일인 경우 그대로 제출하고, 여러 개의 파이썬 파일인 경우 하나의 ZIP 파일로 압축하여 제출합니다.

핵심 정보

보고서 작성 팁: 과제를 수행하면서 중요한 코드 부분, 중간 결과, 최종 플롯 등을 스냅샷으로 찍어 보고서에 삽입하고, 각 결과에 대한 간략한 설명을 덧붙이세요. 이는 나중에 자신의 작업을 다시 검토할 때 매우 유용합니다.

과제 해결 가이드

- Canvas에 게시된 예제 파일을 다운로드하여 실행해봅니다.
- 예제 코드를 기반으로 문제에서 요구하는 수정 사항을 적용합니다.
- 다양한 최적화 알고리즘(예: SGD, Adam)과 하이퍼파라미터(예: 학습률, 미니 배치 크기)를 실험하여 성능 변화를 관찰합니다.

예제

최적화 알고리즘 변경 예시 (Keras):

```

1 from tensorflow.keras import optimizers
2
3 # SGD 옵티마이저 사용 학습률( 0.01, 미니 배치 크기 32)
4 # model.compile(optimizer=optimizers.SGD(learning_rate=0.01),
5 #               loss='categorical_crossentropy',
6 #               metrics=['accuracy'])
7
8 # Adam 옵티마이저 사용 기본( 학습률)
9 model.compile(optimizer='adam', # 또
10              # optimizers.Adam(learning_rate=0.001)
11              loss='categorical_crossentropy',
12              metrics=['accuracy'])

```

Listing 1.3: Keras 옵티마이저 설정 예시

optimizer 매개변수에 'sgd' 또는 'adam' 과 같은 문자열을 직접 전달하거나, optimizers.SGD() 또는 optimizers.Adam() 객체를 생성하여 learning_rate 등의 하이퍼파라미터를 명시적으로 설정할 수 있습니다.

체크리스트

학습 점검표

- 강의 운영 방식 및 소통 채널을 이해했는가?
- 과제, 퀴즈, 최종 프로젝트의 마감일과 평가 기준을 숙지했는가?
- AI 도구 사용 정책을 이해하고 인용 의무를 인지했는가?
- 신경망의 기본 아이디어(자동 특징 학습, 비선형성)를 이해했는가?
- 활성화 함수의 역할과 주요 종류(ReLU, Leaky ReLU, Softmax)를 설명할 수 있는가?
- 손실 함수의 개념과 회귀/분류 문제에 적합한 손실 함수(MSE, MAE, Cross-Entropy)를 구분할 수

있는가?

- 최적화 알고리즘(GD, SGD, Mini-Batch GD)의 원리와 장단점을 이해했는가?
- 미니 배치 크기가 최적화에 미치는 영향을 설명할 수 있는가?

과제 제출 점검표

- 과제 마감일을 확인하고 보스턴 시간 기준으로 제출했는가?
- PDF 보고서에 모든 요구사항(코드 스냅샷, 결과, 플롯, 설명)을 포함했는가?
- 원본 코드(Jupyter Notebook 또는 ZIP 파일)를 함께 제출했는가?
- AI 도구를 사용했다면 보고서에 명확히 인용했는가?
- 특정 성능 목표가 있다면 달성했는지 확인했는가?

시험 대비 로드맵

- 기초 개념 숙지: 신경망, 활성화 함수, 손실 함수, 최적화 알고리즘의 정의와 '왜 필요한지'를 이해합니다.
- 원리 이해: 각 개념의 작동 원리(예: 경사 하강법의 파라미터 업데이트 방식, 교차 엔트로피의 손실 계산)를 파악합니다.
- 코드 실습: 제공된 예제 코드를 직접 실행하고 수정하며 개념을 코드로 구현하는 방법을 익힙니다.
- 하이퍼파라미터 실험: 다양한 하이퍼파라미터(학습률, 배치 크기)를 변경하며 모델 성능에 미치는 영향을 관찰합니다.
- 오류 분석: 과제 수행 중 발생한 오류의 원인을 파악하고 해결하는 과정을 통해 문제 해결 능력을 기릅니다.

FAQ

- Q: 퀴즈 마감 기한을 놓쳤는데, 재응시할 수 있나요? A: 아니요, 퀴즈 마감 기한은 매우 엄격하며, 마감 후에는 재응시나 대체 퀴즈가 제공되지 않습니다. 퀴즈 질문이 다음 강의에서 논의되기 때문입니다. 7일의 응시 기간 동안 시간을 내어 제출해야 합니다.
- Q: 최종 프로젝트 마감 기한도 퀴즈처럼 엄격한가요? A: 네, 최종 프로젝트 마감 기한은 학교 정책상 매우 엄격합니다. 마감 기한을 놓칠 경우, 학교의 승인을 받아야만 연장이 가능합니다. 미리 계획하고 준비하는 것이 중요합니다.
- Q: 강의에서 특정 교재를 따라가야 하나요? A: 아니요, 특정 교재를 필수로 지정하지 않습니다. 강의는 '주제' 중심으로 진행되므로, 여러 책이나 온라인 자료, 연구 논문 등 다양한 출처를 통해 해당 주제를 깊이 이해하는 데 집중하는 것을 권장합니다.
- Q: 신경망에서 활성화 함수를 사용하는 이유가 무엇인가요? A: 활성화 함수는 신경망에 '비선형성'을 도입하여 복잡한 패턴과 관계를 학습할 수 있도록 합니다. 만약 활성화 함수가 없다면, 아무리 층을 깊게 쌓아도 결국 선형 모델과 동일한 결과를 내게 됩니다.
- Q: 분류 문제에서 MSE(평균 제곱 오차) 대신 교차 엔트로피(Cross-Entropy)를 사용하는 이유가 무엇인가요? A: MSE는 연속적인 예측 확률과 이산적인 실제 레이블(원-핫 인코딩) 사이의 차이를 측정하므로, 비용 함수가 매우 비선형적이고 최적화하기 어렵습니다. 반면 교차 엔트로피는 확률 분포

간의 차이를 효과적으로 측정하며, 잘못된 예측에 큰 페널티를 부여하여 최적화가 훨씬 용이합니다.

- **Q: 미니 배치 크기는 어떻게 선택해야 하나요? A:** 미니 배치 크기는 하이퍼파라미터이며, 문제와 데이터셋의 특성에 따라 최적의 값이 달라집니다. 일반적으로 32, 64, 128과 같은 2의 거듭제곱이 많이 사용됩니다. 작은 배치 크기는 무작위성을 높여 지역 최저점 탈출에 유리하고, 큰 배치 크기는 수렴 경로를 안정화하지만 지역 최저점에 갇힐 위험이 있습니다. 실험을 통해 최적의 값을 찾아야 합니다.

빠르게 훑어보기 (1 페이지 요약)

CSCI89 Lecture 1 핵심 요약, sharp corners, boxsep=3mm, left=2mm, right=2mm, top=2mm, bottom=2mm

강의 목표: 신경망(NN) 및 자연어 처리(NLP) 소개.

소통: Piazza (공식 Q&A), WhatsApp (학생 소통), Canvas (공지).

세션: 강의(화), 교수(금), TA(주 2회). 모두 녹화 제공.

평가: 과제(65%), 퀴즈(20%), 최종 프로젝트(15%).

마감: 퀴즈(엄격), 최종 프로젝트(매우 엄격).

AI 사용: 허용, 인용 필수, 책임은 본인.

NN 동기: 수동 특징 설계 한계 극복, 자동 특징 학습.

NN 구조: 입력-은닉-출력층, 가중치, 편향, 활성화 함수.

활성화 함수: 비선형성 도입. ReLU, Leaky ReLU, Sigmoid, Softmax 등.

손실 함수: 예측 오차 측정.

– **회귀:** MSE (제곱 오차), MAE (절대 오차).

– **분류:** Cross-Entropy (교차 엔트로피). Softmax와 함께 사용. MSE는 분류에 부적합.

최적화: 비용 함수 최소화.

– **GD:** 전체 데이터 사용, 느림, 지역 최저점.

– **SGD:** 단일 데이터 사용, 빠름, 노이즈, 지역 최저점 탈출.

– **Mini-Batch GD:** 작은 배치 사용, GD와 SGD 장점 결합, 가장 일반적.

– **Adam:** 인기 있는 고급 옵티마이저.

하이퍼파라미터: 학습률(α), 미니 배치 크기. 실험을 통해 최적화.

Chapter 2

Lecture 2

강의명: CSCI E-89: Introduction to AI

주차: Lecture 2

교수명: Prof. John Doe

목적: 인공지능 핵심 개념, 신경망, RNN, TF-IDF 이해 및 실습

핵심 요약

본 문서는 CSCI89 Lecture 2 강의 자료를 통합하여 인공지능의 핵심 개념을 설명합니다. 신경망의 순전파와 역전파 원리, 학습률의 중요성, 순환 신경망(RNN)의 구조와 한계, LSTM 및 GRU와 같은 개선 모델, 그리고 텍스트 분류를 위한 TF-IDF 기법과 T-검정 기반 특징 선택 방법을 다룹니다. 실제 특히 데이터 분류 및 시계열 예측 예시를 통해 이론을 실습에 적용하는 방법을 제시하며, 초심자도 완벽히 이해할 수 있도록 상세한 설명과 예시를 제공합니다.

2.1 개요

이 문서는 CSCI E-89: Introduction to Artificial Intelligence 과목의 두 번째 강의 내용을 담고 있습니다. 신경망의 기본적인 동작 원리부터 시작하여, 복잡한 시퀀스 데이터를 처리하는 순환 신경망(RNN)과 그 변형들, 그리고 텍스트 데이터를 수치화하는 TF-IDF 기법까지 폭넓게 다룹니다. 각 개념은 비전공자도 쉽게 이해할 수 있도록 배경 설명, 직관적인 비유, 그리고 구체적인 예시를 포함하여 설명합니다.

2.2 용어 정리

인공지능 및 머신러닝 분야에서 사용되는 주요 용어들을 정리했습니다. 각 용어의 쉬운 설명과 함께 원어 및 추가적인 비고를 제공하여 이해를 돕습니다.

Table 2.1: 주요 용어 정리

용어	위문 설명	원어	비고
순전파	입력 데이터가 신경망을 통과하여 최종 출력을 계산하는 과정	Forward Propagation	신경망의 '계산' 단계
역전파	출력 오차를 기반으로 신경망의 가중치를 업데이트하기 위한 기울기를 계산하는 과정	Backward Propagation	신경망의 '학습' 단계
ReLU	입력이 양수이면 그대로 출력하고, 음수이면 0을 출력하는 활성화 함수	Rectified Linear Unit	가장 널리 사용되는 활성화 함수 중 하나
Softmax	다중 클래스 분류 문제에서 각 클래스에 속할 확률을 출력하는 활성화 함수	Softmax	모든 출력 확률의 합은 1
MSE	예측 값과 실제 값의 차이(오차)를 제곱하여 평균 낸 값	Mean Squared Error	회귀 문제에서 주로 사용되는 손실 함수
경사 하강법	손실 함수를 최소화하기 위해 함수의 기울기를 따라 파라미터를 점진적으로 업데이트하는 최적화 알고리즘	Gradient Descent	신경망 학습의 기본 원리
학습률 (α)	경사 하강법에서 파라미터 업데이트 시 기울기에 곱해지는 스케일 값	Learning Rate	하이퍼파라미터, 학습 속도 조절
하이퍼파라미터	모델 학습 전에 사용자가 직접 설정하는 값 (예: 학습률, 은닉층 수)	Hyperparameter	모델의 성능에 큰 영향
학습 가능한 파라미터	모델 학습 과정에서 데이터로부터 자동으로 최적화되는 값 (예: 가중치, 편향)	Trainable Parameter	모델의 '지식'을 구성
기울기 소실	역전파 과정에서 기울기 값이 너무 작아져 초기 층의 가중치가 거의 업데이트되지 않는 문제	Vanishing Gradient	RNN의 주요 문제점
기울기 폭주	역전파 과정에서 기울기 값이 너무 커져 가중치가 비정상적으로 업데이트되는 문제	Exploding Gradient	RNN의 또 다른 문제점
RNN	시퀀스 데이터를 처리하기 위해 이전 단계의 정보를 현재 단계로 전달하는 신경망	Recurrent Neural Network	시계열, 자연어 처리
LSTM	RNN의 기울기 소실 문제를 해결하기 위해 고안된 복잡한 구조의 순환 신경망	Long Short-Term Memory	장기의존성 학습에 효과적
GRU	LSTM보다 간소화된 구조를 가지면서도 유사한 성능을 내는 순환 신경망	Gated Recurrent Unit	파라미터 수가 적어 효율적
TF-IDF	문서 내 단어의 중요도를 측정하는 통계적 수치	Term Frequency-Inverse Document Frequency	텍스트 벡터화 기법
T-검정	두 그룹 간의 평균 차이가 통계적으로 유의미한지 검정하는 통계 방법	T-test	특정 선택에 활용 가능
Epoch	전체 훈련 데이터셋을 신경망에 한 번 통과시키는 과정	Epoch	여러 번의 가중치 업데이트 포함
미니 배치	전체 훈련 데이터셋을 작은 묶음으로 나누어 한 번에 처리하는 데이터 단위	Mini-batch	효율적인 학습을 위한
Dropout	신경망 학습 시 과적합을 방지하기 위해 무작위로 뉴런의 연결을 끊는 정규화 기법	Dropout	훈련 시에만 적용
RMSProp	학습률을 파라미터별로 동적으로 조절하여 수렴을 돕는 최적화 알고리즘	Root Mean Square Propagation	Adam과 함께 널리 사용
Adam	RMSProp과 Momentum을 결합하여 학습률을 효율적으로 조절하는 최적화 알고리즘	Adaptive Moment Estimation	가장 널리 사용되는 최적화 알고리즘

2.3 핵심 개념 및 원리

2.3.1 신경망의 기본 동작: 순전파 (Forward Propagation)

신경망은 입력 데이터를 받아 여러 층을 거쳐 최종 출력을 만들어냅니다. 이 과정을 **순전파(Forward Propagation)**라고 합니다. 마치 공장에서 원재료(입력)가 여러 공정(신경망 층)을 거쳐 완제품(출력)이 되는 것과 같습니다.

- 한 줄 핵심 요약:** 입력 데이터가 신경망의 각 층을 순서대로 통과하며 계산되어 최종 출력을 생성하는 과정입니다.
- 직관적 예시/비유:** 계산기를 사용하여 복잡한 수식을 단계별로 푸는 것과 같습니다. 각 단계의 결과가 다음 단계의 입력이 됩니다.
- 기술적 정확한 설명:** 신경망의 각 뉴런은 이전 층의 출력에 가중치(Weight, W)를 곱하고 편향(Bias, b)을 더한 후, 활성화 함수(Activation Function, f)를 적용하여 자신의 출력을 계산합니다. 이 출력이 다음 층의 입력이 됩니다. 최종 층의 출력이 신경망의 최종 예측 값($\hat{y}$)이 됩니다.

$$\text{뉴런 출력} = f\left(\sum(\text{이전 층 출력} \times W) + b\right)$$

예제

ReLU (Rectified Linear Unit) 활성화 함수 ReLU는 신경망에서 가장 흔히 사용되는 활성화 함수 중 하나입니다.

- 정의:** 입력 값이 0보다 크면 그 값을 그대로 출력하고, 0보다 작거나 같으면 0을 출력합니다.
- 수식:** $f(z) = \max(0, z)$
- 특징:**
 - 양수 입력:** 입력이 양수일 때는 기울기가 1이므로 정보 손실 없이 전달됩니다.
 - 음수 입력:** 입력이 음수일 때는 기울기가 0이므로 해당 뉴런은 비활성화됩니다. 이는 신경망을 희소하게 만들어 계산 효율을 높일 수 있습니다.

예제

퀴즈 1번: 순전파 계산 예시 주어진 신경망에서 최종 출력 $\hat{y}$ 를 계산하는 과정입니다. 활성화 함수는 ReLU를 사용합니다.

1단계: 첫 번째 은닉 뉴런 U_1 계산 U_1 은 입력 X_1, X_2 와 편향 $W_{01}^{(1)}$ 을 받습니다.

- $W_{01}^{(1)} = -1.2$ (편향)
- $W_{11}^{(1)} = 0.1$
- $W_{21}^{(1)} = 0.5$
- $X_1 = 1.3$
- $X_2 = 0.7$

선형 결합 $Z_1 = W_{01}^{(1)} + W_{11}^{(1)} \cdot X_1 + W_{21}^{(1)} \cdot X_2$ $Z_1 = -1.2 + (0.1 \cdot 1.3) + (0.5 \cdot 0.7) = -1.2 + 0.13 + 0.35 = -0.72$ 활성화 함수 적용 $U_1 = f(Z_1) = \max(0, -0.72) = 0$ **결과: $U_1 = 0$**

2단계: 두 번째 은닉 뉴런 U_2 계산 U_2 은 입력 X_1, X_2 와 편향 $W_{02}^{(1)}$ 을 받습니다.

- $W_{02}^{(1)} = 0.9$ (편향)
- $W_{12}^{(1)} = 0.8$
- $W_{22}^{(1)} = 0.3$
- $X_1 = 1.3$
- $X_2 = 0.7$

선형 결합 $Z_2 = W_{02}^{(1)} + W_{12}^{(1)} \cdot X_1 + W_{22}^{(1)} \cdot X_2$ $Z_2 = 0.9 + (0.8 \cdot 1.3) + (0.3 \cdot 0.7) = 0.9 + 1.04 + 0.21 = 2.15$ 활성화 함수 적용 $U_2 = f(Z_2) = \max(0, 2.15) = 2.15$ **결과: $U_2 = 2.15$**

3단계: 최종 출력 $\hat{y}$ 계산 $\hat{y}$ 는 은닉 뉴런 U_1, U_2 와 편향 $W_0^{(2)}$ 을 받습니다.

- $W_0^{(2)} = 0.2$ (편향)
- $W_1^{(2)} = 0.8$
- $W_2^{(2)} = 1.2$
- $U_1 = 0$
- $U_2 = 2.15$

선형 결합 $Z_{\hat{y}} = W_0^{(2)} + W_1^{(2)} \cdot U_1 + W_2^{(2)} \cdot U_2$ $Z_{\hat{y}} = 0.2 + (0.8 \cdot 0) + (1.2 \cdot 2.15) = 0.2 + 0 + 2.58 = 2.78$ 활성화 함수 적용 $\hat{y} = f(Z_{\hat{y}}) = \max(0, 2.78) = 2.78$ **최종 결과: $\hat{y} = 2.78$**

팁

가중치(W)와 편향(Bias)의 역할

- **가중치 (W):** 입력 신호의 중요도를 조절합니다. 가중치가 크면 해당 입력이 출력에 미치는 영향이 커집니다.
- **편향 (Bias):** 뉴런의 활성화 임계값을 조절합니다. 입력 신호가 없어도 뉴런이 활성화될 수 있도록 하거나, 특정 임계값 이상이어야 활성화되도록 돕습니다. 이는 모델이 데이터를 더 유연하게 학습할 수 있도록 합니다.
- **학습 가능한 파라미터 (Trainable Parameters):** 가중치와 편향은 신경망이 데이터를 통해 스스로 학습하여 최적의 값을 찾아내는 '학습 가능한 파라미터'입니다. 사용자가 직접 설정하는 '하이퍼파라미터'와는 다릅니다.

2.3.2 신경망 학습의 핵심: 역전파 (Backward Propagation)

신경망이 예측한 값과 실제 값 사이에 오차가 발생하면, 이 오차를 줄이기 위해 신경망의 가중치와 편향을 조정해야 합니다. 이 조정을 위해 각 파라미터가 오차에 얼마나 기여했는지 계산하는 과정이 **역전파**

(Backward Propagation)입니다.

1. **한 줄 핵심 요약:** 신경망의 출력 오차를 기반으로, 연쇄 법칙을 사용하여 각 가중치와 편향에 대한 손실 함수의 기울기를 효율적으로 계산하는 알고리즘입니다.
2. **직관적 예시/비유:** 복잡한 기계에서 최종 결과물이 잘못 나왔을 때, 어느 부품(가중치)이 얼마나 잘못 되었는지 역으로 추적하여 고치는 과정과 같습니다.
3. **기술적 정확한 설명:** 역전파는 경사 하강법(Gradient Descent)과 함께 사용되어 신경망을 학습시킵니다. 손실 함수(L)의 값을 최소화하기 위해 각 가중치(W)에 대한 손실 함수의 기울기($\frac{\partial L}{\partial W}$)를 계산합니다. 이때, 순전파 과정에서 계산된 중간 값들을 재사용하여 계산 효율을 극대화합니다.

예제

연쇄 법칙 (Chain Rule)을 이용한 기울기 계산 최종 출력 $\hat{y}$ 에 대한 가중치 W 의 기울기를 계산할 때 연쇄 법칙을 사용합니다. 예를 들어, $\frac{\partial \hat{y}}{\partial W_1^{(2)}}$ 를 계산하려면:

$$\frac{\partial \hat{y}}{\partial W_1^{(2)}} = \frac{\partial \hat{y}}{\partial Z_{\hat{y}}} \cdot \frac{\partial Z_{\hat{y}}}{\partial W_1^{(2)}}$$

여기서 $\frac{\partial \hat{y}}{\partial Z_{\hat{y}}}$ 는 활성화 함수의 미분(f')이고, $\frac{\partial Z_{\hat{y}}}{\partial W_1^{(2)}}$ 는 U_1 이 됩니다.

$$\frac{\partial \hat{y}}{\partial W_1^{(2)}} = f'(Z_{\hat{y}}) \cdot U_1$$

만약 $U_1 = 0$ 이라면, $\frac{\partial \hat{y}}{\partial W_1^{(2)}}$ 도 0이 되어 해당 가중치는 업데이트되지 않습니다. 이처럼 순전파의 중간 결과(U_1)를 역전파에 재사용하는 것이 핵심입니다.

팁

손실 함수 (Loss Function)의 미분 신경망 학습의 궁극적인 목표는 손실 함수를 최소화하는 것입니다. 손실 함수는 예측 값($\hat{y}$)과 실제 값(y)의 차이를 측정합니다.

- 전체 기울기: $\frac{\partial L}{\partial W} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial W}$
- **MSE (Mean Squared Error) 예시:** 만약 손실 함수 $L = (\hat{y} - y)^2$ 이라면, $\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y)$ 가 됩니다. Keras나 TensorFlow 같은 딥러닝 프레임워크는 사용자가 손실 함수를 지정하면, 이처럼 분석적으로 미분된 $\frac{\partial L}{\partial \hat{y}}$ 값을 자동으로 계산하여 역전파에 활용합니다.

2.3.3 신경망 아키텍처 설계: 파라미터 수 계산

신경망의 복잡도는 파라미터(가중치와 편향) 수로 가늠할 수 있습니다. 파라미터 수가 많을수록 모델의 표현력은 높아지지만, 학습에 필요한 데이터와 계산량도 증가합니다.

1. **한 줄 핵심 요약:** 신경망의 파라미터 수는 각 층의 뉴런 간 연결 수와 각 뉴런의 편향수를 합한 값입니다.
2. **직관적 예시/비유:** 복잡한 건물을 짓는 데 필요한 벽돌(연결)과 기둥(편향)의 총 개수를 세는 것과 같습니다.
3. **기술적 정확한 설명:** 완전 연결 신경망(Fully Connected Network, Dense Layer)에서 각 뉴런은 이전 층의 모든 뉴런과 연결됩니다. 따라서 파라미터 수는 다음과 같이 계산됩니다.
 - 가중치 수: (이전 층 뉴런 수) $\times$ (현재 층 뉴런 수)

- 편향 수: (현재 층 뉴런 수)
- 총 파라미터 수: (이전 층 뉴런 수 + 1 (편향)) $\times$ (현재 층 뉴런 수)

예제

퀴즈 2번: 완전 연결 신경망 파라미터 수 계산

- 입력층: 700개 입력 ($X_1, \dots, X_{700}$)
- 첫 번째 은닉층: 8개 뉴런
- 두 번째 은닉층: 8개 뉴런
- 출력층: 1개 뉴런 ($\hat{y}$)

1단계: 입력층 $\rightarrow$ 첫 번째 은닉층

- 입력 수: 700개
- 편향: 1개 (입력층에 편향은 없지만, 다음 층의 각 뉴런이 편향을 가짐)
- 첫 번째 은닉층 뉴런 수: 8개
- 파라미터 수: $(700 \text{ (입력)} + 1 \text{ (편향)}) \times 8 \text{ (뉴런)} = 701 \times 8 = 5608$

2단계: 첫 번째 은닉층 $\rightarrow$ 두 번째 은닉층

- 이전 층 (첫 번째 은닉층) 뉴런 수: 8개
- 편향: 1개
- 두 번째 은닉층 뉴런 수: 8개
- 파라미터 수: $(8 \text{ (이전 층 뉴런)} + 1 \text{ (편향)}) \times 8 \text{ (뉴런)} = 9 \times 8 = 72$

3단계: 두 번째 은닉층 $\rightarrow$ 출력층

- 이전 층 (두 번째 은닉층) 뉴런 수: 8개
- 편향: 1개
- 출력층 뉴런 수: 1개
- 파라미터 수: $(8 \text{ (이전 층 뉴런)} + 1 \text{ (편향)}) \times 1 \text{ (뉴런)} = 9 \times 1 = 9$

총 파라미터 수: $5608 + 72 + 9 = 5689$

팁

Keras model.summary() 활용 실제 모델을 구축한 후 model.summary() 명령을 사용하면 각 층의 파라미터 수를 확인할 수 있습니다. 수동으로 계산한 값과 summary()의 결과가 일치하는지 확인하는 것은 신경망 아키텍처를 정확히 이해하고 있는지 점검하는 좋은 방법입니다.

2.3.4 출력층 활성화 함수 및 손실 함수 선택

신경망의 마지막 층(출력층)에서 어떤 활성화 함수를 사용하고, 어떤 손실 함수를 최소화할지는 해결하려는 문제의 종류(분류 또는 회귀)에 따라 달라집니다.

1. **한 줄 핵심 요약:** 문제 유형(분류/회귀)에 따라 출력층의 활성화 함수와 손실 함수를 적절히 선택해야 합니다.
2. **직관적 예시/비유:** 요리할 때 어떤 요리(문제 유형)를 만들지에 따라 마지막 양념(활성화 함수)과 맛 평가 기준(손실 함수)이 달라지는 것과 같습니다.
3. **기술적 정확한 설명:**

- **분류 문제 (Classification):**
 - **다중 클래스 분류 (Multi-class Classification):** 16개의 클래스를 분류하는 문제처럼 여러 개의 범주 중 하나를 예측할 때 사용합니다.
 - * **출력층 뉴런 수:** 분류할 클래스 수와 동일하게 설정합니다 (예: 16개 클래스 → 16개 뉴런).
 - * **활성화 함수:** **Softmax**를 사용합니다. Softmax는 각 뉴런의 출력을 0과 1 사이의 확률 값으로 변환하며, 모든 출력 확률의 합이 1이 되도록 합니다.
 - * **손실 함수:** **Categorical Cross-Entropy**를 주로 사용합니다.
 - **이진 클래스 분류 (Binary Classification):** 두 개의 범주 중 하나를 예측할 때 사용합니다.
 - * **출력층 뉴런 수:** 1개 또는 2개.
 - * **활성화 함수:** 1개 뉴런일 경우 **Sigmoid**를 사용합니다 (0과 1 사이의 확률 출력). 2개 뉴런일 경우 **Softmax**를 사용할 수도 있습니다.
 - * **손실 함수:** **Binary Cross-Entropy**를 주로 사용합니다.
- **회귀 문제 (Regression):** 연속적인 숫자 값을 예측할 때 사용합니다 (예: 주택 가격 예측).
 - **출력층 뉴런 수:** 예측하려는 값의 차원과 동일하게 설정합니다 (예: 하나의 숫자 예측 → 1개 뉴런).
 - **활성화 함수:** 일반적으로 활성화 함수를 사용하지 않거나 **선형 활성화 함수 (Linear Activation Function)**를 사용합니다.
 - **손실 함수:** **Mean Squared Error (MSE)** 또는 **Mean Absolute Error (MAE)**를 주로 사용합니다.

2.3.5 신경망 학습의 조절자: 학습률 (Learning Rate, α)

학습률(α)은 경사 하강법 (Gradient Descent)에서 손실 함수의 기울기를 따라 파라미터를 업데이트할 때, 한 번에 얼마나 크게 이동할지를 결정하는 하이퍼파라미터입니다.

1. **한 줄 핵심 요약:** 학습률은 신경망이 손실 함수의 최저점을 찾아가는 과정에서 한 걸음씩 움직이는 보폭을 결정하는 값입니다.
2. **직관적 예시/비유:** 산 정상에서 가장 낮은 계곡으로 내려갈 때, 한 걸음의 크기를 조절하는 것과 같습니다. 너무 작으면 시간이 오래 걸리고, 너무 크면 계곡을 건너뛰거나 엉뚱한 곳으로 갈 수 있습니다.
3. **기술적 정확한 설명:** 경사 하강법의 파라미터 업데이트 공식은 다음과 같습니다.

$$W_{\text{새로운}} = W_{\text{이전}} - \alpha \times \nabla J(W_{\text{이전}})$$

여기서 W 는 가중치 벡터, α 는 학습률, $\nabla J(W)$ 는 손실 함수 J 의 기울기 벡터입니다.

Figure 2.1: 학습률에 따른 손실 함수의 변화 (개념도)

학습률 α 값에 따른 학습 양상

- **α 가 너무 작을 때 (Small Learning Rate):**
 - **문제점:** 파라미터 업데이트 보폭이 너무 작아 손실 함수의 최저점에 도달하는 데 매우 오랜 시간이 걸립니다. 심지어 학습이 거의 진행되지 않아 초기 값에 고착될 수도 있습니다.
 - **시각적 특징:** 손실 함수 그래프가 거의 평평하게 유지되거나 매우 느리게 감소합니다.
 - **예상 오해:** 모델이 학습되지 않는다고 오해할 수 있지만, 실제로는 너무 느리게 학습되고 있는

것입니다.

- α 가 최적일 때 (Optimal Learning Rate):
 - 특징: 적절한 보폭으로 손실 함수의 최저점을 향해 효율적으로 이동합니다. 수렴 속도가 빠르고 안정적으로 최적점에 도달합니다.
 - 시각적 특징: 손실 함수 그래프가 꾸준히 감소하다가 안정적으로 수렴합니다.
 - 실제: 완벽한 '최적'은 찾기 어렵지만, 충분히 좋은 값을 통해 효율적인 학습이 가능합니다.
- α 가 너무 클 때 (Large Learning Rate):
 - 문제점: 파라미터 업데이트 보폭이 너무 커서 손실 함수의 최저점을 건너뛰거나, 오히려 손실 값이 증가하며 발산(Divergence)할 수 있습니다.
 - 시각적 특징: 손실 함수 그래프가 불안정하게 진동하거나 급격히 증가합니다.
 - 예상 오해: 학습이 전혀 안 된다고 생각할 수 있지만, 실제로는 너무 과도하게 업데이트되어 최적점을 찾지 못하고 있는 것입니다.
 - 진동하며 수렴: 때로는 α 가 약간 커서 최저점 주변에서 진동(Oscillation)하며 수렴하는 경우도 있습니다. 이는 완전히 발산하는 것보다는 낫지만, 최적의 수렴은 아닙니다.

주의

데이터 스케일링의 중요성 학습률의 기본값(예: 0.01 또는 0.001)은 다양한 문제에 적용될 수 있도록 설정되어 있습니다. 하지만 입력 데이터의 스케일이 매우 크거나 작으면 이 기본값이 적절하지 않을 수 있습니다.

- 문제: 예를 들어, 주택 가격(수백만 달러)을 입력으로 사용하는데, 학습률이 작은 경우 가중치 업데이트가 거의 이루어지지 않을 수 있습니다.
- 해결책: 입력 데이터를 스케일링(Scaling)하여 0과 1 사이 또는 평균 0, 표준편차 1(Z-score 표준화)과 같은 일정한 범위로 맞춰주는 것이 중요합니다. 이렇게 하면 기본 학습률로도 안정적인 학습이 가능해집니다.

학습률 조절 기법

- 학습률 스케줄링 (Learning Rate Scheduling): 학습이 진행됨에 따라 학습률을 점진적으로 감소시키는 방법입니다. 초반에는 빠르게 탐색하고, 후반에는 미세 조정을 통해 안정적인 수렴을 돕습니다.
- 최적화 기법 (Optimizers): Adam, RMSprop과 같은 고급 최적화 알고리즘들은 학습률을 단순히 고정된 값으로 사용하는 대신, 각 파라미터에 대해 학습률을 동적으로 조절하거나, 기울기 벡터의 방향을 조절(회전)하여 더욱 효율적인 학습을 가능하게 합니다.
 - RMSprop (Root Mean Square Propagation): 각 파라미터의 기울기 제곱의 이동 평균을 사용하여 학습률을 조절합니다. 기울기가 큰 파라미터는 학습률을 줄이고, 기울기가 작은 파라미터는 학습률을 늘려줍니다.
 - Adam (Adaptive Moment Estimation): RMSprop과 Momentum(이전 기울기 정보를 활용하여 업데이트 방향을 부드럽게 하는 기법)을 결합한 최적화 기법으로, 가장 널리 사용됩니다.

2.4 순환 신경망 (Recurrent Neural Networks, RNN)의 이해

일반적인 신경망(완전 연결 신경망, 합성곱 신경망 등)은 각 입력이 독립적이라고 가정합니다. 하지만 시계열 데이터나 자연어와 같은 시퀀스 데이터는 이전 정보가 현재 정보에 영향을 미치는 '순서'와 '맥락'

이 중요합니다. 순환 신경망(RNN)은 이러한 시퀀스 데이터를 처리하기 위해 고안된 신경망입니다.

2.4.1 RNN의 필요성: 시퀀스 데이터 처리

1. **한 줄 핵심 요약:** RNN은 순서가 중요한 시퀀스 데이터의 장기적인 의존성을 학습하기 위해 고안된 신경망입니다.
2. **직관적 예시/비유:** 문장을 이해할 때, 단어 하나하나를 독립적으로 보는 것이 아니라 앞선 단어들의 맥락을 기억하며 다음 단어를 이해하는 것과 같습니다.
3. **기술적 정확한 설명:**
 - **시계열 데이터 (Time Series Data):** 주식 가격, 기온 변화와 같이 시간 순서에 따라 관측되는 데이터입니다.
 - **자연어 (Natural Language):** 단어들이 문장이라는 순서를 이루어 의미를 전달하는 데이터입니다. 각 단어는 벡터로 표현될 수 있습니다.
 - **예측 문제:** 이전 관측 값들($X_1, X_2, \dots, X_T$)을 기반으로 다음 관측 값(X_{T+1})을 예측하는 것이 일반적인 시퀀스 예측 문제입니다.

기존 모델의 한계

- **자기회귀 (AutoRegressive, AR) 모델:** 고전적인 시계열 모델로, 다음 관측 값이 이전 관측 값들의 선형 함수라고 가정합니다.

$$X_{T+1} = \beta_0 + \beta_1 X_T + \beta_2 X_{T-1} + \dots$$

- **한계:** 선형 관계만 모델링 가능하며, 시계열의 분산이 시간에 따라 변하지 않는 '정상성(Stationarity)'을 가정합니다. 금융 데이터처럼 변동성이 크게 변하는 경우(예: GARCH 모델이 필요한 경우)에는 적합하지 않습니다.
- **완전 연결 신경망 (Fully Connected Network):** 시퀀스 데이터를 완전 연결 신경망으로 처리하려면, 모든 이전 관측 값들을 한꺼번에 입력으로 넣어야 합니다.
 - **문제점:** 시퀀스 길이가 길어지면 입력 뉴런 수가 폭증하고, 파라미터 수도 기하급수적으로 늘어나 학습이 매우 어려워집니다. 또한, 시퀀스 길이가 가변적일 때 유연하게 대처하기 어렵습니다.

2.4.2 순환 뉴런 (Recurrent Neuron)의 구조

RNN의 핵심은 가중치 공유(Weight Sharing)와 은닉 상태(Hidden State)입니다. 이는 시퀀스의 각 시간 단계에서 동일한 계산 로직(동일한 가중치)을 사용하고, 이전 시간 단계의 정보를 현재 시간 단계로 전달하여 맥락을 유지합니다.

1. **한 줄 핵심 요약:** 순환 뉴런은 각 시간 단계에서 동일한 가중치를 공유하며, 이전 시간 단계의 은닉 상태를 현재 시간 단계의 입력으로 받아 시퀀스의 맥락을 학습합니다.
2. **직관적 예시/비유:** 기억력이 있는 사람과 같습니다. 새로운 정보를 들을 때(현재 입력), 이전에 들었던 정보(은닉 상태)를 바탕으로 새로운 정보를 해석하고, 그 결과를 다음 정보 해석에 활용합니다.
3. **기술적 정확한 설명:** 순환 뉴런은 현재 시간 단계의 입력 X_t 와 이전 시간 단계의 은닉 상태 Y_{t-1} 을 받아 현재 시간 단계의 은닉 상태 Y_t 를 계산합니다. 이 과정에서 사용되는 가중치(W_X, W_Y)와 편향(b)은 모든 시간 단계에서 동일하게 공유됩니다.

$$Y_t = f(W_X \cdot X_t + W_Y \cdot Y_{t-1} + b)$$

여기서 f 는 활성화 함수입니다.

Figure 2.2: 순환 뉴런의 전개된 구조 (Unfolded RNN)

순환 뉴런의 전개 (Unfolding) 위 그림처럼, 하나의 순환 뉴런은 시간 축을 따라 여러 개의 복사본이 연결된 형태로 이해할 수 있습니다.

- 초기 은닉 상태 (Y_0): 시퀀스의 시작점에서는 이전 은닉 상태가 없으므로, 보통 0으로 초기화됩니다.
- 가중치 공유: 그림에서 W_X, W_Y, b 는 모든 시간 단계에서 동일하게 사용됩니다. 이것이 파라미터 수를 크게 줄이는 핵심입니다.
- 입력 (X_t): 각 시간 단계 t 에서 새로운 입력 X_t 가 들어옵니다.
- 출력 (Y_t): 각 시간 단계 t 에서 은닉 상태 Y_t 가 출력됩니다. 이 Y_t 는 다음 시간 단계의 입력으로 다시 사용됩니다.

예제

단일 순환 뉴런의 파라미터 수 계산

- 입력 X_t 의 차원: 1 (스칼라)
- 이전 은닉 상태 Y_{t-1} 의 차원: 1 (스칼라)
- 현재 은닉 상태 Y_t 의 차원: 1 (스칼라)
- X_t 에 대한 가중치 (W_X): 1개
- Y_{t-1} 에 대한 가중치 (W_Y): 1개
- 편향 (b): 1개
- 총 파라미터 수: $1 + 1 + 1 = 3$ 개

이처럼 단일 순환 뉴런은 시퀀스 길이에 관계없이 고정된 수의 파라미터를 가집니다.

2.4.3 RNN 학습의 난제: 기울기 소실 및 폭주 (Vanishing/Exploding Gradient)

RNN은 장기 의존성(Long-term Dependencies)을 학습하는 데 이론적으로 적합하지만, 실제 학습 과정에서는 기울기 소실(Vanishing Gradient) 또는 기울기 폭주(Exploding Gradient) 문제가 발생하여 어려움을 겪습니다.

1. **한 줄 핵심 요약:** 역전파 시 활성화 함수의 미분 값들이 반복적으로 곱해지면서 기울기가 너무 작아지거나(소실) 너무 커지는(폭주) 현상으로, RNN이 장기 의존성을 학습하기 어렵게 만듭니다.
2. **직관적 예시/비유:**
 - **기울기 소실:** 멀리 떨어진 과거의 사건(시퀀스 초반의 입력)이 현재 결과에 미치는 영향(기울기)이 너무 미미해져서, 마치 기억 상실증처럼 과거를 잊어버리는 것과 같습니다.
 - **기울기 폭주:** 아주 작은 변화가 기하급수적으로 증폭되어 통제 불능 상태가 되는 것과 같습니다.
3. **기술적 정확한 설명:** 역전파 과정에서 기울기는 연쇄 법칙에 따라 여러 활성화 함수의 미분 값과 가중치 값들이 곱해지면서 전달됩니다.
 - **기울기 소실 (Vanishing Gradient):**
 - **원인:** Sigmoid나 Tanh와 같은 활성화 함수는 미분 값이 0과 1 사이(Sigmoid의 경우 최대 0.25)에 존재합니다. 이 작은 값들이 긴 시퀀스를 따라 반복적으로 곱해지면, 기울기 값이 0에 매우 가까워져 초기 시간 단계의 가중치 업데이트가 거의 이루어지지 않습니다.
 - **결과:** RNN이 멀리 떨어진 과거의 정보를 학습하기 어려워져 장기 의존성을 포착하지 못합니다.

- 기울기 폭주 (Exploding Gradient):

- 원인: 반대로, 활성화 함수의 미분 값이나 가중치 값이 1보다 크고, 이 값들이 반복적으로 곱해지면 기울기가 기하급수적으로 커져 발산할 수 있습니다.
- 결과: 가중치 업데이트가 너무 커져 모델이 불안정해지고 학습이 중단될 수 있습니다.
- 해결책: 기울기 클리핑 (Gradient Clipping)을 통해 기울기 값이 특정 임계값을 넘지 않도록 제한하여 폭주를 방지할 수 있습니다.

2.4.4 순환 뉴런 층 (Layer of Recurrent Neurons)

단일 순환 뉴런만으로는 복잡한 시퀀스 패턴을 학습하기 어렵습니다. 여러 개의 순환 뉴런을 병렬로 배치하여 순환 뉴런 층 (Layer of Recurrent Neurons)을 구성하면, 각 뉴런이 서로 다른 특징을 학습하며 더 풍부한 은닉 상태를 만들 수 있습니다.

1. **한 줄 핵심 요약:** 여러 개의 순환 뉴런을 병렬로 배치하여 시퀀스 데이터에서 더 복잡하고 다양한 특징을 학습하는 구조입니다.
2. **직관적 예시/비유:** 한 사람이 하나의 기억만 가지고 정보를 처리하는 것이 아니라, 여러 명의 전문가 (각 뉴런)가 각자의 관점에서 정보를 처리하고 서로의 의견 (교차 연결)을 교환하며 더 종합적인 결론 (벡터 은닉 상태)을 내는 것과 같습니다.
3. **기술적 정확한 설명:** 순환 뉴런 층은 여러 개의 순환 뉴런으로 구성되며, 각 뉴런은 현재 입력 X_t 와 이전 시간 단계의 모든 은닉 상태 (Y_{t-1} 벡터)를 입력으로 받습니다. 그리고 현재 시간 단계의 은닉 상태 벡터 Y_t 를 출력합니다.
 - **벡터 은닉 상태:** Y_t 는 더 이상 스칼라가 아닌 벡터가 됩니다. 각 요소는 층 내의 개별 순환 뉴런의 출력을 나타냅니다.
 - **교차 연결 (Cross-connections):** 층 내의 모든 순환 뉴런은 이전 시간 단계의 모든 순환 뉴런의 출력 (은닉 상태 벡터의 모든 요소)을 입력으로 받습니다. 이는 완전 연결 신경망과 유사하게 층 내에서 정보가 교환됨을 의미합니다.
 - **입력 벡터 (X_t):** 입력 X_t 또한 스칼라가 아닌 벡터일 수 있습니다 (예: 단어를 나타내는 임베딩 벡터). 이 경우, 각 X_t 의 요소는 층 내의 모든 순환 뉴런에 입력됩니다.

예제

순환 뉴런 층의 파라미터 수 계산

- 입력 X_t 의 차원: 2 (벡터)
 - 이전 은닉 상태 Y_{t-1} 의 차원: 2 (벡터, 층 내 뉴런 수)
 - 현재 은닉 상태 Y_t 의 차원: 2 (벡터)
- 각 순환 뉴런은 다음과 같은 입력을 받습니다.
- X_t 의 2개 요소
 - Y_{t-1} 의 2개 요소
 - 편향 1개

따라서 각 뉴런당 $(2 + 2 + 1) = 5$ 개의 파라미터가 필요합니다. 층 내에 2개의 뉴런이 있으므로, 총 파라미터 수는 $5 \times 2 = 10$ 개입니다.

팁

Keras SimpleRNN 층 사용 Keras에서 SimpleRNN 층을 정의할 때 `units` 인수는 해당 층에 몇 개의 순환 뉴런이 있을지(즉, 은닉 상태 벡터의 차원)를 지정합니다.

```

1 from tensorflow.keras.layers import SimpleRNN, Dense
2 from tensorflow.keras.models import Sequential
3
4 # 입력 데이터 형태: 샘플 (수, 시간 단계, 특징 수)
5 # 예: (None, 200, 2) -> 200 시간 단계, 각 시간 단계마다 차원 2 특징 벡터
6 input_shape = (200, 2)
7
8 model = Sequential([
9     SimpleRNN(units=3, activation='relu', input_shape=input_shape),
10    Dense(1) # 최종 출력을 위한 완전 연결 층
11 ])
12 model.summary()
13
```

Listing 2.1: Keras SimpleRNN 층 정의 예시

위 예시에서 `SimpleRNN(units=3)`은 3개의 순환 뉴런으로 구성된 층을 의미하며, 은닉 상태 Y_t 는 3차원 벡터가 됩니다.

2.5 RNN의 고급 변형: LSTM, GRU, 양방향 RNN

기울기 소실/폭주 문제와 장기 의존성 학습의 어려움을 극복하기 위해 다양한 RNN 변형 모델들이 개발되었습니다. 대표적으로 LSTM과 GRU는 '게이트(Gate)' 메커니즘을 도입하여 중요한 정보를 선택적으로 기억하고 불필요한 정보를 잊어버리도록 설계되었습니다.

2.5.1 장단기 기억 (Long Short-Term Memory, LSTM)

LSTM은 RNN의 기울기 소실 문제를 해결하고 장기 의존성을 효과적으로 학습하기 위해 1997년에 제안된 순환 신경망의 한 종류입니다.

1. **한 줄 핵심 요약:** 게이트 메커니즘을 통해 정보를 선택적으로 기억하고 잊어버리면서 장기적인 패턴을 학습할 수 있는 RNN의 고급 형태입니다.
2. **직관적 예시/비유:** 중요한 정보는 노트에 적어두고(장기 기억), 당장 필요한 정보는 잠시 머릿속에 두었다가(단기 기억) 활용하며, 불필요한 정보는 지워버리는 사람의 기억 시스템과 유사합니다.
3. **기술적 정확한 설명:** LSTM은 '셀 상태(Cell State, C_t)'라는 장기 기억 경로와 '은닉 상태(Hidden State, H_t)'라는 단기 기억 경로를 가집니다. 이 두 기억 경로는 세 가지 종류의 게이트(Gate)에 의해 조절됩니다.
 - **망각 게이트(Forget Gate):** 이전 셀 상태에서 어떤 정보를 잊어버릴지 결정합니다.
 - **입력 게이트(Input Gate):** 현재 입력에서 어떤 정보를 셀 상태에 저장할지 결정합니다.
 - **출력 게이트(Output Gate):** 셀 상태의 어떤 부분을 현재 은닉 상태로 출력할지 결정합니다.
 각 게이트는 시그모이드 활성화 함수를 사용하는 완전 연결 서브 네트워크로 구성되어 0과 1 사이의

값을 출력하며, 이를 통해 정보의 흐름을 제어합니다.

Figure 2.3: LSTM 셀의 내부 구조 (개념도)

팁

LSTM의 파라미터 수 LSTM은 내부적으로 4개의 완전 연결 서브 네트워크(게이트)를 가집니다. 각 서브 네트워크는 입력 X_t , 이전 은닉 상태 H_{t-1} , 그리고 편향을 받습니다. 만약 LSTM(units=16) 이라면, 각 서브 네트워크는 16개의 뉴런을 가지며, 은닉 상태와 셀 상태 모두 16차원이 됩니다. 이로 인해 SimpleRNN보다 훨씬 많은 파라미터를 가집니다.

활용 분야 및 장단점

- **장점:** 기율기 소실 문제를 효과적으로 완화하여 긴 시퀀스의 장기 의존성을 학습하는 데 매우 강력합니다.
- **단점:** 복잡한 내부 구조로 인해 SimpleRNN보다 계산 비용이 높고 파라미터 수가 많습니다.
- **활용:** 과거에는 기계 번역, 음성 인식 등 다양한 NLP 분야에서 지배적이었으나, 최근에는 Transformer 모델에 밀리는 경향이 있습니다. 하지만 짧은 시퀀스, 제한된 컴퓨팅 자원(예: 모바일 기기), 실시간 처리 등 특정 상황에서는 여전히 유용하게 사용됩니다.

2.5.2 게이트 순환 유닛 (Gated Recurrent Unit, GRU)

GRU는 LSTM의 복잡성을 줄이면서도 유사한 성능을 내기 위해 2014년에 제안된 순환 신경망의 또 다른 변형입니다.

1. **한 줄 핵심 요약:** LSTM보다 간소화된 게이트 구조를 가지며, 셀 상태와 은닉 상태를 하나로 통합하여 효율성을 높인 RNN 모델입니다.
2. **직관적 예시/비유:** LSTM이 여러 개의 서류함(셀 상태)과 메모(은닉 상태)를 사용하는 반면, GRU는 하나의 서류함(통합된 은닉 상태)만으로 필요한 정보를 관리하는 것과 같습니다.
3. **기술적 정확한 설명:** GRU는 LSTM의 셀 상태와 은닉 상태를 '은닉 상태(H_t)' 하나로 통합하고, 게이트 수도 두 개(업데이트 게이트, 리셋 게이트)로 줄였습니다.
 - **업데이트 게이트 (Update Gate):** 이전 은닉 상태를 얼마나 유지하고, 새로운 정보를 얼마나 받아 들일지 결정합니다.
 - **리셋 게이트 (Reset Gate):** 이전 은닉 상태 중 어떤 부분을 무시하고 새로운 정보를 계산할지 결정합니다.

이러한 간소화 덕분에 GRU는 LSTM보다 파라미터 수가 적고 계산 속도가 빠르면서도, 많은 경우 LSTM과 비슷한 성능을 보여줍니다.

팁

GRU의 장점

- **효율성:** LSTM보다 적은 파라미터와 빠른 계산 속도를 가집니다.
- **성능:** 많은 벤치마크에서 LSTM과 유사하거나 때로는 더 나은 성능을 보여줍니다.

2.5.3 양방향 순환 신경망 (Bidirectional Recurrent Neural Networks)

일반적인 RNN은 시퀀스를 순방향으로만 처리하여 과거 정보만을 활용합니다. 하지만 어떤 시퀀스 데이터(특히 자연어)는 미래 정보(문장의 뒷부분)가 현재의 의미를 이해하는 데 중요할 수 있습니다. **양방향 순환 신경망(Bidirectional RNN)**은 이러한 필요성을 충족시키기 위해 고안되었습니다.

1. **한 줄 핵심 요약:** 시퀀스를 순방향과 역방향으로 동시에 처리하는 두 개의 RNN을 사용하여, 과거와 미래의 모든 맥락 정보를 활용하는 신경망입니다.
2. **직관적 예시/비유:** 탐정이 사건을 조사할 때, 사건 발생 순서대로(순방향) 단서를 추적하는 동시에, 최종 결과(미래)를 바탕으로 역으로(역방향) 단서의 의미를 재해석하는 것과 같습니다.
3. **기술적 정확한 설명:** 양방향 RNN은 다음과 같이 구성됩니다.
 - **순방향 RNN:** 시퀀스를 $X_1, X_2, \dots, X_T$ 순서로 처리하여 은닉 상태 H_t^{forward} 를 생성합니다.
 - **역방향 RNN:** 시퀀스를 $X_T, X_{T-1}, \dots, X_1$ 순서로 처리하여 은닉 상태 H_t^{backward} 를 생성합니다.
 - **출력 결합:** 각 시간 단계 t 에서 순방향 은닉 상태 H_t^{forward} 와 역방향 은닉 상태 H_t^{backward} 를 결합(보통 연결(Concatenate))하여 최종 은닉 상태 $H_t^{\text{bidirectional}}$ 를 만듭니다.

이 최종 은닉 상태는 과거와 미래의 모든 정보를 담고 있으므로, 시퀀스의 특정 시점에서의 맥락을 더 풍부하게 이해할 수 있습니다.

예제

언어 처리에서의 양방향 RNN 독일어와 같은 일부 언어에서는 문장의 부정(negation)이 문장 끝에 나타나는 경우가 있습니다. 이 경우, 문장의 시작 부분만 보고서는 전체 의미를 정확히 파악하기 어렵습니다. 양방향 RNN은 문장의 끝 부분 정보까지 활용하여 문장 전체의 의미를 더 정확하게 해석할 수 있습니다.

팁

파라미터 수 증가 양방향 RNN은 두 개의 독립적인 RNN(순방향 및 역방향)을 사용하므로, 단방향 RNN에 비해 파라미터 수가 대략 두 배로 증가합니다. 이는 계산 비용 증가로 이어질 수 있습니다.

2.6 텍스트 데이터 전처리: TF-IDF (Term Frequency-Inverse Document Frequency)

텍스트 데이터를 신경망이나 머신러닝 모델에 입력하려면, 텍스트를 숫자로 변환하는 과정이 필요합니다. **TF-IDF (Term Frequency-Inverse Document Frequency)**는 문서 내 특정 단어의 중요도를 수치화하는 널리 사용되는 기법입니다.

2.6.1 TF-IDF의 개념 및 계산

1. **한 줄 핵심 요약:** TF-IDF는 특정 단어가 한 문서에 얼마나 자주 나타나고(TF), 전체 문서 집합에서 얼마나 희귀한지(IDF)를 종합하여 단어의 중요도를 측정하는 방법입니다.
2. **직관적 예시/비유:** 책에서 특정 단어가 얼마나 중요한지 평가하는 것과 같습니다. 한 챕터에 자주 나오면서(TF), 그 책 전체에서는 잘 나오지 않는(IDF) 단어일수록 그 챕터의 핵심 주제를 나타낼 가능성이 높습니다.

3. 기술적 정확한 설명: TF-IDF는 두 가지 구성 요소의 곱으로 계산됩니다.

- **용어 빈도 (Term Frequency, TF):** 특정 문서(d) 내에서 단어(t)가 나타나는 빈도를 측정합니다.

$$\text{TF}(t, d) = \frac{\text{문서 } d \text{ 내 단어 } t \text{의 출현 횟수}}{\text{문서 } d \text{ 내 전체 단어 수}}$$

- **역문서 빈도 (Inverse Document Frequency, IDF):** 단어가 전체 문서 집합에서 얼마나 희귀한지를 측정합니다. 희귀한 단어일수록 높은 IDF 값을 가집니다.

$$\text{IDF}(t, D) = \log \left(\frac{\text{전체 문서 수 } |D|}{\text{단어 } t \text{가 포함된 문서 수 } |\{d \in D : t \in d\}|} \right) + 1$$

(여기서 +1은 분모가 0이 되는 것을 방지하고, 로그 스케일을 조절하기 위한 일반적인 변형입니다.)

- **TF-IDF:** TF와 IDF의 곱입니다.

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

예제

TF-IDF 계산 예시

- 문서 집합 (D): 3개의 문서
 - 문서 1: "The cat sat on the mat." (총 6단어)
 - 문서 2: "The cat again did something." (총 5단어)
 - 문서 3: "Just some sentence." (총 3단어)

- 대상 단어: "cat"

- 대상 문서: 문서 1

1단계: TF 계산 (문서 1에서 "cat"의 용어 빈도)

- 문서 1에서 "cat" 출현 횟수: 1회
- 문서 1의 총 단어 수: 6개
- $\text{TF}(\text{"cat"}, \text{문서 1}) = 1/6$

2단계: IDF 계산 (전체 문서 집합에서 "cat"의 역문서 빈도)

- 전체 문서 수 ($|D|$): 3개
- "cat"이 포함된 문서 수: 문서 1, 문서 2 (총 2개)
- $\text{IDF}(\text{"cat"}, D) = \log(3/2) + 1 \approx 0.405 + 1 = 1.405$

3단계: TF-IDF 계산

- $\text{TF-IDF}(\text{"cat"}, \text{문서 1}, D) = (1/6) \times (\log(3/2) + 1) \approx 0.167 \times 1.405 \approx 0.234$

팁

TF-IDF 벡터화 과정

- **훈련 (fit):** `TfidfVectorizer`는 훈련 데이터셋을 기반으로 모든 고유 단어(어휘집)를 구축하고, 각 단어의 IDF 값을 계산합니다.
- **변환 (transform):** `transform` 메서드는 새로운 문서(훈련 또는 테스트)에 대해 TF를 계산하고, `fit` 과정에서 학습된 IDF 값을 적용하여 TF-IDF 벡터를 생성합니다. 테스트 데이터에 대해서는 IDF를 새로 계산하지 않고 훈련 데이터에서 학습된 IDF를 그대로 사용합니다.

- **정규화 (Normalization):** 일반적으로 TF-IDF 벡터는 L2 정규화(벡터의 길이를 1로 만듦)를 거쳐 스케일이 조정됩니다.

주의

단어 순서 정보 손실 TF-IDF는 각 단어의 중요도를 측정하지만, 문서 내에서 단어들이 나타나는 순서 정보는 완전히 무시합니다. 이는 텍스트 분류와 같이 단어의 존재 여부가 중요한 문제에는 적합하지만, 문맥이나 문법이 중요한 문제(예: 기계 번역)에는 한계가 있습니다.

2.7 특허 데이터 분류 실습 (TF-IDF + 로지스틱 회귀/신경망)

실제 특허 데이터를 사용하여 TF-IDF와 머신러닝 모델을 이용해 문서 분류를 수행하는 과정을 살펴봅니다. 목표는 특정 특허가 자동차 산업과 관련이 있는지 여부를 분류하는 것입니다.

2.7.1 문제 정의 및 데이터셋 구성

1. **한 줄 핵심 요약:** 19세기 말에서 20세기 초의 특허 문서들을 자동차 산업 관련 여부에 따라 분류하는 이진 분류 문제입니다.
2. **데이터 소스:** Google Patents와 같은 공개 데이터베이스에서 수집된 특허 데이터 (출판 번호, 제목, 설명).
3. **훈련 데이터셋 생성:**
 - **긍정 레이블 (1):** 'Wheels within Wheels'와 같은 역사 서적에서 확인된 자동차 산업 관련 기업들이 출원한 특허들을 '자동차 산업 관련'으로 레이블 1을 부여합니다.
 - **부정 레이블 (0):** 전체 특허 중 무작위로 샘플링한 특허들에 대해 '자동차 산업 비관련'으로 레이블 0을 부여합니다. 이 방법은 100% 정확하지 않을 수 있지만, 방대한 특허 수에 비추어 대부분의 무작위 샘플은 자동차 산업과 무관할 것이라는 가정에 기반합니다.

2.7.2 차원 축소 및 특징 선택: T-검정 (T-test) 활용

특허 문서에는 수많은 고유 단어가 포함되어 있어, 이를 모두 특징으로 사용하면 모델의 차원이 너무 높아져 학습이 어렵고 과적합될 위험이 있습니다. **T-검정 (T-test)**을 사용하여 가장 중요한 단어들을 선별하여 차원을 축소합니다.

1. **한 줄 핵심 요약:** T-검정을 통해 자동차 산업 관련 특허와 비관련 특허 그룹 간에 TF-IDF 값이 통계적으로 유의미한 차이를 보이는 단어들을 선별하여 특징으로 사용합니다.
2. **직관적 예시/비유:** 두 그룹의 학생들에게 시험을 치르게 한 후, 어떤 문제(단어)가 두 그룹의 실력 (TF-IDF 값) 차이를 가장 잘 보여주는지 통계적으로 분석하는 것과 같습니다.
3. **기술적 정확한 설명:**
 - **T-검정의 목적:** 두 모집단(여기서는 '자동차 관련 특허'와 '비관련 특허' 그룹)의 평균이 통계적으로 유의미하게 다른지 여부를 검정합니다.
 - **절차:**
 - (a) **가설 설정:** 귀무가설 (H_0): 두 그룹 간 특정 단어의 TF-IDF 평균은 같다. 대립가설 (H_1): 두 그룹 간 특정 단어의 TF-IDF 평균은 다르다.

- (b) **T-통계량 계산:** 두 그룹의 평균 차이를 표준 오차로 나눈 값입니다. 이 값이 클수록 평균 차이가 크다는 의미입니다.
- (c) **P-값 (P-value) 계산:** T-통계량이 관측될 확률을 나타냅니다. P-값이 작을수록 귀무가설을 기각하고 대립가설을 채택할 강력한 증거가 됩니다. 즉, 두 그룹 간의 평균 차이가 우연히 발생했을 가능성이 낮다는 의미입니다.
- **특징 선택:** 각 단어에 대해 T-검정을 수행하고 P-값을 계산합니다. P-값이 가장 작은 상위 N개의 단어(예: 제목에서 300개, 설명에서 600개)를 최종 특징으로 선택합니다. 이는 두 그룹을 가장 잘 구분하는 단어들이라고 볼 수 있습니다.

2.7.3 모델 학습 및 평가

TF-IDF 벡터화 및 T-검정을 통해 선별된 특징들을 사용하여 로지스틱 회귀 모델과 신경망 모델을 학습시키고 성능을 평가합니다.

1. **한 줄 핵심 요약:** TF-IDF 특징을 기반으로 로지스틱 회귀 및 신경망 모델을 훈련하고, 검증 정확도 (Validation Accuracy)를 통해 모델의 성능을 평가합니다.
2. **모델 유형:**
 - **로지스틱 회귀 (Logistic Regression):**
 - **제목 기반:** 제목에서 추출된 상위 300개 TF-IDF 특징 사용.
 - **설명 기반:** 설명에서 추출된 상위 600개 TF-IDF 특징 사용.
 - **결합 기반:** 제목(300개)과 설명(600개)의 TF-IDF 특징을 연결(Concatenate)하여 총 900개 특징 사용.
 - **신경망 (Neural Network):**
 - **아키텍처:** 3개의 은닉층 (각 16개 뉴런), 출력층 (2개 뉴런, Softmax 활성화).
 - **최적화:** RMSprop (또는 Adam) 옵티마이저.
 - **손실 함수:** Categorical Cross-Entropy.
 - **평가 지표:** 정확도 (Accuracy).
 - **정규화:** Dropout (과적합 방지).

팁

훈련/검증 정확도 곡선 분석 모델 학습 시 훈련 정확도 (Training Accuracy)와 검증 정확도 (Validation Accuracy)를 함께 모니터링하는 것이 중요합니다.

- **훈련 정확도:** 훈련 데이터에 대한 모델의 성능. 에포크가 진행될수록 일반적으로 계속 증가합니다.
- **검증 정확도:** 모델이 보지 못한 새로운 데이터(검증 데이터)에 대한 성능. 훈련 정확도와 함께 증가하다가, 모델이 과적합(Overfitting)되기 시작하면 감소하거나 정체될 수 있습니다.
- **최적 모델 선택:** 검증 정확도가 가장 높은 시점의 모델을 선택하는 것이 가장 좋습니다. 훈련 정확도가 계속 높아지더라도 검증 정확도가 더 이상 개선되지 않거나 감소한다면, 과적합이 발생하고 있으므로 학습을 중단해야 합니다.

2.8 시계열 예측 실습 (RNN/LSTM)

RNN과 LSTM을 사용하여 GARCH 모델과 유사한 비선형 시계열 데이터를 예측하는 실습 과정을 다룹니다.

2.8.1 데이터 생성 및 준비

1. **한 줄 핵심 요약:** GARCH와 유사한 비선형 시계열 데이터를 생성하고, 이를 RNN 모델의 입력 형태로 변환하여 훈련 및 테스트 데이터셋을 구성합니다.
2. **시계열 생성 함수:** GARCH(Generalized AutoRegressive Conditional Heteroskedasticity) 모델은 금융 시계열의 변동성 클러스터링(Volatility Clustering)을 모델링하는 데 사용됩니다. 본 실습에서는 GARCH와 유사한 구조에 비선형 상호작용 항을 추가하여 더 복잡한 시계열을 생성합니다.

```

1 import numpy as np
2
3 def generate_garch_like_series(n_steps=1000, alpha=0.1, beta=0.8, gamma
    =0.05, interaction=0.1):
4     series = np.zeros(n_steps)
5     volatility = np.zeros(n_steps)
6     volatility[0] = 0.1 # 초기 변동성
7
8     for t in range(1, n_steps):
9         # 와 GARCH 유사한 변동성 업데이트
10        volatility[t] = np.sqrt(alpha * series[t-1]**2 + beta * volatility[
            t-1]**2 + gamma)
11        # 비선형 상호작용 항 추가
12        series[t] = np.random.normal(0, volatility[t]) + interaction *
            series[t-1] * volatility[t-1]
13    return series, volatility
14

```

Listing 2.2: GARCH 유사 시계열 생성 함수 (개념 코드)

3. **데이터셋 체크 분할:** RNN 모델은 고정된 길이의 시퀀스를 입력으로 받습니다. 따라서 생성된 시계열을 '입력 시퀀스'와 '예측할 다음 값'으로 구성된 청크(Chunk)로 분할해야 합니다.
 - 예시: $\text{time_steps} = 14$ 라면, $[X_t, X_{t+1}, \dots, X_{t+13}]$, X_{t+14} .
4. **RNN 입력 형태 재구성:** Keras의 RNN 층은 입력 데이터를 (샘플 수, 시간 단계, 특징 수)의 3차원 형태로 기대합니다.
 - 문제: 1차원 시계열 데이터(예: (1000,))를 (샘플 수, 시간 단계) 형태로만 제공하면 오류가 발생합니다.
 - 해결: `reshape` 함수를 사용하여 (샘플 수, 시간 단계, 1) 형태로 명시적으로 특징 차원을 추가해야 합니다.
5. **훈련/테스트 데이터 분할:** 시계열 데이터는 시간 순서가 중요하므로, 무작위로 섞지 않고 시계열의 앞부분을 훈련 데이터로, 뒷부분을 테스트 데이터로 사용합니다 (예: 80% 훈련, 20% 테스트).

2.8.2 RNN/LSTM 모델 구축 및 학습

1. 한 줄 핵심 요약: SimpleRNN 또는 LSTM 층을 사용하여 시계열 예측 모델을 구축하고, Dropout으로 과적합을 방지하며 학습시킵니다.
2. 모델 아키텍처 예시 (Keras):

```

1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import SimpleRNN, LSTM, Dense, Dropout
3
4 # 입력 형태: 시간( 단계, 특징 수)
5 # 예: (14, 1) -> 14 시간 단계, 각 시간 단계마다 차원1 특징
6 n_time_steps = 14
7 n_features = 1
8
9 model = Sequential([
10     # 첫 번째 RNN 층: 전체 시퀀스를 다음 층으로 전달 (return_sequences=True)
11     SimpleRNN(units=2, activation='relu', input_shape=(n_time_steps,
12     n_features), return_sequences=True),
13     Dropout(0.1), # 과적합 방지를 위한 Dropout (10% 연결 드롭)
14     # 두 번째 RNN 층: 마지막 은닉 상태만 다음 층으로 전
15     달 (return_sequences=False)
16     SimpleRNN(units=4, activation='relu', return_sequences=False),
17     Dropout(0.1),
18     # 최종 예측을 위한 Dense 층 회귀이므로( 활성화 함수 없음)
19     Dense(1)
20 ])
21
22 model.compile(optimizer='adam', loss='mse') # 회귀 문제이므로 MSE 손실 함수 사
    용
23 model.summary()

```

Listing 2.3: RNN 시계열 예측 모델 (개념 코드)

팁

return_sequences 옵션

- **return_sequences=True:** 해당 RNN 층의 모든 시간 단계에 대한 은닉 상태 시퀀스를 출력합니다. 이는 다음 RNN 층이 전체 시퀀스 정보를 입력으로 받아야 할 때 사용됩니다.
- **return_sequences=False (기본값):** 해당 RNN 층의 마지막 시간 단계의 은닉 상태만 출력합니다. 이는 RNN 층 다음에 완전 연결 층(Dense Layer)을 연결하여 최종 예측 값을 만들 때 주로 사용됩니다.

팁

Dropout (드롭아웃) 정규화

- **목적:** 신경망의 과적합(Overfitting)을 방지하기 위한 정규화 기법입니다.
- **동작 방식:** 훈련 과정에서 각 에포크마다 무작위로 일부 뉴런(또는 RNN의 경우 연결)을 비활성화(드롭)시킵니다. 이는 모델이 특정 뉴런에 과도하게 의존하는 것을 방지하고, 더 강건한 특징을 학습하도록 돕습니다.

- **훈련 시에만 적용:** 드롭아웃은 훈련 시에만 적용되며, 테스트 또는 예측 시에는 모든 뉴런이 활성화됩니다.

3. 학습 및 평가:

- **옵티마이저:** Adam (효율적인 학습률 조절).
- **손실 함수:** MSE (회귀 문제).
- **배치 크기 (Batch Size):** 한 번의 가중치 업데이트에 사용되는 샘플 수.
- **검증 데이터 (Validation Data):** 훈련 중 모델의 일반화 성능을 모니터링하기 위해 사용됩니다.
- **평가:** 훈련 및 테스트 데이터에 대한 MSE 값을 에포크별로 플로팅하여 모델의 학습 진행 상황과 과적합 여부를 확인합니다.

2.9 절차/방법

신경망 모델을 구축하고 학습시키는 일반적인 절차와 특정 문제 해결을 위한 단계별 방법을 정리했습니다.

2.9.1 신경망 모델 학습 및 평가 절차

1. 데이터 준비:

- 원본 데이터 수집 및 탐색.
- 결측치 처리, 이상치 제거 등 데이터 정제.
- **데이터 스케일링:** 입력 특징들을 0-1 범위 또는 표준 정규 분포로 변환하여 학습 안정성 및 속도 향상.
- **데이터 분할:** 훈련(Train), 검증(Validation), 테스트(Test) 세트로 분할. 시계열 데이터는 시간 순서 유지.

2. 모델 아키텍처 정의:

- 문제 유형(분류/회귀)에 따라 적절한 신경망 층(Dense, SimpleRNN, LSTM 등) 선택.
- 각 층의 뉴런 수(units), 활성화 함수(activation) 설정.
- 출력층의 뉴런 수와 활성화 함수를 문제에 맞게 설정 (예: 다중 분류 → 클래스 수, Softmax).
- `input_shape` 지정.

3. 모델 컴파일:

- **옵티마이저(optimizer):** Adam, RMSprop 등 선택.
- **손실 함수(loss):** 문제 유형에 맞게 선택 (예: 회귀 → 'mse', 다중 분류 → 'categorical_crossentropy').
- **평가 지표(metrics):** 훈련 중 모니터링할 지표 설정 (예: 'accuracy', 'mse').

4. 모델 학습 (model.fit()):

- **훈련 데이터(x_train, y_train):** 모델 학습에 사용될 데이터.
- **에포크(epochs):** 전체 훈련 데이터를 몇 번 반복하여 학습할지 지정.
- **배치 크기(batch_size):** 한 번의 가중치 업데이트에 사용될 샘플 수.
- **검증 데이터(validation_data):** 훈련 중 모델의 일반화 성능을 평가하기 위한 데이터.

- 콜백(callbacks): EarlyStopping, ModelCheckpoint 등을 사용하여 학습 과정 제어.

5. 모델 평가(model.evaluate()):

- 테스트 데이터(x_test, y_test): 모델의 최종 성능을 평가하기 위한 데이터.
- 훈련 및 검증 과정에서 사용되지 않은 완전히 새로운 데이터로 평가하여 모델의 실제 일반화 성능을 측정합니다.

6. 예측(model.predict()):

- 학습된 모델을 사용하여 새로운 입력에 대한 예측 값을 생성합니다.

2.9.2 TF-IDF 기반 텍스트 벡터화 및 특징 선택 (T-test 활용)

1. 텍스트 데이터 로드 및 전처리:

- 텍스트 데이터(예: 특허 제목, 설명)를 불러옵니다.
- 불용어(stop words) 제거, 토큰화(tokenization), 표제어 추출(lemmatization) 등 기본적인 텍스트 전처리를 수행합니다.

2. TF-IDF 벡터화 객체 생성 및 학습:

- TfidfVectorizer 객체를 생성합니다.
- 훈련 데이터셋의 텍스트를 사용하여 fit() 메서드를 호출하여 어휘집과 IDF 값을 학습시킵니다.

3. TF-IDF 특징 추출:

- transform() 메서드를 사용하여 훈련 데이터셋의 텍스트를 TF-IDF 벡터로 변환합니다.
- 각 단어(특징)에 대한 TF-IDF 값을 얻습니다.

4. T-검정을 통한 특징 선택:

- 훈련 데이터셋을 레이블(예: 0과 1)에 따라 두 그룹으로 나눕니다.
- 각 단어(TF-IDF 특징)에 대해 두 그룹 간의 TF-IDF 값 평균 차이에 대한 T-검정을 수행하고 P-값을 계산합니다.
- P-값이 가장 작은 상위 N개의 단어를 최종 특징으로 선택합니다.

5. 선택된 특징으로 데이터셋 재구성:

- 선택된 N개의 단어에 해당하는 TF-IDF 값만 사용하여 훈련 및 테스트 데이터셋을 재구성합니다.
- 이를 통해 모델의 입력 차원을 줄이고 학습 효율을 높입니다.

2.10 실습/코드/오류와 해결

2.10.1 Keras 신경망 모델 정의 예시

다음은 Keras를 사용하여 간단한 완전 연결 신경망을 정의하는 코드 예시입니다.

```
1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import Dense
3
4 # 입력 특징의 수 예 (: TF-IDF 특징 개 900)
5 input_dim = 900
6
7 model = Sequential([
8     Dense(16, activation='relu', input_shape=(input_dim,)), # 첫 번째 은닉층
```

```

9     Dense(16, activation='relu'), # 두 번째 은닉층
10    Dense(16, activation='relu'), # 세 번째 은닉층
11    Dense(2, activation='softmax') # 출력층 개 (2 클래스 분류, Softmax)
12 ])
13
14 model.compile(optimizer='rmsprop', loss='categorical_crossentropy', metrics=['
    accuracy'])
15 model.summary()

```

Listing 2.4: Keras 완전 연결 신경망 모델 정의

2.10.2 TF-IDF Vectorizer 사용 예시

sklearn의 TfidfVectorizer를 사용하여 텍스트를 TF-IDF 벡터로 변환하는 예시입니다.

```

1  from sklearn.feature_extraction.text import TfidfVectorizer
2  import numpy as np
3
4  documents = [
5      "The cat sat on the mat.",
6      "The cat again did something.",
7      "Just some sentence but no cat."
8  ]
9
10 # TfidfVectorizer 객체 생성
11 # min_df: 최소 문서 빈도 너무 (희귀한 단어 제거)
12 # max_df: 최대 문서 빈도 너무 (흔한 단어 제거)
13 vectorizer = TfidfVectorizer(min_df=1, max_df=1.0)
14
15 # 훈련 데이터에 fit (IDF 값 학습)
16 vectorizer.fit(documents)
17
18 # 훈련 데이터 변환 (TF-IDF 벡터 생성)
19 tfidf_matrix = vectorizer.transform(documents)
20
21 print("어휘집:", vectorizer.get_feature_names_out())
22 print("TF-IDF 행렬:\n", tfidf_matrix.toarray())
23
24 # 테스트 데이터 새로운 (문서) 변환
25 test_document = ["A cat sat on a tree."]
26 test_tfidf = vectorizer.transform(test_document)
27 print("\테스트\n 문서 TF-IDF:\n", test_tfidf.toarray())

```

Listing 2.5: TfidfVectorizer 사용 예시

2.10.3 시계열 데이터 재구성 (Reshape) 오류 및 해결

RNN 모델은 입력 데이터의 형태에 매우 민감합니다. 특히 1차원 시계열 데이터를 처리할 때 (샘플 수, 시간 단계, 특징 수) 형태로 정확히 맞춰주지 않으면 오류가 발생합니다.

- **증상:** `ValueError: Input 0 of layer "simple_rnn" is incompatible with the layer: expected ndim=3, found ndim=2. Full shape received: (None, 14)`
- **원인 가설:** 1차원 시계열 데이터를 (샘플 수, 시간 단계) 형태의 2차원으로 제공하여, RNN 층이 기대하는 (샘플 수, 시간 단계, 특징 수) 형태의 3차원 입력과 불일치합니다. RNN은 각 시간 단계의 입력이 벡터(특징)라고 가정합니다. 1차원 시계열의 경우, 각 시간 단계의 특징 수가 1인 벡터로 명시해야 합니다.
- **수정 방법:** `numpy.reshape()` 함수를 사용하여 데이터의 마지막 차원에 1을 추가하여 3차원으로 만듭니다.

```

1 import numpy as np
2
3 # 가상의 시계열 데이터 샘플 (개 100, 각 샘플은 개의 14 시간 단계)
4 # 현재 형태: (100, 14) -> 이 RNN 기대하는 (100, 14, 1)이 아님
5 data_2d = np.random.rand(100, 14)
6
7 print("원본 데이터 형태:", data_2d.shape)
8
9 # 수정: 마지막 차원에 1을 추가하여 차원으로 3 재구성
10 # -1 알아서 '계산을' 의미하며, 여기서는 샘플 수를 유지
11 # 14 시간 단계를 유지
12 # 1 특징 수를 명시
13 data_3d = data_2d.reshape(-1, 14, 1)
14
15 print("재구성된 데이터 형태:", data_3d.shape)
16
17 # 이제 이 data_3d RNN 모델의 input 사용할 수 있습니다.
18 # model.fit(data_3d, labels)

```

Listing 2.6: 시계열 데이터 재구성 예시

Table 2.2: 시계열 데이터 재구성 전/후 비교

항목	재구성 전 (오류 발생)	재구성 후 (정상 작동)
데이터 형태	(샘플 수, 시간 단계)	(샘플 수, 시간 단계, 특징 수)
예시	(100, 14)	(100, 14, 1)
RNN 층 기대	ndim=3	ndim=3
실제 제공	ndim=2	ndim=3
해결 방법	<code>data.reshape(-1, time_steps)</code>	<code>data.reshape(-1, time_steps, 1)</code>

2.10.4 학습률 관련 문제 시각화 및 해결 방안

학습률이 모델 학습에 미치는 영향은 손실 함수 그래프를 통해 시각적으로 확인할 수 있습니다.

Figure 2.4: 다양한 학습률에 따른 손실 함수 변화 시각화

- **과소적합 (Underfitting) 또는 느린 수렴 (Too Small Learning Rate):**

- **증상:** 손실 함수가 거의 감소하지 않거나 매우 느리게 감소합니다. 훈련 및 검증 정확도가 낮게 유지됩니다.
- **해결 방안:**
 - * 학습률을 증가시킵니다.
 - * 데이터 스케일링이 올바르게 되었는지 확인합니다.
 - * 더 강력한 옵티마이저(Adam 등)를 사용합니다.
- **과적합 (Overfitting) 또는 발산 (Too Large Learning Rate):**
 - **증상:** 훈련 손실은 감소하지만 검증 손실이 증가하거나 불안정하게 진동합니다. 훈련 정확도는 높지만 검증 정확도는 낮습니다. 학습률이 너무 크면 손실이 발산할 수 있습니다.
 - **해결 방안:**
 - * 학습률을 감소시킵니다.
 - * Dropout, L1/L2 정규화와 같은 정규화 기법을 적용합니다.
 - * Early Stopping을 사용하여 검증 손실이 증가하기 시작하면 학습을 중단합니다.
 - * 데이터 증강(Data Augmentation)을 통해 훈련 데이터의 양을 늘립니다.

2.11 체크리스트

본 강의 내용을 학습하고 실습을 수행하며 점검해야 할 사항들입니다.

학습 및 이해도 점검

- 신경망 순전파 과정을 수동으로 계산할 수 있는가?
- 역전파의 필요성과 연쇄 법칙의 역할을 설명할 수 있는가?
- ReLU, Softmax, MSE의 용도를 구분하고 설명할 수 있는가?
- 신경망의 파라미터 수를 계산할 수 있는가?
- 학습률(α)이 모델 학습에 미치는 영향을 설명하고, 적절한 α 를 선택하는 기준을 아는가?
- RNN이 시퀀스 데이터를 처리하는 원리(가중치 공유, 은닉 상태)를 설명할 수 있는가?
- RNN의 기울기 소실/폭주 문제를 설명하고, LSTM/GRU가 이를 어떻게 해결하는지 아는가?
- TF-IDF의 TF와 IDF를 각각 계산하고, 텍스트 벡터화 과정을 설명할 수 있는가?
- T-검정을 이용한 특징 선택의 원리와 P-값의 의미를 설명할 수 있는가?
- 에포크, 배치 크기, 훈련/검증/테스트 데이터의 역할을 구분할 수 있는가?

실습 및 코드 구현 점검

- Keras를 사용하여 완전 연결 신경망 및 RNN/LSTM 모델을 구축할 수 있는가?
- TfidfVectorizer를 사용하여 텍스트 데이터를 TF-IDF 벡터로 변환할 수 있는가?
- 시계열 데이터를 RNN 입력 형태((샘플 수, 시간 단계, 특징 수))로 올바르게 재구성할 수 있는가?
- `model.compile()`에서 옵티마이저, 손실 함수, 평가 지표를 올바르게 설정할 수 있는가?
- `model.fit()`을 사용하여 모델을 학습시키고, 훈련/검증 성능을 모니터링할 수 있는가?
- Dropout과 같은 정규화 기법을 모델에 적용할 수 있는가?
- 학습률 문제(느린 수렴, 발산, 과적합) 발생 시 손실 곡선을 분석하고 해결 방안을 제시할 수 있는가?

2.12 FAQ

초심자들이 자주 혼동하거나 궁금해할 만한 질문들을 QA 형식으로 정리했습니다.

자주 묻는 질문

Q1: ReLU의 입력이 음수일 때 기울기가 0이 되는데, 이때 가중치 업데이트는 어떻게 되나요?

A1: ReLU의 입력이 음수이면 출력은 0이 되고, 해당 뉴런의 기울기 또한 0이 됩니다. 역전파 시 연쇄 법칙에 따라 이 0의 기울기가 이전 층으로 전달되므로, 해당 뉴런과 관련된 가중치들은 업데이트되지 않습니다. 이를 '죽은 ReLU(Dying ReLU)' 문제라고도 하며, 학습이 정체될 수 있습니다.

Q2: 가중치(W)는 왜 하이퍼파라미터가 아닌가요? 학습률(α)은 하이퍼파라미터인데 왜 W는 아닌가요? A2: 하이퍼파라미터는 모델 학습 '전에' 사용자가 직접 설정하는 값(예: 학습률, 층 수, 뉴런 수)입니다. 반면, 가중치(W)는 모델 학습 '과정에서' 데이터로부터 자동으로 최적의 값을 찾아내는 '학습 가능한 파라미터'입니다. W는 모델이 데이터를 통해 배우는 '지식'을 나타냅니다.

Q3: 학습률이 너무 작으면 왜 수렴이 느린가요? 그냥 오래 기다리면 되는 것 아닌가요? A3: 학습률이 너무 작으면 가중치 업데이트 보폭이 매우 작아져 손실 함수의 최저점에 도달하는 데 엄청난 시간이 걸립니다. 단순히 오래 기다리는 것을 넘어, 실제로는 학습이 거의 진행되지 않아 초기 값에 고착될 수도 있습니다. 이는 컴퓨팅 자원과 시간 낭비로 이어지므로, 적절한 학습률을 찾는 것이 중요합니다.

Q4: RNN에서 가중치 공유는 왜 필요한가요? 각 시간 단계마다 다른 가중치를 사용하면 안 되나요? A4: 각 시간 단계마다 다른 가중치를 사용하면 시퀀스 길이가 길어질수록 파라미터 수가 기하급수적으로 증가하여 학습이 불가능해집니다. 가중치 공유는 시퀀스 길이에 관계없이 고정된 수의 파라미터를 유지하게 하여 모델의 효율성을 높이고, 시퀀스 내에서 동일한 패턴이 반복될 수 있다는 가정을 반영합니다.

Q5: TF-IDF는 단어 순서를 고려하지 않는데, 텍스트 분류에 사용해도 괜찮은가요? A5: 네, 텍스트 분류와 같은 많은 문제에서는 단어의 '존재 여부'와 '중요도'가 '순서'보다 더 중요할 수 있습니다. TF-IDF는 단어의 중요도를 효과적으로 측정하여 문서의 주제를 파악하는 데 강점을 가집니다. 하지만 문맥이나 문법이 중요한 문제(예: 기계 번역, 감성 분석)에서는 RNN이나 Transformer와 같이 순서를 고려하는 모델이 더 적합합니다.

Q6: 훈련 정확도는 계속 높아지는데 검증 정확도가 낮아지면 어떻게 해야 하나요? A6: 이는 모델이 훈련 데이터에 과적합(Overfitting)되고 있다는 명확한 신호입니다. 모델이 훈련 데이터의 노이즈까지 학습하여 새로운 데이터에 대한 일반화 성능이 떨어진 것입니다. 해결책으로는 Dropout, L1/L2 정규화 적용, Early Stopping 사용, 훈련 데이터 증강, 모델 복잡도 줄이기 등이 있습니다.

Q7: Epoch와 Batch Size의 관계는 무엇인가요? 왜 둘 다 필요한가요? A7:

- **Epoch:** 전체 훈련 데이터셋을 모델이 한 번 학습하는 단위입니다. 1 Epoch 동안 모델은 전체 데이터를 한 번 봅니다.
- **Batch Size:** 전체 훈련 데이터를 작은 묶음(미니 배치)으로 나누어, 한 번의 가중치 업데이트에 사용되는 샘플 수입니다.

예를 들어, 훈련 데이터가 1000개이고 배치 크기가 100이라면, 1 Epoch 동안 10번의 가중치 업데이트가 일어납니다. Epoch는 모델이 전체 데이터를 몇 번 반복 학습할지, Batch Size는 한 번에 얼마나 많은 데이터를 보고 가중치를 업데이트할지를 결정하여 학습의 효율성과 안정성에 영향을 미칩니다.

2.13 빠르게 훑어보기 (Quick Glance)

핵심 개념들을 한눈에 파악할 수 있도록 요약했습니다.

신경망 기본

순전파: 입력 $\rightarrow$ 출력 계산. ReLU 활성화 함수($\max(0, z)$). **역전파:** 오차 $\rightarrow$ 기울기 계산. 연쇄 법칙, 중간 값 재사용. **파라미터:** 가중치(W), 편향(b). 학습 가능한 값. **학습률 (α):** 경사 하강법 보폭. 너무 작으면 느림, 너무 크면 발산. 데이터 스케일링 필수! **활성화/손실 함수:** 분류(Softmax/Cross-Entropy), 회귀(Linear/MSE).

시퀀스 데이터와 RNN

RNN 필요성: 시계열, 자연어 등 순서 있는 데이터 처리. **핵심:** 가중치 공유, 은닉 상태($Y_t = f(W_X X_t + W_Y Y_{t-1} + b)$). **문제점:** 기울기 소실/폭주 (장기 의존성 학습 어려움). **LSTM:** 게이트 (망각, 입력, 출력)로 장단기 기억 조절. 기울기 소실 완화. **GRU:** LSTM 간소화 버전. 효율성 높음. **양방향 RNN:** 순방향 + 역방향 처리로 모든 맥락 활용.

텍스트 처리 및 모델 평가

TF-IDF: 단어 중요도 수치화. $TF(\text{문서 내 빈도}) \times IDF(\text{문서 집합 내 희귀도})$. 단어 순서 무시. **특징 선택:** T-검정으로 유의미한 단어 선별 (P-값 활용). 차원 축소. **모델 학습:** 로지스틱 회귀, 신경망. 옵티마이저(Adam, RMSprop), 정규화(Dropout). **평가:** 훈련/검증/테스트 정확도(MSE). 최대 검증 정확도 시점 선택. **시계열 입력 형태:** (샘플 수, 시간 단계, 특징 수)로 재구성 필수.

Chapter 3

Lecture 3

이후에 문서별로 재정의됩니다.

, keywordstyle=, commentstyle=, stringstyle=, numberstyle=, numbers=left, numbersep=8pt, breaklines=true, breakatwhitespace=false, frame=single, frameround=tttt, rulecolor=, captionpos=b, showstringspaces=false, tabsize=2, xleftmargin=15pt, xrightmargin=5pt, escapeinside=%*)

,

,

[table] labelfont=bf, textfont=it, skip=5pt [figure] labelfont=bf, textfont=it, skip=5pt

정리[section] [theorem]보조정리 [theorem]명제 [theorem]따름정리 정의[section] 예제[section]

강의명: *CSCI89*

주차: *Lecture 3*

교수명: *UIUC Student*

목적: 본 문서는 *CSCI E-89* 강의의 세 번째 내용을 통합하여 순환 신경망(*RNN*)의 기본 개념부터 자연어 처리(*NLP*)의 핵심 기술까지 다룹니다. 특히 *RNN*의 구조, 파라미터 계산, *LSTM*의 작동 원리, 그리고 텍스트 데이터를 모델에 입력하기 위한 전처리 기법(토큰화, 어간 추출, 표제어 추출, 임베딩)을 상세히 설명합니다. 실제 텍스트 분류 예제를 통해 이론적 지식이 어떻게 적용되는지 보여주며, 비전 공자도 쉽게 이해할 수 있도록 풍부한 설명과 예시를 제공합니다.

3.1 용어 정리

다음 표는 본 문서에서 사용되는 주요 용어들을 정리하고 쉽게 설명합니다.

용어	쉬운 설명	원어	비고
순환 신경망	시퀀스(순서가 있는) 데이터를 처리하는 데 특화된 신경망.	<i>Recurrent Neural Network (RNN)</i>	이전 단계의 정보를 기억하여 다음 단계에 활용
장단기 기억망	<i>RNN</i> 의 한 종류로, 장기 의존성 문제를 해결하기 위해 고안된 복잡한 구조.	<i>Long Short-Term Memory (LSTM)</i>	셀 상태(<i>Cell State</i>)와 게이트(<i>Gate</i>)를 통해 정보 흐름 제어
자연어 처리	컴퓨터가 인간의 언어를 이해하고 생성하도록 하는 인공지능 분야.	<i>Natural Language Processing (NLP)</i>	텍스트 분류, 번역, 요약 등 다양한 응용
토큰화	텍스트를 의미 있는 작은 단위(토큰)로 나누는 과정.	<i>Tokenization</i>	단어, 문장, 문자 등이 토큰이 될 수 있음
어간 추출	단어의 접미사나 접두사를 제거하여 단어의 '어간'을 찾는 과정.	<i>Stemming</i>	결과가 실제 단어가 아닐 수 있으며, 자원 축소에 유용
표제어 추출	단어를 사전적 의미를 가지는 '표제어'로 변환하는 과정.	<i>Lemmatization</i>	어간 추출보다 정확하며, 품사 정보 활용
임베딩	단어나 문장 같은 이산적인 데이터를 연속적인 숫자 벡터로 변환하는 기법.	<i>Embedding</i>	고차원 희소 벡터를 저차원 밀집 벡터로 효율적으로 표현
원-핫 인코딩	단어를 고유한 위치에 1을, 나머지는 0을 가지는 벡터로 표현하는 방식.	<i>One-Hot Encoding</i>	희소(<i>Sparse</i>)하며, 단어 간 유사성 표현 어려움
희소 벡터	대부분의 요소가 0인 벡터.	<i>Sparse Vector</i>	메모리 비효율적, 계산 낭비 발생 가능
밀집 벡터	대부분의 요소가 0이 아닌 벡터.	<i>Dense Vector</i>	정보 압축 및 효율적인 계산 가능
<i>OOV</i> 토큰	모델 학습 시 사용된 어휘 사전에 없는 단어를 나타내는 특별한 토큰.	<i>Out-Of-Vocabulary (OOV) Token</i>	사전에 없는 단어 처리 방식 중 하나
드롭아웃	신경망 훈련 중 일부 뉴런을 무작위로 비활성화하여 과적합을 방지하는 기법.	<i>Dropout</i>	모델의 일반화 성능 향상
이진 교차 엔트로피	이진 분류 문제에서 모델의 예측 확률과 실제 레이블 간의 차이를 측정하는 손실 함수.	<i>Binary Cross-Entropy</i>	로지스틱 회귀와 같은 이진 분류 모델에 적합
시그모이드 함수	입력 값을 0과 1 사이의 확률 값으로 변환하는 활성화 함수.	<i>Sigmoid Function</i>	이진 분류 모델의 출력 레이어에 주로 사용

3.2 핵심 개념·원리

순환 신경망 (*RNN*) 핵심 요약

*RNN*은 시퀀스 데이터를 처리하는 데 특화된 신경망으로, 이전 단계의 정보를 '기억'하여 현재 단계의 예측에 활용합니다. 이는 텍스트, 음성, 시계열 데이터와 같이 순서가 중요한 데이터에 특히 유용합니다.

*RNN*의 기본 구조와 작동 원리

*RNN*은 일반적인 신경망(피드포워드 신경망)과 달리, 은닉층의 출력이 다음 시점의 은닉층 입력으로 다시 들어가는 '순환(*recurrent*)' 연결을 가집니다. 이 순환 연결 덕분에 *RNN*은 시퀀스 데이터의 시간적 의존성을 학습할 수 있습니다.

*RNN*의 직관적 비유: 문장 읽기

우리가 문장을 읽을 때, 각 단어의 의미는 그 앞의 단어들과의 관계 속에서 이해됩니다. 예를 들어 "나는 사과를 먹었다"라는 문장에서 '먹었다'는 '사과'와 연결되어 의미를 완성합니다. *RNN*도 이와 유사하게, '사과'라는 단어를 처리할 때 '나는'이라는 정보와 '사과를'이라는 정보를 함께 고려하여 '먹었다'를 예측하거나 이해합니다.

Simple *RNN*의 파라미터 계산

RNN 모델의 복잡성을 이해하는 한 가지 방법은 학습 가능한 파라미터(가중치와 바이어스)의 수를 계산하는 것입니다. 이는 모델의 크기와 계산 비용을 가늠하는 데 중요합니다.

강의에서는 *SimpleRNN(2, input_shape=(None, 4))*와 같은 *Keras* 코드를 예시로 들었습니다.

- *SimpleRNN(2)*: 이 순환 레이어에는 2개의 뉴런(또는 은닉 상태)이 있습니다. 이는 출력 y 벡터가 2차원임을 의미합니다.
- *input_shape=(None, 4)*: 입력 데이터의 형태를 나타냅니다.
 - *None*: 시간 스텝(*time steps*)의 수는 가변적임을 의미합니다. 훈련 중 네트워크가 자동으로 시간 스텝 수를 파악합니다.

- 4: 각 시간 스텝에서 입력 x 벡터의 차원이 4임을 의미합니다. 즉, 각 시점에 4개의 특성 (*feature*)이 입력됩니다.

SimpleRNN 파라미터 계산 공식, *colback=myblue!10!white*, *colframe=myblue!75!black*

SimpleRNN 레이어의 총 파라미터 수는 다음과 같이 계산됩니다:

총 파라미터 수 = (입력 특성 수 $\times$ 은닉 뉴런 수) + (은닉 뉴런 수 $\times$ 은닉 뉴런 수) + 은닉 뉴런 수

여기서:

- 입력 특성 수: 각 시간 스텝에서 들어오는 입력 벡터의 차원 (예: 4).
- 은닉 뉴런 수: *RNN* 레이어 내의 뉴런 수 (예: 2).
- 은닉 뉴런 수 (세 번째 항): 각 뉴런에 대한 바이어스 (*bias*) 수.

SimpleRNN 파라미터 계산 예시

SimpleRNN(2, *input_shape*=(None, 4))의 경우:

- 입력 특성 수 = 4
- 은닉 뉴런 수 = 2

계산:

- 입력에서 은닉층으로의 가중치: $4 \times 2 = 8$
- 이전 은닉 상태에서 현재 은닉층으로의 가중치: $2 \times 2 = 4$
- 바이어스: 2 (각 은닉 뉴런에 하나씩)

총 파라미터 수 = $8 + 4 + 2 = 14$

이는 각 뉴런이 4개의 입력(x 벡터)과 2개의 순환 입력(y 벡터)을 받고, 1개의 바이어스를 가지므로 $(4 + 2 + 1) \times 2 = 14$ 로도 계산할 수 있습니다.

RNN의 다양한 입출력 아키텍처

*RNN*은 시퀀스 데이터를 처리하는 방식에 따라 다양한 입출력 구조를 가질 수 있습니다. *Keras*에서는 *return_sequences* 옵션을 통해 출력을 제어합니다.

- 다대다 (*Many-to-Many*): 입력 시퀀스에 대해 출력 시퀀스를 생성합니다.
 - 예시: 기계 번역 (입력 문장 -> 출력 문장).
 - *return_sequences=True* (기본값)로 설정하면 각 시간 스텝의 출력을 모두 반환합니다.
- 다대일 (*Many-to-One*): 입력 시퀀스에 대해 단일 출력을 생성합니다.
 - 예시: 감성 분석 (입력 문장 -> 긍정/부정).
 - *return_sequences=False*로 설정하면 마지막 시간 스텝의 출력만 반환합니다.
- 일대다 (*One-to-Many*): 단일 입력에 대해 출력 시퀀스를 생성합니다.
 - 예시: 이미지 캡셔닝 (입력 이미지 -> 이미지 설명 문장).
 - 이 경우, 단일 입력 후에는 '가짜(*fake*) 0 벡터'를 입력으로 넣어 시퀀스를 생성합니다.

주의: *RNN*의 출력과 *return_sequences*

RNN 레이어는 기본적으로 각 시간 스텝마다 출력을 생성합니다. *return_sequences=True*는 이 모든 출력을 다음 레이어로 전달하는 반면, *return_sequences=False*는 마지막 시간 스텝의 출력만 전달합니다. 이는 모델의 목적에 따라 적절히 선택해야 합니다.

3.2.1 인코더-디코더 (Encoder-Decoder) 아키텍처

인코더-디코더 핵심 요약

인코더-디코더는 입력 시퀀스를 고정된 크기의 '컨텍스트 벡터'로 압축하는 인코더와, 이 컨텍스트 벡터로부터 출력 시퀀스를 생성하는 디코더로 구성됩니다. 이는 정보 압축과 변환에 유용합니다.

정보 압축을 위한 병목 (Bottleneck)

인코더-디코더는 오토인코더(*Autoencoder*)의 한 형태로 볼 수 있으며, 특히 '병목(*bottleneck*)' 구조를 통해 정보 압축을 수행합니다.

인코더-디코더의 비유: 요약과 확장

긴 문장(입력 시퀀스)을 읽고 핵심 아이디어(컨텍스트 벡터)를 한 문장으로 요약한 다음, 그 핵심 아이디어로부터 원래 문장과 유사하거나 다른 언어의 문장(출력 시퀀스)을 다시 작성하는 것과 같습니다. 핵심 아이디어가 바로 '병목'을 통과한 압축된 정보입니다.

- 오토인코더 (*Autoencoder*): 입력 데이터를 자기 자신으로 매핑하는 신경망입니다. 이 과정에서 중간에 저차원의 '병목' 레이어를 두어 데이터의 중요한 특징을 압축하여 학습합니다.
 - 노이즈 제거 오토인코더 (*Denoising Autoencoder*): 손상된 이미지나 노이즈가 포함된 텍스트를 입력으로 받아 원본 데이터를 복원하도록 훈련됩니다. 병목 레이어는 노이즈와 같은 중요하지 않은 정보를 걸러내고 핵심 정보만 압축하는 방법을 학습합니다.
- 번역에서의 활용: *RNN* 기반 인코더-디코더는 2014년부터 2017년까지 기계 번역의 지배적인 기술이었습니다.
 - 인코더가 원어 문장(시퀀스)을 하나의 컨텍스트 벡터로 압축합니다.
 - 디코더가 이 컨텍스트 벡터로부터 대상 언어의 문장(시퀀스)을 생성합니다.
 - 한계: 하나의 고정된 컨텍스트 벡터가 긴 문장의 모든 정보를 담기 어렵다는 '정보 병목' 문제가 있었습니다. 이는 나중에 어텐션 메커니즘(*Attention Mechanism*)을 사용하는 트랜스포머(*Transformer*) 모델로 해결됩니다.

3.2.2 장단기 기억망 (Long Short-Term Memory, LSTM)의 이해

LSTM 핵심 요약

*LSTM*은 *RNN*의 '장기 의존성 (*long-term dependency*)' 문제를 해결하기 위해 고안된 특수한 *RNN* 구조입니다. '셀 상태 (*Cell State*)'와 '게이트 (*Gate*)' 메커니즘을 통해 정보를 선택적으로 기억하거나 잊어버리면서 장기적인 패턴을 학습할 수 있습니다.

LSTM의 두 가지 기억: 셀 상태 (C)와 은닉 상태 (H)

*LSTM*은 두 가지 주요 '기억'을 가집니다.

- 셀 상태 (*Cell State*, *C*): 장기 기억의 역할을 합니다. 정보가 셀 상태를 따라 흐르면서 게이트에 의해 추가되거나 제거됩니다.
- 은닉 상태 (*Hidden State*, *H*): 단기 기억의 역할을 하며, 현재 시점의 출력과 다음 시점의 입력으로 사용됩니다.

*C*와 *H*의 차원 관계, *colback=myblue!10!white*, *colframe=myblue!75!black*

LSTM 구조 내에서 셀 상태 (*C*)와 은닉 상태 (*H*)는 항상 동일한 차원 (*dimension*)을 가집니다. 이는 게이트를 통한 연산(요소별 곱셈, 덧셈)이 차원을 변경하지 않기 때문입니다.

*C*와 *H*의 차원 관계 직관적 이해

LSTM 셀 내부의 연산들을 살펴보면, 셀 상태 (*C*)와 은닉 상태 (*H*)는 서로 더해지거나 (*summation*), 요소별로 곱해지는 (*element-wise multiplication*) 방식으로 상호작용합니다. 벡터의 덧셈이나 요소별 곱셈은 벡터의 차원을 변경하지 않습니다. 따라서, *C*와 *H*는 항상 같은 차원을 유지해야 합니다. 예를 들어, *LSTM(5)*라고 지정하면 *C*와 *H* 모두 5 차원 벡터가 됩니다.

LSTM의 게이트 메커니즘

*LSTM*은 세 가지 주요 게이트를 통해 셀 상태의 정보 흐름을 제어합니다. 각 게이트는 시그모이드 활성화 함수를 사용하여 0과 1 사이의 값을 출력하며, 이는 정보의 '얼마나'를 통과시킬지 결정합니다.

- 망각 게이트 (*Forget Gate*): 이전 셀 상태에서 어떤 정보를 '잊어버릴지' 결정합니다.
- 입력 게이트 (*Input Gate*): 현재 입력과 이전 은닉 상태에서부터 어떤 새로운 정보를 '셀 상태에 추가할지' 결정합니다.
- 출력 게이트 (*Output Gate*): 셀 상태를 기반으로 현재 시점의 은닉 상태 (*H*)와 출력 (*Y*)을 '얼마나 내보낼지' 결정합니다.

각 게이트는 독립적인 완전 연결 (*Fully Connected, Dense*) 서브 네트워크로 구성되며, *LSTM(N)*에서 *N*은 각 서브 네트워크의 뉴런 수를 의미합니다. 따라서 *LSTM(N)*은 총 $4 \times N$ 개의 뉴런을 내부적으로 가집니다.

3.2.3 양방향 순환 신경망 (Bidirectional RNN/LSTM)

양방향 RNN/LSTM 핵심 요약

양방향 RNN/LSTM은 입력 시퀀스를 순방향과 역방향으로 모두 처리하여, 각 시점의 출력에 대해 과거와 미래의 모든 문맥 정보를 활용할 수 있도록 합니다. 이는 특히 텍스트와 같이 전체 문맥이 중요한 데이터에 유용합니다.

양방향 처리의 필요성

일반적인 RNN은 과거의 정보만을 기반으로 현재를 예측합니다. 하지만 많은 시퀀스 데이터, 특히 자연어에서는 미래의 정보(문장의 뒷부분)가 현재의 의미를 이해하는 데 중요할 수 있습니다.

양방향 RNN의 비유: 문장 전체 읽기

독일어와 같이 문장의 부정어가 마지막에 오는 언어의 경우, 문장의 끝을 읽어야만 전체 문장이 긍정적인지 부정적인지 정확히 판단할 수 있습니다. 순방향으로만 읽는다면 문장 끝에 도달하기 전까지는 정확한 의미를 파악하기 어렵습니다. 양방향 RNN은 문장을 앞에서부터 읽고(순방향), 뒤에서부터도 읽어(역방향) 두 정보를 모두 활용하여 더 풍부한 문맥 이해를 가능하게 합니다.

구조와 파라미터 계산

양방향 RNN/LSTM은 두 개의 독립적인 RNN/LSTM 레이어를 가집니다. 하나는 입력 시퀀스를 순방향으로 처리하고, 다른 하나는 역방향으로 처리합니다. 두 레이어의 출력은 최종적으로 연결(concatenate)되어 다음 레이어의 입력으로 사용됩니다.

양방향 RNN/LSTM의 파라미터 및 출력 차원, `colback=myblue!10!white`, `colframe=myblue!75!black`

- 파라미터 수: 양방향 레이어는 두 개의 독립적인 단방향 레이어를 포함하므로, 단방향 레이어의 파라미터 수의 정확히 두 배가 됩니다.
- 출력 차원: 각 단방향 레이어의 출력이 연결되므로, 양방향 레이어의 출력 차원 또한 단방향 레이어 출력 차원의 두 배가 됩니다.

양방향 LSTM 파라미터 계산 예시

만약 LSTM(128) 레이어가 8320개의 파라미터를 가진다면, Bidirectional(LSTM(128)) 레이어는 $8320 \times 2 = 16640$ 개의 파라미터를 가집니다.

이후에 연결되는 Dense 레이어의 파라미터는 단순히 두 배가 되지 않을 수 있습니다. 예를 들어, Dense(1) 레이어는 단일 바이어스를 가지므로, 양방향 LSTM의 출력이 2배가 되어 입력이 2배가 되더라도 바이어스는 하나만 추가되어 파라미터가 $32 \times 2 + 1 = 65$ 와 같이 계산됩니다.

3.2.4 자연어 처리 (Natural Language Processing, NLP) 개요

NLP 핵심 요약

NLP는 컴퓨터가 인간의 자연어를 이해하고, 해석하고, 생성할 수 있도록 하는 인공지능 분야입니다. 언어학적 지식과 인공지능 기술을 결합하여 다양한 언어 관련 문제를 해결합니다.

AI, 머신러닝, NLP의 관계

- 인공지능 (*Artificial Intelligence, AI*): 인간의 지능을 모방하는 광범위한 분야입니다. 규칙 기반 시스템(하드코딩된 규칙)부터 복잡한 학습 알고리즘까지 모두 포함합니다.
 - 예시: 의사가 특정 증상에 대한 처방 규칙을 컴퓨터에 입력하여 진단하는 시스템.
- 머신러닝 (*Machine Learning, ML*): 데이터와 결과(레이블)를 입력으로 받아 '규칙'을 스스로 학습하는 *AI*의 하위 분야입니다.
 - 예시: 선형 회귀 모델이 데이터로부터 최적의 β_0, β_1 계수를 학습하여 예측 규칙을 생성하는 것.
- 자연어 처리 (*NLP*): 언어학(언어의 구조, 의미)과 인공지능(특히 머신러닝, 딥러닝)이 결합된 다학제 분야입니다. 컴퓨터가 인간 언어를 이해하고 생성하도록 하는 것이 목표입니다.
 - *NLP*는 때로는 언어학적 규칙(예: 단어를 어근 형태로 줄이는 규칙)을 하드코딩하여 사용하기도 하므로, 순수 머신러닝의 범주를 넘어 *AI*의 더 넓은 영역에 걸쳐 있습니다.

NLP의 도전 과제

인간의 언어는 모호하고, 문맥에 따라 의미가 달라지며, 배경 지식을 요구하는 경우가 많아 컴퓨터가 이해하기 매우 어렵습니다.

- 모호성 (*Ambiguity*): "*Court to try shooting defendant.*" (법원이 피고를 총살하려 한다? 아니면 법원이 피고의 총격 사건을 심리한다?)와 같이 문장의 해석이 여러 가지일 수 있습니다.
- 배경 지식 (*Background Knowledge*): 특정 분야의 전문 지식이나 상식이 없으면 문장의 의미를 정확히 파악하기 어렵습니다.

NLP의 주요 응용 분야

- 텍스트 분류 (*Text Classification*): 스팸 메일 감지, 뉴스 기사 주제 분류, 감성 분석 (긍정/부정).
- 개체명 인식 (*Named Entity Recognition, NER*): 텍스트에서 사람 이름, 조직, 장소 등 고유한 개체를 식별합니다. (예: 번역 시 고유명사는 번역하지 않고 그대로 두는 것이 중요).
- 감성 분석 (*Sentiment Analysis*): 텍스트에 담긴 감정(긍정, 부정, 중립)을 파악합니다. (예: 영화 리뷰 분석). 비꼬는 표현(*sarcasm*) 등은 매우 어려운 문제입니다.
- 정보 검색 (*Information Retrieval*): 대규모 텍스트 데이터에서 관련성 높은 정보를 찾아냅니다. (예: 검색 엔진). *TF-IDF*, *BM25* 등의 기법이 사용됩니다.

- 광학 문자 인식 (*Optical Character Recognition, OCR*): 이미지 형태의 텍스트를 컴퓨터가 편집 가능한 텍스트로 변환합니다. (예: 문서 스캔).
- 기계 번역 (*Machine Translation*): 한 언어의 문장을 다른 언어로 번역합니다. (예: 구글 번역).
- 텍스트 요약 (*Text Summarization*): 긴 텍스트의 핵심 내용을 추출하여 짧은 요약문을 생성합니다.
- 음성 인식 (*Speech Recognition*): 음성 데이터를 텍스트로 변환합니다. (예: 음성 비서).
- 질의응답 시스템 (*Question Answering Systems*): 질문에 대한 답변을 텍스트에서 찾아 제공합니다. (예: "프랑스의 수도는?").
- 챗봇 (*Chatbots*): 사용자와 대화하며 질문에 답하거나 작업을 수행합니다. (예: *ChatGPT*). 질의응답 시스템과 유사하지만 대화의 연속성이 중요합니다.
- 토픽 모델링 (*Topic Modeling*): 텍스트 문서 집합에서 숨겨진 주제(*topic*)를 찾아내고, 각 문서가 어떤 주제로 구성되어 있는지 분석합니다. 비지도 학습의 일종입니다.
- 임상 응용 (*Clinical Applications*): 의료 문헌 분석, 증상 기반 질병 진단 지원 등 의료 분야에서 *NLP* 활용.
- 언어 생성 (*Language Generation*): 인간과 유사한 자연스러운 텍스트를 생성합니다.

3.3 절차/방법: 텍스트 전처리 및 모델 구축

3.3.1 텍스트 전처리 (Text Preprocessing) 과정

신경망에 텍스트를 입력하기 전에, 텍스트를 컴퓨터가 이해할 수 있는 형태로 변환하는 전처리 과정이 필수적입니다. 주요 단계는 토큰화, 어간 추출, 표제어 추출입니다.

토큰화 (Tokenization)

토큰화 핵심 요약

토큰화는 텍스트를 의미 있는 최소 단위인 '토큰(*token*)'으로 분할하는 과정입니다. 일반적으로 단어가 토큰이 되지만, 문장, 문자, *N*-그램 등 다양한 단위로 토큰화할 수 있습니다.

- 정의: 텍스트를 개별 단위(토큰)로 분리하는 과정입니다.
- 토큰의 종류:
 - 단어 토큰화 (*Word Tokenization*): 가장 일반적인 형태로, 텍스트를 개별 단어로 나눕니다. 구두점도 별도의 토큰으로 처리될 수 있습니다.
 - 문장 토큰화 (*Sentence Tokenization*): 텍스트를 개별 문장으로 나눕니다.
 - *N*-그램 토큰화 (*N-gram Tokenization*): *N*개의 연속된 단어를 하나의 토큰으로 간주합니다 (예: 2-그램은 두 단어).
 - 문자 토큰화 (*Character Tokenization*): 텍스트를 개별 문자로 나눕니다.
 - 서브워드 토큰화 (*Subword Tokenization*): 단어를 더 작은 의미 단위(서브워드)로 분할

합니다. (예: 'un-happy'를 'un'과 'happy'로).

- 불용어 (Stop Words): 'the', 'a', 'is'와 같이 자주 등장하지만 텍스트의 핵심 의미를 파악하는 데 크게 기여하지 않는어들입니다. 텍스트 분류와 같은 작업에서는 불용어를 제거하는 것이 모델 성능 향상에 도움이 될 수 있습니다.

NLTK를 이용한 토큰화 예시

NLTK (Natural Language Toolkit) 라이브러리는 파이썬에서 텍스트를 토큰화하는 데 널리 사용됩니다.

```

1 import nltk
2 from nltk.tokenize import word_tokenize, sent_tokenize
3
4 text = "Henry Ford's innovation profoundly impacted the automotive
        industry. He revolutionized mass production."
5
6 # 단어 토큰화
7 word_tokens = word_tokenize(text)
8 print("단어 토큰:", word_tokens)
9 # 출력: ['Henry', 'Ford', "'s", 'innovation', 'profoundly', 'impacted', '
        the', 'automotive', 'industry', '.', 'He', 'revolutionized', 'mass',
        'production', '.']
10
11 # 문장 토큰화
12 sent_tokens = sent_tokenize(text)
13 print("문장 토큰:", sent_tokens)
14 # 출력: ["Henry Ford's innovation profoundly impacted the automotive
        industry.", "He revolutionized mass production."]

```

Listing 3.1: NLTK 단어 토큰화 예시

라이브러리 사용 권장

직접 토큰화 로직을 구현하는 것보다 *NLTK*와 같은 라이브러리를 사용하는 것이 좋습니다. 라이브러리는 'can't'와 같은 축약형이나 구두점 처리 등 복잡한 예외 상황을 더 잘 처리합니다.

어간 추출 (Stemming)

어간 추출 핵심 요약

어간 추출은 단어의 접미사나 접두사를 제거하여 단어의 '어간(stem)'을 찾는 과정입니다. 이는 단어의 다양한 형태를 하나의 공통된 형태로 줄여 어휘 사전을 축소하고 모델의 차원을 줄이는 데 사용됩니다.

- 정의: 단어의 형태학적 변형(예: *running*, *runs*, *ran*)을 무시하고 단어의 '어간'을 추출하는 기법입니다. 주로 접미사를 제거하는 방식으로 작동합니다.
- 목적: 어휘 사전에 포함될 단어의 수를 줄여 모델의 복잡성을 낮춥니다.

- 특징: 어간 추출의 결과는 종종 실제 사전에 없는 단어 형태일 수 있습니다. (예: 'electric' -> 'electr', 'octopus' -> 'octop').
- 종류:
 - *Porter Stemmer* (포터 스테머): 1979년에 개발된 가장 유명한 어간 추출 알고리즘 중 하나입니다. 규칙 기반으로 접미사를 제거합니다. (예: 'transportation' -> 'transport', 'electric' -> 'electr').
 - *Snowball Stemmer* (스노볼 스테머): *Porter Stemmer*의 개선 버전으로, 여러 언어를 지원하며 정확도가 향상되었습니다.
 - *Lancaster Stemmer* (랭커스터 스테머): *Porter Stemmer*보다 더 공격적으로 단어를 자릅니다.

NLTK를 이용한 어간 추출 예시

```

1  from nltk.stem import PorterStemmer, SnowballStemmer
2  from nltk.tokenize import word_tokenize
3
4  text = "Henry Ford's innovation profoundly impacted the automotive
5         industry. He revolutionized mass production."
6  tokens = word_tokenize(text)
7
8  porter_stemmer = PorterStemmer()
9  snowball_stemmer = SnowballStemmer("english") # 언어 지정
10
11 stemmed_porter = [porter_stemmer.stem(token) for token in tokens]
12 stemmed_snowball = [snowball_stemmer.stem(token) for token in tokens]
13
14 print("원본 토큰:", tokens)
15 print("Porter Stemmer 결과:", stemmed_porter)
16 # 예시 출력: ['henri', 'ford', "'s", 'innov', 'profoundli', 'impact', 'the', '
17               automot', 'industri', '.', 'he', 'revolut', 'mass', 'product', '.']
18 print("Snowball Stemmer 결과:", stemmed_snowball)
19 # 예시 출력: ['henri', 'ford', "'s", 'innov', 'profound', 'impact', 'the', '
20               automot', 'industri', '.', 'he', 'revolut', 'mass', 'product', '.']

```

Listing 3.2: NLTK Porter Stemmer 예시

어간 추출의 특징

'Henry'가 'Henri'로, 'profoundly'가 'profoundli' 또는 'profound'로 변하는 것을 볼 수 있습니다. 이는 어간 추출이 단어를 사전형으로 만드는 것이 아니라, 단순히 규칙에 따라 접미사를 제거하여 '어간'을 찾는 과정이기 때문입니다. 결과가 실제 사전에 없는 단어일 수 있습니다.

표제어 추출 (Lemmatization)

표제어 추출 핵심 요약

표제어 추출은 단어를 사전적 의미를 가지는 '표제어(lemma)'로 변환하는 과정입니다. 어간 추출보다 더 정교하며, 단어의 품사(명사, 동사 등) 정보를 활용하여 정확한 원형을 찾습니다.

- 정의: 단어의 다양한 형태를 문법적 규칙에 따라 '표제어(lemma)' 또는 사전형으로 변환하는 기법입니다. (예: 'driving', 'drove', 'driven' -> 'drive').
- 목적: 어간 추출과 마찬가지로 차원 축소를 목표로 하지만, 결과가 항상 실제 사전에 있는 단어 형태를 유지합니다.
- 특징: 어간 추출보다 계산 비용이 더 많이 듭니다. 단어의 품사(Part-of-Speech, POS) 정보를 먼저 식별해야 하는 경우가 많습니다.
- 도구:
 - *WordNetLemmatizer (NLTK)*: NLTK에서 제공하는 표제어 추출 도구입니다. 품사 태깅이 필요하며, 주로 영어에 사용됩니다. 학술 연구에서 인기가 많습니다.
 - *SpaCy*: 산업 환경에서 많이 사용되는 라이브러리로, *WordNetLemmatizer*보다 빠르고 효율적입니다.

NLTK와 SpaCy를 이용한 표제어 추출 예시

```

1 import nltk
2 from nltk.stem import WordNetLemmatizer
3 from nltk.tokenize import word_tokenize
4 # nltk.download('wordnet')
5 # nltk.download('omw-1.4')
6 # nltk.download('averaged_perceptron_tagger') # 품사 태깅을 위해 필요
7
8 import spacy
9 # python -m spacy download en_core_web_sm # SpaCy 모델 다운로드
10
11 text = "Henry Ford's innovation profoundly impacted the automotive
12         industry. He revolutionized mass production."
13
14 # NLTK WordNetLemmatizer
15 lemmatizer = WordNetLemmatizer()
16 lemmatized_nltk = [lemmatizer.lemmatize(token, pos='v') for token in
17                     tokens] # 동사(v)로 가정
18 # 실제 사용 시에는 품사 태깅 후 해당 품사에 맞는 pos 인자를 전달해야 합니다.
19 # 예: nltk.pos_tag(tokens) 후 각 단어의 품사를 기반으로 lemmatize 호출
20
21 print("NLTK Lemmatizer 동사(가정) 결과:", lemmatized_nltk)
22 # 예시 출력: ['Henry', 'Ford', "'s", 'innovation', 'profoundly', 'impact',
23               'the', 'automotive', 'industry', '.', 'He', 'revolutionize', 'mass',

```

```

22     'production', '.']
23 # SpaCy Lemmatization
24 nlp = spacy.load("en_core_web_sm")
25 doc = nlp(text)
26 lemmatized_spacy = [token.lemma_ for token in doc]
27
28 print("SpaCy Lemmatizer 결과:", lemmatized_spacy)
29 # 예시 출력: ['Henry', 'Ford', "'s", 'innovation', 'profoundly', 'impact',
    'the', 'automotive', 'industry', '.', 'he', 'revolutionize', 'mass',
    'production', '.']

```

Listing 3.3: NLTK WordNetLemmatizer 및 SpaCy 예시

어간 추출 vs. 표제어 추출

- 어간 추출 (*Stemming*): 'driving' -> 'driv'. 빠르고 간단하지만, 결과가 실제 단어가 아닐 수 있습니다.
- 표제어 추출 (*Lemmatization*): 'driving' -> 'drive'. 느리고 복잡하지만, 결과가 항상 실제 사전에 있는 단어입니다.

어떤 방법을 사용할지는 작업의 목적과 요구되는 정확도에 따라 달라집니다. 텍스트 분류와 같이 의미의 정확한 복원보다 차원 축소가 중요한 경우에는 어간 추출도 충분히 효과적일 수 있습니다.

3.3.2 텍스트 분류 모델 구축 (실습)

강의에서는 *20 Newsgroups* 데이터셋을 사용하여 텍스트 분류 모델을 구축하는 과정을 설명했습니다. 목표는 게시물의 내용을 기반으로 '하키(hockey)' 관련 게시물인지 '판매(for sale)' 관련 게시물인지 분류하는 것입니다.

데이터셋 준비

- 데이터셋: `sklearn.datasets.fetch_20newsgroups`를 사용하여 20개의 뉴스그룹 중 'rec.sport.hockey'와 'misc.forsale' 두 가지 카테고리만 선택합니다.
- 분리: 훈련(*train*) 데이터셋과 테스트(*test*) 데이터셋으로 나눕니다.
- 레이블: 'for sale'은 0, 'hockey'는 1로 레이블링됩니다.

텍스트를 숫자로 변환: 임베딩 (Embedding)

임베딩 레이어 핵심 요약

임베딩 레이어는 단어와 같은 이산적인(*discrete*) 데이터를 저차원의 연속적인(*continuous*) 숫자 벡터로 변환하는 신경망 레이어입니다. 이는 고차원의 희소(*sparse*)한 원-핫 인코딩의 비효율성을 해결하고, 단어 간의 의미적 유사성을 벡터 공간에 반영하여 모델의 학습 효율과 성능을 높입니다.

- 원-핫 인코딩의 한계:
 - 희소성 (*Sparsity*): 어휘 사전의 크기가 커질수록 벡터의 대부분이 0이 되어 메모리 낭비가 심합니다. (예: 10,000개 단어 사전에서 각 단어는 10,000차원 벡터).
 - 의미 관계 부족: 단어 간의 유사성이나 관계를 표현할 수 없습니다. 모든 단어는 서로 독립적으로 간주됩니다.
- 임베딩의 필요성:
 - 차원 축소 (*Dimensionality Reduction*): 고차원의 희소 벡터를 저차원의 밀집(*dense*) 벡터로 변환하여 효율성을 높입니다.
 - 의미 학습 (*Semantic Learning*): 학습 가능한 파라미터(가중치)를 통해 단어의 의미적 유사성을 벡터 공간에 반영합니다. (예: '왕'과 '여왕'은 벡터 공간에서 가까이 위치).
- 작동 원리:
 - 임베딩 레이어는 입력으로 단어의 인덱스(정수)를 받습니다.
 - 내부적으로는 각 단어 인덱스에 해당하는 '임베딩 벡터'를 가집니다. 이 벡터들은 모델 훈련 과정에서 함께 학습됩니다.
 - `Embedding(input_dim, output_dim)`:
 - * `input_dim`: 어휘 사전의 크기 (총 단어 수 + *OOV* 토큰).
 - * `output_dim`: 임베딩 벡터의 차원 (예: 128).

임베딩의 비유: 사람의 특징 벡터

사람을 '남성'과 '여성'으로 분류할 때, 원-핫 인코딩은 '남성'을 [1, 0], '여성'을 [0, 1]로 표현합니다. 이는 두 범주가 완전히 독립적임을 나타냅니다.

하지만 임베딩은 각 범주에 대해 학습 가능한 숫자(예: 남성 -> 0.8, 여성 -> 0.2)를 할당하여, 이 숫자가 모델의 다른 부분에서 의미 있는 방식으로 사용되도록 합니다. 더 복잡한 경우, '성별', '나이', '직업' 등 여러 특징을 나타내는 벡터로 사람을 표현할 수 있습니다. 임베딩은 이러한 특징 벡터를 데이터로부터 스스로 학습합니다.

임베딩 레이어의 효율성

수학적으로 임베딩 레이어는 선형 활성화 함수와 바이어스가 없는 완전 연결(*Dense*) 레이어와 유사하게 볼 수 있습니다. 그러나 실제 구현에서는 단어 인덱스를 직접 사용하여 해당 임베딩 벡터를 '조회(*lookup*)'하는 방식으로 작동하므로, 고차원 원-핫 벡터에 대한 행렬 곱셈을 수행하는 것보다 훨씬 계산적으로 효율적입니다.

과적합 방지: 드롭아웃 (Dropout)

드롭아웃 핵심 요약

드롭아웃은 신경망의 과적합(*overfitting*)을 방지하기 위한 정규화(*regularization*) 기법입니다. 훈련 과정에서 무작위로 일부 뉴런의 연결을 끊어 모델이 특정 특징에 과도하게 의존하는 것을 막고 일반화 성능을 향상시킵니다.

- 정의: 훈련 단계에서 각 미니 배치(*mini-batch*)마다 무작위로 특정 비율의 뉴런을 비활성화 (출력을 0으로 설정)합니다.
- 작동 방식:
 - *Dropout(0.2)*: 각 뉴런이 20%의 확률로 비활성화됩니다.
 - 비활성화된 뉴런은 해당 훈련 스텝에서 가중치 업데이트에 참여하지 않습니다.
 - 매 훈련 스텝마다 무작위로 다른 뉴런들이 비활성화되므로, 모델은 다양한 '하위 네트워크'를 학습하는 효과를 얻습니다.
- 종류:
 - 일반 드롭아웃: 입력 레이어나 *LSTM*의 출력에 적용되어 특정 특성 값들을 0으로 만듭니다.
 - 순환 드롭아웃 (*Recurrent Dropout*): *LSTM* 내부의 순환 연결에 적용되어 은닉 상태의 일부를 0으로 만듭니다.

드롭아웃의 비유: 팀워크 훈련

한 팀의 모든 구성원이 항상 완벽하게 제 역할을 한다고 가정하면, 특정 구성원이 없으면 팀 전체가 무너질 수 있습니다. 드롭아웃은 훈련 중에 무작위로 팀원 몇 명을 쉬게 하는 것과 같습니다. 이렇게 하면 나머지 팀원들은 어떤 상황에서도 팀의 목표를 달성하는 방법을 학습하게 되어, 특정 개인에게 의존하지 않는 더 강하고 유연한 팀(모델)이 됩니다.

모델 아키텍처

강의에서 제시된 텍스트 분류 모델은 다음과 같은 간단한 *LSTM* 기반 구조를 가집니다.

```

1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import Embedding, LSTM, Dropout, Dense
3
4 def build_model(max_features, embedding_dim, max_len):
5     model = Sequential()
6     # 1. 임베딩 레이어: 단어 인덱스를 밀집 벡터로 변환
7     model.add(Embedding(input_dim=max_features + 1, output_dim=embedding_dim,
8                           input_length=max_len))
9     # 2. 드롭아웃: 임베딩 레이어 출력에 적용하여 과적합 방지
10    model.add(Dropout(0.2))
11    # 3. LSTM 레이어: 시퀀스 데이터의 장기 의존성 학습
12    model.add(LSTM(128)) # 개의128 은닉 상태
13    # 4. 드롭아웃: LSTM 레이어 출력에 적용하여 과적합 방지
14    model.add(Dropout(0.2))

```



```

14 # 5. 완전 연결 레이어: 최종 분류를 위한 Dense 레이어
15 # 이진 분류이므로 개1 뉴런과 시그모이드 활성화 함수 사용
16 model.add(Dense(1, activation='sigmoid'))
17
18 # 모델 컴파일
19 model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['
accuracy'])
20 return model

```

Listing 3.4: 텍스트 분류 모델 아키텍처

- *Embedding Layer*: $max_features + 1$ (어휘 사전 크기 + *OOV*) 차원의 입력을 *embedding_dim* (예: 128) 차원의 밀집 벡터로 변환합니다. *input_length*는 각 시퀀스의 최대 길이입니다.
- *Dropout Layer*: 임베딩 레이어의 출력에 적용됩니다.
- *LSTM Layer*: 128개의 은닉 상태를 가지는 *LSTM* 레이어입니다. *return_sequences*가 지정되지 않았으므로 기본값인 *False*가 적용되어 마지막 시간 스텝의 출력만 다음 레이어로 전달됩니다.
- *Dropout Layer*: *LSTM* 레이어의 출력에 적용됩니다.
- *Dense Layer*: 1개의 뉴런과 *sigmoid* 활성화 함수를 사용하여 이진 분류를 수행합니다.

손실 함수 (Loss Function)와 활성화 함수 (Activation Function)

- 시그모이드 활성화 함수 (*Sigmoid Activation Function*):
 - 정의: 입력 값을 0과 1 사이의 값으로 변환하는 함수입니다.
 - 공식: $\sigma(z) = \frac{1}{1+e^{-z}}$
 - 활용: 이진 분류 문제에서 출력 레이어에 사용되어 예측 확률을 나타냅니다. 로지스틱 회귀 (*Logistic Regression*)의 핵심입니다.
- 이진 교차 엔트로피 (*Binary Cross-Entropy*):
 - 정의: 이진 분류 문제에서 모델의 예측 확률과 실제 레이블(0 또는 1) 간의 차이를 측정하는 손실 함수입니다.
 - 공식: $L = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}))$
 - 활용: 시그모이드 활성화 함수와 함께 이진 분류 모델의 손실 함수로 사용됩니다.
 - 주의: 이진 분류 문제에서 평균 제곱 오차(*Mean Squared Error, MSE*)는 적절하지 않습니다. *MSE*는 확률 분포가 아닌 연속적인 값의 예측에 더 적합하며, 이진 분류에서는 경사 하강법의 수렴을 방해하는 등의 문제가 발생할 수 있습니다.

데이터 전처리 상세 과정

모델에 텍스트를 입력하기 위해 다음 단계를 거칩니다.

- 최대 특성 수 (*max_features*): 어휘 사전에 포함할 가장 빈번한 단어의 수를 지정합니다 (예: 10,000개). 이는 모델의 복잡성을 제어하고 희소성을 줄입니다.

- 최대 길이 (*max_len*): 각 텍스트 시퀀스의 최대 길이를 지정합니다 (예: 500). 길이가 긴 텍스트는 잘리고, 짧은 텍스트는 0으로 채워집니다(패딩).
- *OOV* (*Out-Of-Vocabulary*) 토큰: *max_features*에 포함되지 않는 단어들을 처리하기 위한 특별한 토큰입니다. 일반적으로 인덱스 0을 *OOV* 토큰에 할당합니다.
- 단어 빈도 계산 및 인덱싱:
 1. 훈련 텍스트를 토큰화합니다.
 2. 각 토큰의 빈도를 계산합니다.
 3. 빈도 순으로 정렬하여 상위 *max_features*개의 단어만 선택합니다.
 4. 각 단어에 고유한 인덱스를 부여합니다 (*OOV*는 0, 나머지는 1부터 시작).
- 텍스트를 인덱스 시퀀스로 변환: 각 텍스트를 해당 단어 인덱스의 시퀀스로 변환합니다. 사전에 없는 단어는 *OOV* 인덱스(0)로 대체됩니다.

텍스트를 인덱스 시퀀스로 변환 예시

원본 텍스트: "This is a test sentence." 어휘 사전: 'this': 1, 'is': 2, 'a': 3, 'test': 4, 'sentence': 5, 'OOV': 0

변환된 인덱스 시퀀스: [1, 2, 3, 4, 5]

만약 "new word here"라는 텍스트가 들어오고 'new', 'word', 'here'가 사전에 없다면:
변환된 인덱스 시퀀스: [0, 0, 0]

모델 훈련 및 평가

- 훈련: *model.fit()* 함수를 사용하여 훈련 데이터와 레이블로 모델을 훈련합니다. *epochs* (훈련 반복 횟수)와 *batch_size* (미니 배치 크기)를 지정합니다.
- 평가: *model.evaluate()* 함수를 사용하여 테스트 데이터셋에서 모델의 성능을 평가합니다. *accuracy* (정확도)는 분류 문제에서 모델의 예측이 얼마나 정확한지를 나타내는 지표입니다.

전처리 기법별 성능 비교 (예시), *colback=myblue!10!white*, *colframe=myblue!75!black*

강의에서 제시된 결과는 다음과 같습니다.

- 토큰화만 적용: 테스트 정확도 약 93.5%
- 토큰화 + 어간 추출 적용: 테스트 정확도 약 96.8%
- 토큰화 + 표제어 추출 적용: 테스트 정확도 약 93.8%

이 예시에서는 어간 추출이 가장 좋은 성능을 보였습니다. 이는 어간 추출이 어휘 사전을 효과적으로 줄여 모델이 더 일반적인 패턴을 학습하도록 도왔기 때문일 수 있습니다. 하지만 이는 데이터셋과 모델에 따라 달라질 수 있으므로, 항상 여러 방법을 실험해봐야 합니다.

3.4 실습/코드: 텍스트 분류 모델 구현

이 섹션에서는 *20 Newsgroups* 데이터셋을 사용하여 텍스트 분류 모델을 구현하는 과정을 단계별로 보여줍니다. 토큰화, 어간 추출, 표제어 추출을 적용한 세 가지 시나리오를 비교합니다.

과제 2번 및 3번 관련 힌트

과제 2번은 '*Bag of Words*' 방식을 사용하여 텍스트 분류를 수행합니다. 이는 시간 정보를 잃지만, 특정 분류 문제에서 효율적일 수 있습니다. 과제 3번은 순환 신경망을 사용하여 텍스트 분류를 수행하며, 본 실습과 유사한 접근 방식을 사용합니다.

3.4.1 환경 설정 및 데이터 로딩

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import fetch_20newsgroups
4 from tensorflow.keras.preprocessing.text import Tokenizer
5 from tensorflow.keras.preprocessing.sequence import pad_sequences
6 from tensorflow.keras.models import Sequential
7 from tensorflow.keras.layers import Embedding, LSTM, Dropout, Dense
8 from nltk.tokenize import word_tokenize
9 from nltk.stem import PorterStemmer, SnowballStemmer, WordNetLemmatizer
10 # from nltk.corpus import wordnet # WordNetLemmatizer 사용 시 필요
11 # import spacy # SpaCy 사용 시 필요
12
13 # NLTK 데이터 다운로드 최초(회1 실행)
14 # nltk.download('punkt')
15 # nltk.download('wordnet')
16 # nltk.download('omw-1.4')
17 # nltk.download('averaged_perceptron_tagger')
18
19 # 20 Newsgroups 데이터셋 로드
20 categories = ['rec.sport.hockey', 'misc.forsale']
21 train_data = fetch_20newsgroups(subset='train', categories=categories, shuffle=True, random_state=42)
22 test_data = fetch_20newsgroups(subset='test', categories=categories, shuffle=True, random_state=42)
23
24 train_texts = train_data.data
25 train_labels = train_data.target
26 test_texts = test_data.data
27 test_labels = test_data.target
28
29 print(f"훈련 데이터 개수: {len(train_texts)}")
30 print(f"테스트 데이터 개수: {len(test_texts)}")
31 print(f"카테고리: {train_data.target_names}")
32
33 # 예시 데이터 확인

```

```

34 print("\n--- 훈련 데이터 예시 ---")
35 print(f"텍스트 번째(53): {train_texts[52][:200]}...")
36 print(f"레이블 번째(53): {train_labels[52]} ({train_data.target_names[
    train_labels[52]])") # 0: for sale
37 print(f"텍스트 번째(65): {train_texts[64][:200]}...")
38 print(f"레이블 번째(65): {train_labels[64]} ({train_data.target_names[
    train_labels[64]])") # 1: hockey

```

Listing 3.5: 환경 설정 및 데이터 로딩

3.4.2 모델 정의 함수

앞서 설명한 *LSTM* 기반 모델 아키텍처를 함수로 정의합니다.

```

1 def build_lstm_model(max_features, embedding_dim, max_len):
2     model = Sequential()
3     model.add(Embedding(input_dim=max_features + 1, output_dim=embedding_dim,
4         input_length=max_len))
5     model.add(Dropout(0.2))
6     model.add(LSTM(128))
7     model.add(Dropout(0.2))
8     model.add(Dense(1, activation='sigmoid'))
9     model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['
    accuracy'])
10    return model

```

Listing 3.6: 텍스트 분류 모델 정의 함수

3.4.3 시나리오 1: 토큰화만 적용

텍스트를 단어 토큰으로만 분할하고, 어간 추출이나 표제어 추출은 적용하지 않습니다.

```

1 # 전처리 파라미터
2 max_features = 10000 # 가장 빈번한 개10000 단어 사용
3 max_len = 500 # 시퀀스 최대 길이
4 embedding_dim = 128 # 임베딩 벡터 차원
5
6 # 1. 토큰화 및 단어 인덱스 생성
7 # Keras 는Tokenizer 자동으로 단어 빈도를 계산하고 인덱스를 부여합니다.
8 # 는num_words max_features + 1 (OOV 포함)
9 tokenizer = Tokenizer(num_words=max_features, oov_token="<OOV>")
10 tokenizer.fit_on_texts(train_texts)
11
12 # 텍스트를 인덱스 시퀀스로 변환
13 train_sequences = tokenizer.texts_to_sequences(train_texts)
14 test_sequences = tokenizer.texts_to_sequences(test_texts)
15
16 # 시퀀스 패딩 최대( 길이 맞추기)

```

```

17 train_padded = pad_sequences(train_sequences, maxlen=max_len, padding='post',
    truncating='post')
18 test_padded = pad_sequences(test_sequences, maxlen=max_len, padding='post',
    truncating='post')
19
20 print("\n--- 시나리오 1: 토큰화만 적용 ---")
21 print(f"훈련 데이터 패딩 후 형태: {train_padded.shape}")
22 print(f"테스트 데이터 패딩 후 형태: {test_padded.shape}")
23 print(f"단어 사전 크기: {len(tokenizer.word_index)}") # 실제 단어 인덱스 수
24
25 # OOV 예시 강의에서( 언급된 philip_ws@spk.hp.com)
26 # 이메일 주소는 빈번하지 않아 로 OOV 처리될 가능성이 높음
27 example_text = "From: philip_ws@spk.hp.com (Philip W. S)"
28 example_sequence = tokenizer.texts_to_sequences([example_text])
29 print(f"예시 텍스트: '{example_text}'")
30 print(f"변환된 시퀀스 (OOV 포함): {example_sequence}")
31 # 출력에서 은0 OOV 토큰을 의미합니다.
32
33 # 2. 모델 구축 및 훈련
34 model_tokenize = build_lstm_model(max_features, embedding_dim, max_len)
35 print("\n--- 모델 요약 토큰화만() ---")
36 model_tokenize.summary()
37
38 history_tokenize = model_tokenize.fit(
39     train_padded, train_labels,
40     epochs=10,
41     batch_size=32,
42     validation_split=0.2 # 훈련 데이터의 20% 검증 데이터로 사용
43 )
44
45 # 3. 모델 평가
46 loss_tokenize, accuracy_tokenize = model_tokenize.evaluate(test_padded,
    test_labels)
47 print(f"\테스트 {n} 정확도 토큰화만(): {accuracy_tokenize:.4f}")
48
49 # 결과 시각화
50 plt.figure(figsize=(12, 4))
51 plt.subplot(1, 2, 1)
52 plt.plot(history_tokenize.history['accuracy'], label='Train Accuracy')
53 plt.plot(history_tokenize.history['val_accuracy'], label='Validation Accuracy')
54
55 plt.title('Accuracy (Tokenization Only)')
56 \xlabel('Epoch')
57 \ylabel('Accuracy')
58 \legend()
59
60 \subplot(1, 2, 2)
61 \plot(history_tokenize.history['loss'], label='Train Loss')
62 \plot(history_tokenize.history['val_loss'], label='Validation Loss')
63 \title('Loss (Tokenization Only)')

```

```

63 \xlabel('Epoch')
64 \ylabel('Loss')
65 \legend()
66 \tight_layout()
67 \show()

```

Listing 3.7: 시나리오 1: 토큰화만 적용

3.4.4 시나리오 2: 토큰화 + 어간 추출 적용

텍스트를 토큰화한 후, *Porter Stemmer*를 사용하여 각 단어의 어간을 추출합니다.

```

1 porter_stemmer = PorterStemmer()
2
3 def stem_text(text):
4     tokens = word_tokenize(text)
5     stemmed_tokens = [porter_stemmer.stem(token) for token in tokens]
6     return " ".join(stemmed_tokens)
7
8 # 훈련 및 테스트 텍스트에 어간 추출 적용
9 train_texts_stemmed = [stem_text(text) for text in train_texts]
10 test_texts_stemmed = [stem_text(text) for text in test_texts]
11
12 print("\n--- 시나리오 2: 토큰화 + 어간 추출 적용 ---")
13 # 예시 텍스트 확인 강의에서( 언급된 octopus -> octop)
14 example_original = train_texts[64][:200]
15 example_stemmed = stem_text(example_original)
16 print(f"원본 텍스트 번째(65): {example_original}...")
17 print(f"어간 추출 후 텍스트: {example_stemmed}...")
18 # 'octopus가' 'octop으로', 'years가' 'year로' 변환되는 것을 볼 수 있습니다.
19
20 # 1. 토큰화 및 단어 인덱스 생성 어간( 추출된 텍스트 사용)
21 tokenizer_stem = Tokenizer(num_words=max_features, oov_token="<OOV>")
22 tokenizer_stem.fit_on_texts(train_texts_stemmed)
23
24 train_sequences_stem = tokenizer_stem.texts_to_sequences(train_texts_stemmed)
25 test_sequences_stem = tokenizer_stem.texts_to_sequences(test_texts_stemmed)
26
27 train_padded_stem = pad_sequences(train_sequences_stem, maxlen=max_len,
28                                   padding='post', truncating='post')
29 test_padded_stem = pad_sequences(test_sequences_stem, maxlen=max_len, padding=
30                                   'post', truncating='post')
31
32 print(f"훈련 데이터 패딩 후 형태 어간( 추출): {train_padded_stem.shape}")
33 print(f"단어 사전 크기 어간( 추출): {len(tokenizer_stem.word_index)}")
34
35 # 2. 모델 구축 및 훈련
36 model_stem = build_lstm_model(max_features, embedding_dim, max_len)
37 print("\n--- 모델 요약 어간( 추출) ---")
38 model_stem.summary()

```

```

37
38 history_stem = model_stem.fit(
39     train_padded_stem, train_labels,
40     epochs=10,
41     batch_size=32,
42     validation_split=0.2
43 )
44
45 # 3. 모델 평가
46 loss_stem, accuracy_stem = model_stem.evaluate(test_padded_stem, test_labels)
47 print(f"\테스트 정확도 어간( 추출): {accuracy_stem:.4f}")
48
49 # 결과 시각화
50 plt.figure(figsize=(12, 4))
51 plt.subplot(1, 2, 1)
52 plt.plot(history_stem.history['accuracy'], label='Train Accuracy')
53 plt.plot(history_stem.history['val_accuracy'], label='Validation Accuracy')
54 \title('Accuracy (Stemming)')
55 \xlabel('Epoch')
56 \ylabel('Accuracy')
57 \legend()
58
59 \subplot(1, 2, 2)
60 \plot(history_stem.history['loss'], label='Train Loss')
61 \plot(history_stem.history['val_loss'], label='Validation Loss')
62 \title('Loss (Stemming)')
63 \xlabel('Epoch')
64 \ylabel('Loss')
65 \legend()
66 \tight_layout()
67 \show()

```

Listing 3.8: 시나리오 2: 토큰화 + 어간 추출 적용

3.4.5 시나리오 3: 토큰화 + 표제어 추출 적용

텍스트를 토큰화한 후, *NLTK*의 *WordNetLemmatizer*를 사용하여 각 단어의 표제어를 추출합니다. (품사 태깅이 필요하지만, 여기서는 단순화를 위해 동사로 가정합니다.)

```

1 lemmatizer = WordNetLemmatizer()
2
3 def lemmatize_text(text):
4     tokens = word_tokenize(text)
5     # 실제 사용 시에는 nltk.를 pos_tag 사용하여 각 단어의 정확한 품사를 파악해야 합니
    다.
6     # 여기서는 단순화를 위해 모든 단어를 동사(v)로 가정합니다.
7     lemmatized_tokens = [lemmatizer.lemmatize(token, pos='v') for token in
    tokens]
8     return " ".join(lemmatized_tokens)
9

```

```

10 # 훈련 및 테스트 텍스트에 표제어 추출 적용
11 train_texts_lemmatized = [lemmatize_text(text) for text in train_texts]
12 test_texts_lemmatized = [lemmatize_text(text) for text in test_texts]
13
14 print("\n--- 시나리오 3: 토큰화 + 표제어 추출 적용 ---")
15 # 예시 텍스트 확인 강의에서( 언급된 driving -> drive)
16 example_text_drive = "I was driving my car."
17 example_lemmatized_drive = lemmatize_text(example_text_drive)
18 print(f"원본 텍스트: '{example_text_drive}'")
19 print(f"표제어 추출 후 텍스트: '{example_lemmatized_drive}'")
20 # 'driving'이 'drive'로 변환되는 것을 볼 수 있습니다.
21
22 # 1. 토큰화 및 단어 인덱스 생성 표제어( 추출된 텍스트 사용)
23 tokenizer_lem = Tokenizer(num_words=max_features, oov_token="<OOV>")
24 tokenizer_lem.fit_on_texts(train_texts_lemmatized)
25
26 train_sequences_lem = tokenizer_lem.texts_to_sequences(train_texts_lemmatized)
27 test_sequences_lem = tokenizer_lem.texts_to_sequences(test_texts_lemmatized)
28
29 train_padded_lem = pad_sequences(train_sequences_lem, maxlen=max_len, padding=
    'post', truncating='post')
30 test_padded_lem = pad_sequences(test_sequences_lem, maxlen=max_len, padding='
    post', truncating='post')
31
32 print(f"훈련 데이터 패딩 후 형태 표제어( 추출): {train_padded_lem.shape}")
33 print(f"단어 사전 크기 표제어( 추출): {len(tokenizer_lem.word_index)}")
34
35 # 2. 모델 구축 및 훈련
36 model_lem = build_lstm_model(max_features, embedding_dim, max_len)
37 print("\n--- 모델 요약 표제어( 추출) ---")
38 model_lem.summary()
39
40 history_lem = model_lem.fit(
41     train_padded_lem, train_labels,
42     epochs=10,
43     batch_size=32,
44     validation_split=0.2
45 )
46
47 # 3. 모델 평가
48 loss_lem, accuracy_lem = model_lem.evaluate(test_padded_lem, test_labels)
49 print(f"\n테스트 정확도 표제어( 추출): {accuracy_lem:.4f}")
50
51 # 결과 시각화
52 plt.figure(figsize=(12, 4))
53 plt.subplot(1, 2, 1)
54 plt.plot(history_lem.history['accuracy'], label='Train Accuracy')
55 plt.plot(history_lem.history['val_accuracy'], label='Validation Accuracy')
56 \title('Accuracy (Lemmatization)')
57 \xlabel('Epoch')

```



```

58 \ylabel('Accuracy')
59 \legend()
60
61 \subplot(1, 2, 2)
62 \plot(history_lem.history['loss'], label='Train Loss')
63 \plot(history_lem.history['val_loss'], label='Validation Loss')
64 \title('Loss (Lemmatization)')
65 \xlabel('Epoch')
66 \ylabel('Loss')
67 \legend()
68 \tight_layout()
69 \show()

```

Listing 3.9: 시나리오 3: 토큰화 + 표제어 추출 적용

3.5 체크리스트

본 문서를 학습하고 과제를 수행하며 점검할 수 있는 체크리스트입니다.

학습 점검 체크리스트

- *RNN* 기본 이해: 순환 신경망이 시퀀스 데이터를 어떻게 처리하는지 이해했는가?
- *RNN* 파라미터 계산: *SimpleRNN* 레이어의 파라미터 수를 직접 계산할 수 있는가?
- *LSTM* 작동 원리: *LSTM*의 셀 상태(C)와 은닉 상태(H), 그리고 게이트 메커니즘을 설명할 수 있는가? C 와 H 의 차원이 왜 같은지 이해했는가?
- 양방향 *RNN*: 양방향 *RNN*이 필요한 이유와 파라미터/출력 차원 변화를 이해했는가?
- *NLP* 개요: *NLP*의 주요 응용 분야와 도전 과제를 설명할 수 있는가?
- 텍스트 전처리: 토큰화, 어간 추출, 표제어 추출의 정의, 목적, 차이점을 명확히 구분할 수 있는가? 각 기법의 장단점은 무엇인가?
- 임베딩 레이어: 임베딩 레이어가 왜 필요하며, 원-핫 인코딩의 한계를 어떻게 극복하는지 설명할 수 있는가?
- 드롭아웃: 드롭아웃이 과적합을 어떻게 방지하는지 이해했는가?
- 손실/활성화 함수: 이진 분류 문제에서 시그모이드와 이진 교차 엔트로피를 사용하는 이유를 설명할 수 있는가?

과제 수행 체크리스트

- 데이터 로딩: *20 Newsgroups* 데이터셋을 올바르게 로드하고 훈련/테스트 데이터로 분리했는가?
- 텍스트 전처리 구현: 토큰화, 어간 추출, 표제어 추출 함수를 정확히 구현했는가?
- *OOV* 처리: *OOV* 토큰을 포함하여 단어 인덱싱을 올바르게 처리했는가?
- 시퀀스 패딩: *pad_sequences*를 사용하여 시퀀스 길이를 통일했는가?

- 모델 구축: *Embedding, LSTM, Dropout, Dense* 레이어를 사용하여 모델을 올바르게 정의했는가?
- 모델 훈련 및 평가: 모델을 훈련하고 테스트 데이터셋에서 정확도를 평가했는가?
- 결과 분석: 각 전처리 기법(토큰화만, 어간 추출, 표제어 추출)에 따른 모델 성능 변화를 분석하고 설명할 수 있는가?

3.6 FAQ (자주 묻는 질문)

Q1: RNN 파라미터 계산은 어떻게 하나요?

A1: *SimpleRNN*(은닉 뉴런 수, *input_shape*=(None, 입력 특성 수))의 경우, 파라미터 수는 (입력 특성 수 * 은닉 뉴런 수) + (은닉 뉴런 수 * 은닉 뉴런 수) + 은닉 뉴런 수로 계산됩니다. 이는 입력 가중치, 순환 가중치, 바이어스의 합입니다.

Q2: LSTM의 셀 상태(*C*)와 은닉 상태(*H*)는 왜 같은 차원인가요?

A2: *LSTM* 내부에서 *C*와 *H*는 요소별 덧셈(*summation*)과 요소별 곱셈(*element-wise multiplication*) 연산을 통해 상호작용합니다. 이러한 연산들은 벡터의 차원을 변경하지 않으므로, *C*와 *H*는 항상 동일한 차원을 유지해야 합니다.

Q3: 양방향 RNN은 왜 필요한가요?

A3: 일반 *RNN*은 과거 정보만으로 현재를 예측하지만, 텍스트와 같은 시퀀스 데이터에서는 미래 정보(문장의 뒷부분)가 현재의 의미를 이해하는 데 중요할 수 있습니다. 양방향 *RNN*은 순방향과 역방향으로 모두 학습하여 과거와 미래의 모든 문맥 정보를 활용하여 더 정확한 예측을 가능하게 합니다.

Q4: 어간 추출(*Stemming*)과 표제어 추출(*Lemmatization*)의 차이는 무엇인가요?

A4: 어간 추출은 단어의 접미사를 단순히 잘라내어 '어간'을 찾습니다. 빠르지만 결과가 실제 사전에 없는 단어일 수 있습니다 (예: '*driving*' -> '*driv*'). 표제어 추출은 단어의 품사 정보를 활용하여 사전적 의미를 가지는 '표제어'를 찾습니다. 느리지만 결과가 항상 실제 단어입니다 (예: '*driving*' -> '*drive*').

Q5: 임베딩 레이어는 왜 사용하나요?

A5: 임베딩 레이어는 고차원의 희소(*sparse*)한 원-핫 인코딩의 비효율성을 해결하고, 단어 간의 의미적 유사성을 저차원의 밀집(*dense*) 벡터 공간에 반영하기 위해 사용됩니다. 이는 모델의 학습 효율과 성능을 높이는 데 기여합니다.

Q6: OOV(Out-Of-Vocabulary) 단어는 어떻게 처리하나요?

A6: OOV 단어는 모델 학습 시 사용된 어휘 사전에 없는 단어를 의미합니다. 일반적으로 OOV 토큰을 위한 특별한 인덱스(예: 0)를 사전에 할당하고, 사전에 없는 단어가 나타나면 이 OOV 인덱스로 대체합니다. 이는 모델이 새로운 단어를 만나도 처리할 수 있도록 합니다.

Q7: 드롭아웃(Dropout)은 왜 사용하나요?

A7: 드롭아웃은 신경망의 과적합을 방지하기 위한 정규화 기법입니다. 훈련 과정에서 무작위로 일부 뉴런의 연결을 끊어 모델이 특정 특징에 과도하게 의존하는 것을 막고, 다양한 하위 네트워크를 학습하는 효과를 통해 모델의 일반화 성능을 향상시킵니다.

3.7 빠르게 훑어보기 (1페이지 요약)

CSCI89 Lecture 3 핵심 요약

1. 순환 신경망 (RNN)의 이해
 - 시퀀스 데이터 처리 특화 (텍스트, 음성, 시계열).
 - 이전 정보 기억 (은닉 상태).
 - 파라미터 계산: (입력 특성 수 + 은닉 뉴런 수 + 1) * 은닉 뉴런 수.
 - 입출력 유형: 다대다, 다대일, 일대다 (*Keras return_sequences* 옵션).
2. 장단기 기억망 (LSTM)
 - RNN의 장기 의존성 문제 해결.
 - 두 가지 기억: 셀 상태(C , 장기 기억)와 은닉 상태(H , 단기 기억).
 - 게이트 메커니즘: 망각, 입력, 출력 게이트로 정보 흐름 제어.
 - C 와 H 차원: 항상 동일. $LSTM(N)$ 은 내부적으로 $4 \times N$ 뉴런.
3. 양방향 RNN/LSTM
 - 순방향/역방향 시퀀스 동시 처리로 전체 문맥 활용.
 - 파라미터 수: 단방향의 2배.
 - 출력 차원: 단방향의 2배 (연결(concatenate) 때문).
4. 자연어 처리 (NLP) 개요
 - 컴퓨터가 인간 언어를 이해/생성하는 AI 분야.
 - 도전 과제: 언어의 모호성, 문맥, 배경 지식 필요.
 - 주요 응용: 텍스트 분류, *NER*, 감성 분석, 기계 번역, 챗봇 등.
5. 텍스트 전처리 기법
 - 토큰화: 텍스트를 의미 단위(단어, 문장 등)로 분할. 불용어 처리.
 - 어간 추출 (*Stemming*): 단어의 접미사 제거 (예: 'running' → 'runn'). 빠르지만 실제 단어가 아닐 수 있음. 차원 축소.

- 표제어 추출 (*Lemmatization*): 단어를 사전형(표제어)으로 변환 (예: '*driving*' → '*drive*'). 정확하지만 품사 정보 필요, 비용 높음.
- 6. 텍스트를 숫자로: 임베딩 (*Embedding*)
 - 고차원 희소(*Sparse*) 원-핫 인코딩의 한계 극복.
 - 단어 인덱스를 저차원 밀집(*Dense*) 벡터로 변환.
 - 단어 간 의미적 유사성 학습.
 - *OOV* (*Out-Of-Vocabulary*): 사전에 없는 단어 처리 (특별 인덱스 할당).
- 7. 모델 정규화: 드롭아웃 (*Dropout*)
 - 훈련 중 무작위로 뉴런 연결 끊어 과적합 방지.
 - 모델의 일반화 성능 향상.
- 8. 텍스트 분류 모델 (*LSTM* 기반)
 - 아키텍처: *Embedding* → *Dropout* → *LSTM* → *Dropout* → *Dense*(*Sigmoid*).
 - 손실 함수: 이진 분류에 *Binary Cross-Entropy* 사용 (*MSE* 부적합).
 - 평가 지표: *Accuracy*.
 - 전처리 영향: 토큰화, 어간 추출, 표제어 추출 등 전처리 방식에 따라 모델 성능 변화.

Chapter 4

Lecture 4

강의명: *CSCI89*

주차: *Lecture 4*

교수명: *UIUC Student*

목적: 텍스트 표현 및 합성곱 신경망

4.1 개요

강

4.2 용어 정리

다음 표는 본 문서에서 사용되는 주요 용어들을 쉽게 설명하고 원어 및 비고를 함께 제시합니다.

용어	쉬운 설명	원어	비고
토큰화	텍스트를 의미 있는 작은 단위(단어, 문자 등)로 나누는 과정	<i>Tokenization</i>	<i>NLP</i> 의 첫 단계
스테밍	단어의 어미나 접사를 제거하여 어간을 추출하는 과정	<i>Stemming</i>	실제 단어가 아닐 수 있음
표제어 추출	단어를 사전형(기본형)으로 변환하는 과정	<i>Lemmatization</i>	품사 정보 활용, 스테밍보다 정확
단어 가방	문서에서 단어의 출현 빈도만으로 텍스트를 표현하는 방식	<i>Bag-of-Words (BoW)</i>	단어 순서 정보 손실
<i>N</i> -그램	<i>N</i> 개의 연속된 단어를 하나의 토큰으로 간주하는 방식	<i>N-grams</i>	단어 간의 지역적 문맥 포착
단어 임베딩	단어를 저차원의 밀집 벡터로 표현하여 의미를 반영하는 방식	<i>Word Embedding</i>	희소성 문제 해결, 의미 유사성 반영
희소 표현	대부분의 값이 0인 벡터나 행렬로 데이터를 표현하는 방식	<i>Sparse Representation</i>	<i>BoW</i> 의 주요 단점
어휘 집합	텍스트 데이터에서 추출된 고유한 단어들의 집합	<i>Vocabulary</i>	모델 학습의 기준
<i>Oov</i>	어휘 집합에 없는 단어	<i>Out-of-Vocabulary</i>	새로운 단어 처리 방식
합성곱 신경망	이미지나 시퀀스 데이터의 지역적 특징을 추출하는 신경망	<i>Convolutional Neural Network (CNN)</i>	가중치 공유, 지역적 연결
필터/커널	합성곱 연산에 사용되는 작은 행렬	<i>Filter / Kernel</i>	특징 추출기 역할
패딩	합성곱 연산 시 이미지 경계에 0을 추가하는 기법	<i>Padding</i>	출력 크기 조절
스트라이드	필터가 이동하는 간격	<i>Stride</i>	특징 맵의 크기 조절
풀링	특징 맵의 크기를 줄이고 중요한 특징을 강조하는 기법	<i>Pooling</i>	<i>Max Pooling, Average Pooling</i>
<i>Flatten</i>	다차원 데이터를 1차원 벡터로 변환하는 계층	<i>Flatten</i>	완전 연결 계층으로의 입력 준비

4.3 과제 2 피드백 및 논의

이번 강의는 지난 과제 2에 대한 피드백과 함께 시작되었습니다. 학생들은 과제를 수행하면서 겪었던 기술적인 문제와 개념적인 어려움에 대해 질문하고, 강사는 이에 대한 설명을 제공했습니다.

4.3.1 기술적 문제 및 해결

- *TensorFlow* 설치 문제: 한 학생은 로컬 *TensorFlow* 설치 문제가 아님을 확인했음에도 불구하고 기술적인 문제로 과제를 완료하지 못했다고 언급했습니다. 다른 노트북에서는 신경망이 정상적으로 작동했기 때문에, 문제는 로컬 환경이 아닌 다른 곳에 있었을 가능성이 높습니다.
- 커널 충돌: 과제의 첫 번째 부분이 계속해서 커널 충돌을 일으켰다고 합니다. 이는 코드나 환경 설정의 특정 부분에서 발생할 수 있는 문제입니다.
- 정확도 75% 미달: 과제의 두 번째, 세 번째 부분에서는 순환 신경망(*Recurrent Network*)을 사용하여 정확도를 75% 이상으로 올리지 못했다고 합니다. 이는 모델 설계나 하이퍼파라미터 튜닝의 어려움을 시사합니다.

4.3.2 희소 표현(Sparse Representation)의 어려움

예

강사는 원-핫 인코딩을 사용하면 벡터가 매우 길고 대부분 0으로 채워지는 '희소 표현'이 된다고 설명했습니다. 이는 다음과 같은 문제점을 야기합니다.

- 높은 차원: 어휘 집합(*Vocabulary*)의 크기가 커질수록 벡터의 차원이 기하급수적으로 증가합니다.
- 저장 공간 낭비: 대부분의 값이 0이므로, 실제 유용한 정보는 적는데도 많은 저장 공간을 차지합니다.
- 계산 비용 증가: 신경망의 입력으로 사용될 때, 많은 수의 0을 처리해야 하므로 계산이 비효율적입니다.
- 의미 손실: 단어 간의 의미적 관계를 전혀 반영하지 못합니다.

손

4.3.3 손실(Loss) 폭발 문제 (Sigmoid + Cross Entropy)

한 학생이 10번째 에포크(*epoch*)에서 손실(*loss*)이 비정상적으로 폭발하는 현상(*ludicrous loss*)에 대해 질문했습니다. 이는 딥러닝 모델 학습에서 흔히 발생하는 문제 중 하나입니다.

- 문제 발생 원인:
 - 모델의 마지막 계층이 시그모이드(*sigmoid*) 함수이고, 손실 함수로 교차 엔트로피(*cross entropy*)를 사용했을 때 발생할 수 있습니다.

- 시그모이드 함수는 출력이 0 또는 1에 매우 가까워질 때, 모델이 특정 답에 대해 극도로 확신하지만 실제로는 틀린 경우(오답 또는 오음성)에 문제가 됩니다.
- 이때 교차 엔트로피 손실 함수는 로그(log) 함수를 포함하는데, 로그 함수의 입력이 0에 매우 가까워지면 손실 값이 무한대로 발산하게 됩니다($\log(0) \rightarrow -\infty$). 예를 들어, 모델이 정답을 1로 확신했지만 실제 정답은 0인 경우, 손실 계산 시 $\log(0)$ 과 유사한 상황이 발생하여 손실이 폭발합니다.
- 특히, 단 하나의 데이터 포인트에서 모델이 잘못된 답에 대해 과도하게 확신할 때 이러한 현상이 발생할 수 있습니다.
- 흥미로운 관찰: 손실이 폭발했음에도 불구하고, 해당 모델이 75% 이상의 정확도를 달성한 첫 번째 모델이었다는 점입니다. 이는 손실 함수가 특정 이상치(anomaly)에 매우 민감하게 반응할 수 있음을 보여줍니다.
- 해결책: 학생은 드롭아웃(Dropout) 계층을 추가하여 이 문제를 해결했다고 보고했습니다. 드롭아웃은 과적합(overfitting)을 방지하고 모델의 일반화 성능을 향상시키는 데 도움이 되는 기법입니다.

4.4 텍스트 전처리 및 표현 기법

자연어 처리(NLP)에서 텍스트 데이터를 모델이 이해할 수 있는 형태로 변환하는 것은 매우 중요합니다. 이를 위해 다양한 전처리 및 표현 기법이 사용됩니다.

4.4.1 토큰화(Tokenization)

정의 및 목적

토큰화의 주된 목적은 텍스트를 분석 가능한 단위로 나누는 것입니다. 어떤 단위로 나눌지는 특정 NLP 작업의 특성과 데이터셋에 따라 달라집니다.

- 단어(Words): 텍스트 분류(text classification)와 같은 작업에서는 단어 단위의 토큰화가 흔히 사용됩니다. 특정 데이터셋에 모델을 적용할 때 효과적입니다.
- 문자(Characters): 모델을 다른 데이터셋으로 전이 학습(transfer learning) 시키거나, 어휘 집합(vocabulary)에 없는 단어(OOV) 문제를 처리할 때 문자 단위 토큰화가 유용할 수 있습니다. 모든 문자가 토큰이 되므로, 새로운 단어에 대해서도 유연하게 대처할 수 있습니다.
- 서브워드(Subwords): 단어와 문자 토큰화의 장점을 결합한 방식으로, 자주 등장하는 단어는 그대로 사용하고 희귀한 단어는 더 작은 의미 단위(서브워드)로 분리합니다.

NLTK를 이용한 토큰화 예시

*NLTK(Natural Language Toolkit)*는 파이썬에서 널리 사용되는 *NLP* 라이브러리입니다. *NLTK*의 *word_tokenize* 함수는 문장을 단어 단위로 토큰화합니다. 이때 단순히 공백이나 구두점을 기준으로 나누는 것이 아니라, 문맥을 고려하여 똑똑하게 토큰을 분리합니다.

4.4.2 스테밍(Stemming)과 표제어 추출(Lemmatization)

정의 및 목적

두 기법 모두 단어의 형태를 정규화하여 같은 의미를 가진 단어들을 하나로 묶는 데 사용됩니다. 예를 들어, "*running*", "*runs*", "*ran*"과 같은 단어들을 "*run*"이라는 하나의 형태로 만듭니다.

- 스테밍(*Stemming*):
 - 정의: 단순히 규칙 기반으로 접두사/접미사를 잘라내는 방식입니다.
 - 예시: "*running*" → "*run*", "*octopus*" → "*octop*"
 - 장점: 구현이 간단하고 계산 비용이 매우 저렴합니다.
 - 단점: 추출된 어간이 실제 사전에 없는 단어일 수 있습니다. (예: "*octop*"은 실제 단어가 아님)
- 표제어 추출(*Lemmatization*):
 - 정의: 단어의 품사(*part of speech*)를 고려하여 사전에 있는 유효한 기본형으로 변환합니다.
 - 예시: "*running*" → "*run*", "*better*" → "*good*"
 - 장점: 항상 유효한 단어를 반환하므로 의미 보존에 더 유리합니다.
 - 단점: 품사 태깅(*POS tagging*)과 같은 추가적인 분석이 필요하여 스테밍보다 계산 비용이 높습니다.

스테밍의 단점

스

4.4.3 불용어(Stop Words) 처리

일

정의 및 목적

불용어는 텍스트 데이터의 차원을 줄이고, 모델이 더 중요한 단어에 집중하도록 돕기 위해 제거될 수 있습니다.

- 차원 축소: 불용어를 제거하면 어휘 집합의 크기가 줄어들어 모델의 계산 효율성이 향상됩니다.
- 노이즈 감소: 의미 없는 단어들을 제거함으로써 모델이 텍스트의 핵심 내용을 더 잘 학습할 수 있습니다.
- 적용 여부: 불용어 제거는 항상 필수는 아닙니다. 예를 들어, *"not funny"*와 같은 부정어 표현은 *"not"*이 불용어일지라도 문맥상 매우 중요하므로 제거하지 않는 것이 좋습니다. *N-*그램을 사용하면 이러한 문제를 어느 정도 해결할 수 있습니다.

4.5 텍스트를 숫자로 표현하는 방법

기계 학습 모델은 텍스트를 직접 처리할 수 없으므로, 텍스트를 숫자로 변환하는 과정이 필수적입니다. 여기서는 대표적인 텍스트 표현 기법들을 살펴봅니다.

4.5.1 단어 가방(Bag-of-Words, BoW)

정의 및 원리

BoW 모델은 각 문서를 고정된 길이의 벡터로 변환합니다. 이 벡터의 각 요소는 어휘 집합 (*Vocabulary*) 내 특정 단어의 출현 빈도를 나타냅니다.

- 어휘 집합 생성: 먼저 전체 훈련 데이터셋 (*corpus*)에서 모든 고유한 단어들을 수집하여 어휘 집합을 만듭니다. 이 어휘 집합은 모델의 참조 사전 역할을 합니다.
- 벡터 표현: 각 문서는 어휘 집합의 길이와 같은 차원의 벡터로 표현됩니다. 벡터의 각 위치에는 해당 위치에 해당하는 어휘 집합의 단어가 문서에 나타난 횟수(빈도)가 기록됩니다.
- 시간 순서 손실: *BoW*의 가장 큰 특징이자 한계는 단어의 순서 정보가 완전히 무시된다는 점입니다. "영화가 재미있다"와 "재미있는 영화가 아니다"는 *BoW* 관점에서는 유사하게 표현될 수 있습니다.

장점 및 응용 분야

- 간단하고 강력함: 구현이 비교적 간단하며, 특정 문제(예: 텍스트 분류)에서는 강력한 성능을 보여줍니다.
- 텍스트 분류 (*Text Classification*): 문서의 주제를 분류하거나 스팸 메일을 감지하는 등의 작업에 유용합니다.

- 감성 분석(*Sentiment Analysis*): 영화 리뷰나 상품 평가와 같이 텍스트의 긍정/부정 감성을 판단하는 데 사용될 수 있습니다. 단어의 순서보다는 특정 감성 단어의 출현 여부나 빈도가 중요할 때 효과적입니다.

단점 및 한계

- 시간 순서 손실: 단어의 순서나 문법적 구조를 무시하므로, 문맥(*context*) 정보를 잃게 됩니다. 이는 번역이나 텍스트 생성과 같이 순서가 중요한 작업에는 부적합합니다.
- 희소 표현(*Sparse Representation*): 어휘 집합의 크기가 클수록 대부분의 문서 벡터는 많은 0을 포함하는 희소 벡터가 됩니다. 이는 저장 공간 낭비와 계산 비효율성을 초래합니다.
- 의미 손실: 단어들을 독립적인 특징으로 간주하므로, 단어 간의 의미적 관계(예: 동의어, 유의어)를 포착하지 못합니다.
- 높은 차원: 어휘 집합이 커지면 벡터의 차원이 매우 높아져 모델의 파라미터 수가 증가하고 학습이 어려워질 수 있습니다.

BoW 생성 단계

BoW를 생성하는 일반적인 단계는 다음과 같습니다.

1. 토큰화(*Tokenization*): 문서를 단어 단위의 토큰으로 분리합니다.
2. 어휘 집합 생성(*Vocabulary Creation*): 전체 코퍼스(*corpus*)에서 모든 고유한 토큰을 수집하여 어휘 집합을 만듭니다. 이때 중복된 단어는 한 번만 포함됩니다.
3. 빈도 계산(*Frequency Counting*): 각 문서에 대해 어휘 집합의 각 단어가 몇 번 나타나는지 계산하여 벡터를 생성합니다.

f

BoW 구현 예시

scikit-learn의 `CountVectorizer`는 BoW 모델을 쉽게 생성할 수 있도록 해 주는 도구입니다.

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
corpus = [ "Henry Ford introduced the Model T.", "Ford's Model T was revolutionary." ]
```

```
CountVectorizer 인스턴스 생성 vectorizer = CountVectorizer()
```

```
코퍼스에 fit_transform BoW X = vectorizer.fit_transform(corpus)
```

```
어휘 집합 확인 vocabulary = vectorizer.get_feature_names_out() print(" : ", vocabulary)
```

```
BoW 벡터 확인 (희소 행렬을 일반 배열로 변환) print("BoW 표현:", X.toarray())
```

```
새 문서에 대한 BoW 생성 new_document = ["The impact of Model T was huge."] X_new = vectorizer.transform(new_document).toarray()
```

예시

출력 예시 및 설명: `CountVectorizer`는 자동으로 토큰화하고 어휘 집합을 생성합니다.

- 어휘 집합: `['ford', 'henry', 'impact', 'introduced', 'model', 'of', 'revolutionary', 't', 'the', 'was']` (알파벳 순으로 정렬됨)
- 첫 번째 문서 *BoW*: `[1, 1, 0, 1, 1, 0, 0, 1, 1, 0]` (각 단어의 빈도)
- 두 번째 문서 *BoW*: `[1, 0, 0, 0, 1, 0, 1, 1, 0, 1]`
- 새 문서 *BoW*: `[0, 0, 1, 0, 1, 1, 0, 1, 1, 1]`
 - `'impact', 'of', 'huge'`와 같은 단어는 훈련 코퍼스에 없었으므로, 해당 위치의 값은 `0` 이 됩니다. (OOV 처리)
 - `'Model', 'T', 'the', 'was'` 등 기존 어휘에 있는 단어는 빈도수가 반영됩니다.

NLTK를 이용한 BoW NLTK는 `CountVectorizer`보다 더 수동적인 접근 방식을 제공하여, 토큰화, 어휘 집합 생성, 빈도 계산 과정을 직접 제어할 수 있습니다. 이는 더 많은 유연성을 제공합니다.

```

1 from nltk.tokenize import word_tokenize
2 import collections
3
4 corpus = [
5     "Henry Ford introduced the Model T.",
6     "Ford's Model T was revolutionary."
7 ]
8
9 all_tokens = []
10 tokenized_documents = []
11
12 # 모든 문서 토큰화 및 토큰 리스트 생성
13 for doc in corpus:
14     tokens = word_tokenize(doc.lower()) # 소문자 변환 후 토큰화
15     tokenized_documents.append(tokens)
16     all_tokens.extend(tokens)
17
18 # 어휘 집합 생성 중복( 제거 및 OOV 토큰 추가)
19 vocabulary = sorted(list(set(all_tokens)))
20 vocabulary.append('OOV') %* OOV 토큰 추가 %*
21
22 def create_bag_of_words_nltk(tokens, vocab):
23     bow_vector = [0] * len(vocab)
24     oov_count = 0
25     for token in tokens:
26         try:
27             idx = vocab.index(token)
28             bow_vector[idx] += 1
29         except ValueError:
30             oov_count += 1 %* 어휘 집합에 없는 단어는 로OOV 카운트 %*
31     bow_vector[vocab.index('OOV')] = oov_count

```

```

32     return bow_vector
33
34 print("어휘 집합:", vocabulary)
35
36 # 각 문서에 대한 BoW 생성
37 bow_vectors = []
38 for tokens in tokenized_documents:
39     bow_vectors.append(create_bag_of_words_nltk(tokens, vocabulary))
40 print("훈련 문서 BoW 표현:\n", bow_vectors)
41
42 # 새 문서에 대한 BoW 생성
43 new_document_text = "The impact of Model T was huge."
44 new_tokens = word_tokenize(new_document_text.lower())
45 new_bow_vector = create_bag_of_words_nltk(new_tokens, vocabulary)
46 print("새 문서 BoW 표현:\n", new_bow_vector)

```

Listing 4.1: NLTK를 이용한 BoW 생성

예시

설명:

- *NLTK*는 *word_tokenize*를 사용하여 텍스트를 토큰화합니다.
- 어휘 집합은 수동으로 생성되며, 'OOV' 토큰을 추가하여 어휘 집합에 없는 단어를 처리할 수 있습니다.
- *create_bag_of_words_nltk* 함수는 주어진 토큰 리스트와 어휘 집합을 바탕으로 *BoW* 벡터를 생성합니다.
- 새 문서에 대해 *transform* 메서드 없이 직접 함수를 호출하여 *BoW*를 생성합니다. 이때 어휘 집합에 없는 단어들은 'OOV' 카운트에 합산됩니다.

SpaCy를 이용한 BoW *SpaCy*는 *NLTK*와 유사하게 유연성을 제공하면서도, 더 빠르고 효율적인 처리를 위한 기능을 내장하고 있습니다.

```

1  import spacy
2  from collections import Counter
3
4  # SpaCy 모델 로드 예(: 영어 소형 모델)
5  # python -m spacy download en_core_web_sm
6  nlp = spacy.load("en_core_web_sm")
7
8  corpus = [
9      "Henry Ford introduced the Model T.",
10     "Ford's Model T was revolutionary."
11 ]
12
13 all_tokens = []
14 tokenized_documents = []
15
16 for doc_text in corpus:

```

```

17     doc = nlp(doc_text.lower())  %* SpaCy Doc 객체 생성 %*
18     tokens = [token.text for token in doc if not token.is_punct] %* 구두점 제
    외 %*
19     tokenized_documents.append(tokens)
20     all_tokens.extend(tokens)
21
22     vocabulary = sorted(list(set(all_tokens)))
23     vocabulary.append('OOV')
24
25     def create_bag_of_words_spacy(tokens, vocab):
26         bow_vector = [0] * len(vocab)
27         oov_count = 0
28         for token in tokens:
29             try:
30                 idx = vocab.index(token)
31                 bow_vector[idx] += 1
32             except ValueError:
33                 oov_count += 1
34         bow_vector[vocab.index('OOV')] = oov_count
35         return bow_vector
36
37     print("어휘 집합:", vocabulary)
38
39     bow_vectors = []
40     for tokens in tokenized_documents:
41         bow_vectors.append(create_bag_of_words_spacy(tokens, vocabulary))
42     print("훈련 문서 BoW 표현:\n", bow_vectors)
43
44     new_document_text = "The impact of Model T was huge."
45     new_doc = nlp(new_document_text.lower())
46     new_tokens = [token.text for token in new_doc if not token.is_punct]
47     new_bow_vector = create_bag_of_words_spacy(new_tokens, vocabulary)
48     print("새 문서 BoW 표현:\n", new_bow_vector)

```

Listing 4.2: SpaCy를 이용한 BoW 생성

예시**설명:**

- *SpaCy*는 *nlp* 객체를 통해 텍스트를 처리하고, *token.text*를 사용하여 토큰을 추출합니다.
- *NLTK*와 유사하게 수동으로 어휘 집합을 만들고 *BoW* 벡터를 생성합니다.
- *SpaCy*는 품사 태깅(*POS tagging*)이나 개체명 인식(*NER*)과 같은 고급 *NLP* 기능을 기본적으로 제공하므로, 이를 활용하여 더 정교한 토큰화나 전처리를 수행할 수 있습니다.

4.5.2 N-그램(N-grams)

핵심 개념

N-그램(*N-grams*)은 텍스트에서 *N*개의 연속된 단어(또는 문자)를 하나의 토큰으로 간주하는 기법입니다. 이는 단어 가방(*BoW*) 모델의 단점인 단어 순서 정보 손실을 부분적으로 보완하여 지역적인 문맥(*local context*)을 포착하는 데 도움을 줍니다.

정의 및 목적

N-그램은 단어의 순서를 고려하여 특정 단어 시퀀스가 함께 나타나는 빈도를 측정합니다.

- 유니그램(*Unigram, 1-gram*): 단어 하나하나를 토큰으로 간주합니다. 이는 일반적인 *BoW* 모델과 동일합니다.
- 바이그램(*Bigram, 2-gram*): 두 개의 연속된 단어를 하나의 토큰으로 간주합니다.
- 트라이그램(*Trigram, 3-gram*): 세 개의 연속된 단어를 하나의 토큰으로 간주합니다.

예시

예시 문장: "*This movie was not funny.*"

유니그램: ['*This*', '*movie*', '*was*', '*not*', '*funny*']

바이그램: ['*This movie*', '*movie was*', '*was not*', '*not funny*']

트라이그램: ['*This movie was*', '*movie was not*', '*was not funny*']

문맥 포착의 중요성: 감성 분석에서 "*funny*"는 긍정적이지만, "*not funny*"는 부정적인 의미를 가집니다. 바이그램은 이러한 부정적인 문맥을 포착하는 데 유용합니다.

장점 및 단점

- 장점:
 - 지역적 문맥 포착: 인접한 단어들 간의 관계를 반영하여 *BoW*보다 풍부한 정보를 제공합니다.
 - 감성 분석 개선: "*not good*", "*very bad*"와 같은 표현의 의미를 더 정확하게 파악할 수 있습니다.
- 단점:
 - 어휘 집합 폭발: *N*이 커질수록 가능한 *N*-그램의 수가 기하급수적으로 증가하여 어휘 집합의 크기가 매우 커집니다. 이는 희소성 문제를 더욱 심화시키고, 저장 공간 및 계산 비용을 증가시킵니다.
 - 장거리 의존성 한계: *N*-그램은 인접한 *N*개의 단어만을 고려하므로, 멀리 떨어진 단어들 간의 관계(장거리 의존성)는 포착할 수 없습니다.
 - 최적의 *N* 선택: *N*의 크기를 결정하는 것이 중요합니다. 일반적으로 2-그램이나 3-그램이 가장 유용하며, 그 이상은 효율성이 떨어집니다.

N-그램 구현 예시

scikit-learn의 CountVectorizer CountVectorizer는 ngram_range 파라미터를 통해 N-그램을 쉽게 생성할 수 있습니다.

```

1 from sklearn.feature_extraction.text import CountVectorizer
2
3 corpus = [
4     "Henry Ford introduced the Model T.",
5     "Ford's Model T was revolutionary."
6 ]
7
8 # ngram_range=(2, 2)로 바이그램만 사용
9 vectorizer = CountVectorizer(ngram_range=(2, 2))
10
11 X = vectorizer.fit_transform(corpus)
12
13 vocabulary = vectorizer.get_feature_names_out()
14 print("바이그램 어휘 집합:", vocabulary)
15 print("바이그램 BoW 표현:\n", X.toarray())
16
17 # 새 문서에 대한 바이그램 BoW 생성
18 new_document = ["The impact of Model T was huge."]
19 X_new = vectorizer.transform(new_document)
20 print("새 문서 바이그램 BoW 표현:\n", X_new.toarray())

```

Listing 4.3: scikit-learn CountVectorizer를 이용한 N-그램 생성

예시

출력 예시 및 설명:

- 바이그램 어휘 집합: ['ford introduced', 'introduced model', 'model revolutionary', 'model was', 't was', 'the model']
- 새 문서 BoW: 대부분의 값이 0이 됩니다. 훈련 코퍼스가 작기 때문에 새 문서에서 훈련된 바이그램이 거의 나타나지 않을 수 있습니다.

NLTK를 이용한 N-그램 NLTK는 ngrams 함수를 제공하여 N-그램을 생성합니다.

```

1 from nltk.tokenize import word_tokenize
2 from nltk.util import ngrams
3 import collections
4
5 corpus = [
6     "Henry Ford introduced the Model T.",
7     "Ford's Model T was revolutionary."
8 ]
9
10 all_ngrams = []
11 tokenized_documents = []

```

```

12
13 for doc in corpus:
14     tokens = word_tokenize(doc.lower())
15     doc_ngrams = list(ngrams(tokens, 2)) %* 바이그램 생성 %*
16     tokenized_documents.append(doc_ngrams)
17     all_ngrams.extend(doc_ngrams)
18
19 vocabulary = sorted(list(set(all_ngrams)))
20 vocabulary.append(('OOV',)) %* OOV 토큰은 튜플 형태로 추가 %*
21
22 def create_ngram_bow_nltk(ngrams_list, vocab):
23     bow_vector = [0] * len(vocab)
24     oov_count = 0
25     for ngram_item in ngrams_list:
26         try:
27             idx = vocab.index(ngram_item)
28             bow_vector[idx] += 1
29         except ValueError:
30             oov_count += 1
31     bow_vector[vocab.index(('OOV',))] = oov_count
32     return bow_vector
33
34 print("바이그램 어휘 집합:", vocabulary)
35
36 ngram_bow_vectors = []
37 for doc_ngrams in tokenized_documents:
38     ngram_bow_vectors.append(create_ngram_bow_nltk(doc_ngrams, vocabulary))
39 print("훈련 문서 바이그램 BoW 표현:\n", ngram_bow_vectors)
40
41 new_document_text = "The impact of Model T was huge."
42 new_tokens = word_tokenize(new_document_text.lower())
43 new_ngrams = list(ngrams(new_tokens, 2))
44 new_ngram_bow_vector = create_ngram_bow_nltk(new_ngrams, vocabulary)
45 print("새 문서 바이그램 BoW 표현:\n", new_ngram_bow_vector)

```

Listing 4.4: NLTK를 이용한 N-그램 생성

SpaCy를 이용한 N-그램 *SpaCy*는 토큰화된 문서 객체에서 *N*-그램을 생성할 수 있습니다.

```

1 import spacy
2 from collections import Counter
3
4 nlp = spacy.load("en_core_web_sm")
5
6 corpus = [
7     "Henry Ford introduced the Model T.",
8     "Ford's Model T was revolutionary."
9 ]
10
11 all_ngrams = []

```



```

12 tokenized_documents = []
13
14 for doc_text in corpus:
15     doc = nlp(doc_text.lower())
16     %* 의SpaCy 는ngrams 직접 제공되지 않으므로, 수동으로 생성 *
17     tokens = [token.text for token in doc if not token.is_punct]
18     doc_ngrams = [(tokens[i], tokens[i+1]) for i in range(len(tokens)-1)] %*
    바이그
    램 %*
19     tokenized_documents.append(doc_ngrams)
20     all_ngrams.extend(doc_ngrams)
21
22 vocabulary = sorted(list(set(all_ngrams)))
23 vocabulary.append(('OOV',))
24
25 def create_ngram_bow_spacy(ngrams_list, vocab):
26     bow_vector = [0] * len(vocab)
27     oov_count = 0
28     for ngram_item in ngrams_list:
29         try:
30             idx = vocab.index(ngram_item)
31             bow_vector[idx] += 1
32         except ValueError:
33             oov_count += 1
34     bow_vector[vocab.index(('OOV',))] = oov_count
35     return bow_vector
36
37 print("바이그램 어휘 집합:", vocabulary)
38
39 ngram_bow_vectors = []
40 for doc_ngrams in tokenized_documents:
41     ngram_bow_vectors.append(create_ngram_bow_spacy(doc_ngrams, vocabulary))
42 print("훈련 문서 바이그램 BoW 표현:\n", ngram_bow_vectors)
43
44 new_document_text = "The impact of Model T was huge."
45 new_doc = nlp(new_document_text.lower())
46 new_tokens = [token.text for token in new_doc if not token.is_punct]
47 new_ngrams = [(new_tokens[i], new_tokens[i+1]) for i in range(len(new_tokens)
    -1)]
48 new_ngram_bow_vector = create_ngram_bow_spacy(new_ngrams, vocabulary)
49 print("새 문서 바이그램 BoW 표현:\n", new_ngram_bow_vector)

```

Listing 4.5: SpaCy를 이용한 N-그램 생성

4.5.3 단어 임베딩(Word Embedding)

핵심 개념

단어 임베딩(*Word Embedding*)은 단어를 저차원의 밀집(*dense*) 벡터로 표현하는 기법입니다. 이는 단어 간의 의미적 유사성을 벡터 공간에 반영하여, 희소 표현의 문제점을 해결하고 모델이 단어의 문맥적 의미를 학습할 수 있도록 돕습니다.

정의 및 필요성

*BoW*나 원-핫 인코딩은 단어를 독립적인 존재로 취급하며, 벡터가 매우 희소하고 고차원이라는 단점이 있습니다. 임베딩은 이러한 문제를 해결하기 위해 등장했습니다.

- 희소성 문제 해결: 원-핫 인코딩은 어휘 집합의 크기만큼 차원을 가지며 대부분 0으로 채워지지만, 임베딩은 훨씬 낮은 차원(예: 50차원, 100차원)의 밀집 벡터를 사용합니다.
- 의미 유사성 반영: 임베딩 벡터 공간에서 의미적으로 유사한 단어들은 서로 가까운 위치에 배치됩니다. 예를 들어, "왕(*king*)"과 "여왕(*queen*)"의 벡터는 "남자(*man*)"와 "여자(*woman*)"의 벡터 관계와 유사한 패턴을 보일 수 있습니다.
- 문맥 학습: 임베딩은 단어가 사용되는 문맥을 통해 학습되므로, 단어의 의미를 더 잘 포착할 수 있습니다.

원-핫 인코딩과의 비교

- 원-핫 인코딩:
 - 표현: 어휘 집합 크기만큼의 차원을 가지는 희소 벡터. 특정 단어 위치에만 1, 나머지는 0.
 - 정보: 단어의 존재 여부만 표현. 단어 간의 관계나 의미는 반영하지 못함.
 - 문제점: 고차원, 희소성, 계산 비효율성.
- 단어 임베딩:
 - 표현: 훨씬 낮은 차원(예: 32, 64, 128차원)의 밀집 벡터. 모든 요소가 0이 아닌 실수 값.
 - 정보: 단어의 의미적, 문맥적 정보를 벡터 공간에 압축하여 표현.
 - 장점: 차원 축소, 희소성 해결, 의미 유사성 반영, 효율적인 계산.

임베딩의 원리

임베딩은 고차원의 희소 벡터(원-핫 인코딩)를 저차원의 밀집 벡터로 매핑하는 함수라고 생각할 수 있습니다.

예시

고양이(*Cat*) 단어 예시:

- 원-핫 인코딩: 어휘 집합에서 '*Cat*'이 두 번째 단어라면, $[0, 1, 0, \dots, 0]$ 과 같은 200차원(예시)의 희소 벡터로 표현됩니다. 이는 200차원 공간의 한 축에 있는 점과 같습니다.
- 임베딩: 이 200차원 벡터를 3차원(예시)의 밀집 벡터로 매핑합니다. 예를 들어, $[0.7, 1.2, 0.2]$ 와 같은 벡터가 됩니다. 이 벡터는 3차원 공간의 한 점을 나타내며, 더 효율적이고 의미 있는 표현이 됩니다.

실제 구현 (인덱스 직접 사용)

신경망에서 임베딩 계층(*Embedding Layer*)을 사용할 때, 우리는 단어를 직접 원-핫 인코딩 벡터로 변환하여 입력하지 않습니다. 대신, 각 단어의 고유한 정수 인덱스(*index*)를 직접 입력으로 사용합니다.

- 인덱스 시퀀스: 문장은 $[1, 2, 5, 7, 4, \dots]$ 와 같은 단어 인덱스들의 시퀀스로 표현됩니다.
- 임베딩 계층: 임베딩 계층은 이 인덱스를 입력으로 받아, 내부적으로 학습된 임베딩 행렬에서 해당 인덱스에 해당하는 행(*row*)을 찾아 밀집 벡터로 출력합니다.
- 계산 효율성: 이는 원-핫 인코딩 벡터를 명시적으로 생성하고 이를 선형 변환하는 것과 수학적으로 동등하지만, 훨씬 더 효율적인 계산 방식입니다. 임베딩 계층은 필요한 행만 선택하므로, 불필요한 0과의 곱셈 연산을 피할 수 있습니다.

예시

임베딩 계층의 파라미터 수: 어휘 집합의 크기가 200이고, 각 단어를 3차원 벡터로 임베딩하고 싶다고 가정해 봅시다.

- 임베딩 계층은 내부적으로 200×3 크기의 행렬을 가집니다. 이 행렬의 각 행은 어휘 집합의 한 단어에 대한 3차원 임베딩 벡터를 나타냅니다.
- 따라서 총 파라미터 수는 $200 \times 3 = 600$ 개가 됩니다.
- 단어 인덱스(예: 1)가 입력되면, 임베딩 계층은 이 행렬의 첫 번째 행을 찾아 출력합니다. 이는 원-핫 벡터 $[1, 0, \dots, 0]$ 에 200×3 행렬을 곱하는 것과 동일한 효과를 냅니다.

주의

임베딩은 모델 학습 과정에서 역전파(*backpropagation*)를 통해 최적화됩니다. 즉, 임베딩 벡터의 값들은 특정 작업(예: 분류)의 손실 함수를 최소화하는 방향으로 학습됩니다. 따라서 임베딩 자체를 독립적으로 해결하는 것이 아니라, 전체 신경망의 일부로서 학습됩니다.

4.6 합성곱 신경망(Convolutional Neural Networks, CNNs)

핵심 개념

합성곱 신경망(*Convolutional Neural Networks, CNNs*)은 주로 이미지 처리 분야에서 뛰어난 성능을 보이는 신경망 아키텍처입니다. 하지만 텍스트 데이터를 배열(*sequence of vectors*)로 간주하여 자연어 처리(*NLP*) 문제에도 성공적으로 적용될 수 있습니다.

4.6.1 CNN 개요 및 NLP 적용

이미지 처리에서 NLP로의 확장

*CNN*은 이미지의 지역적 특징(*local features*)을 효과적으로 추출하도록 설계되었습니다. 이러한 특성은 텍스트 데이터에도 적용될 수 있습니다.

- 이미지: 픽셀들의 2차원 배열로, 인접한 픽셀들 간에 강한 상관관계(*correlation*)가 존재합니다. *CNN*은 필터(*filter*)를 사용하여 이러한 지역적 패턴(예: 모서리, 질감)을 감지합니다.
- 텍스트: 단어 임베딩을 통해 각 단어를 벡터로 표현하면, 문장은 벡터들의 시퀀스(*sequence of vectors*)가 됩니다. 이 시퀀스를 1차원 배열처럼 간주할 수 있습니다.
- *NLP* 적용: 텍스트에서 인접한 단어들(*N*-그램과 유사)은 문맥적 의미를 형성합니다. *CNN*은 필터를 사용하여 이러한 지역적 단어 패턴(예: 구문, 특정 표현)을 추출할 수 있습니다. 광학 문자 인식(*OCR*)도 이미지 처리의 일종으로 *CNN*이 활용됩니다.

예시

텍스트를 벡터 시퀀스 $[X_1, X_2, X_3, \dots, X_T]$ 로 표현할 때, 각 X_i 는 단어 임베딩 벡터입니다. 이 벡터 시퀀스는 마치 1차원 이미지처럼 간주될 수 있으며, *CNN* 필터는 이 시퀀스 위를 슬라이딩하며 지역적 특징을 추출합니다.

4.6.2 합성곱(Convolution) 연산

핵심 개념

합성곱(*Convolution*)은 입력 데이터(예: 이미지)와 필터(*filter*) 또는 커널(*kernel*)이라는 작은 행렬을 사용하여 새로운 특징 맵(*feature map*)을 생성하는 연산입니다. 필터는 입력 데이터 위를 슬라이딩하며 지역적 패턴을 감지합니다.

정의 및 직관적 이해

합성곱 연산은 신호 처리에서 유래한 개념입니다. 물리적인 센서가 특정 지점의 신호를 정확히 측정하기 어렵고, 항상 주변의 신호를 통합하여 측정하는 것과 유사합니다.

- 신호 처리 비유: 어떤 슬릿(*slit*)을 통해 들어오는 빛의 강도를 측정할 때, 슬릿의 크기 때문에 특정 한 점의 강도만 측정할 수 없습니다. 슬릿을 통과하는 빛의 누적된(*cumulative*) 강도를

측정하게 됩니다. 이때 슬릿의 특성(필터)과 실제 신호(입력)가 겹치는 영역의 합이 측정값(출력)이 됩니다.

- 신경망에서의 합성곱: 입력 데이터의 지역적인 부분에 필터의 가중치(*weights*)를 곱하고 모두 더하는 연산입니다. 이는 마치 작은 돋보기(필터)를 들고 이미지의 각 부분을 살펴보며 특징을 추출하는 것과 같습니다.

완전 연결 신경망(FCNN)과의 비교

*CNN*은 완전 연결 신경망(*Fully Connected Neural Network, FCNN*)과 비교하여 두 가지 중요한 특징을 가집니다.

특징	완전 연결 신경망 (<i>FCNN</i>)	합성곱 신경망 (<i>CNN</i>)
연결 방식	모든 입력 뉴런이 다음 계층의 모든 뉴런과 연결 (각 뉴런이 전체 입력 신호를 받음)	지역적 연결 (<i>Local Connectivity</i>) (각 뉴런이 입력의 특정 지역만 연결)
가중치 공유	각 뉴런이 고유한 가중치 집합을 가짐 (파라미터 수가 매우 많음)	가중치 공유 (<i>Weight Sharing</i>) (동일한 필터 가중치를 입력의 여러 위치에 적용)
파라미터 수	매우 많음	훨씬 적음
특징 감지	전역적 패턴 감지	지역적 패턴 감지 (위치 불변성)

- 지역적 연결 (*Local Connectivity*): *CNN*의 뉴런은 입력 데이터의 전체가 아닌 특정 지역 (*receptive field*)에만 연결됩니다. 이는 이미지에서 특정 픽셀이 주변 픽셀과 더 강한 상관 관계를 가지는 것과 유사합니다.
- 가중치 공유 (*Weight Sharing*): 동일한 필터(가중치 집합)가 입력 데이터의 모든 지역에 걸쳐 슬라이딩하며 적용됩니다.
 - 파라미터 감소: *FCNN*은 모든 연결에 대해 고유한 가중치를 가지지만, *CNN*은 필터의 가중치만 학습하면 되므로 파라미터 수가 획기적으로 줄어듭니다.
 - 위치 불변성 (*Translation Invariance*): 이미지의 특정 특징(예: 고양이의 눈)이 어디에 나타나든 동일한 필터로 감지할 수 있습니다. 이는 "어떤 위치에 있든 같은 눈"이라는 직관을 반영합니다.

필터(Filter) / 커널(Kernel)

필터는 합성곱 연산의 핵심입니다. 이는 입력 데이터에서 특정 특징(예: 수직선, 수평선, 모서리)을 추출하는 역할을 하는 작은 행렬입니다. 필터의 각 요소는 학습 가능한 가중치입니다.

바이어스(Bias)

각 필터는 합성곱 연산 결과에 더해지는 바이어스(*bias*) 값을 가집니다.

활성화 함수(Activation Function)

합성곱 연산 결과에 비선형성을 추가하기 위해 활성화 함수(예: *ReLU*)가 적용됩니다.

패딩 (Padding)

합성곱 연산 시 입력 데이터의 경계에 0을 추가하는 기법입니다.

- *Valid Padding*: 패딩을 사용하지 않는 경우입니다. 필터가 입력 데이터의 경계를 넘어가지 않으므로, 출력 특징 맵의 크기가 입력보다 작아집니다. (예: $N \times N$ 입력에 $F \times F$ 필터를 적용하면 $(N - F + 1) \times (N - F + 1)$ 크기의 출력이 나옴)
- *Same Padding*: 입력 데이터의 크기와 동일한 크기의 특징 맵을 출력하기 위해 경계에 0을 추가합니다.

스트라이드 (Stride)

필터가 입력 데이터 위를 이동하는 간격입니다.

- *Stride = 1*: 필터가 한 칸씩 이동합니다.
- *Stride > 1*: 필터가 여러 칸씩 건너뛰며 이동합니다. 이는 특징 맵의 크기를 줄이는 효과가 있습니다.

주의

입력 데이터의 차원을 보존하려면 패딩은 '*Same*'으로, 스트라이드는 '*1*'로 설정해야 합니다. 그렇지 않으면 특징 맵의 크기가 줄어듭니다.

합성곱 연산 예시 (2D 배열)

다음은 2D 배열에 필터를 적용하는 합성곱 연산의 예시입니다.

예시

입력 배열 (*Input Array*):

1	2	3
4	5	6
7	8	9

필터 (*Filter*):

1	1
2	2

바이어스 (*Bias*): 100

연산 과정:

1. 필터를 입력 배열의 좌측 상단에 배치합니다 (1, 2, 4, 5).
2. 필터의 각 요소와 겹치는 입력 배열의 요소를 곱하고 모두 더합니다: $(1 \times 1) + (1 \times 2) + (2 \times 4) + (2 \times 5) = 1 + 2 + 8 + 10 = 21$.
3. 여기에 바이어스를 더합니다: $21 + 100 = 121$.
4. 활성화 함수(예: *ReLU*)를 적용합니다. 만약 $ReLU(\max(z, 0))$ 를 사용하면, 121이 출력됩니다.
5. 필터를 스트라이드(기본값 1)만큼 이동하여 다음 위치에 대해 동일한 연산을 반복합니다.

출력 특징 맵 (*Output Feature Map*): (*Valid Padding, Stride=1* 가정)

121	128
135	142

4.6.3 풀링(Pooling) 계층

핵심 개념

풀링(*Pooling*) 계층은 합성곱 계층의 출력인 특징 맵의 크기를 줄여(다운샘플링) 모델의 파라미터 수를 감소시키고, 과적합을 방지하며, 특징의 위치 변화에 대한 강건성(*robustness*)을 높이는 역할을 합니다.

목적

- 차원 축소: 특징 맵의 공간적 크기를 줄여 다음 계층의 계산량을 감소시킵니다.
- 과적합 방지: 모델의 복잡도를 줄여 일반화 성능을 향상시킵니다.
- 위치 불변성 강화: 특징의 미세한 위치 변화에도 동일한 특징으로 인식하도록 돕습니다.

Max Pooling

가장 널리 사용되는 풀링 기법입니다.

- 원리: 특징 맵의 특정 지역(예: 2×2 영역)에서 가장 큰 값(최대값)을 추출하여 해당 지역을 대표하는 값으로 사용합니다.

- 장점: 가장 두드러진 특징을 보존하고, 이미지의 흐릿함(*blur*)을 줄이는 데 효과적입니다. 평균 풀링(*Average Pooling*)은 특징 맵을 흐릿하게 만들 수 있습니다.

예시

Max Pooling 예시 (2×2 필터, *Stride*=2): 입력 특징 맵:

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Max Pooling 적용:

- 좌측 상단 2×2 영역 [1, 2, 5, 6]에서 최대값은 6.
- 우측 상단 2×2 영역 [3, 4, 7, 8]에서 최대값은 8.
- 좌측 하단 2×2 영역 [9, 10, 13, 14]에서 최대값은 14.
- 우측 하단 2×2 영역 [11, 12, 15, 16]에서 최대값은 16.

출력 특징 맵:

6	8
14	16

 입력 크기가 4×4 에서 2×2 로 줄어들었습니다.

4.6.4 CNN 아키텍처 예시 및 Flatten

CNN 모델은 여러 합성곱 계층과 풀링 계층을 쌓아 올린 후, 최종적으로 완전 연결 계층을 통해 분류나 회귀와 같은 작업을 수행합니다.

예시

CNN 모델 아키텍처 예시:

- 입력 계층 (*Input Layer*): *Input Shape*: (32, 32, 3)
 - 32×32 픽셀 크기의 이미지.
 - 3개의 채널(예: *Red, Green, Blue*)을 가집니다.
- 합성곱 계층 1 (*Conv2D Layer 1*): *Conv2D*(32, (3, 3))
 - 32개의 필터(*feature map*)를 사용합니다.
 - 각 필터의 크기는 3×3 입니다.
 - 입력 채널이 3개이므로, 각 필터는 $3 \times 3 \times 3$ 의 가중치를 가집니다.
 - 패딩이 '*valid*'이고 스트라이드가 1인 경우, 출력 크기는 $(32 - 3 + 1) \times (32 - 3 + 1) = 30 \times 30$ 이 됩니다.
 - 출력: (30, 30, 32) (높이, 너비, 채널 수)
- 맥스 풀링 계층 1 (*MaxPooling2D Layer 1*): *MaxPooling2D*((2, 2))
 - 2×2 크기의 풀링 필터를 사용합니다.
 - 각 특징 맵의 크기를 절반으로 줄입니다.
 - 출력: (15, 15, 32) (채널 수는 유지)
- 합성곱 계층 2 (*Conv2D Layer 2*): *Conv2D*(64, (3, 3))
 - 64개의 필터를 사용합니다.
 - 각 필터의 크기는 3×3 입니다.
 - 입력 채널이 32개이므로, 각 필터는 $3 \times 3 \times 32$ 의 가중치를 가집니다.
 - 출력 크기는 $(15 - 3 + 1) \times (15 - 3 + 1) = 13 \times 13$ 이 됩니다.
 - 출력: (13, 13, 64)
- 맥스 풀링 계층 2 (*MaxPooling2D Layer 2*): *MaxPooling2D*((2, 2))
 - 출력: (6, 6, 64) (13을 2로 나누면 6.5이므로, 반올림 또는 버림에 따라 6이 됨)
- *Flatten* 계층:
 - 다차원 특징 맵을 1차원 벡터로 평탄화(*reshape*)합니다.
 - $(6, 6, 64) \rightarrow 6 \times 6 \times 64 = 2304$ 차원의 벡터.
 - 이 계층은 파라미터가 없으며, 단순히 데이터의 형태를 변경합니다.
- 완전 연결 계층 (*Dense Layer*): *Dense*(10, *activation*='softmax')
 - 10개의 출력 뉴런을 가집니다 (예: 10가지 클래스 분류).
 - 소프트맥스(*softmax*) 활성화 함수를 사용하여 각 클래스에 대한 확률을 출력합니다.

Flatten 계층

핵심 개념

Flatten 계층은 합성곱 계층이나 풀링 계층에서 생성된 다차원의 특징 맵을 완전 연결 계층(*Dense Layer*)의 입력으로 사용하기 위해 1차원 벡터로 변환하는 역할을 합니다.

- 목적: *CNN*의 특징 추출 부분(합성곱 + 풀링)과 분류/회귀 부분(완전 연결 계층)을 연결합니다.
- 비용: *Flatten* 계층 자체는 학습 가능한 파라미터가 없으므로, 계산 비용이 발생하지 않습니다. 단순히 데이터의 형태를 재구성하는 작업입니다.

실제 이미지 데이터셋(MNIST)에서의 CNN vs. Flattening 비교

*MNIST*와 같이 깔끔하게 중앙에 정렬된 숫자 이미지 데이터셋에서는 이미지를 바로 *Flatten* 하여 완전 연결 신경망에 입력해도 좋은 성능을 얻을 수 있습니다.

주의

현실 세계 데이터의 문제점: 현실 세계의 이미지(예: 고양이 사진)는 숫자가 항상 중앙에 있거나 크기가 일정하지 않습니다. 고양이가 이미지의 왼쪽 상단에 있을 수도 있고, 오른쪽 하단에 있을 수도 있으며, 크기도 다양합니다.

- 만약 이미지를 바로 *Flatten*하여 *FCNN*에 입력하면, 모델은 특정 위치에서만 특징을 인식하도록 학습될 수 있습니다.
- 고양이가 다른 위치로 이동하면, 이전에 학습된 뉴런들은 해당 특징을 인식하지 못하게 되어 성능이 저하됩니다.

*CNN*의 장점: *CNN*의 가중치 공유와 지역적 연결 특성은 이러한 위치 불변성 문제를 해결합니다. 필터는 이미지의 어느 위치에 있든 동일한 특징(예: 고양이의 귀)을 감지할 수 있으므로, 현실 세계의 다양한 이미지에 더 강건하게 작동합니다.

1x1 필터 (Inception Module)

핵심 개념

1x1 필터는 합성곱 계층에서 1×1 크기의 필터를 사용하는 것을 말합니다. 이는 특징 맵의 공간적 해상도를 유지하면서 채널(깊이) 차원을 조절하는 데 주로 사용됩니다.

- 목적:
 - 채널 차원 축소/확대: 1×1 필터는 각 픽셀에 대해 채널 방향으로 선형 변환을 수행합니다. 이를 통해 특징 맵의 채널 수를 줄이거나 늘릴 수 있습니다. 이는 계산 비용을 줄이는 데 매우 효과적입니다.
 - 비선형성 추가: 1×1 합성곱 뒤에 활성화 함수를 적용하여 비선형성을 추가할 수 있습니다.
- *Inception Module*: *GoogLeNet(Inception)*과 같은 고급 *CNN* 아키텍처에서 널리 사용됩니다. *Inception* 모듈은 1×1 , 3×3 , 5×5 필터와 풀링을 병렬로 적용하여 다양한 스케일의 특징을 추출하며, 이때 1×1 필터는 주로 채널 차원을 줄여 계산 비용을 제어하는 데 사용됩니다.

4.7 학습 로드맵 및 시험 대비 체크리스트

4.7.1 학습 로드맵

본 강의 내용을 효과적으로 학습하기 위한 권장 로드맵입니다.

1. 기초 다지기: 텍스트 전처리(토큰화, 스테밍, 표제어 추출, 불용어)의 정의와 목적을 명확히 이해합니다.
 2. 핵심 개념 이해: 단어 가방(*BoW*)과 *N*-그램의 원리, 장단점, 그리고 희소 표현의 문제점을 파악합니다.
 3. 문제 해결 방안: 단어 임베딩이 희소 표현 문제를 어떻게 해결하고 단어의 의미를 어떻게 반영하는지 깊이 이해합니다.
 4. 새로운 아키텍처: 합성곱 신경망(*CNN*)의 기본 원리(합성곱, 필터, 풀링, 가중치 공유, 지역적 연결)를 학습하고, *NLP*에 어떻게 적용되는지 파악합니다.
 5. 실습 및 코드: 제공된 코드 예시(*scikit-learn*, *NLTK*, *SpaCy*)를 직접 실행하고, 각 도구의 사용법과 유연성 차이를 경험합니다.
 6. 개념 비교: *BoW* vs. 임베딩, 스테밍 vs. 표제어 추출, *FCNN* vs. *CNN*의 차이점을 명확히 구분합니다.
-

4.7.2 시험 대비 체크리스트

No.	항목	완료 여부
1	토큰화의 정의와 다양한 토큰 단위(단어, 문자, 서브워드)를 설명할 수 있는가?	☐
2	스테밍과 표제어 추출의 차이점, 장단점, 그리고 적용 시기를 설명할 수 있는가?	☐
3	단어 가방(<i>BoW</i>) 모델의 원리, 장단점, 그리고 희소 표현 문제를 설명할 수 있는가?	☐
4	<i>N</i> -그램의 정의와 지역적 문맥 포착 원리를 설명하고, <i>N</i> -그램의 장단점을 열거할 수 있는가?	☐
5	단어 임베딩이 희소 표현 문제를 어떻게 해결하고, 단어의 의미를 어떻게 반영하는지 설명할 수 있는가?	☐
6	임베딩 계층의 파라미터 수를 계산할 수 있는가?	☐
7	합성곱 신경망(<i>CNN</i>)이 완전 연결 신경망(<i>FCNN</i>)과 다른 두 가지 주요 특징(지역적 연결, 가중치 공유)을 설명할 수 있는가?	☐
8	합성곱 연산의 필터, 패딩(<i>Valid/Same</i>), 스트라이드의 역할을 설명할 수 있는가?	☐
9	맥스 풀링(<i>Max Pooling</i>)의 목적과 작동 방식을 설명할 수 있는가?	☐
10	<i>CNN</i> 아키텍처 예시(<i>Conv2D</i> , <i>MaxPooling2D</i> , <i>Flatten</i> , <i>Dense</i>)를 보고 각 계층의 입출력 형태와 파라미터 변화를 이해할 수 있는가?	☐
11	<i>Flatten</i> 계층의 역할과 파라미터 수를 설명할 수 있는가?	☐
12	<i>1x1</i> 필터의 역할과 <i>Inception Module</i> 에서의 활용을 설명할 수 있는가?	☐
13	<i>scikit-learn</i> , <i>NLTK</i> , <i>SpaCy</i> 를 이용한 <i>BoW</i> 및 <i>N</i> -그램 구현 코드를 이해하고 수정할 수 있는가?	☐

4.8 FAQ

- *Q: 단어 가방(BoW)에서 단어의 빈도를 사용하지 않고 단순히 단어의 존재 여부(0 또는 1)만 사용해도 되나요? A: 네, 가능합니다. 단어의 존재 여부만 사용하는 것은 '이진 단어 가방(Binary Bag-of-Words)'이라고 불리며, 계산 비용을 줄일 수 있습니다. 하지만 "great"과 같은 긍정적인 단어가 여러 번 반복될 때의 강한 긍정적 감성을 포착하지 못할 수 있습니다. 빈도를 사용하면 이러한 정보를 더 잘 반영할 수 있습니다.*
- *Q: N-그램을 사용할 때 N의 크기를 어떻게 결정해야 하나요? A: 일반적으로 2-그램(바이그램)이나 3-그램(트라이그램)이 가장 유용합니다. N이 너무 커지면 어휘 집합이 폭발적으로 증가하여 희소성 문제가 심화되고 계산 비용이 매우 높아집니다. 감성 분석과 같은 작업에서는 2-그램과 3-그램을 함께 사용하는 것이 최상의 성능을 낼 수 있습니다.*
- *Q: 임베딩 계층에서 입력으로 원-핫 인코딩 벡터 대신 정수 인덱스를 직접 사용하는 이유는 무엇인가요? A: 계산 효율성 때문입니다. 원-핫 인코딩 벡터를 명시적으로 생성하여 입력하는 것은 많은 0과의 불필요한 곱셈 연산을 유발합니다. 정수 인덱스를 직접 사용하면, 임베딩 계층이 내부적으로 학습된 임베딩 행렬에서 해당 인덱스에 해당하는 행(단어 벡터)을 바로 선택하여 출력할 수 있어 훨씬 빠르고 효율적입니다.*
- *Q: 과제 채점은 언제 이루어지나요? A: 과제 채점은 일관성을 위해 모든 학생의 과제를 채점한 후 일괄적으로 공개됩니다. 여러 채점자가 있을 수 있으므로, 모든 채점 결과를 확인하고 일관성을 유지하기 위해 시간이 소요됩니다.*
- *Q: 불용어(Stop Words)는 항상 제거해야 하나요? A: 아닙니다. 불용어 제거는 텍스트의 차원을 줄이고 노이즈를 감소시키는 데 도움이 되지만, 때로는 문맥상 중요한 정보를 제거할 수도 있습니다. 예를 들어, "not good"과 같은 부정 표현에서 "not"을 제거하면 의미가 완전히 달라집니다. 따라서 작업의 특성과 데이터셋을 고려하여 불용어 제거 여부를 결정해야 합니다. N-그램을 사용하면 불용어가 포함된 구문도 하나의 토큰으로 처리하여 문맥을 보존할 수 있습니다.*

4.9 빠르게 훑어보기 (1페이지 요약)

CSCI89 Lecture 4 핵심 요약, fonttitle=, arc=5mm

과제 피드백: 원-핫 인코딩의 희소성 문제, 손실 폭발(*Sigmoid+Cross Entropy*) 및 *Dropout* 해결.

텍스트 전처리:

- 토큰화: 텍스트를 단어/문자 단위로 분리. *NLTK/SpaCy* 활용.
- 스테밍 *vs.* 표제어 추출: 어간 추출 *vs.* 사전형 변환. 정확도와 비용의 트레이드오프.
- 불용어: 의미 없는 단어 제거로 차원 축소. 문맥 고려하여 선택적 적용.

텍스트 표현 기법:

- 단어 가방(*BoW*): 단어 빈도 기반. 순서 정보 손실, 희소성 문제. 텍스트 분류에 유용.
- *N*-그램: *N*개 연속 단어 토큰화. 지역적 문맥 포착. 어휘 집합 폭발 주의.
- 단어 임베딩: 단어를 저차원 밀집 벡터로 표현. 희소성 해결, 의미 유사성 반영. 인덱스 직접 입력으로 효율성 증대.

합성곱 신경망(*CNN*):

- *NLP* 적용: 텍스트를 벡터 시퀀스(*1D* 배열)로 간주하여 지역적 패턴 추출.
- 합성곱 연산: 필터(가중치)가 입력 위를 슬라이딩하며 특징 추출.
- *FCNN vs. CNN*: *CNN*은 지역적 연결과 가중치 공유로 파라미터 감소 및 위치 불변성 확보.
- 필터/패딩/스트라이드: 특징 추출 및 특징 맵 크기 조절.
- 풀링 계층: 특징 맵 크기 축소(*Max Pooling*), 과적합 방지.
- *Flatten* 계층: 다차원 특징 맵을 1차원 벡터로 변환하여 완전 연결 계층에 연결. 파라미터 없음.
- *1x1* 필터: 채널 차원 조절, 계산 효율성 증대.

Chapter 5

용어 정리

강의명: *CSCI89*

주차: *Lecture 5*

교수명: *UIUC Student*

목적: 이 문서는 *CSCI89 Lecture 5* 강의 내용을 바탕으로 텍스트 처리의 핵심 개념과 기술을 다룹니다. 주요 목표는 *Bag-of-Words*의 한계를 극복하고, 단어의 의미론적 관계를 포착하는 방법을 이해하는 것입니다. 가중치 공유, *N*-그램 토큰화, *TF-IDF*, 단어 임베딩 (*Word2Vec*, *GloVe*) 등의 개념을 상세히 설명하고, 실제 코드 예시를 통해 구현 방법을 제시합니다. 시험 대비를 위해 비전공자도 쉽게 이해할 수 있도록 직관적인 설명과 풍부한 예시를 포함합니다.

5.1 용어 정리

용어	쉬운 설명	원어	비고
가중치 공유	신경망의 일부 가중치를 여러 위치에서 재사용하여 모델 복잡도 감소	<i>Weight Sharing</i>	<i>RNN</i> , <i>CNN</i> 에서 주로 사용
<i>N</i> -그램	텍스트에서 <i>N</i> 개의 연속된 단어 묶음	<i>N-gram</i>	단어 순서 정보를 일부 보존
<i>Bag-of-Words</i>	문서 내 단어의 출현 빈도만으로 문서를 표현하는 방식	<i>Bag-of-Words (BoW)</i>	단어 순서 정보 손실, 희소 표현
<i>TF-IDF</i>	단어의 중요도를 측정하는 통계적 방법	<i>Term Frequency-Inverse Document Frequency</i>	<i>TF</i> (빈도)와 <i>IDF</i> (희귀성)의 곱
<i>Term Frequency (TF)</i>	특정 문서 내에서 단어가 나타나는 빈도	<i>Term Frequency</i>	문서 길이에 대한 상대적 빈도
<i>Inverse Document Frequency (IDF)</i>	전체 문서 집합에서 단어가 얼마나 희귀한지 나타내는 값	<i>Inverse Document Frequency</i>	희귀할수록 높은 값
원-핫 인코딩	단어를 고유한 위치에 <i>1</i> 을, 나머지는 <i>0</i> 으로 표현하는 방식	<i>One-Hot Encoding</i>	희소하고 의미 관계를 반영 못함
단어 임베딩	단어를 저차원의 밀집 벡터로 표현하여 의미 관계를 포착	<i>Word Embedding</i>	<i>Word2Vec</i> , <i>GloVe</i> 등
임베딩 레이어	신경망에서 원-핫 벡터를 임베딩 벡터로 변환하는 층	<i>Embedding Layer</i>	학습 가능한 가중치 행렬
<i>Word2Vec</i>	단어 임베딩을 학습하는 모델 (<i>Google</i>)	<i>Word2Vec</i>	<i>CBOW</i> , <i>Skip-gram</i> 방식
<i>GloVe</i>	단어 임베딩을 학습하는 모델 (<i>Stanford</i>)	<i>Global Vectors for Word Representation</i>	동시 출현 행렬 기반
<i>CBOW</i>	주변 단어를 이용해 중심 단어를 예측하는 <i>Word2Vec</i> 방식	<i>Continuous Bag-of-Words</i>	
<i>Skip-gram</i>	중심 단어를 이용해 주변 단어를 예측하는 <i>Word2Vec</i> 방식	<i>Skip-gram</i>	
코사인 유사성	두 벡터 간의 각도를 이용해 유사도를 측정하는 방법	<i>Cosine Similarity</i>	임베딩 벡터 간 유사도 측정에 활용
패딩 (<i>Padding</i>)	합성곱 연산 시 이미지 가장자리에 <i>0</i> 을 추가하는 기법	<i>Padding</i>	출력 차원 유지 또는 경계 정보 보존
스트라이드 (<i>Stride</i>)	합성곱 필터가 이동하는 간격	<i>Stride</i>	출력 차원 조절
오토인코더	입력 데이터를 압축 후 다시 복원하는 신경망	<i>Autoencoder</i>	데이터 압축, 노이즈 제거 등에 활용
스테밍 (<i>Stemming</i>)	단어의 어간을 추출하여 단어를 정규화하는 기법	<i>Stemming</i>	'cats' → 'cat'
표제어 추출	단어의 기본형(표제어)을 찾아 단어를 정규화하는 기법	<i>Lemmatization</i>	'better' → 'good'

5.2 강의 및 과제 일정 안내

과제 마감일 변경

다음 과제 마감일이 변경되었습니다. 원래 이번 주 일요일(*This Sunday*)이 아닌, 다다음 주 일요일(*The following Sunday*)로 연기되었습니다. 이는 여러분이 과제를 작업할 수 있는 시간을 약 2주 정도 확보해 주기 위함입니다.

- 과제 게시: 이미 과제는 게시되었지만, 마감일이 연기되었습니다.
- 변경 이유: 강의에서 다루는 내용(예: *TF-IDF*)이 과제에 바로 적용되기 때문에, 강의와 과제 마감일 사이에 충분한 시간(최소 1주)을 두어 학생들이 편안하게 학습하고 과제를 수행할 수 있도록 하기 위함입니다.
- 전체 과제 일정 조정: 이번 과제 마감일 연기로 인해 전체 과제 일정이 뒤로 밀리게 됩니다. 이는 *Excel* 문서에 업데이트된 스케줄로 공지되었습니다. 과제 개수 자체가 줄어드는 것이 아니라, 단순히 마감일이 순차적으로 뒤로 밀리는 것입니다.
- 누적 과제 없음: 다음 주에 두 개의 과제를 제출해야 하는 등의 누적은 없습니다. 각 과제에 충분한 시간을 주기 위해 전체 일정이 조정된 것입니다.

5.3 핵심 개념 및 원리

5.3.1 퀴즈 4 해설

1. 가중치 공유 (Weight Sharing)

가중치 공유는 신경망 모델의 파라미터 수를 줄이고, 특정 패턴을 여러 위치에서 재사용할 수 있도록 하는 중요한 개념입니다. 이는 모델의 일반화 성능을 높이고 과적합을 방지하는 데 도움을 줍니다.

가중치 공유의 개념

가중치 공유 (*Weight Sharing*)는 신경망의 여러 연결에서 동일한 가중치(W)를 사용하는 기법입니다. 이는 모델의 파라미터 수를 크게 줄여 계산 효율성을 높이고, 특정 특징(패턴)이 입력 데이터의 위치에 관계없이 중요하다고 가정할 때 유용합니다.

- 완전 연결 신경망 (*Fully Connected Neural Network, FNN*):
 - 설명: 가장 기본적인 신경망으로, 이전 레이어의 모든 뉴런이 다음 레이어의 모든 뉴런과 연결됩니다. 각 연결에는 고유한 가중치(W)가 할당됩니다.
 - 가중치 공유 여부: 가중치 공유가 없습니다. 모든 W_{ij} 는 고유합니다.
 - 문제점: 입력 차원이 커지면 파라미터 수가 기하급수적으로 늘어나 학습이 어렵고 과적합 위험이 높습니다.
- 순환 신경망 (*Recurrent Neural Network, RNN*):
 - 설명: 시퀀스 데이터(시간, 텍스트 등) 처리에 특화된 신경망입니다. 현재 시점의 출력이 다음 시점의 입력으로 다시 들어가는 '순환' 구조를 가집니다.

- 가중치 공유 여부: 가중치 공유가 발생합니다. 서로 다른 시점의 입력에 대해 동일한 가중치 집합을 사용합니다.
- 이유: 시퀀스 데이터에서 시간 독립적인 패턴(예: 문법 규칙)을 학습하기 위함입니다. 즉, 특정 패턴이 시퀀스의 어느 시점에서 나타나든 동일하게 인식되어야 한다고 가정합니다. 이는 차원 축소의 한 방법이기도 합니다.
- 예시: "나는 학교에 간다"라는 문장에서 '나는' 다음에 '학교에'가 오는 패턴은 문장의 시작이든 중간이든 동일한 의미를 가질 수 있습니다. *RNN*은 이 패턴을 학습하는 가중치를 모든 시점에서 공유합니다.
- 합성곱 신경망 (*Convolutional Neural Network, CNN*):
 - 설명: 이미지와 같은 그리드(*grid*) 형태의 데이터 처리에 특화된 신경망입니다. 필터(커널)를 입력 데이터에 슬라이딩하며 특징 맵(*feature map*)을 생성합니다.
 - 가중치 공유 여부: 가중치 공유가 발생합니다. 하나의 필터가 이미지의 여러 영역에 걸쳐 동일한 가중치 집합을 사용하여 특징을 추출합니다.
 - 이유: 이미지에서 특정 특징(예: 모서리, 특정 모양)이 이미지의 어느 위치에 나타나든 동일한 특징으로 인식되어야 한다고 가정합니다. 필터가 이미지를 '슬라이딩'하는 과정 자체가 가중치 공유를 의미합니다.
 - 예시: 고양이 이미지에서 '눈'을 인식하는 필터는 이미지의 왼쪽 상단에서든 오른쪽 하단에서든 동일한 가중치를 사용하여 '눈'이라는 특징을 찾아냅니다.

2. N-그램 토큰화 (N-gram Tokenization)

텍스트를 분석할 때 단어 하나하나를 보는 것을 넘어, 여러 단어의 연속된 묶음을 함께 고려하는 것이 *N*-그램 토큰화입니다. 이는 단어의 순서 정보를 일부 보존하여 문맥을 더 잘 이해하는 데 도움을 줍니다.

N-그램 토큰화의 개념

N-그램 (*N-gram*)은 텍스트에서 *N*개의 연속된 단어(또는 토큰)를 하나의 단위로 묶는 방법입니다. 2-그램 (*bi-gram*)은 두 개의 연속된 단어를 묶으며, 이는 단어의 순서 정보를 부분적으로 보존하여 문맥을 파악하는 데 유용합니다.

- *Bag-of-Words*의 한계: *Bag-of-Words*는 단어의 출현 빈도만 고려하고 순서는 무시합니다. "not good"과 "good not"이 동일하게 처리될 수 있습니다.
- *N*-그램의 필요성: *N*-그램은 이러한 순서 정보를 일부 보존하여 "not good"과 같은 부정적인 표현을 하나의 의미 단위로 인식할 수 있게 합니다.
- 오버랩핑 *N*-그램 (*Overlapping N-grams*): *N*-그램을 추출할 때는 일반적으로 오버랩핑 방식을 사용합니다. 즉, 한 *N*-그램의 마지막 단어가 다음 *N*-그램의 첫 단어가 될 수 있습니다. 이는 모든 가능한 *N*-그램 조합을 포착하기 위함입니다.
- 예시 문장 분석:

"Port assembly line one reduced car costs"

이 문장에서 2-그램을 추출하면 다음과 같습니다 (오버랩핑 방식):

- "Port assembly"

- “assembly line”
- “line one”
- “one reduced”
- “reduced car”
- “car costs”

강의에서 제시된 퀴즈의 정답은 이와 같이 모든 오버랩핑 2-그램을 포함하는 첫 번째 옵션이었습니다.

3. N-그램 개수 계산

주어진 텍스트에서 N -그램의 총 개수를 계산하는 것은 N -그램 모델을 구축할 때 중요한 단계입니다.

N -그램 개수 계산 공식

길이가 L 인 텍스트에서 N -그램의 개수는 일반적으로 $L - N + 1$ 입니다. 단, 모든 N -그램이 고유하다는 가정 하에 계산됩니다.

- 예시: 5개의 연속된 단어로 구성된 텍스트 “A B C D E”
 - 1-그램 (단어): A, B, C, D, E (총 5개)
 - 2-그램 (bi-gram): AB, BC, CD, DE (총 $5 - 2 + 1 = 4$ 개)
 - 3-그램 (tri-gram): ABC, BCD, CDE (총 $5 - 3 + 1 = 3$ 개)
- 경계 효과 (Boundary Effect): 텍스트의 시작과 끝 부분에서는 N -그램을 만들 수 있는 단어의 수가 줄어들기 때문에, N -그램의 개수가 1-그램(단어)의 개수보다 적을 수 있습니다.
- 실제 어휘 수: 이론적으로는 N -그램의 개수가 단어의 개수보다 적을 수 있지만, 실제 대규모 텍스트(코퍼스)에서는 N -그램의 고유한(unique) 개수가 1-그램의 고유한 개수보다 훨씬 많습니다.
 - 예시: ‘computer’라는 단어 뒤에는 ‘is fast’, ‘does not respond’, ‘does’ 등 다양한 단어가 올 수 있습니다. ‘computer’는 1-그램으로 하나지만, ‘computer is’, ‘computer does’ 등 2-그램은 훨씬 많아질 수 있습니다.
 - 의미: N -그램을 사용하면 어휘(vocabulary)의 크기가 매우 커지므로, 모델의 복잡도가 증가하고 학습에 더 많은 데이터와 계산 자원이 필요합니다.

4. 합성곱 신경망 (CNN) 패딩과 스트라이드

CNN에서 패딩(padding)과 스트라이드(stride)는 합성곱 연산의 출력 차원을 조절하는 중요한 하이퍼파라미터입니다.

패딩과 스트라이드의 역할

패딩(*Padding*)은 입력 데이터의 가장자리에 0을 추가하여 출력 특징 맵의 크기를 조절합니다. 스트라이드(*Stride*)는 필터가 입력 데이터를 가로지르는 간격을 의미하며, 이 또한 출력 특징 맵의 크기에 영향을 줍니다.

- `padding='same'`과 `strides=1`:
 - 목표: 입력 데이터의 공간적 차원(가로, 세로)을 출력 특징 맵에서 동일하게 유지하는 것입니다.
 - 방법: `padding='same'`은 필터 크기에 맞춰 자동으로 0을 추가하여 입력과 출력의 차원을 같게 만듭니다. `strides=1`은 필터가 한 칸씩 이동하므로, 모든 픽셀을 한 번씩 처리합니다.
 - 활용: 오토인코더(*Autoencoder*)와 같이 입력과 출력의 크기를 동일하게 유지해야 하는 대칭적인 신경망 구조에서 널리 사용됩니다.
- `padding='valid'`와 `strides > 1`:
 - `padding='valid'`: 패딩을 전혀 추가하지 않습니다. 이 경우 출력 특징 맵의 크기는 입력보다 작아집니다.
 - `strides > 1`: 필터가 여러 칸씩 건너뛰며 이동합니다. 이 경우 출력 특징 맵의 크기는 입력보다 크게 줄어듭니다.
 - 목표: 주로 특징 맵의 차원을 줄여서 모델의 복잡도를 낮추고 계산량을 줄이는 데 사용됩니다.
- *Max Pooling* (최대 풀링):
 - 설명: 합성곱 레이어 다음에 오는 다운샘플링(*downsampling*) 기법으로, 특정 영역(예: 2×2) 내에서 가장 큰 값을 추출하여 특징 맵의 크기를 줄입니다.
 - 기본 동작: *Max Pooling*은 기본적으로 `strides`가 풀링 필터 크기와 동일하게 설정되어 영역들이 겹치지 않도록 합니다(예: 2×2 풀링에 `strides=2`).
 - 비대칭성: 입력 데이터의 크기가 풀링 필터 크기로 나누어 떨어지지 않을 경우, 남는 가장자리 픽셀들은 버려집니다. 이는 패딩이 없는 경우에도 발생하는 비대칭적인 동작입니다.
- 오토인코더(*Autoencoder*)에서의 활용:
 - 오토인코더는 입력 이미지를 압축(인코딩)했다가 다시 원본 이미지로 복원(디코딩)하는 신경망입니다.
 - 인코더 부분에서는 합성곱 레이어와 풀링을 통해 차원을 줄이고, 디코더 부분에서는 역합성곱(*deconvolution*) 또는 전치 합성곱(*transposed convolution*)을 통해 차원을 다시 늘립니다.
 - 이때, `padding='same'`과 `strides=1`을 사용하여 중간 합성곱 레이어들의 크기를 동일하게 유지하면, 원본 이미지 크기를 복원하기가 훨씬 용이해집니다. 이는 '샌드위치' 아키텍처라고도 불리며, 이미지 노이즈 제거(*denoising autoencoder*)와 같은 작업에 유용합니다.

5.4 5. TF-IDF (Term Frequency-Inverse Document Frequency)

*TF-IDF*는 텍스트 마이닝에서 단어의 중요도를 측정하는 데 널리 사용되는 통계적 가중치 기법입니다. 단순히 단어의 출현 빈도만 보는 *Bag-of-Words*의 한계를 극복합니다.

TF-IDF의 개념 및 필요성

TF-IDF는 특정 단어가 문서 내에서 얼마나 자주 나타나는지(**TF**)와 전체 문서 집합에서 얼마나 희귀하게 나타나는지(**IDF**)를 결합하여 단어의 중요도를 측정합니다. 이는 '흔한 단어는 중요하지 않고, 특정 문서에서 자주 나오면서 다른 문서에서는 잘 나오지 않는 단어가 중요하다'는 직관을 반영합니다.

- **Bag-of-Words**의 한계:
 - 단순히 단어의 출현 횟수(*count*)만 사용합니다.
 - 'the', 'a', 'is'와 같은 불용어(*stop words*)나 'Boston'과 같이 특정 도메인에서 너무 흔한 단어들은 모든 문서에 자주 나타나지만, 문서의 내용을 구별하는 데는 거의 도움이 되지 않습니다. **Bag-of-Words**는 이러한 단어들에 높은 가중치를 부여하게 됩니다.
- **TF-IDF**의 목표:
 - 문서 내에서 자주 나타나는 단어에 높은 가중치를 부여합니다 (**TF**).
 - 전체 문서 집합에서 희귀하게 나타나는 단어에 높은 가중치를 부여합니다 (**IDF**).
 - 결과적으로, 특정 문서에서 자주 나타나지만 다른 문서에서는 잘 나타나지 않는 단어에 가장 높은 가중치를 부여하여 해당 문서의 특징을 잘 나타내도록 합니다.

5.1. Term Frequency (TF)

TF는 특정 문서 내에서 단어가 얼마나 자주 나타나는지를 측정합니다. 문서의 길이에 따라 단어 빈도가 달라질 수 있으므로, 일반적으로 문서 길이로 정규화된 빈도를 사용합니다.

Term Frequency (TF) 정의

Term Frequency (TF)는 특정 단어(t)가 특정 문서(d) 내에서 나타나는 횟수를 해당 문서의 총 단어 수로 나눈 값입니다.

$$TF(t, d) = \frac{\text{단어 } t \text{의 문서 } d \text{ 내 출현 횟수}}{\text{문서 } d \text{의 총 단어 수}}$$

- 예시: 문서 d 가 총 3개의 단어로 구성되어 있고, 'cat'이라는 단어가 2번 나타났다면, $TF('cat', d) = 2/3$ 입니다.
- 목표: 문서의 길이가 길수록 특정 단어가 더 많이 나타날 가능성이 높으므로, 문서 길이로 나누어 상대적인 빈도를 측정합니다.

5.2. Inverse Document Frequency (IDF)

IDF는 특정 단어가 전체 문서 집합(코퍼스)에서 얼마나 희귀하게 나타나는지를 측정합니다. 희귀한 단어일수록 높은 **IDF** 값을 가집니다.

Inverse Document Frequency (IDF) 정의

*Inverse Document Frequency (IDF)*는 전체 문서 집합(N)에서 특정 단어(t)를 포함하는 문서의 수($DF(t)$)에 반비례하는 값입니다. 일반적으로 로그 스케일로 계산됩니다.

$$IDF(t) = \log \left(\frac{\text{전체 문서 수 } N}{\text{단어 } t \text{를 포함하는 문서 수 } DF(t)} \right)$$

- 로그 스케일링의 이유:
 - 매우 희귀한 단어(예: 100만 개 문서 중 1개 문서에만 출현)에 과도하게 높은 *IDF* 값이 부여되는 것을 방지합니다. 로그 함수는 값이 커질수록 증가율이 둔화되므로, 희귀 단어의 가중치를 적절히 조절합니다.
 - 또한, *IDF* 값이 너무 커지면 *TF-IDF* 값 전체가 특정 희귀 단어에 의해 지배될 수 있습니다.
- 분모가 0이 되는 경우 방지:
 - $DF(t)$ 는 단어 t 를 포함하는 문서의 수이므로, 우리가 사용하는 어휘(*vocabulary*)에 있는 단어라면 최소한 1개 이상의 문서에 나타나므로 0이 될 일은 없습니다.
 - 하지만 안전을 위해 $DF(t)$ 에 1을 더하는 변형($\log(N/(1 + DF(t)))$)을 사용하기도 합니다.
- 훈련 데이터셋 기반 계산:
 - *IDF*는 단어 자체의 특성(코퍼스 내 희귀성)을 나타내므로, 훈련 데이터셋(*train data set*)을 기반으로 한 번만 계산됩니다.
 - 이후 테스트 데이터셋(*test data set*)의 문서를 처리할 때도 훈련 데이터셋에서 계산된 *IDF* 값을 그대로 사용합니다.
 - 이유: 테스트 데이터셋은 모델이 처음 보는 데이터이므로, 테스트 데이터셋의 정보(예: 테스트 문서 내 단어의 희귀성)를 미리 사용하여 모델을 조정하는 것은 허용되지 않습니다. 또한, 테스트 데이터셋이 단일 문서일 경우 *IDF*를 계산하는 것이 불가능하거나 의미가 없습니다.

5.3. TF-IDF 계산 및 예시

*TF-IDF*는 *TF*와 *IDF*를 곱하여 최종 단어 중요도 점수를 산출합니다.

TF-IDF 계산

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

- 예시 시나리오: 4개의 문서로 구성된 코퍼스
 - 문서 1: "cat cat dog" (총 3단어)
 - 문서 2: "cat dog" (총 2단어)
 - 문서 3: "dog mouse" (총 2단어)
 - 문서 4: "dog" (총 1단어)
- *TF* 계산 (각 문서별):
 - 문서 1:

- * $TF('cat', D1) = 2/3$
- * $TF('dog', D1) = 1/3$
- * $TF('mouse', D1) = 0/3 = 0$
- 문서 3:
 - * $TF('cat', D3) = 0/2 = 0$
 - * $TF('dog', D3) = 1/2$
 - * $TF('mouse', D3) = 1/2$
- **IDF 계산 (코퍼스 전체):** (총 문서 수 $N = 4$)
 - 단어 'cat': 문서 1, 2에 출현 ($DF('cat') = 2$)
 - * $IDF('cat') = \log(4/2) = \log(2) \approx 0.69$
 - * 해석: 'cat'은 4개 문서 중 2개에만 나타나므로, 문서 구별에 유용한 단어입니다.
 - 단어 'dog': 문서 1, 2, 3, 4에 출현 ($DF('dog') = 4$)
 - * $IDF('dog') = \log(4/4) = \log(1) = 0$
 - * 해석: 'dog'은 모든 문서에 나타나므로, 문서 구별에 전혀 도움이 되지 않는 단어입니다. $TF-IDF$ 값은 0이 됩니다.
 - 단어 'mouse': 문서 3에 출현 ($DF('mouse') = 1$)
 - * $IDF('mouse') = \log(4/1) = \log(4) \approx 1.39$
 - * 해석: 'mouse'는 4개 문서 중 1개에만 나타나므로, 매우 희귀하며 문서 구별에 매우 유용한 단어입니다.
- **$TF-IDF$ 계산 (문서 1의 'cat'):**
 - $TF-IDF('cat', D1) = TF('cat', D1) \times IDF('cat') = (2/3) \times 0.69 \approx 0.46$
 - 해석: 'cat'은 문서 1에서 자주 나타나고(TF 높음), 코퍼스 전체에서는 적당히 희귀하므로(IDF 높음), 문서 1을 대표하는 중요한 단어가 됩니다.

5.4. $TF-IDF$ 변형 및 정규화

$TF-IDF$ 는 다양한 변형과 정규화 기법이 존재하며, 이는 실제 적용 시 성능에 영향을 미칠 수 있습니다.

$TF-IDF$ 변형 및 정규화의 목적

$TF-IDF$ 값의 계산 방식을 미세 조정하거나, 계산된 $TF-IDF$ 벡터를 특정 스케일로 맞추어 모델의 안정성과 성능을 향상시키는 것이 목적입니다.

- **TF (Term Frequency) 변형:**
 - **Raw Count** (단순 빈도): $TF(t, d) =$ 단어 t 의 문서 d 내 출현 횟수 (*Bag-of-Words*와 유사)
 - **Boolean** (이진 빈도): $TF(t, d) = 1$ (단어 t 가 문서 d 에 출현하면), 0 (아니면)
 - **Logarithmic** (로그 빈도): $TF(t, d) = 1 + \log(\text{단어 } t \text{의 출현 횟수})$ (빈도가 높을수록 증가율 둔화)
 - **Double Normalization** (이중 정규화): $TF(t, d) = 0.5 + 0.5 \times \frac{\text{단어 } t \text{의 출현 횟수}}{\text{문서 } d \text{ 내 최대 단어 출현 횟수}}$ (과

도한 빈도 편향 완화)

- *IDF (Inverse Document Frequency) 변형:*

- *Smooth IDF (스무스 IDF):* $IDF(t) = \log\left(\frac{N+1}{DF(t)+1}\right) + 1$

- * 분모가 0이 되는 것을 방지하고, *IDF* 값이 0이 되는 것을 막아 모든 단어에 최소한의 가중치를 부여합니다.

- * *scikit-learn*의 *TfidfVectorizer*에서 기본값으로 사용됩니다.

- *Probabilistic IDF (확률적 IDF):* 확률 이론에 기반한 변형으로, 단어가 나타나지 않는 문서의 비율을 고려합니다.

- *Maximal IDF:* 특정 단어의 *IDF* 값을 코퍼스 내 최대 *IDF* 값으로 정규화하는 방식입니다.

- 벡터 정규화 (*Vector Normalization*):

- 목적: 계산된 *TF-IDF* 벡터의 길이를 1로 만들어 단위 벡터(*unit vector*)로 변환합니다.

- 방법: 각 벡터의 모든 요소를 해당 벡터의 *L2* 노름(*Euclidean distance*, 유클리드 거리)으로 나눕니다.

$$\text{정규화된 벡터 } \mathbf{v}' = \frac{\mathbf{v}}{\|\mathbf{v}\|_2}$$

- *L2* 노름: $\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \dots + v_k^2}$

- 효과:

- * 문서 길이의 영향 감소: 문서의 길이가 길다고 해서 *TF-IDF* 벡터의 길이가 무조건 길어지는 것을 방지합니다.

- * 방향성 강조: 벡터의 '방향'에 초점을 맞춰 문서 간의 유사도를 측정할 때 '코사인 유사성 (*Cosine Similarity*)'을 직접적으로 사용할 수 있게 합니다. 두 단위 벡터의 내적(*dot product*)은 두 벡터 사이의 코사인 값과 동일합니다.

- * 모델 안정성: 신경망의 입력으로 사용될 때, 입력 값의 스케일을 일정하게 유지하여 학습의 안정성을 높일 수 있습니다.

- 논의: *TF* 계산 시 이미 문서 길이로 나누어 정규화하는데, 최종적으로 다시 벡터 정규화를 하는 것이 중복이 아닌가 하는 의문이 있을 수 있습니다. 하지만 *TF*는 '단어 빈도'를 문서 길이로 정규화하는 것이고, 최종 벡터 정규화는 '문서 전체의 *TF-IDF* 벡터'의 길이를 정규화하는 것으로, 서로 다른 목적을 가집니다. 경험적으로 두 가지 정규화를 모두 적용하는 것이 좋은 성능을 보이는 경우가 많습니다.

- 활용 분야:

- 키워드 추출 (*Keyword Extraction*): *TF-IDF* 값이 높은 단어는 문서의 핵심 키워드로 간주될 수 있습니다.

- 문서 분류 (*Document Classification*): *TF-IDF* 벡터를 신경망이나 머신러닝 모델의 입력으로 사용하여 문서의 카테고리를 분류합니다.

- 검색 엔진 (*Search Engine*): 사용자의 쿼리와 문서 간의 유사도를 *TF-IDF* 기반으로 계산하여 관련성 높은 문서를 검색 결과로 제공합니다.

5.5 6. 단어 임베딩 (Word Embedding)

단어 임베딩은 텍스트 데이터를 신경망이 이해할 수 있는 형태로 변환하는 핵심 기술입니다. 단어의 의미론적 관계를 저차원 밀집 벡터로 표현하여 텍스트 처리 모델의 성능을 혁신적으로 향상시켰습니다.

단어 임베딩의 개념

단어 임베딩 (*Word Embedding*)은 단어를 고차원의 희소한 원-핫 벡터가 아닌, 저차원의 밀집 (*dense*) 벡터로 표현하는 기법입니다. 이 과정에서 단어의 의미론적, 문맥적 관계가 벡터 공간에 반영되어, 유사한 의미를 가진 단어들은 벡터 공간에서 가까운 위치에 배치됩니다.

6.1. 원-핫 인코딩 (One-Hot Encoding)의 한계

단어 임베딩이 등장하기 전에는 주로 원-핫 인코딩을 사용했습니다.

- 표현 방식: 어휘 (*vocabulary*)의 크기만큼의 차원을 가진 벡터에서, 해당 단어의 인덱스에만 1을 표시하고 나머지는 0으로 채웁니다.
 - 예: 어휘 = {'cat', 'dog', 'mouse'}
 - 'cat' $\rightarrow [1, 0, 0]$
 - 'dog' $\rightarrow [0, 1, 0]$
 - 'mouse' $\rightarrow [0, 0, 1]$
- 문제점:
 - 희소성 (*Sparsity*): 어휘 크기가 커질수록 벡터의 대부분이 0이 되어 메모리 비효율적입니다.
 - 의미 관계 미반영: 'cat'과 'dog'은 동물이라는 점에서 유사하지만, 원-핫 벡터 상에서는 서로 직교 (*orthogonal*)하여 아무런 의미론적 관계를 나타내지 못합니다. 모든 단어 간의 거리가 동일합니다.
 - 차원 문제: 어휘 크기가 수만 개가 되면 입력 벡터의 차원이 너무 커져 신경망 학습에 부담이 됩니다.

6.2. 임베딩 레이어 (Embedding Layer)

단어 임베딩은 신경망의 한 부분으로 학습될 수 있습니다.

- 개념: 원-핫 인코딩된 단어 벡터를 입력으로 받아, 이를 저차원의 밀집 벡터(임베딩 벡터)로 변환하는 신경망의 첫 번째 층입니다.
- 동작 원리:
 - 입력: 어휘 크기(V)의 원-핫 벡터 (예: $[0, \dots, 1, \dots, 0]$)
 - 출력: 임베딩 차원(D)의 밀집 벡터 (예: $[0.1, -0.5, 0.3, \dots]$)
 - 내부적으로 $V \times D$ 크기의 임베딩 행렬 (*Embedding Matrix*) W_E 를 가집니다. 원-핫 벡터가 입력되면, 해당 단어의 인덱스에 해당하는 임베딩 행렬의 행 (*row*)을 찾아 출력합니다.

- 이는 사실상 선형 변환($u = xW_E$)과 동일하며, 활성화 함수나 바이어스(*bias*)는 필요하지 않습니다.
- 학습: 임베딩 행렬 W_E 의 값들은 신경망이 특정 작업(예: 분류, 예측)을 수행하는 과정에서 역전파(*backpropagation*)를 통해 함께 학습됩니다. 즉, 임베딩은 '문제 해결' 과정의 일부로 최적화됩니다.
- 효율적인 구현: 실제로는 고차원 원-핫 벡터를 곱하는 대신, 단어의 인덱스(예: 'cat'이 5번째 단어라면 인덱스 4)를 입력으로 받아 임베딩 행렬에서 해당 행을 직접 조회하는 방식으로 효율적으로 구현됩니다.

6.3. 임베딩 차원 결정

임베딩 벡터의 차원(D)은 중요한 하이퍼파라미터입니다.

- 결정 방법: 정해진 규칙은 없으며, 주로 경험적(*empirical*)으로 결정됩니다.
- 일반적인 범위: 어휘 크기에 따라 다르지만, 일반적으로 100차원, 200차원, 300차원 등 수백 차원 수준으로 설정됩니다.
- 고려 사항:
 - 너무 낮으면 단어의 의미를 충분히 표현하지 못할 수 있습니다.
 - 너무 높으면 모델의 복잡도가 증가하고 과적합 위험이 있습니다.

6.4. 임베딩의 의미론적 특성

잘 학습된 단어 임베딩은 단어 간의 의미론적 관계를 벡터 공간에 반영합니다.

- 벡터 연산의 의미: 임베딩 벡터 간의 덧셈, 뺄셈 연산이 의미 있는 결과를 도출할 수 있습니다.
- 예시: "*King - Man + Woman = Queen*"
 - $vec(King) - vec(Man)$ 은 '남성성'이라는 개념의 벡터를 나타냅니다.
 - 여기에 $vec(Woman)$ 을 더하면, '여성성'을 가진 '왕'의 개념, 즉 '여왕(*Queen*)'에 해당하는 벡터를 얻을 수 있습니다.
 - 이는 벡터 공간에서 '남성-여성'이라는 관계가 일관된 '이동 벡터(*shift vector*)'로 표현된다는 것을 의미합니다.
- 유사성 측정: 두 임베딩 벡터 간의 코사인 유사성(*Cosine Similarity*)을 통해 단어 간의 의미적 유사도를 측정합니다. 코사인 값이 1에 가까울수록 유사하고, 0에 가까울수록 관련성이 적습니다.

6.5. 단어 임베딩의 활용

단어 임베딩은 다양한 텍스트 처리 작업에 활용됩니다.

- 텍스트 표현: 단어의 순서나 시간 정보를 보존하면서 텍스트를 효율적으로 표현할 수 있습니다. (예: 번역 모델)

- 문서 표현 (평균 임베딩): 문서 내 모든 단어 임베딩 벡터의 평균을 계산하여 문서 전체를 하나의 벡터로 표현할 수 있습니다.
 - 장점: 문서의 시간/순서 정보를 잃지만, *Bag-of-Words*나 *TF-IDF*보다 의미론적 정보를 더 잘 보존합니다.
 - 활용: 문서 분류와 같이 순서 정보가 덜 중요하거나, 순환 신경망(*RNN*)과 같은 복잡한 모델 학습 비용이 부담될 때 사용될 수 있습니다.
- 다양한 *NLP* 태스크: 감성 분석, 기계 번역, 질의응답 시스템 등 거의 모든 자연어 처리(*NLP*) 분야에서 핵심적인 입력 특징으로 사용됩니다.

6.6. 단어 임베딩의 한계

단어 임베딩은 강력하지만 몇 가지 한계점도 존재합니다.

- *OOV (Out-Of-Vocabulary)* 문제:
 - 훈련 데이터셋에 없는 새로운 단어(*OOV*)가 나타나면 해당 단어에 대한 임베딩 벡터를 생성할 수 없습니다.
 - 해결책: *OOV* 단어를 위한 특별한 토큰(<UNK>)을 사용하거나, 서브워드(*subword*) 임베딩(예: *FastText*)을 사용합니다.
- 정적 임베딩 (*Static Embedding*):
 - 전통적인 *Word2Vec*이나 *GloVe* 임베딩은 한 단어에 대해 하나의 고정된 벡터를 할당합니다.
 - 문제점: 'bank'와 같이 여러 의미를 가지는 다의어(*polysemy*)의 경우, 문맥에 따라 다른 의미를 가지더라도 동일한 벡터로 표현됩니다.
 - 해결책: 문맥을 고려하여 동적으로 임베딩을 생성하는 문맥 기반 임베딩(*Contextualized Embedding*, 예: *ELMo*, *BERT*)이 등장했습니다.
- 편향 반영 (*Bias Reflection*):
 - 임베딩은 훈련 데이터셋의 텍스트에서 단어의 사용 패턴을 학습합니다. 만약 훈련 텍스트에 사회적 편향(예: '의사'는 남성과, '간호사'는 여성과 자주 연결)이 있다면, 임베딩 벡터에도 이러한 편향이 반영됩니다.
 - 문제점: 이는 모델이 편향된 예측이나 번역을 수행하게 만들 수 있습니다.
 - 해결책: 편향을 제거하기 위한 다양한 후처리 기법이 연구되고 있습니다.
- 데이터 부족 언어:
 - 임베딩 모델은 대규모 텍스트 코퍼스에서 학습되어야 좋은 성능을 냅니다.
 - 전자 형식의 텍스트 데이터가 부족한 언어의 경우, 고품질의 임베딩을 학습하기 어렵습니다.

6.7. 인기 있는 임베딩 모델

*Word2Vec*과 *GloVe*는 가장 널리 사용되는 단어 임베딩 모델입니다.

- *Word2Vec (Google)*:
 - 개념: 단어의 '주변 문맥(*context*)'을 기반으로 단어 임베딩을 학습합니다.

- 두 가지 학습 방식:
 1. *CBOW (Continuous Bag-of-Words)*: 주변 단어들을 입력으로 받아 중심 단어를 예측합니다.
 2. *Skip-gram*: 중심 단어를 입력으로 받아 주변 단어들을 예측합니다. (일반적으로 더 좋은 성능을 보임)
- 학습 과정: 슬라이딩 윈도우(*sliding window*)를 텍스트에 적용하여, 각 윈도우 내의 단어들을 입력/출력 쌍으로 사용하여 신경망을 훈련합니다. 이 신경망의 은닉층 가중치가 임베딩 벡터가 됩니다.
- 특징: 단어 간의 의미론적 관계(예: *King - Man + Woman = Queen*)를 잘 포착합니다.
- *GloVe (Stanford)*:
 - 개념: 코퍼스 전체의 '동시 출현 통계(*co-occurrence statistics*)'를 활용하여 단어 임베딩을 학습합니다.
 - 동작 원리:
 - * 코퍼스 내 모든 단어 쌍의 동시 출현 빈도를 계산하여 거대한 동시 출현 행렬(*co-occurrence matrix*)을 만듭니다.
 - * 이 행렬의 정보(로그 동시 출현 확률)를 임베딩 벡터의 내적(*dot product*)으로 모델링하도록 학습합니다.
 - 특징: *Word2Vec*이 지역적인 문맥(*sliding window*)에 집중하는 반면, *GloVe*는 전역적인 코퍼스 통계를 활용합니다. 결과적으로 *Word2Vec*과 유사하게 의미론적 관계를 잘 포착합니다.

5.6 절차/방법: 텍스트 처리 실습

이 섹션에서는 *scikit-learn*의 *TfidfVectorizer*와 *gensim*의 *Word2Vec*을 사용하여 텍스트를 처리하고 임베딩을 생성하는 방법을 단계별로 설명합니다.

5.6.1 1. TF-IDF 벡터화 (*scikit-learn TfidfVectorizer*)

*TfidfVectorizer*는 텍스트 문서 집합을 *TF-IDF* 특징 행렬로 변환하는 데 사용됩니다.

□ *TF-IDF* 벡터화 기본 절차

1. *TfidfVectorizer* 객체 초기화 (필요에 따라 파라미터 설정).
2. *fit()* 메서드로 문서 집합에 대한 어휘(*vocabulary*)를 학습.
3. *transform()* 메서드로 문서를 *TF-IDF* 행렬로 변환.
4. *get_feature_names_out()*으로 학습된 어휘 확인.
5. *toarray()*로 희소 행렬을 밀집 행렬로 변환하여 결과 확인.

1.1. TfidfVectorizer 주요 파라미터

*TfidfVectorizer*를 초기화할 때 다양한 파라미터를 설정하여 텍스트 처리 방식을 제어할 수 있습니다.

- *lowercase=True* (기본값): 모든 텍스트를 소문자로 변환합니다.
- *ngram_range=(1, 1)* (기본값): 1-그램(단어 하나)만 사용합니다. (1, 2)로 설정하면 1-그램과 2-그램을 모두 사용합니다.
- *stop_words=None* (기본값): 불용어를 제거하지 않습니다. 'english'로 설정하면 영어 불용어를 제거합니다.
- *token_pattern=r'(?u)\b\w\w+\b'* (기본값): 토큰(단어)을 추출하는 정규 표현식입니다. 기본값은 두 글자 이상의 단어만 토큰으로 인식합니다. (예: 'a'와 같은 한 글자 단어는 제외)
- *norm='l2'* (기본값): *TF-IDF* 벡터를 *L2* 노름으로 정규화하여 단위 벡터로 만듭니다.

□ TF-IDF 벡터화 코드 예시

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 # 문서 집합 코퍼스()
4 corpus = [
5     "a cat a dog and also my cats",
6     "a cat a dog and also my dogs",
7     "a dog a mouse and also my mice",
8     "a dog and also my dogs"
9 ]
10
11 # TfidfVectorizer 초기화 기본값( 사용)
12 # lowercase=True, ngram_range=(1,1), stop_words=None, token_pattern=r'(?u
13     )\b\w\w+\b', norm='l2'
14
15 vectorizer = TfidfVectorizer()
16
17 # 문서 집합에 하고fit 변환 (TF-IDF 행렬 생성)
18 tfidf_matrix = vectorizer.fit_transform(corpus)
19
20 # 학습된 어휘단어( 목록) 확인
21 feature_names = vectorizer.get_feature_names_out()
22 print("어휘:", feature_names)
23
24 # TF-IDF 행렬 출력 밀집( 행렬로 변환)
25 print("TF-IDF 행렬:\n", tfidf_matrix.toarray())

```

Listing 5.1: TF-IDF 벡터화 기본 예시

코드 실행 결과 분석

- 어휘 (*feature_names*): 'a'와 같은 한 글자 단어는 기본 *token_pattern*에 의해 제외되고, 'also', 'and', 'cat', 'cats', 'dog', 'dogs', 'mice', 'mouse', 'my'와 같은 두 글자 이상의 단어들이 알파벳 순서로 어휘에 포함됩니다.
- *TF-IDF* 행렬: 각 행은 하나의 문서를, 각 열은 어휘의 단어를 나타냅니다. 셀 값은 해당 문서에서 단어의 *TF-IDF* 점수입니다.
- 정규화 확인: *norm='l2'*가 기본값이므로, 각 문서 벡터의 *L2* 노름(길이)은 1이 됩니다. (예: 첫 번째 문서 벡터의 각 요소를 제공하여 더한 후 제곱근을 취하면 1이 나옴)

1.2. 불용어 (Stop Words) 제거

문서 구별에 도움이 되지 않는 흔한 단어(불용어)를 제거하여 차원을 줄이고 모델 성능을 향상시킬 수 있습니다.

□ 불용어 제거 코드 예시

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 corpus = [
4     "a cat a dog and also my cats",
5     "a cat a dog and also my dogs",
6     "a dog a mouse and also my mice",
7     "a dog and also my dogs"
8 ]
9
10 # TfidfVectorizer 초기화 영어( 불용어 제거)
11 vectorizer_sw = TfidfVectorizer(stop_words='english')
12
13 tfidf_matrix_sw = vectorizer_sw.fit_transform(corpus)
14 feature_names_sw = vectorizer_sw.get_feature_names_out()
15 print("불용어 제거 후 어휘:", feature_names_sw)
16 print("불용어 제거 후 TF-IDF 행렬:\n", tfidf_matrix_sw.toarray())
17
18 # scikit-learn 영어 불용어 목록 확인
19 from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
20 print("\nscikit-learn 영어 불용어 목록 일
    부():\n", list(ENGLISH_STOP_WORDS)[:10])

```

Listing 5.2: 영어 불용어 제거 예시

코드 실행 결과 분석

- 'a', 'and', 'also', 'my'와 같은 단어들이 불용어 목록에 포함되어 제거됩니다.
- 어휘의 크기가 'cat', 'cats', 'dog', 'dogs', 'mice', 'mouse' 등으로 훨씬 짧아집니다.
- 차원이 줄어들어 계산 효율성이 높아집니다.
- 사용자 정의 불용어: `stop_words` 파라미터에 리스트 형태로 직접 불용어 목록을 전달하여 사용자 정의 불용어를 제거할 수도 있습니다. (예: `stop_words=['cat', 'dog']`)

1.3. N-그램 적용

단어의 순서 정보를 일부 보존하기 위해 *N*-그램을 사용할 수 있습니다.

□ 2-그램 적용 코드 예시

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 corpus = [
4     "a cat a dog and also my cats",
5     "a cat a dog and also my dogs",
6     "a dog a mouse and also my mice",
7     "a dog and also my dogs"
8 ]
9
10 # TfidfVectorizer 초기화 2-그램만(2- 사용)
11 vectorizer_ngram = TfidfVectorizer(ngram_range=(2, 2))
12
13 tfidf_matrix_ngram = vectorizer_ngram.fit_transform(corpus)
14 feature_names_ngram = vectorizer_ngram.get_feature_names_out()
15 print("2-그램 어휘:", feature_names_ngram)
16 print("2-그램 TF-IDF 행렬:\n", tfidf_matrix_ngram.toarray())

```

Listing 5.3: 2-그램 TF-IDF 벡터화 예시

코드 실행 결과 분석

- 어휘가 'also my', 'and also', 'cat a', 'cat and', 'dog a', 'dog and', 'my cats', 'my dogs', 'my mice'와 같은 2-그램으로 구성됩니다.
- 어휘의 크기가 1-그램을 사용할 때보다 훨씬 커질 수 있습니다.
- 'not bad'와 같이 부정적인 의미를 가지는 구문을 하나의 토큰으로 처리하여 문맥을 더 잘 반영할 수 있습니다.
- `ngram_range=(1, 2)`로 설정하면 1-그램과 2-그램을 모두 어휘에 포함시킬 수 있습니다.

5.6.2 2. NLTK를 이용한 텍스트 전처리 (스테밍)

NLTK (Natural Language Toolkit)는 텍스트 전처리에 유용한 도구들을 제공합니다. 여기서는 스템밍 (Stemming)을 예시로 들어봅니다.

□ NLTK 스템밍 코드 예시

```

1  import nltk
2  from nltk.stem import PorterStemmer
3  from nltk.tokenize import word_tokenize
4  from sklearn.feature_extraction.text import TfidfVectorizer
5
6  # NLTK 데이터 다운로드 최초 ( 회1 실행)
7  # nltk.download('punkt')
8
9  corpus = [
10     "a cat a dog and also my cats",
11     "a cat a dog and also my dogs",
12     "a dog a mouse and also my mice",
13     "a dog and also my dogs"
14 ]
15
16 # Porter Stemmer 초기화
17 stemmer = PorterStemmer()
18
19 # 문서 전처리 함수 토큰화( -> 스템밍 -> 재결합)
20 def preprocess_document(doc):
21     tokens = word_tokenize(doc) # 단어 토큰화
22     stemmed_tokens = [stemmer.stem(token) for token in tokens] # 각 토큰
    에 스템밍 적용
23     return " ".join(stemmed_tokens) # 스템밍된 토큰들을 다시 문장으로 결합
24
25 # 코퍼스 내 모든 문서에 전처리 적용
26 preprocessed_corpus = [preprocess_document(doc) for doc in corpus]
27 print("전처리된 코퍼스:", preprocessed_corpus)
28
29 # 전처리된 문서에 TfidfVectorizer 적용
30 vectorizer_stem = TfidfVectorizer(token_pattern=r'(?u)\b\w+\b') # 한 글
    자 단어도 포함하도록 패턴 변경
31 tfidf_matrix_stem = vectorizer_stem.fit_transform(preprocessed_corpus)
32 feature_names_stem = vectorizer_stem.get_feature_names_out()
33 print("스테밍 후 어휘:", feature_names_stem)
34 print("스테밍 후 TF-IDF 행렬:\n", tfidf_matrix_stem.toarray())

```

Listing 5.4: NLTK를 이용한 스템밍 및 TF-IDF 적용

코드 실행 결과 분석

- 스테밍 효과: 'cats' → 'cat', 'dogs' → 'dog', 'mice' → 'mic' (*Porter Stemmer*의 결과)과 같이 단어의 접미사가 제거되어 어간(*stem*)만 남습니다.
- 어휘 감소: 'cat'과 'cats'가 'cat'으로 통합되면서 어휘의 크기가 줄어들고, 동일한 의미를 가진 단어들이 하나의 토큰으로 처리됩니다.
- *token_pattern* 변경: 기본 *token_pattern*은 두 글자 이상의 단어만 인식하므로, 'a'와 같은 단어를 포함시키기 위해 `r'(?u)\b\w+\b'`로 변경했습니다.
- 표제어 추출 (*Lemmatization*): 스테밍보다 더 정교하게 단어의 기본형(표제어)을 찾아주는 기법입니다. (예: 'better' → 'good'). *NLTK*의 *WordNetLemmatizer*를 사용할 수 있습니다.

5.7 3. Word2Vec 임베딩 (gensim Word2Vec)

gensim 라이브러리는 *Word2Vec* 모델을 쉽게 구현할 수 있도록 돕습니다.

□ Word2Vec 임베딩 기본 절차

1. 텍스트 데이터를 문장 단위로 토큰화하고, 각 문장을 단어 리스트로 만듭니다.
2. `gensim.models.Word2Vec` 객체 초기화 (주요 파라미터: `vector_size`, `window`, `min_count`).
3. `build_vocab()` 메서드로 어휘를 구축.
4. `train()` 메서드로 모델을 학습.
5. 학습된 모델에서 단어 임베딩 벡터 추출 및 유사 단어/유추 문제 해결.

3.1. Word2Vec 주요 파라미터

- *vector_size*: 임베딩 벡터의 차원 (예: 2, 100, 200). 실습에서는 시각화를 위해 2차원을 사용하지만, 실제로는 100 300차원을 권장합니다.
- *window*: 중심 단어 주변에서 고려할 단어의 최대 거리. (예: `window=5`는 중심 단어 좌우 5개 단어까지 고려)
- *min_count*: 모델이 학습할 단어의 최소 빈도. 이 값보다 적게 나타나는 단어는 무시됩니다.
- *sg*: 학습 알고리즘 선택. 0은 *CBOW*, 1은 *Skip-gram* (기본값).
- *epochs*: 훈련 반복 횟수.

□ Word2Vec 임베딩 코드 예시

```

1 import nltk
2 from nltk.tokenize import word_tokenize
3 from gensim.models import Word2Vec
4 import numpy as np
5
6 # NLTK 데이터 다운로드 최초 ( 회1 실행)
7 # nltk.download('punkt')
8
9 # 예시 문서 허리케인 ( 관련 뉴스)
10 documents = [
11     "Florida is bracing for Hurricane Ian, which is expected to make
12     landfall as a major storm.",
13     "Residents are urged to evacuate coastal areas as the storm
14     approaches.",
15     "Power outages are widespread, affecting thousands of homes.",
16     "The storm surge could cause significant flooding.",
17     "Emergency services are on high alert, preparing for the worst."
18 ]
19
20 # 문서 전처리 토큰화 ( 및 소문자 변환)
21 tokenized_sentences = [word_tokenize(doc.lower()) for doc in documents]
22 print("토큰화된 문장:", tokenized_sentences)
23
24 # Word2Vec 모델 학습
25 # vector_size=2: 시각화를 위해 차원2 임베딩 사용 실제로는 ( 더 높은 차원 사용)
26 # window=5: 중심 단어 좌우 개5 단어까지 문맥으로 고려
27 # min_count=1: 최소 번1 이상 나타나는 단어만 학습
28 model = Word2Vec(sentences=tokenized_sentences, vector_size=2, window=5,
29                  min_count=1, sg=1)
30
31 # 모델 학습
32 model.train(tokenized_sentences, total_examples=model.corpus_count,
33            epochs=model.epochs)
34
35 # 학습된 어휘 확인
36 vocabulary = list(model.wv.index_to_key)
37 print("\n학습된 n 어휘 일부():", vocabulary[:10])
38
39 # 특정 단어의 임베딩 벡터 추출
40 word_vectors = {word: model.wv[word] for word in ['storm', 'power', '
    residents', 'outages']}
41 print("\n'storm' 임베딩 벡터:", word_vectors['storm'])
42 print("'power' 임베딩 벡터:", word_vectors['power'])
43 print("'residents' 임베딩 벡터:", word_vectors['residents'])
44 print("'outages' 임베딩 벡터:", word_vectors['outages'])

```

Listing 5.5: Word2Vec 모델 학습 및 임베딩 추출

코드 실행 결과 분석

- 토큰화된 문장: 각 문장이 단어 리스트로 분리되고 소문자로 변환됩니다.
- 학습된 어휘: 모델이 학습한 단어들이 어휘에 포함됩니다.
- 임베딩 벡터: 'storm', 'power', 'residents', 'outages'와 같은 단어들이 2차원 벡터로 표현됩니다. 이 벡터 값들은 모델 학습 과정에서 단어의 문맥을 반영하여 결정됩니다.
- 시각화: 2차원 임베딩은 X-Y 평면에 단어를 점으로 찍어 시각화할 수 있습니다. 유사한 단어는 가까이 위치하고, 관련 없는 단어는 멀리 떨어져 있을 것입니다.

3.2. 단어 유사성 계산

Word2Vec 모델은 단어 간의 코사인 유사성을 기반으로 가장 유사한 단어를 찾아주는 기능을 제공합니다.

□ 단어 유사성 계산 코드 예시

```

1 # 'destruction과 ' 가장 유사한 단어 찾기
2 # 예시( 텍스트에 'destruction이' 없으므로, 강의에서 언급된 'distraction으
  로' 대체)
3 # 또는 'storm과 ' 유사한 단어를 찾아봅니다.
4 similar_words_storm = model.wv.most_similar('storm', topn=2)
5 print("\n'storm과 ' 가장 유사한 단어 상위( 개2):", similar_words_storm)
6
7 # 결과 예시: [('waters', 0.99...), ('frustration', 0.98...)]
8 # 이는 'storm과 ' 'waters가' 벡터 공간에서 가장 가까운코사인( 유사성이 높은) 단
  어임을 의미합니다.

```

Listing 5.6: 가장 유사한 단어 찾기

3.3. 단어 유추 (Analogy) 해결

Word2Vec 모델은 벡터 연산을 통해 단어 유추 문제(예: $King - Man + Woman = ?$)를 해결할 수 있습니다.

단어 유추 문제 해결 원리

단어 유추 문제는 "A는 B에 해당하는 것처럼, C는 D에 해당한다"는 관계를 찾는 것입니다. 이를 벡터 연산으로 표현하면 $vec(A) - vec(B) \approx vec(C) - vec(D)$ 가 됩니다. 따라서 D를 찾으려면 $vec(D) \approx vec(C) - vec(A) + vec(B)$ 를 계산한 후, 이 결과 벡터와 가장 가까운 단어를 찾습니다.

□ 단어 유추 해결 코드 예시

```

1 # 예시 텍스트에 'king', 'man', 'woman', 'queen'이 없으므로,
2 # 강의에서 언급된 'power', 'outage', 'fuel'을 사용하여 유추 문제를 만들어보
  다.
3 # "power에 outage 해당하는 것처럼, fuel 무엇에 해당하는가?"
4 # 즉,  $\text{vec}(\text{outage}) - \text{vec}(\text{power}) + \text{vec}(\text{fuel})$ 에 가장 가까운 단어를 찾습니다.
5
6 # model.wv.most_similar(positive=[vec(C), vec(B)], negative=[vec(A)],
  topn=1)
7 #  $(C + B - A)$ 
8 analogy_result = model.wv.most_similar(positive=['outage', 'fuel'],
  negative=['power'], topn=1)
9 print("\n'power'가 'outage'에 해당하는 것처럼, 'fuel'은 무엇에 해당하는
  가?:", analogy_result)
10
11 # 결과 예시: [('remained', 0.99...)]
12 # 이는 텍스트의 문맥상 'power outage'와 'fuel remained'가 유사한 관계를 가
  질 수 있음을 시사합니다.
13 # 매우 (작은 코퍼스이므로 결과가 직관적이지 않을 수 있습니다.)

```

Listing 5.7: 단어 유추 문제 해결 (King - Man + Woman

5.8 실습/코드/오류와 해결

이 섹션에서는 강의 중 발생한 코드 관련 논의와 실제 구현 시 발생할 수 있는 일반적인 오류 및 해결책을 다룹니다.

5.8.1 1. TF-IDF 벡터 정규화 논의

강의 중 *TF-IDF* 벡터의 정규화에 대한 논의가 있었습니다.

TF-IDF 벡터 정규화 불일치 논의

강의 슬라이드에 제시된 *TF-IDF* 값들이 *L2* 노름으로 정규화되지 않은 것처럼 보였으나, *scikit-learn*의 *TfidfVectorizer*는 *norm='l2'*를 기본값으로 사용하여 각 문서 벡터를 *L2* 노름으로 정규화합니다. 학생의 코드 실행 결과, 벡터의 *L2* 노름이 1에 가까운 값(예: $\sqrt{0.8944^2 + 0.4472^2} \approx 1$)이 나왔음을 확인했습니다. 이는 슬라이드의 숫자가 정규화 전의 값이거나, 강사의 타이핑 오류였음을 의미합니다.

- 증상: 슬라이드에 제시된 *TF-IDF* 벡터 값들의 제곱합의 제곱근이 1이 아니었습니다.
- 원인 가설:
 1. *TfidfVectorizer*의 *norm* 파라미터가 *None*으로 설정되었을 가능성.
 2. 슬라이드에 제시된 숫자가 정규화되기 전의 값일 가능성.
 3. 강사의 타이핑 오류.
- 수정 방법 및 결과 검증:

1. *TfidfVectorizer*의 공식 문서를 확인한 결과, *norm='l2'*가 기본값임을 확인했습니다.
 2. 학생이 직접 코드를 실행하여 확인한 결과, *L2* 노름으로 정규화된 단위 벡터가 출력됨을 확인했습니다.
- 결론: *TfidfVectorizer*는 기본적으로 *L2* 노름 정규화를 수행하여 각 문서 벡터의 길이를 1로 만듭니다. 이는 코사인 유사성 계산 시 벡터의 방향성만을 고려하게 하여 문서 간 유사도 측정에 유리합니다.

5.8.2 2. ChatGPT 언어별 성능 논의

강의 말미에 *ChatGPT*의 언어별 성능에 대한 흥미로운 논의가 있었습니다.

□ *ChatGPT* 언어별 성능 및 특징

*ChatGPT*와 같은 대규모 언어 모델(*LLM*)은 훈련 데이터의 양과 질에 따라 언어별로 성능 차이를 보입니다. 영어는 방대한 양의 고품질 텍스트 데이터로 훈련되어 매우 자연스럽게 정확한 응답을 생성하는 반면, 다른 언어들은 상대적으로 데이터가 부족하여 미묘한 차이가 발생할 수 있습니다.

- 영어: 매우 사실적이고 사람과 대화하는 듯한 자연스러움을 보입니다. 문법적 오류가 거의 없으며, 다양한 문맥을 이해하고 적절히 반응합니다.
- 아랍어: 특정 방언까지 구사할 수 있을 정도로 뛰어난 성능을 보입니다. 다만, 남성/여성 구분이 중요한 일부 단어에서 미묘한 오류가 발생할 수 있습니다.
- 힌디어: 매우 세련되고 정제된 언어를 구사하여, 마치 '교수님'이 말하는 듯한 느낌을 줍니다. 이는 훈련 데이터가 주로 책이나 공식 문서와 같은 정제된 텍스트였기 때문으로 추정됩니다. 일상적인 대화체와는 다를 수 있습니다.
- 일반적인 문제점:
 - 데이터 부족: 영어 외의 언어는 훈련 데이터의 양이 적어 성능이 떨어질 수 있습니다.
 - 편향 반영: 훈련 데이터에 존재하는 편향(예: 특정 성별에 대한 고정관념)이 모델의 응답에 반영될 수 있습니다.
 - 최신 정보 부족: 특정 시점까지의 데이터로 훈련되므로, 그 이후의 최신 정보는 알지 못합니다.
- 활용 팁:
 - 프롬프트 개선 요청: *ChatGPT*에게 "프롬프트를 어떻게 개선할 수 있을까요?"라고 물어보면 더 나은 질문을 만드는 데 도움을 받을 수 있습니다.
 - 오류 지적: "어떤 실수를 했나요?" 또는 "어떤 부정확한 정보를 제공했나요?"라고 물으면 스스로 오류를 지적하고 수정하는 모습을 보입니다.
 - 역할 부여: 특정 역할(예: "전문가처럼 설명해 줘")을 부여하면 그에 맞는 어조와 깊이로 답변을 생성합니다.

5.9 체크리스트

□ 학습 및 과제 점검 체크리스트

이 체크리스트를 활용하여 강의 내용을 효과적으로 복습하고 과제를 완벽하게 준비하세요.

- 개념 이해
 - 가중치 공유의 개념과 *RNN*, *CNN*에서의 적용 방식 이해
 - N*-그램 토큰화의 필요성 및 오버랩핑 *N*-그램의 의미 이해
 - TF-IDF*의 *TF*, *IDF* 각각의 정의와 계산 방법 이해
 - TF-IDF* 계산 시 훈련/테스트 데이터셋 활용 방식 이해 (*IDF*는 훈련 데이터 기반)
 - 단어 임베딩의 필요성 (원-핫 인코딩의 한계) 이해
 - 임베딩 레이어의 동작 원리 및 임베딩 벡터의 의미론적 특성 이해
 - Word2Vec* (*CBOW*, *Skip-gram*) 및 *GloVe*의 기본 원리 이해
- 실습 및 코드
 - TfidfVectorizer*를 이용한 *TF-IDF* 행렬 생성 코드 구현 및 결과 해석
 - TfidfVectorizer*의 *ngram_range*, *stop_words*, *token_pattern* 파라미터 활용
 - NLTK*를 이용한 텍스트 토큰화 및 스테밍/표제어 추출 전처리 코드 구현
 - gensim.models.Word2Vec*을 이용한 단어 임베딩 모델 학습 코드 구현
 - 학습된 *Word2Vec* 모델에서 단어 임베딩 벡터 추출 및 시각화 (개념적)
 - Word2Vec* 모델의 *most_similar()* 메서드를 이용한 유사 단어 찾기 및 단어 유추 문제 해결
- 과제 준비
 - 다음 과제 마감일 (다다음 주 일요일) 확인 및 일정 계획
 - 과제에서 요구하는 *TF-IDF* 및 임베딩 적용 방식 확인 (예: 10,000개 단어 사용)
 - 90% 정확도 달성 목표 확인 (지난 과제보다 쉬울 수 있음)
- 심화 학습
 - TF-IDF*의 다양한 변형 (로그 *TF*, 스무스 *IDF* 등) 및 벡터 정규화의 효과 탐색
 - 단어 임베딩의 한계 (*OOV*, 정적 임베딩, 편향) 및 해결 방안 추가 조사
 - Word2Vec*과 *GloVe*의 내부 알고리즘 상세 비교

5.10 FAQ (자주 묻는 질문)

□ FAQ: 초심자를 위한 질문과 답변

초심자들이 텍스트 처리 및 임베딩을 학습하면서 흔히 가질 수 있는 질문들을 모아 답변합니다.

- Q: *TF-IDF*에서 *IDF*를 계산할 때 왜 테스트 데이터셋이 아닌 훈련 데이터셋을 사용하나

요? *A: IDF*는 단어 자체의 희귀성(전체 코퍼스 내에서의 분포)을 측정하는 값입니다. 이는 모델이 학습하는 '지식'의 일부로 간주됩니다. 테스트 데이터셋은 모델이 '처음 보는' 데이터여야 하므로, 테스트 데이터셋의 정보를 미리 사용하여 *IDF*를 계산하는 것은 모델의 일반화 성능을 왜곡할 수 있습니다. 또한, 테스트 데이터셋이 단일 문서일 경우 *IDF*를 계산하는 것이 불가능합니다. 따라서 *IDF*는 훈련 데이터셋에서 한 번만 계산하여 고정적으로 사용합니다.

- *Q: TF-IDF* 벡터를 *L2* 노름으로 정규화하는 것이 왜 중요한가요? *A: L2* 노름 정규화는 *TF-IDF* 벡터의 길이를 1로 만들어 단위 벡터로 변환합니다. 이는 문서의 길이가 길다고 해서 *TF-IDF* 벡터의 길이가 무조건 길어지는 것을 방지하여, 문서 길이의 영향을 줄입니다. 또한, 벡터의 '방향'에 초점을 맞추게 되어, 문서 간의 유사도를 코사인 유사성으로 측정할 때 벡터의 크기가 아닌 순수한 방향성만을 비교할 수 있게 합니다. 이는 모델의 안정성과 성능 향상에 기여합니다.
- *Q: Word2Vec*과 같은 단어 임베딩이 *Bag-of-Words*나 *TF-IDF*보다 왜 더 좋다고 평가되나요? *A: Bag-of-Words*와 *TF-IDF*는 단어의 출현 빈도나 희귀성만을 고려하며, 단어 간의 의미론적 관계(예: '왕'과 '여왕'의 관계)를 전혀 포착하지 못합니다. 반면, 단어 임베딩은 단어를 저차원의 밀집 벡터로 표현하면서 단어의 문맥 정보를 학습하여, 유사한 의미를 가진 단어들이 벡터 공간에서 가깝게 위치하도록 만듭니다. 이는 모델이 텍스트의 의미를 더 깊이 이해하고, 번역, 감성 분석 등 다양한 *NLP* 태스크에서 훨씬 뛰어난 성능을 발휘하게 합니다.
- *Q: 임베딩 레이어를 신경망에 직접 추가하여 학습하는 것과 Word2Vec*처럼 별도로 사전 학습된 임베딩을 사용하는 것의 차이는 무엇인가요? *A: 신경망에 임베딩 레이어를 직접 추가하여 학습하는 방식은 '작업 특정(task-specific)' 임베딩을 생성합니다. 즉, 현재 해결하려는 특정 문제(예: 영화 리뷰 분류)에 최적화된 임베딩을 학습합니다. 이 방식은 문맥 정보를 명시적으로 고려하지 않고, 오직 최종 비용 함수를 최소화하는 방향으로 임베딩을 조정합니다. 반면, Word2Vec과 같은 사전 학습된 임베딩은 대규모 코퍼스에서 '단어의 문맥'을 예측하는 비지도 학습 방식으로 학습됩니다. 이는 단어 간의 일반적인 의미론적 관계를 포착하며, 다양한 작업에 전이 학습(transfer learning)될 수 있습니다. Word2Vec은 단어 간의 상관관계를 명시적으로 고려하므로, 동의어 등이 유사한 벡터를 가질 가능성이 더 높습니다.*
- *Q: 단어 임베딩의 '편향 반영' 문제는 어떻게 발생하며 왜 중요한가요? A: 단어 임베딩은 훈련 데이터셋의 텍스트에서 단어의 사용 패턴을 학습합니다. 만약 훈련 텍스트에 사회적 편향(예: '의사'라는 단어가 남성 대명사와 더 자주 함께 사용됨)이 존재한다면, 임베딩 모델은 이러한 편향을 그대로 학습하여 '의사'와 '남성'의 임베딩 벡터를 가깝게 만들 수 있습니다. 이는 모델이 편향된 예측(예: 직업 추천 시 성별 편향)을 하거나, 번역 시 성별을 잘못 유추하는 등의 문제를 야기할 수 있습니다. 따라서 이러한 편향을 인식하고 제거하려는 노력이 중요합니다.*

5.11 빠르게 훑어보기 (1페이지 요약)

□ 핵심 개념 요약: 텍스트 처리 및 임베딩

CSCI89 Lecture 5

1. 가중치 공유 (Weight Sharing)

- 개념: 신경망의 여러 연결에서 동일 가중치 사용. 파라미터 수 감소, 패턴 재사용.
- 적용:
 - *FNN*: 없음. 각 연결 고유 가중치.
 - *RNN*: 있음. 시퀀스 데이터에서 시간 독립적 패턴 학습.
 - *CNN*: 있음. 이미지에서 위치 독립적 특징 추출 (필터 슬라이딩).

2. N-그램 토큰화 (N-gram Tokenization)

- 개념: N 개의 연속 단어 묶음. 단어 순서 정보 일부 보존.
- 예시 (2-그램): "Port assembly line" → "Port assembly", "assembly line".
- 개수: 길이 L 에서 N -그램은 $L - N + 1$ 개. 실제 어휘는 1-그램보다 N -그램이 훨씬 많음.

3. TF-IDF (Term Frequency-Inverse Document Frequency)

- 목표: 단어의 중요도 측정. *Bag-of-Words*의 한계 극복.
- *TF* (Term Frequency): 문서 내 단어 빈도 / 문서 총 단어 수.
- *IDF* (Inverse Document Frequency): $\log(\text{전체 문서 수} / \text{단어 포함 문서 수})$. 희귀 할수록 높음.
- 계산: $TF \times IDF$.
- 특징: *IDF*는 훈련 데이터셋 기반으로 한 번만 계산.
- 정규화: $L2$ 노름으로 벡터 길이 1로 정규화 (단위 벡터). 코사인 유사성 측정에 유리.

4. 단어 임베딩 (Word Embedding)

- 목표: 단어를 저차원 밀집 벡터로 표현, 의미 관계 포착.
- 원-핫 인코딩 한계: 희소성, 의미 관계 미반영.
- 임베딩 레이어: 신경망의 한 층으로 학습. 가중치 행렬이 임베딩 행렬.
- 의미론적 특성: 벡터 연산으로 단어 관계 표현 (예: $King - Man + Woman = Queen$).
- 한계: *OOV*, 정적 임베딩, 편향 반영, 데이터 부족 언어.
- 주요 모델:
 - *Word2Vec*: 주변 단어로 중심 단어 예측 (*CBOW*) 또는 중심 단어로 주변 단어 예측 (*Skip-gram*).
 - *GloVe*: 코퍼스 전체의 동시 출현 통계 기반.

5. 실습 도구

- *TF-IDF*: `sklearn.feature_extraction.text.TfidfVectorizer`
- 전처리: *NLTK* (토큰화, 스테밍)
- *Word2Vec*: `gensim.models.Word2Vec`

Chapter 6

용어 정리

강의명: *CSCI89*

주차: *Lecture 6*

교수명: *UIUC Student*

목적: 본 문서는 *CSCI89 Lecture 6* 강의 내용을 바탕으로 작성되었습니다. 주요 내용은 다음과 같습니다:

- 퀴즈 복습: *TF-IDF*, 의미론적 드리프트, 코사인 유사도 계산.
- 문자 임베딩 (*Character Embeddings*): 단어 임베딩의 한계를 극복하고, 철자 오류나 형태론적으로 복잡한 언어를 처리하는 방법.
- 오토인코더 (*Autoencoders*): 데이터의 효율적인 표현 학습을 위한 비지도 학습 모델.
- 스택형 오토인코더 (*Stacked Autoencoders*): 여러 계층을 가진 오토인코더의 훈련 방법.
- 희소 오토인코더 (*Sparse Autoencoders*): 잠재 공간의 희소성을 강제하여 효율성을 높이는 방법 (*L1* 정규화, *KL* 발산).
- 변이형 오토인코더 (*Variational Autoencoders, VAEs*): 잠재 공간을 구조화하고 새로운 데이터를 생성하는 방법.

이 문서는 비전공자도 쉽게 이해할 수 있도록 상세한 설명과 예시를 포함합니다.

6.1 용어 정리

다음 표는 본 문서에서 사용되는 주요 용어들을 쉽게 설명합니다.

용어	쉬운 설명	원어	비고
<i>TF-IDF</i>	특정 문서에서 단어가 얼마나 중요한지 나타내는 값	<i>Term Frequency-Inverse Document Frequency</i>	정보 검색 및 텍스트 마이닝에 사용
<i>TF</i>	특정 단어가 문서 내에서 얼마나 자주 나타나는지	<i>Term Frequency</i>	단어의 빈도
<i>IDF</i>	특정 단어가 전체 문서 집합에서 얼마나 희귀한지	<i>Inverse Document Frequency</i>	단어의 희귀성
의미론적 드리프트	단어의 의미가 시간이 지남에 따라 변하는 현상	<i>Semantic Drift</i>	정적 임베딩의 한계
문맥 비민감성	단어가 여러 의미를 가질 때 문맥을 구분하지 못하는 것	<i>Context Insensitivity</i>	정적 임베딩의 한계
코사인 유사도	두 벡터가 얼마나 유사한지 각도를 통해 측정	<i>Cosine Similarity</i>	임베딩 벡터 간 유사도 측정
문자 임베딩	단어가 아닌 문자를 벡터로 표현하는 기법	<i>Character Embedding</i>	<i>OOV</i> , 철자 오류, 형태론적 복잡성 해결
<i>OOV</i>	모델 훈련 시 보지 못했던 단어	<i>Out-Of-Vocabulary</i>	
오토인코더	입력 데이터를 압축하고 다시 복원하는 신경망	<i>Autoencoder</i>	비지도 학습, 차원 축소, 특징 학습
인코더	입력 데이터를 압축된 잠재 표현으로 변환하는 부분	<i>Encoder</i>	
디코더	압축된 잠재 표현을 원래 데이터로 복원하는 부분	<i>Decoder</i>	
잠재 공간	인코더가 생성한 압축된 데이터 표현 공간	<i>Latent Space / Coding Space</i>	
재구성 손실	원본 입력과 복원된 출력 간의 차이	<i>Reconstruction Loss</i>	오토인코더의 학습 목표
스택형 오토인코더	여러 은닉 계층을 가진 오토인코더	<i>Stacked Autoencoder</i>	심층 학습
희소 오토인코더	잠재 표현의 일부 뉴런만 활성화되도록 강제하는 오토인코더	<i>Sparse Autoencoder</i>	과적합 방지, 특징 추출
<i>KL</i> 발산	두 확률 분포 간의 차이를 측정하는 통계량	<i>Kullback-Leibler Divergence</i>	희소성 정규화, VAE에 사용
변이형 오토인코더	잠재 공간을 확률 분포로 모델링하여 생성 능력을 갖춘 오토인코더	<i>Variational Autoencoder (VAE)</i>	데이터 생성, 잠재 공간 구조화
<i>t-SNE</i>	고차원 데이터를 2차원 또는 3차원으로 시각화하는 기법	<i>t-Distributed Stochastic Neighbor Embedding</i>	시각화 도구, 차원 축소 아님

6.2 퀴즈 복습: 핵심 개념 이해

이번 섹션에서는 지난 강의에서 다룬 핵심 개념들을 퀴즈를 통해 복습하고, 각 개념의 중요성과 계산 방법을 상세히 설명합니다.

6.2.1 TF-IDF (Term Frequency-Inverse Document Frequency)

*TF-IDF*는 문서 내 단어의 중요도를 측정하는 통계적 척도입니다. 특정 단어가 문서에 얼마나 자주 나타나는지(*TF*)와 전체 코퍼스에서 얼마나 희귀한지(*IDF*)를 모두 고려합니다.

핵심 정보

단순히 단어 빈도만으로는 문서 내 단어의 중요도를 정확히 파악하기 어렵습니다. 예를 들어, '*the*'와 같은 불용어는 자주 나타나지만 문서의 의미를 크게 좌우하지 않습니다. 반면, 특정 전문 용어는 드물게 나타나지만 해당 문서의 핵심 주제를 나타낼 수 있습니다. *TF-IDF*는 이러한 점을 보완하여 단어의 실제 중요도를 반영합니다.

TF (Term Frequency): 단어 빈도

*TF*는 특정 단어가 주어진 문서 내에서 얼마나 자주 나타나는지를 측정합니다.

- 한 줄 핵심 요약: 특정 문서에서 특정 단어가 얼마나 많이 등장하는가.
- 직관적 예시: "사과 바나나 사과"라는 문서에서 '사과'는 2번 등장합니다.
- 기술적 설명:

$$TF(t, d) = \frac{\text{문서 } d \text{ 내 단어 } t \text{의 등장 횟수}}{\text{문서 } d \text{ 내 전체 단어 수}}$$

여기서 t 는 단어(*term*), d 는 문서(*document*)를 의미합니다.

IDF (Inverse Document Frequency): 역문서 빈도

IDF는 특정 단어가 전체 코퍼스(문서 집합)에서 얼마나 희귀한지를 측정합니다. 희귀한 단어일수록 IDF 값이 높아져 중요도가 커집니다.

- 한 줄 핵심 요약: 전체 문서들 중에서 특정 단어가 얼마나 드물게 등장하는가.
- 직관적 예시: '사과'라는 단어가 100개의 문서 중 2개에만 있다면 매우 희귀하고 중요할 수 있습니다.
- 기술적 설명:

$$IDF(t, D) = \log \left(\frac{\text{전체 문서 수 } (N)}{\text{단어 } t \text{를 포함하는 문서 수 } (df)} \right)$$

여기서 D 는 전체 코퍼스(*corpus*), N 은 전체 문서 수, df 는 단어 t 를 포함하는 문서의 수를 의미합니다. 로그 함수를 사용하는 이유는 단어 빈도수의 차이를 완화하고, 값이 너무 커지는 것을 방지하기 위함입니다.

핵심 정보

강의에서 로그 함수의 밑(*base*)에 대한 질문이 있었습니다. 자연로그($\ln$, 밑이 e)를 사용하든 상용로그($\log_{10}$, 밑이 10)를 사용하든 결과는 본질적으로 동일합니다. 이는 로그의 밑 변환 공식 $\log_a b = \frac{\ln b}{\ln a}$ 에 따라, 밑이 바뀌면 단순히 상수배가 되기 때문입니다. 최종적으로 임베딩 벡터를 정규화(*normalize*)하는 경우, 이 상수배는 결과에 아무런 영향을 미치지 않습니다.

TF-IDF 계산 예시

다음은 주어진 코퍼스에서 'apple' 단어의 TF-IDF를 계산하는 예시입니다.

예제

코퍼스 (전체 문서 집합):

- 문서 1: "apple banana apple"
- 문서 2: "banana orange"
- 문서 3: "apple orange banana"

계산 목표: 문서 1에서 'apple' 단어의 *TF-IDF* 값

1. *TF* (*Term Frequency*) 계산:

- 문서 1 내 'apple' 등장 횟수: 2회
- 문서 1 내 전체 단어 수: 3개 ('apple', 'banana', 'apple')
- $TF(\text{apple}, \text{문서 1}) = \frac{2}{3}$

2. *IDF* (*Inverse Document Frequency*) 계산:

- 전체 문서 수 (*N*): 3개
- 'apple'을 포함하는 문서 수 (*df*): 문서 1, 문서 3 (총 2개)
- $IDF(\text{apple}, \text{코퍼스}) = \log\left(\frac{3}{2}\right) = \log(1.5)$

3. *TF-IDF* 계산:

- $TF-IDF(\text{apple}, \text{문서 1}) = TF(\text{apple}, \text{문서 1}) \times IDF(\text{apple}, \text{코퍼스})$
- $TF-IDF(\text{apple}, \text{문서 1}) = \frac{2}{3} \times \log(1.5) \approx 0.6667 \times 0.4055 \approx 0.2703$

주의

*Python*의 *TfidfVectorizer*와 같은 라이브러리는 *TF-IDF*를 계산할 때 다양한 변형 (예: 스무딩을 위한 +1 추가, 이중 정규화, 로그 밑 변경)을 사용합니다. 따라서 직접 계산한 값과 라이브러리 결과가 다를 수 있습니다. 이는 구현 방식의 차이일 뿐, 기본 원리는 동일합니다.

6.2.2 정적 단어 임베딩의 한계: 의미론적 드리프트와 문맥 비민감성

정적 단어 임베딩 (*Static Word Embeddings*)은 한 번 학습되면 고정된 단어 벡터를 사용합니다. 이는 언어의 변화나 단어의 다의성을 반영하지 못하는 한계를 가집니다.

의미론적 드리프트 (Semantic Drift)

- 한 줄 핵심 요약: 단어의 의미가 시간이 흐름에 따라 변하는 현상.
- 직관적 예시: 과거의 'cool'은 '시원하다'는 의미였지만, 현재는 '멋지다'는 의미로도 널리 사용됩니다.
- 기술적 설명: 정적 임베딩은 특정 시점의 코퍼스로 학습되므로, 10년 후 단어의 의미가 변하면 해당 단어의 임베딩 벡터는 더 이상 정확한 의미를 반영하지 못하게 됩니다. 이는 언어가 끊임없이 진화하기 때문에 발생하는 문제입니다.

문맥 비민감성 (Context Insensitivity)

- 한 줄 핵심 요약: 단어가 여러 의미를 가질 때, 문맥에 따라 다른 의미를 구분하지 못하고 하나의 고정된 벡터를 사용하는 문제.
- 직관적 예시: 'bank'는 '은행'과 '강둑'이라는 두 가지 의미를 가집니다. 정적 임베딩은 이 두 의미를 구분하지 못하고 'bank'에 대해 하나의 벡터만 제공합니다.
- 기술적 설명: 정적 임베딩은 단어 하나당 하나의 고정된 벡터를 할당합니다. 따라서 다의어 (*polysemous words*)의 경우, 문맥에 따라 달라지는 의미를 포착할 수 없습니다. 이는 현재 시점에서도 단어의 다양한 의미를 반영하지 못하는 문제입니다.

Table 6.1: 의미론적 드리프트와 문맥 비민감성 비교

특징	의미론적 드리프트	문맥 비민감성
시간 축	시간 경과에 따른 의미 변화	특정 시점 내 문맥에 따른 의미 변화
원인	언어의 진화	단어의 다의성
영향	오래된 임베딩의 유효성 감소	다의어 처리의 한계

6.2.3 코사인 유사도 (Cosine Similarity)

코사인 유사도는 두 벡터 간의 각도 코사인 값을 측정하여 벡터의 유사도를 판단하는 방법입니다. 각도가 작을수록(코사인 값이 1에 가까울수록) 두 벡터는 더 유사하다고 간주됩니다.

핵심 정보

단어 임베딩에서 단어들은 고차원 공간의 벡터로 표현됩니다. 이때 두 단어의 의미적 유사성은 이들 벡터 간의 '거리'나 '각도'로 측정될 수 있습니다. 코사인 유사도는 벡터의 크기(길이)보다는 방향에 초점을 맞춰, 두 단어가 얼마나 유사한 문맥에서 사용되는지를 효과적으로 나타냅니다. 이는 임베딩 학습 시 손실 함수(*loss function*)가 내적(*dot product*)과 관련되어 있기 때문에 자연스럽게 유사도 척도로 사용됩니다.

코사인 유사도 공식

두 벡터 A와 B 사이의 코사인 유사도는 다음과 같이 정의됩니다.

$$\text{Cosine Similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

여기서 $\mathbf{A} \cdot \mathbf{B}$ 는 두 벡터의 내적(*dot product*)을, $\|\mathbf{A}\|$ 와 $\|\mathbf{B}\|$ 는 각 벡터의 유클리드 노름(*Euclidean norm*, 길이)을 의미합니다.

주의

두 벡터 중 하나라도 0 벡터(0)인 경우, 해당 벡터의 길이가 0이 되어 분모가 0이 됩니다. 이 경우 각도가 정의되지 않으므로, 코사인 유사도를 계산할 수 없습니다. 따라서 벡터의 길이가 0이 아님을 가정합니다.

코사인 유사도 계산 예시

다음은 'cat' 단어와 'dog', 'mouse' 단어 간의 코사인 유사도를 계산하여 어떤 단어가 'cat' 과 가장 유사한지 판단하는 예시입니다.

예제

주어진 단어 벡터:

- **Cat:** $C = (2, 3)$
- **Dog:** $D = (5, 7)$
- **Mouse:** $M = (1, 2)$

1. Cat과 Dog 간의 코사인 유사도:

- 내적: $C \cdot D = (2 \times 5) + (3 \times 7) = 10 + 21 = 31$
- **Cat** 벡터 길이: $\|C\| = \sqrt{2^2 + 3^2} = \sqrt{4 + 9} = \sqrt{13} \approx 3.6056$
- **Dog** 벡터 길이: $\|D\| = \sqrt{5^2 + 7^2} = \sqrt{25 + 49} = \sqrt{74} \approx 8.6023$
- 코사인 유사도: $\frac{31}{\sqrt{13} \times \sqrt{74}} = \frac{31}{\sqrt{962}} \approx \frac{31}{31.0161} \approx 0.99948$
- 강의에서 제시된 값: **0.9948** (계산 도구의 반올림 또는 내부 로직 차이로 인한 미세한 오차 가능성)

2. Cat과 Mouse 간의 코사인 유사도:

- 내적: $C \cdot M = (2 \times 1) + (3 \times 2) = 2 + 6 = 8$
- **Cat** 벡터 길이: $\|C\| = \sqrt{13} \approx 3.6056$ (위와 동일)
- **Mouse** 벡터 길이: $\|M\| = \sqrt{1^2 + 2^2} = \sqrt{1 + 4} = \sqrt{5} \approx 2.2361$
- 코사인 유사도: $\frac{8}{\sqrt{13} \times \sqrt{5}} = \frac{8}{\sqrt{65}} \approx \frac{8}{8.0623} \approx 0.99228$
- 강의에서 제시된 값: **0.9922** (계산 결과와 일치)

결론: Cat과 Dog의 유사도(약 0.9948)가 Cat과 Mouse의 유사도(약 0.9922)보다 높습니다. 따라서 'dog'이 'cat'에 더 가깝다고 할 수 있습니다. 이는 벡터의 길이가 아닌 방향(각도)을 기준으로 판단한 결과입니다.

6.3 문자 임베딩 (Character Embeddings)

이전까지는 단어(word)를 토큰으로 사용하여 임베딩을 생성했지만, 문자 임베딩은 텍스트를 개별 문자(character) 단위로 분해하여 임베딩을 생성하는 기법입니다.

6.3.1 문자 임베딩의 필요성

단어 임베딩은 강력하지만, 다음과 같은 한계가 있습니다.

- **OOV (Out-Of-Vocabulary) 문제:** 훈련 데이터에 없는 새로운 단어가 나타나면 해당 단어에 대한 임베딩을 생성할 수 없습니다. 이는 철자 오류(misspellings)가 있는 단어에도 해당됩니다.
- **형태론적 복잡성:** 터키어, 핀란드어와 같이 어미 변화가 매우 복잡하여 한 단어에 수십 가지

형태가 존재하는 언어의 경우, 단어 단위 토큰화는 어휘 크기를 폭발적으로 증가시킵니다.

- 접두사/접미사 처리: 단어 임베딩은 'run', 'running', 'runner'를 별개의 단어로 취급합니다. 문자 임베딩은 이러한 형태론적 정보를 더 잘 포착할 수 있습니다.
- 철자 오류 교정 및 이름 개체 인식 (*Named Entity Recognition, NER*): 문자 수준에서 텍스트를 분석하면 철자 오류를 감지하고 수정하거나, 혼란 시 보지 못한 고유 명사(이름, 장소 등)를 인식하는 데 유리합니다.

핵심 정보

문자 임베딩은 *OOV* 문제에 강하고, 형태론적 정보를 효과적으로 학습하며, 철자 오류나 새로운 단어에 대한 유연한 처리가 가능합니다. 이는 특히 형태론적으로 풍부한 언어나, 오타가 잦은 사용자 생성 텍스트 분석에 유용합니다.

6.3.2 문자 임베딩의 작동 방식

- 토큰화 (*Tokenization*): 텍스트를 단어 대신 개별 문자로 분리합니다. 예를 들어, "hello"는 'h', 'e', 'l', 'l', 'o'로 분리됩니다.
- 임베딩 벡터 생성: 각 문자는 고유한 임베딩 벡터로 변환됩니다. 이 벡터는 훈련 과정에서 학습됩니다.
- 시퀀스 처리: 문장이나 단어는 문자 임베딩 벡터들의 시퀀스로 표현됩니다. 이후 *RNN* (*Recurrent Neural Network*)이나 *CNN* (*Convolutional Neural Network*)과 같은 시퀀스 모델을 사용하여 처리됩니다.

핵심 정보

단어 임베딩의 경우 보통 100 300차원의 벡터를 사용하지만, 문자 임베딩은 어휘(*vocabulary*) 크기가 훨씬 작기 때문에 (알파벳 26자 + 숫자 + 특수문자 등) 8 20차원과 같이 더 낮은 차원의 벡터로도 충분히 좋은 성능을 낼 수 있습니다.

6.3.3 하이브리드 접근 방식 (Hybrid Approaches)

문자 임베딩과 단어 임베딩의 장점을 결합하기 위해 하이브리드 접근 방식이 자주 사용됩니다.

- 단어를 구성하는 문자들의 임베딩 시퀀스를 *RNN/CNN*으로 처리하여 단어 수준의 문자 임베딩을 생성합니다.
- 이 문자 기반 단어 임베딩과 기존의 단어 임베딩을 결합(예: 연결(*concatenate*))하여 최종 단어 표현을 만듭니다.
- 이렇게 하면 단어의 의미적 정보와 형태론적 정보를 모두 활용할 수 있습니다.

6.3.4 문자 임베딩 모델 구축 예시 (TensorFlow Keras)

다음은 *TensorFlow Keras*를 사용하여 문자 임베딩 기반의 신경망 모델을 구축하는 간단한 예시입니다.

예제

1. 데이터 준비:

- 텍스트 데이터: ["hello world", "machine learning", "deep learning"]
- 레이블: [1, 0, 0] (예시 분류 문제)

2. 문자 수준 토큰화:

```

1 import tensorflow as tf
2 from tensorflow.keras.preprocessing.text import Tokenizer
3 from tensorflow.keras.preprocessing.sequence import pad_sequences
4 from tensorflow.keras.models import Sequential
5 from tensorflow.keras.layers import Embedding, LSTM, Dense
6 import numpy as np
7
8 texts = ["hello world", "machine learning", "deep learning"]
9 labels = np.array([1, 0, 0])
10
11 # character_level=로 True 문자 수준 토큰화 설정
12 tokenizer = Tokenizer(char_level=True)
13 tokenizer.fit_on_texts(texts)
14
15 # 어휘 크기 계산 패딩( 토큰을 위해 +1)
16 vocab_size = len(tokenizer.word_index) + 1
17 print(f"문자 어휘 크기: {vocab_size}")
18 # 출력 예시: 문자 어휘 크
    기: 16 (h, e, l, o, w, r, d, m, a, c, i, n, g, p, _ 공
    백(), +1 for padding)
19
20 # 텍스트를 시퀀스로 변환
21 sequences = tokenizer.texts_to_sequences(texts)
22
23 # 시퀀스 패딩 가장( 긴 시퀀스 길이에 맞춰 으로 0 채움)
24 max_len = max(len(s) for s in sequences)
25 padded_sequences = pad_sequences(sequences, maxlen=max_len, padding='post
    ')
26 print(f"패딩된 시퀀스:\n{padded_sequences}")
27 # 출력 예시:
28 # [[ 1  2  3  3  4  5  6  7  0  0  0  0  0  0  0]
29 #  [ 8  9 10 11 12 13 14 15  0  0  0  0  0  0]
30 #  [15  2  2 14  5 12  2 14  6  0  0  0  0  0  0]]

```

Listing 6.1: 문자 수준 토큰화

- `tokenizer.word_index`는 각 문자에 할당된 인덱스를 보여줍니다. 예를 들어, 'e'는 1, 'l'은 2와 같이 빈도순으로 인덱스가 할당될 수 있습니다.
- `vocab_size`에 +1을 하는 이유는 패딩(padding)을 위한 특별한 토큰(보통 0번 인덱스)을 추가하기 위함입니다. `padding='post'`는 시퀀스 뒤에 0을 추가합니다.

3. 모델 구축:

```

1 embedding_dim = 8 # 문자 임베딩 차원 단어보다( 작음)
2
3 model = Sequential([
4     Embedding(vocab_size, embedding_dim, input_length=max_len),
5     LSTM(16), # LSTM 레이어
6     Dense(1, activation='sigmoid') # 이진 분류를 위한 출력 레이어

```


핵심 정보

`tokenizer.word_index`를 출력해보면, 'e'가 첫 번째, 'l'이 두 번째, 'n'이 세 번째와 같이 나타날 수 있습니다. 이는 토큰라이저가 기본적으로 코퍼스 내에서 문자의 빈도수에 따라 인덱스를 할당하기 때문입니다. 가장 자주 나타나는 문자가 가장 낮은 인덱스를 받습니다. 이는 나중에 차원 축소나 특징 처리를 할 때 편리할 수 있지만, 신경망 학습 자체에는 순서가 크게 중요하지 않습니다.

6.4 오토인코더 (Autoencoders)

오토인코더는 입력 데이터를 효율적인 저차원 표현(잠재 표현 또는 인코딩)으로 압축한 다음, 이 압축된 표현으로부터 원본 데이터를 최대한 가깝게 복원하도록 훈련되는 신경망입니다.

6.4.1 오토인코더의 동기 부여

- 효율적인 정보 표현: 복잡한 데이터를 더 간결하고 의미 있는 형태로 저장하거나 전달하는 방법을 찾는 것이 목표입니다. 예를 들어, 긴 숫자 시퀀스 70, 68, 66, 64, 62, 60은 70에서 시작하여 2씩 감소하는 6개의 숫자로 훨씬 효율적으로 기억할 수 있습니다.
- 데이터의 구조 이해: 오토인코더는 데이터 내재된 중요한 특징이나 패턴을 학습하여, 이를 통해 데이터의 본질적인 구조를 파악합니다.

핵심 정보

오토인코더는 입력 데이터 x 를 받아 이를 잠재 표현 c 로 인코딩한 다음, 이 c 로부터 다시 원본과 유사한 $\hat{x}$ 를 디코딩합니다. 이때 x 와 $\hat{x}$ 의 차이(재구성 손실)를 최소화하도록 학습됩니다. 인코딩 과정에서 병목(*bottleneck*) 구조를 강제하여, 모델이 데이터의 가장 중요한 특징만을 압축하도록 만듭니다.

6.4.2 오토인코더의 구조

오토인코더는 크게 두 부분으로 구성됩니다.

1. 인코더 (*Encoder*): 입력 데이터 x 를 받아 저차원의 잠재 표현 c (또는 코딩, 인코딩)로 변환합니다.
2. 디코더 (*Decoder*): 잠재 표현 c 를 받아 원본 데이터 x 와 유사한 재구성된 데이터 $\hat{x}$ 로 변환합니다.

Figure 6.1: 오토인코더의 기본 구조 (입력-인코더-병목-디코더-출력)

위 그림은 오토인코더의 기본 구조를 나타냅니다. 입력 데이터가 인코더를 통해 저차원의 병목(*bottleneck*) 계층으로 압축되고, 이 병목 계층의 표현이 디코더를 통해 다시 원본과 유사한 형태로 복원됩니다.

재구성 손실 (Reconstruction Loss)

오토인코더의 학습 목표는 입력 x 와 재구성된 출력 $\hat{x}$ 사이의 차이를 최소화하는 것입니다. 이를 재구성 손실 (*Reconstruction Loss*)이라고 합니다.

$$Loss = \sum_{k=1}^N (x_k - \hat{x}_k)^2 \quad (L2 \text{ 노름 예시})$$

이 손실 함수를 최소화함으로써, 인코더는 입력 데이터를 가장 효율적으로 압축하는 방법을 학습하게 됩니다.

언더컴플리트 오토인코더 (Undercomplete Autoencoder)

잠재 표현의 차원(병목 계층의 뉴런 수)이 입력 데이터의 차원보다 작은 경우를 언더컴플리트 오토인코더라고 합니다. 이는 모델이 입력 데이터의 중요한 특징만을 학습하도록 강제하여 차원 축소 효과를 얻습니다.

핵심 정보

만약 오토인코더의 활성화 함수가 선형이고, 손실 함수가 $L2$ 노름이라면, 이 오토인코더는 주성분 분석(*PCA*)과 매우 유사하게 작동합니다. *PCA*는 데이터의 분산을 가장 잘 설명하는 직교 축을 찾는 반면, 선형 오토인코더는 데이터를 가장 잘 재구성할 수 있는 저차원 공간을 학습합니다.

6.4.3 오토인코더의 활용

1. 차원 축소 (*Dimensionality Reduction*): 훈련된 오토인코더에서 디코더 부분을 제거하고 인코더만 사용하여 고차원 데이터를 저차원의 잠재 표현으로 변환합니다. 이 잠재 표현은 다른 머신러닝 모델의 입력으로 사용될 수 있습니다.
2. 특징 학습 (*Feature Learning*): 오토인코더는 데이터의 본질적인 특징을 비지도 방식으로 학습합니다. 이렇게 학습된 특징은 분류, 군집화 등 다양한 다운스트림 태스크에 활용될 수 있습니다.
3. 데이터 압축: 잠재 표현 자체가 원본 데이터의 압축된 형태이므로, 데이터 저장 및 전송에 활용될 수 있습니다.

핵심 정보

오토인코더는 입력 신호 자체를 재구성하는 것을 목표로 합니다. 반면, 단어 임베딩(예: *Word2Vec*)은 단어를 주변 문맥에 매핑하거나 분류하는 등 다른 태스크를 통해 단어 벡터를 학습합니다. 하지만 둘 다 고차원 데이터를 저차원 벡터로 표현한다는 공통점이 있습니다.

6.4.4 노이즈 제거 오토인코더 (Denoising Autoencoders)

노이즈 제거 오토인코더는 입력 데이터에 의도적으로 노이즈를 추가한 후, 이 노이즈가 제거된 원본 데이터를 복원하도록 훈련됩니다.

- 작동 방식:
 1. 원본 데이터 x 에 인위적으로 노이즈를 추가하여 $\tilde{x}$ 를 만듭니다.
 2. 오토인코더의 입력으로 $\tilde{x}$ 를 사용하고, 출력으로는 노이즈가 없는 원본 x 를 사용합니다.
 3. 모델은 노이즈가 있는 입력에서 노이즈를 제거하고 원본을 복원하는 방법을 학습합니다.
- 장점:
 - 강건한 특징 학습: 모델이 노이즈에 강인한 특징을 학습하게 됩니다.
 - 비지도 학습: 레이블된 데이터가 필요 없습니다. 수많은 이미지나 텍스트 데이터에 노이즈를 추가하여 스스로 학습할 수 있습니다.
 - 데이터 정제: 훈련된 모델은 실제 노이즈가 포함된 데이터를 정제하는 데 사용될 수 있습니다.

예제

텍스트 데이터의 경우, 문장에 의도적으로 오타를 삽입하거나 단어를 무작위로 삭제하는 방식으로 노이즈를 추가할 수 있습니다. 예를 들어, "Hello world"를 "Helo wrld"로 만든 후, 오토인코더가 이를 다시 "Hello world"로 복원하도록 학습시킵니다. 이렇게 훈련된 모델은 실제 오타가 포함된 문장을 교정하는 데 활용될 수 있습니다.

6.5 스택형 오토인코더 (Stacked Autoencoders)

스택형 오토인코더는 여러 개의 은닉 계층을 가진 심층 신경망 형태의 오토인코더입니다. 인코더와 디코더 모두 여러 계층으로 구성됩니다.

6.5.1 구조 및 훈련의 어려움

- 구조: 입력 계층, 여러 은닉 계층(인코더), 병목 계층, 여러 은닉 계층(디코더), 출력 계층으로 구성됩니다. 일반적으로 인코더와 디코더는 계층 수와 뉴런 수가 대칭적인 형태를 가집니다.
- 심층 신경망 훈련의 어려움: 심층 신경망은 기울기 소실(*vanishing gradients*) 또는 기울기 폭발(*exploding gradients*)과 같은 문제로 인해 훈련이 어렵습니다. 손실 함수의 표면이 매우 복잡하여 최적화가 쉽지 않습니다.

Figure 6.2: 스택형 오토인코더의 구조 예시 (784 -> 100 -> 30 -> 100 -> 784)

위 그림은 784차원 입력이 100차원, 30차원으로 압축된 후 다시 100차원, 784차원으로 복원되는 스택형 오토인코더의 예시를 보여줍니다. 30차원 계층이 병목 역할을 합니다.

6.5.2 탐욕적 계층별 사전 훈련 (Greedy Layer-wise Pre-training)

심층 스택형 오토인코더를 한 번에 훈련하는 대신, 각 계층을 개별적으로 사전 훈련하는 방법입니다. 이는 심층 신경망 훈련의 어려움을 완화하는 데 도움이 됩니다.

1. 1단계: 첫 번째 오토인코더를 훈련합니다. 이 오토인코더는 원본 입력을 받아 첫 번째 은닉 계층의 크기(예: $784 \rightarrow 100 \rightarrow 784$)로 압축 및 복원합니다.
2. 2단계: 1단계에서 훈련된 인코더의 출력(100차원 잠재 표현)을 새로운 입력으로 사용하여 두 번째 오토인코더를 훈련합니다. 이 오토인코더는 100차원 입력을 받아 두 번째 은닉 계층의 크기(예: $100 \rightarrow 30 \rightarrow 100$)로 압축 및 복원합니다. 이때 1단계의 인코더 가중치는 고정(freeze)할 수 있습니다.
3. N단계: 이 과정을 반복하여 모든 계층을 순차적으로 훈련합니다.
4. 미세 조정 (Fine-tuning): 모든 계층이 사전 훈련된 후, 전체 스택형 오토인코더를 다시 한 번 훈련하여 모든 가중치를 미세 조정할 수 있습니다.

핵심 정보

- 훈련 안정성: 각 계층을 얇은 오토인코더로 개별 훈련하므로, 심층 신경망의 훈련 불안정성 문제를 완화합니다.
- 더 나은 초기화: 사전 훈련을 통해 가중치가 무작위 초기화보다 더 의미 있는 값으로 초기화되어, 전체 네트워크의 최종 훈련 성능을 향상시킬 수 있습니다.

주의

최근에는 Adam과 같은 개선된 최적화 알고리즘, ReLU와 같은 활성화 함수, 배치 정규화 (Batch Normalization), 잔차 연결 (Residual Connections) 등 다양한 기술의 발전으로 인해 심층 신경망을 한 번에 훈련하는 것이 과거보다 훨씬 쉬워졌습니다. 따라서 계층별 사전 훈련의 필요성이 예전만큼 크지는 않지만, 여전히 특정 상황에서는 유용한 전략입니다.

6.5.3 스택형 오토인코더 예시 (MNIST/Fashion MNIST)

28×28 픽셀 이미지(784차원)를 스택형 오토인코더로 압축 및 복원하는 예시입니다.

예제

모델 구조:

- 입력: 784차원 (28×28 픽셀 이미지)
- 인코더: 100개 뉴런 $\rightarrow$ 30개 뉴런 (병목 계층)
- 디코더: 100개 뉴런 $\rightarrow$ 784개 뉴런 (출력 계층)

훈련:

- 입력 데이터: X_{train} (원본 이미지)
- 출력 데이터: X_{train} (원본 이미지, 재구성 목표)
- 에포크: 10회

결과: 훈련된 모델은 784차원의 이미지를 30차원의 잠재 표현으로 압축한 후 다시 784차원으로 복원합니다.

- 원본 이미지: 첫 번째 행에 원본 이미지가 표시됩니다.
 - 재구성 이미지: 두 번째 행에 오토인코더가 복원한 이미지가 표시됩니다.
- 결과를 보면, 30차원이라는 매우 낮은 차원으로 압축되었음에도 불구하고 원본 이미지의 형태(예: 신발, 가방 등)를 상당 부분 유지하고 있음을 확인할 수 있습니다. 이는 오토인코더가 이미지의 핵심 특징을 성공적으로 학습했음을 의미합니다.

6.5.4 t-SNE (t-Distributed Stochastic Neighbor Embedding)를 이용한 시각화

t-SNE는 고차원 데이터를 2차원 또는 3차원으로 시각화하는 데 사용되는 비선형 차원 축소 기법입니다.

핵심 정보

t-SNE는 주로 시각화를 위해 사용됩니다. 오토인코더가 생성한 고차원 잠재 표현(예: 30차원 인코딩)이 실제로 의미 있는 군집을 형성하는지, 데이터의 구조를 잘 반영하는지를 눈으로 확인하는 데 매우 유용합니다. t-SNE는 모델의 입력으로 사용되는 차원 축소 기법이 아니라는 점이 중요합니다.

t-SNE의 작동 원리

- 이웃 보존: t-SNE는 고차원 공간에서 가까운 점들은 저차원 공간에서도 가깝게, 멀리 떨어진 점들은 멀리 떨어지도록 매핑하려고 시도합니다. 특히 '지역적 이웃(local neighborhood)' 관계를 잘 보존하는 데 중점을 둡니다.
- 확률적 접근: 데이터 포인트 간의 유사도를 확률 분포로 모델링하고, 저차원 공간에서도 이 확률 분포를 최대한 유사하게 만들려고 합니다.
- 비결정론적: t-SNE는 확률적 알고리즘이므로, 실행할 때마다 결과가 약간씩 달라질 수 있습니다.

예제

스택형 오토인코더가 생성한 30차원 잠재 표현에 t -SNE를 적용하여 2차원 평면에 시각화하면, 유사한 이미지들(예: 모든 신발, 모든 티셔츠)이 서로 가까이 군집을 형성하는 것을 볼 수 있습니다.

- 군집 형성: 신발에 해당하는 잠재 표현들은 한 곳에 모여 클러스터를 형성하고, 바지에 해당하는 것들은 다른 클러스터를 형성합니다.
- 의미 있는 잠재 공간: 이는 오토인코더가 30차원 잠재 공간에서 이미지의 중요한 의미적 특징을 성공적으로 분리하여 표현했음을 의미합니다.

6.5.5 오토인코더를 이용한 분류기 사전 훈련

오토인코더는 레이블이 없는 대량의 데이터로 사전 훈련되어, 레이블이 부족한 분류 문제에서 강력한 성능을 발휘할 수 있습니다.

1. 1단계: 오토인코더 훈련 (비지도 학습):

- 레이블이 없는 대량의 데이터(예: 수백만 장의 이미지)를 사용하여 오토인코더를 훈련합니다.
- 이 과정에서 오토인코더는 데이터의 효율적인 저차원 잠재 표현(인코딩)을 학습합니다.

2. 2단계: 인코더 추출 및 분류기 구축 (지도 학습):

- 훈련된 오토인코더에서 디코더 부분을 제거하고 인코더 부분만 추출합니다.
- 이 인코더의 가중치는 고정(*freeze*)합니다.
- 인코더의 출력(저차원 잠재 표현)을 새로운 분류 모델의 입력으로 사용합니다.
- 레이블이 있는 소량의 데이터(예: 수천 장의 이미지)를 사용하여 이 분류 모델을 훈련합니다.

핵심 정보

- 레이블 데이터 부족 문제 해결: 고차원 입력(예: 784차원 이미지)에 직접 분류기를 훈련하려면 매우 많은 레이블 데이터가 필요합니다. 하지만 오토인코더를 통해 저차원 잠재 표현(예: 30차원)을 얻으면, 훨씬 적은 수의 레이블 데이터로도 효과적인 분류기를 훈련할 수 있습니다.
 - 강력한 특징 추출: 오토인코더가 비지도 방식으로 학습한 잠재 표현은 데이터의 핵심 특징을 잘 나타내므로, 분류 성능 향상에 기여합니다.
- 이 접근 방식은 분류기의 일부(특징 추출 부분)를 레이블 없이 사전 훈련하는 것으로 볼 수 있습니다.

6.6 희소 오토인코더 (Sparse Autoencoders)

일반적인 오토인코더는 병목 계층의 뉴런 수가 입력 차원보다 작을 때 차원 축소 효과를 얻습니다. 하지만 데이터가 매우 이질적(예: 숫자, 동물, 자동차 이미지가 섞여 있음)인 경우, 병목 계층의 뉴런 수가 충분히 작지 않으면 모든 뉴런이 항상 활성화되어 희소성이 사라질 수 있습니다. 희소

오토인코더는 이러한 문제를 해결하기 위해 잠재 표현의 희소성을 강제합니다.

6.6.1 희소 오토인코더의 필요성

- **이질적인 데이터 처리:** 만약 오토인코더가 다양한 종류의 데이터(예: 숫자, 동물, 자동차)를 처리해야 하고, 병목 계층의 뉴런 수가 충분히 크다면(예: 90개), 각 입력(숫자, 동물, 자동차)에 대해 모든 90개 뉴런이 활성화될 수 있습니다.
- **특징 분리:** 이렇게 되면 특정 뉴런 그룹이 특정 유형의 데이터(예: 숫자)에만 반응하도록 학습되지 않고, 모든 뉴런이 모든 유형의 데이터에 대해 조금씩 활성화되어 특징 분리가 어려워집니다.
- **과적합 방지 및 효율성:** 잠재 표현이 희소하다는 것은 각 입력에 대해 소수의 뉴런만 활성화된다는 의미입니다. 이는 모델의 과적합을 방지하고, 데이터의 핵심 특징을 더 명확하게 추출하는 데 도움이 됩니다.

핵심 정보

희소 오토인코더의 목표는 병목 계층의 뉴런 중 일부만 활성화되도록 강제하는 것입니다. 예를 들어, 90개의 뉴런 중 10%인 9개만 활성화되도록 하여, 각 입력에 대해 특정 뉴런 그룹이 전문화되도록 유도합니다.

6.6.2 희소성 강제 방법

희소성을 강제하는 주요 방법은 손실 함수에 추가적인 정규화 항을 도입하는 것입니다.

L1 활동 정규화 (L1 Activity Regularization)

- **작동 방식:** 잠재 표현 c 의 각 요소(C_i)의 절댓값 합을 손실 함수에 추가합니다.

$$Loss = Reconstruction\ Loss + \lambda \sum_{i=1}^N |C_i|$$

여기서 λ 는 정규화 강도를 조절하는 하이퍼파라미터입니다.

- **효과:** L1 정규화는 가중치를 0으로 만드는 경향이 있으므로, 잠재 표현의 일부 뉴런이 완전히 비활성화(0 값)되도록 유도합니다. 이는 희소성을 직접적으로 증가시킵니다.

쿨백-라이블러 (Kullback-Leibler, KL) 발산 정규화

KL 발산은 두 확률 분포 간의 차이를 측정하는 통계량입니다. 희소 오토인코더에서는 잠재 계층의 실제 평균 활성화 분포를 우리가 설정한 목표 희소성 분포에 가깝게 만들도록 손실 함수에 추가됩니다.

- **목표 희소성 설정:** 잠재 계층의 뉴런들이 평균적으로 얼마나 활성화되기를 원하는지 목표 비율 P 를 설정합니다 (예: $P = 0.1$ 은 10%의 뉴런만 활성화되기를 원함).
- **실제 활성화 측정:** 훈련 중 잠재 계층 뉴런들의 실제 평균 활성화 비율 $\hat{P}$ 를 계산합니다.

- **KL 발산 추가:** 손실 함수에 P 와 $\hat{P}$ 사이의 **KL 발산** 항을 추가합니다.

$$Loss = Reconstruction\ Loss + \lambda \cdot D_{KL}(P||\hat{P})$$

여기서 $D_{KL}(P||\hat{P})$ 는 다음과 같이 정의됩니다 (이진 교차 엔트로피 기반):

$$D_{KL}(P||\hat{P}) = P \log \left(\frac{P}{\hat{P}} \right) + (1 - P) \log \left(\frac{1 - P}{1 - \hat{P}} \right)$$

- **효과:** **KL 발산**은 $\hat{P}$ 가 P 에 가까워지도록 강제합니다. 만약 $\hat{P}$ 가 P 보다 너무 크거나 작으면 **KL 발산** 값이 커져 손실이 증가하고, 모델은 $\hat{P}$ 를 P 에 맞추기 위해 뉴런 활성화를 조절하게 됩니다. 이는 **L1 정규화**보다 더 정교하게 목표 희소성을 제어할 수 있습니다.

핵심 정보

우리는 "잠재 계층 뉴런의 10%만 활성화되도록 해!"라는 목표($P = 0.1$)를 설정합니다. 모델이 훈련되면서 실제 활성화되는 비율($\hat{P}$)이 계산됩니다. 만약 $\hat{P}$ 가 10%와 다르면, **KL 발산** 항이 손실을 증가시키고, 모델은 $\hat{P}$ 를 10%에 가깝게 만들기 위해 뉴런의 활성화를 조절합니다. 재구성 손실과 **KL 발산** 항은 서로 경쟁하며, 모델은 두 손실을 모두 최소화 하는 방향으로 학습됩니다.

예제

목표: 300개의 인코딩 뉴런 중 10%만 활성화되도록 함 ($P = 0.1$). **구현:** *Keras*에서 `activity_regularizer`를 사용하여 **KL 발산 정규화**를 적용할 수 있습니다.

```
1 from tensorflow.keras.regularizers import KLDivergence
2
3 # 목표 희소성 (target sparsity) 설정
4 target_sparsity = 0.1
5 # 정규화 강도 (lambda) 설정
6 regularization_weight = 0.05
7
8 model = Sequential([
9     # ... 이전 레이어 ...
10     Dense(300, activation='sigmoid',
11           activity_regularizer=KLDivergence(target_sparsity,
12                                             regularization_weight)),
13     # ... 이후 레이어 ...
14 ])
```

Listing 6.4: KL 발산 정규화 적용 예시

결과: 훈련 후, 평균적으로 300개 뉴런 중 약 10% (30개)만 활성화되는 경향을 보입니다. 특정 입력에 대해 재구성이 더 중요하면 10%보다 많은 뉴런이 활성화될 수도 있지만, 전체적으로는 희소성이 유지됩니다.

6.7 변이형 오토인코더 (Variational Autoencoders, VAEs)

일반적인 오토인코더의 잠재 공간은 '비구조적(*unstructured*)'입니다. 즉, 잠재 공간의 한 지점에서 아주 조금만 이동해도 완전히 다른, 심지어 무의미한 데이터가 재구성될 수 있습니다. 변이형 오토인코더(*VAE*)는 이러한 문제를 해결하고, 잠재 공간을 '구조화'하여 새로운 데이터를 생성할 수 있도록 설계되었습니다.

6.7.1 일반 오토인코더 잠재 공간의 문제점

- 불연속성: 일반 오토인코더는 입력 데이터를 잠재 공간의 특정 '점'으로 매핑합니다. 이 점들 사이의 공간(틈새)은 훈련 데이터에 해당하지 않으므로, 이 틈새에 있는 잠재 표현으로부터 재구성된 데이터는 의미가 없거나 이상할 수 있습니다.
- 생성 능력 부족: 잠재 공간이 구조화되어 있지 않기 때문에, 임의의 잠재 벡터를 샘플링하여 의미 있는 새로운 데이터를 생성하기 어렵습니다.

6.7.2 변이형 오토인코더의 핵심 아이디어

*VAE*는 입력 데이터를 잠재 공간의 특정 '점'으로 매핑하는 대신, '확률 분포'로 매핑합니다.

- 인코더의 출력: *VAE*의 인코더는 잠재 공간의 각 차원에 대해 평균(μ)과 분산(σ^2)을 출력합니다. 즉, 입력 데이터는 잠재 공간에서 가우시안 분포(정규 분포)로 표현됩니다.
- 잠재 벡터 샘플링: 이 분포($N(\mu, \sigma^2)$)에서 잠재 벡터 z 를 샘플링합니다. 이때 $z = \mu + \sigma \cdot \epsilon$ 공식을 사용하는데, ϵ 은 표준 정규 분포($N(0, 1)$)에서 샘플링된 노이즈입니다. 이를 '재매개변수화 트릭(*reparameterization trick*)'이라고 합니다.
- 디코더: 샘플링된 z 를 디코더에 입력하여 원본 데이터를 재구성합니다.

Figure 6.3: 변이형 오토인코더의 기본 구조 (인코더가 평균과 분산을 출력)

위 그림은 *VAE*의 구조를 보여줍니다. 인코더는 입력으로부터 잠재 공간의 평균(μ)과 로그 분산($\log \sigma^2$)을 예측하고, 이로부터 잠재 벡터 z 를 샘플링합니다. 디코더는 z 로부터 원본을 재구성합니다.

6.7.3 VAE의 손실 함수

*VAE*의 손실 함수는 두 가지 항으로 구성됩니다.

1. 재구성 손실 (*Reconstruction Loss*): 일반 오토인코더와 동일하게, 원본 입력과 재구성된 출력 간의 차이를 최소화합니다. 이는 모델이 데이터를 정확하게 복원하도록 합니다.
2. *KL* 발산 정규화 (*KL Divergence Regularization*): 인코더가 출력하는 잠재 분포($N(\mu, \sigma^2)$)가 표준 정규 분포($N(0, 1)$)와 얼마나 다른지를 측정하는 *KL* 발산 항을 추가합니다.

$$Loss = Reconstruction\ Loss + D_{KL}(N(\mu, \sigma^2) || N(0, 1))$$

이 KL 발산 항은 다음과 같이 계산됩니다:

$$D_{KL}(N(\mu, \sigma^2) || N(0, 1)) = \frac{1}{2} \sum_{i=1}^D (\sigma_i^2 + \mu_i^2 - \log(\sigma_i^2) - 1)$$

여기서 D 는 잠재 공간의 차원입니다.

핵심 정보

- **잠재 공간 구조화:** KL 발산 항은 인코더가 학습하는 잠재 분포를 표준 정규 분포에 가깝게 만듭니다. 이는 잠재 공간을 연속적이고 부드럽게 (*continuous and smooth*) 만듭니다. 즉, 잠재 공간의 어떤 지점을 샘플링하더라도 의미 있는 데이터가 생성될 가능성이 높아집니다.
- **분산 붕괴 방지:** 만약 KL 발산 항이 없다면, 모델은 재구성 손실만 최소화하기 위해 σ 를 0에 가깝게 만들려고 할 것입니다. 이는 VAE 를 일반 오토인코더처럼 만들고, 잠재 공간의 확률적 특성을 잃게 합니다. KL 발산 항은 σ 가 0으로 수축하는 것을 방지하고, 충분한 분산을 유지하도록 강제합니다.
- **평균을 0으로:** 또한, μ 가 0에 가깝도록 유도하여 잠재 공간이 원점을 중심으로 분포하도록 만듭니다.

6.7.4 VAE의 장점

- **생성 능력:** 잠재 공간이 표준 정규 분포를 따르도록 학습되므로, 표준 정규 분포에서 임의의 잠재 벡터 z 를 샘플링하여 디코더에 입력하면 새로운, 그럴듯한 데이터를 생성할 수 있습니다.
- **연속적인 잠재 공간:** 잠재 공간에서 한 지점에서 다른 지점으로 이동하면, 재구성된 데이터도 부드럽게 변화합니다. 예를 들어, '티셔츠'에 해당하는 잠재 벡터에서 '바지'에 해당하는 잠재 벡터로 이동하면, 이미지가 점진적으로 티셔츠에서 바지로 변형되는 것을 관찰할 수 있습니다.
- **이미지 편집:** 잠재 공간의 특정 차원이 데이터의 특정 속성(예: 이미지의 색상, 객체의 크기)과 연관되도록 학습될 수 있으며, 이를 조작하여 이미지 편집에 활용할 수 있습니다.

주의

만약 VAE 훈련 시 σ 에 대한 제한이 없다면, 모델은 재구성 손실만 최소화하기 위해 σ 를 0으로 만들려고 할 것입니다. 이는 노이즈를 추가하지 않아 재구성이 쉬워지기 때문입니다. KL 발산 항은 σ 가 0으로 수축하는 것을 방지하고, 잠재 공간에 의미 있는 분산을 유지하도록 강제하는 중요한 역할을 합니다.

6.8 체크리스트

본 문서를 학습하거나 작성할 때 활용할 수 있는 점검표입니다.

6.8.1 학습 점검표

- $TF-IDF$ 의 개념과 계산 방법을 이해했는가?
- 정적 임베딩의 한계(의미론적 드리프트, 문맥 비민감성)를 구분하고 설명할 수 있는가?
- 코사인 유사도 계산 방법을 이해하고 직접 계산할 수 있는가?
- 문자 임베딩이 필요한 이유와 작동 방식을 설명할 수 있는가?
- 오토인코더의 기본 구조와 재구성 손실의 의미를 이해했는가?
- 노이즈 제거 오토인코더의 목적과 작동 방식을 설명할 수 있는가?
- 스택형 오토인코더의 훈련 어려움과 계층별 사전 훈련 방법을 이해했는가?
- $t-SNE$ 가 시각화 도구임을 이해하고 그 역할을 설명할 수 있는가?
- 희소 오토인코더가 필요한 이유와 $L1$ 정규화, KL 발산 정규화의 차이를 설명할 수 있는가?
- 변이형 오토인코더가 일반 오토인코더와 다른 점, 그리고 생성 능력을 갖는 이유를 설명할 수 있는가?
- VAE 의 KL 발산 항이 잠재 공간을 구조화하고 분산 붕괴를 막는 역할을 이해했는가?

6.8.2 문서 작성 점검표

- 제공된 강의 자료의 모든 내용을 빠짐없이 포함했는가?
- 출처, 파일명, 페이지 언급을 금지했는가?
- $LaTeX$ 소스 전체를 제공했는가?
- 한국어로 작성되었으며, 전문용어는 쉬운 풀이와 원어 병기를 병행했는가?
- 문장은 짧고 명확하며, 단락은 4-6줄 내외로 유지되었는가?
- 나열은 불릿/번호 목록, 소제목, 박스, 공백을 적극 활용했는가?
- A4 규격에 맞는 레이아웃(여백, 줄간격, 단락 설정)을 준수했는가?
- 헤더/푸터(문서명, 날짜, 쪽수)를 $fancyhdr$ 로 표시했는가?
- 요약, 주의, 예시 박스를 $tcolorbox$ 로 구성했는가?
- 표와 그림에는 캡션과 라벨을 필수로 포함하고, 설명과 가까이 배치했는가?
- 모든 표는 A4 폭(160mm)에 맞추고 또는 $adjustbox$ 를 사용했는가?
- 코드 예시는 $listings$ 를 사용하고 $breaklines=true$, 행번호, 캡션/라벨을 포함했는가?
- 각 섹션 시작 전

를 사용하여 페이지 경계를 명확히 했는가?

- 초심자도 완전히 이해할 수 있도록 '왜 이런 개념이 필요한가'부터 설명했는가?
 - 공식/절차/명령문에는 반드시 최소 1개의 예시를 붙였는가?
 - 모를 수 있는 포인트를 예측하여 미리 설명하고, 오해 가능성을 고려했는가?
 - 한 개념을 '한 줄 핵심 요약', '직관적 예시/비유', '기술적 정확한 설명'의 세 단계로 설명했는가?
 - 문맥 의존 표현 없이 문장 자체로 의미가 통하도록 '단락 독립성'을 확보했는가?
 - 필요한 경우 '개념 간 비교표'를 추가했는가?
 - *LaTeX* 안전 가드레일(특수문자 이스케이프, 백틱 금지 등)을 준수했는가?
-

6.9 FAQ (자주 묻는 질문)

핵심 요약

Q1: *TF-IDF*에서 로그 함수의 밑이 중요하지 않다는 게 무슨 의미인가요? **A1:** 로그 함수의 밑이 e 든 10이든, 밑이 바뀌면 *IDF* 값 전체에 단순히 상수배가 됩니다. 예를 들어 $\log_{10} x = \frac{\ln x}{\ln 10}$ 이므로, $\ln 10$ 이라는 상수가 곱해지는 것과 같습니다. 최종적으로 임베딩 벡터를 정규화(*normalize*)하면 이 상수배는 사라지므로, 최종 결과에 영향을 미치지 않습니다.

Q2: 문자 임베딩이 단어 임베딩보다 항상 좋은가요? **A2:** 항상 그렇지는 않습니다. 문자 임베딩은 *OOV* 문제나 형태론적으로 복잡한 언어에 강하지만, 단어 임베딩은 단어 수준의 의미론적 정보를 더 직접적으로 포착할 수 있습니다. 일반적으로는 두 가지를 결합한 하이브리드 접근 방식이 가장 좋은 성능을 보이는 경우가 많습니다.

Q3: 오토인코더의 잠재 공간 차원을 얼마나 작게 설정해야 하나요? **A3:** 잠재 공간의 차원(병목 계층의 뉴런 수)은 데이터의 복잡성과 원하는 압축률에 따라 달라집니다. 너무 작으면 정보 손실이 커지고, 너무 크면 압축 효과가 줄어들고 과적합 위험이 있습니다. 일반적으로는 실험을 통해 최적의 차원을 찾습니다.

Q4: 스택형 오토인코더를 계층별로 사전 훈련하는 것이 요즘도 필요한가요? **A4:** 과거에는 심층 신경망 훈련의 어려움 때문에 필수적이었지만, 최근에는 *Adam*과 같은 개선된 최적화 기법, 배치 정규화, 잔차 연결 등 다양한 기술 덕분에 한 번에 훈련하는 것이 더 일반적입니다. 하지만 특정 복잡한 모델이나 데이터셋에서는 여전히 유용한 전략이 될 수 있습니다.

Q5: 희소 오토인코더에서 *L1* 정규화와 *KL* 발산 정규화 중 어떤 것을 사용해야 하나요?

A5: *L1* 정규화는 단순히 활성화를 0으로 만드는 경향이 있어 희소성을 유도합니다. *KL* 발산 정규화는 특정 목표 활성화 비율(P)을 설정하고, 실제 활성화 비율($\hat{P}$)이 이 목표에 가깝도록 더 정교하게 제어합니다. *KL* 발산이 더 세밀한 제어가 가능하며, *VAE*와 같은 생성 모델에서도 활용됩니다.

Q6: *VAE*의 잠재 공간이 '구조화'되었다는 것은 정확히 무엇을 의미하나요? **A6:** 일반 오토인코더의 잠재 공간은 듬성듬성한 점들로 이루어져 있어, 점들 사이의 공간은 의미가 없습니다. *VAE*는 잠재 공간을 연속적인 확률 분포로 채워 넣기 때문에, 잠재 공간의 어떤 지점을 선택하더라도 그럴듯한 데이터가 생성됩니다. 이는 잠재 공간에서 부드럽게 이동하며 데이터가 점진적으로 변형되는 것을 관찰할 수 있다는 의미입니다.

6.10 빠르게 훑어보기 (1페이지 요약)

핵심 개념 요약

TF-IDF: 문서 내 단어 중요도 측정 (*TF*: 문서 내 빈도, *IDF*: 코퍼스 내 희귀도). 정적 임베딩 한계:

- 의미론적 드리프트: 시간 경과에 따른 단어 의미 변화.
- 문맥 비민감성: 동의어의 문맥별 의미 구분 불가.

코사인 유사도: 두 벡터의 각도를 이용한 유사도 측정 (방향성 중요).

문자 임베딩

목적: *OOV*, 철자 오류, 형태론적 복잡성 해결. 작동: 문자를 토큰으로 임베딩, *RNN/CNN*으로 시퀀스 처리. 장점: 유연성, 형태론적 정보 포착. 특징: 단어 임베딩보다 낮은 차원 사용 (예: 8차원).

오토인코더 (Autoencoders)

목표: 입력 데이터를 효율적인 잠재 표현으로 압축 후 재구성. 구조: 인코더 (압축) → 병목 → 디코더 (복원). 손실: 재구성 손실 (원본과 복원본의 차이 최소화). 활용: 차원 축소, 특징 학습, 사전 훈련 (레이블 부족 시). 노이즈 제거: 노이즈 입력 → 원본 출력 (강건한 특징 학습).

스택형 β 희소 오토인코더

스택형: 여러 은닉 계층을 가진 심층 오토인코더. 훈련: 계층별 사전 훈련으로 심층망 훈련 어려움 완화. 희소: 잠재 표현의 일부 뉴런만 활성화되도록 강제. 희소성 강제:

- *L1* 정규화: 활성화 값의 절댓값 합을 손실에 추가.
- *KL* 발산 정규화: 실제 활성화 분포를 목표 희소성 분포에 맞춤.

날짜 변수 처리

$publish_{date}$ 는 *YYYYMMDD*

```
1 연도와 월 추출 news_atayear <- as.numeric(format(news_atadate_obj, "news_atamonth <-  
  as.numeric(format(news_atadate_obj, "  
2 연속적인 숫자형 날짜 변수 생성 (연도 + (월-1)/12) 예: 2003년 2월은 2003 + (2-1)/12  
  = 2003.083... news_atanumeric_date <- news_atayear + (news_atamonth - 1) / 12  
3 첫 몇 개 값 확인 head(news_atanumeric_date)
```

Listing 7.7: 날짜 변수 전처리

이렇게 생성된

```
4 연도와 월 추출 news_atayear <- as.numeric(format(news_atadate_obj, "news_atamonth <-  
  as.numeric(format(news_atadate_obj, "  
5 연속적인 숫자형 날짜 변수 생성 (연도 + (월-1)/12) 예: 2003년 2월은 2003 + (2-1)/12  
  = 2003.083... news_atanumeric_date <- news_atayear + (news_atamonth - 1) / 12  
6 첫 몇 개 값 확인 head(news_atanumeric_date)
```

Listing 7.8: 날짜 변수 전처리

이렇게 생성된 형식의 숫자로 되어 있습니다. 이를 *STM* 모델에서 공변량으로 사용하기 위해 연속적인 숫자형 변수로 변환합니다.

```
7 연도와 월 추출 news_atayear <- as.numeric(format(news_atadate_obj, "news_atamonth <-  
  as.numeric(format(news_atadate_obj, "  
8 연속적인 숫자형 날짜 변수 생성 (연도 + (월-1)/12) 예: 2003년 2월은 2003 + (2-1)/12  
  = 2003.083... news_atanumeric_date <- news_atayear + (news_atamonth - 1) / 12  
9 첫 몇 개 값 확인 head(news_atanumeric_date)
```

Listing 7.9: 날짜 변수 전처리

이렇게 생성된 `numeric_date`

데이터 샘플링

백만 개 이상의 데이터를 모두 처리하는 데는 시간이 오래 걸리므로, 개발 단계에서는 데이터의 일부만 샘플링하여 사용합니다.

```
10 샘플링된 데이터의 차원 확인 dim(news_sample)
```

Listing 7.10: 데이터 샘플링

7.3.1 텍스트 전처리

STM 모델을 훈련하기 전에 텍스트 데이터를 전처리해야 합니다. `stm` 패키지는 `textProcessor`와 `prepareDocuments`

```
11 텍스트 전처리: 소문자화, 불용어 제거, 숫자 제거, 구두점 제거, 어간 추출 processed <-
    textProcessor( documents = docs, metadata = meta, lowercase = TRUE, removestopwords =
        TRUE, removenumbers = TRUE, removepunctuation = TRUE, stem = TRUE )
12 전처리 결과 요약 summary(processed)
```

Listing 7.11: 텍스트 전처리 (textProcessor)

함수를 제공하여 이 과정을 자동화합니다.

```
13 텍스트 전처리: 소문자화, 불용어 제거, 숫자 제거, 구두점 제거, 어간 추출 processed <-
    textProcessor( documents = docs, metadata = meta, lowercase = TRUE, removestopwords =
        TRUE, removenumbers = TRUE, removepunctuation = TRUE, stem = TRUE )
14 전처리 결과 요약 summary(processed)
```

Listing 7.12: 텍스트 전처리 (textProcessor)

`textProcessor`의 `summary()`

```
15 준비된 문서, 어휘, 메타데이터 추출 docs_final <- outdocuments vocab_final <- outvocab
    vocab_final <- vocab_final[vocab_final$meta_final <- outmeta 전처리 과정에서 변경될 수 있
    으므로 새로 추출
```

Listing 7.13: 문서 준비 (prepareDocuments)

출력은 제거된 단어, 빈 문서 수, 최종 어휘 크기 등을 보여줍니다. 예를 들어, "removing 9000 observations from 21000 due to frequency"는 빈번하지 않은 단어들 때문에 9000개의 단어가 어휘에서 제거되었음을 의미합니다. "50 documents become empty"는 전처리 후 50개의 문서가 비어있게 되어 제거되었음을 의미합니다. 이 경우, 해당 문서에 대응하는 메타데이터도 함께 제거됩니다.

```
16 준비된 문서, 어휘, 메타데이터 추출 docs_final <- outdocuments vocab_final <- outvocab
    vocab_final <- vocab_final[vocab_final$meta_final <- outmeta 전처리 과정에서 변경될 수 있
    으므로 새로 추출
```

Listing 7.14: 문서 준비 (prepareDocuments)

`prepareDocuments`는 `lower.thresh`와 같은 파라미터를 통해 희귀 단어를 추가로 제거할 수 있습니다. 이 과정에서 문서나 어휘가 더 줄어들 수 있으며, `meta_final`은 d

7.3.2 STM 모델 실행 및 결과 해석

이제 전처리된 데이터를 사용하여 STM 모델을 훈련하고 결과를 해석합니다.

STM 모델 훈련

`stm()` 함수를 사용하여 모델을 훈련합니다. K 는 토픽의 수, `formula`

```
17 STM 모델 훈련 (K=5로 가정) formula =
18 ~ date는 'date' 변수를 공변량으로 사용하겠다는 의미
    stm_model <- stm(documents = docs_final, vocab = vocab_final, K = 5, ( 5 ) formula =
    ~date, data = meta_final, init.type = "Spectral", max.em.iter = 100EM)
```

Listing 7.15: STM 모델 훈련

는 공변량을 지정합니다.

```
19 STM 모델 훈련 (K=5로 가정) formula =
20 ~ date는 'date' 변수를 공변량으로 사용하겠다는 의미
    stm_model <- stm(documents = docs_final, vocab = vocab_final, K = 5, ( 5 ) formula =
    ~date, data = meta_final, init.type = "Spectral", max.em.iter = 100EM)
```

Listing 7.16: STM 모델 훈련

`stm()` 함수는 EM 알고리즘을 사용하여 모델 파라미터를 추정합니다. *E-step*과 *M-step*이 반복적으로 수행되며, 수렴할 때까지 또는 `max.em.iter`

모델 결과 요약

훈련된 모델의 에 도달할 때까지 진행됩니다.

모델 결과 요약

훈련된 모델의 `summary()`

```
1 stlisting
2 를 통해 각 토픽과 가장 연관된 단어들을 확인할 수 있습니다.
3
4 1 stlisting
5 2
6 3 \subsubsection토픽{ prevalence 시각화}
7 4 }plot()\texttt{ 함수는 각 토픽의 평균 출현 빈도(prevalence)를 시각화하여, 어
   5   6   7   8   9   10   11   12   13   14   15   16   17   18   19   20   21   22   23   24   25   26   27   28   29   30   31   32   33   34   35   36   37   38   39   40   41   42   43   44   45   46   47   48   49   50   51   52   53   54   55   56   57   58   59   60   61   62   63   64   65   66   67   68   69   70   71   72   73   74   75   76   77   78   79   80   81   82   83   84   85   86   87   88   89   90   91   92   93   94   95   96   97   98   99   100   101   102   103   104   105   106   107   108   109   110   111   112   113   114   115   116   117   118   119   120   121   122   123   124   125   126   127   128   129   130   131   132   133   134   135   136   137   138   139   140   141   142   143   144   145   146   147   148   149   150   151   152   153   154   155   156   157   158   159   160   161   162   163   164   165   166   167   168   169   170   171   172   173   174   175   176   177   178   179   180   181   182   183   184   185   186   187   188   189   190   191   192   193   194   195   196   197   198   199   200   201   202   203   204   205   206   207   208   209   210   211   212   213   214   215   216   217   218   219   220   221   222   223   224   225   226   227   228   229   230   231   232   233   234   235   236   237   238   239   240   241   242   243   244   245   246   247   248   249   250   251   252   253   254   255   256   257   258   259   260   261   262   263   264   265   266   267   268   269   270   271   272   273   274   275   276   277   278   279   280   281   282   283   284   285   286   287   288   289   290   291   292   293   294   295   296   297   298   299   300   301   302   303   304   305   306   307   308   309   310   311   312   313   314   315   316   317   318   319   320   321   322   323   324   325   326   327   328   329   330   331   332   333   334   335   336   337   338   339   340   341   342   343   344   345   346   347   348   349   350   351   352   353   354   355   356   357   358   359   360   361   362   363   364   365   366   367   368   369   370   371   372   373   374   375   376   377   378   379   380   381   382   383   384   385   386   387   388   389   390   391   392   393   394   395   396   397   398   399   400   401   402   403   404   405   406   407   408   409   410   411   412   413   414   415   416   417   418   419   420   421   422   423   424   425   426   427   428   429   430   431   432   433   434   435   436   437   438   439   440   441   442   443   444   445   446   447   448   449   450   451   452   453   454   455   456   457   458   459   460   461   462   463   464   465   466   467   468   469   470   471   472   473   474   475   476   477   478   479   480   481   482   483   484   485   486   487   488   489   490   491   492   493   494   495   496   497   498   499   500   501   502   503   504   505   506   507   508   509   510   511   512   513   514   515   516   517   518   519   520   521   522   523   524   525   526   527   528   529   530   531   532   533   534   535   536   537   538   539   540   541   542   543   544   545   546   547   548   549   550   551   552   553   554   555   556   557   558   559   560   561   562   563   564   565   566   567   568   569   570   571   572   573   574   575   576   577   578   579   580   581   582   583   584   585   586   587   588   589   590   591   592   593   594   595   596   597   598   599   600   601   602   603   604   605   606   607   608   609   610   611   612   613   614   615   616   617   618   619   620   621   622   623   624   625   626   627   628   629   630   631   632   633   634   635   636   637   638   639   640   641   642   643   644   645   646   647   648   649   650   651   652   653   654   655   656   657   658   659   660   661   662   663   664   665   666   667   668   669   670   671   672   673   674   675   676   677   678   679   680   681   682   683   684   685   686   687   688   689   690   691   692   693   694   695   696   697   698   699   700   701   702   703   704   705   706   707   708   709   710   711   712   713   714   715   716   717   718   719   720   721   722   723   724   725   726   727   728   729   730   731   732   733   734   735   736   737   738   739   740   741   742   743   744   745   746   747   748   749   750   751   752   753   754   755   756   757   758   759   760   761   762   763   764   765   766   767   768   769   770   771   772   773   774   775   776   777   778   779   780   781   782   783   784   785   786   787   788   789   790   791   792   793   794   795   796   797   798   799   800   801   802   803   804   805   806   807   808   809   810   811   812   813   814   815   816   817   818   819   820   821   822   823   824   825   826   827   828   829   830   831   832   833   834   835   836   837   838   839   840   841   842   843   844   845   846   847   848   849   850   851   852   853   854   855   856   857   858   859   860   861   862   863   864   865   866   867   868   869   870   871   872   873   874   875   876   877   878   879   880   881   882   883   884   885   886   887   888   889   890   891   892   893   894   895   896   897   898   899   900   901   902   903   904   905   906   907   908   909   910   911   912   913   914   915   916   917   918   919   920   921   922   923   924   925   926   927   928   929   930   931   932   933   934   935   936   937   938   939   940   941   942   943   944   945   946   947   948   949   950   951   952   953   954   955   956   957   958   959   960   961   962   963   964   965   966   967   968   969   970   971   972   973   974   975   976   977   978   979   980   981   982   983   984   985   986   987   988   989   990   991   992   993   994   995   996   997   998   999   1000
   11 plot(stm_model, type = "summary", n = 5) # 개5 토픽 모두 표시
```

Listing 7.18: STM 모델 요약

이 그래프는 각 토픽이 전체 문서에서 차지하는 평균 비율을 보여줍니다. 예를 들어, 'Topic 2'가 30%로 가장 높다면, 이 토픽이 전체 뉴스 헤드라인에서 가장 자주 다루어지는 주제임을 의미합니다.

워드 클라우드 시각화

`cloud()` 함수는 각 토픽에 대한 워드 클라우드를 생성합니다.

```
1 cloud(stm_model, topic = 1, scale = c(2.2, .2))Topic1
```

Listing 7.19: 토픽별 워드 클라우드

워드 클라우드는 토픽과 연관된 단어들을 시각적으로 보여주지만, `summary()`와 마찬가지로 토픽의 의미를 명확히 해석하는 데 한계가 있을 수 있습니다.

7.3.3 토픽 효과 추정 및 시각화

STM의 강력한 기능 중 하나는 공변량(메타데이터)이 토픽의 prevalence에 미치는 영향을 추정하고 시각화하는 것입니다.

`estimateEffect()` 함수 `estimateEffect()` 함수는 훈련된 STM 모델의 파라미터를 사용하여, 공변량의 변화에 따른 토픽 prevalence의 변화를 추정합니다.

```
1 1번부터 5번까지 모든 토픽에 대해 효과 추정 formula =
2
3 ~ date는 'date' 변수가 토픽 prevalence에 미치는 영향을 분석하겠다는 의미effect <-
  estimateEffect(1:5
4
5 ~ date, 모든 토픽 (1:5)에 대해 date 변수의 효과 추정stmobj =
  stm_model, STM metadata = meta_final, uncertainty =
  "Global" ) summary(effect)
```

Listing 7.20: 토픽 효과 추정

`estimateEffect()`는 추정된 파라미터(사후 분포)를 기반으로 토픽 prevalence의 기대값을 반복적으로 샘플링하고, 이 샘플링된 값들을 공변량에 대해 회귀 분석하여 효과를 추정합니다.

토픽 prevalence 변화 시각화

`plot()` 함수를 `estimateEffect()`의 결과와 함께 사용하여 공변량에 따른 토픽 prevalence의 변화를 시각화할 수 있습니다.

```
1 plot(effect, covariate = "date", 공변량 지정topics = 1:5, 모든 토픽 표시model =
  stm_model, STM method = "continuous", xlab = "Year", x main =
  "TopicPrevalenceOverTime", linecol = "blue", printlegend = TRUE )
```

Listing 7.21: 토픽 prevalence 변화 시각화

이 그래프는 `numeric_date`

7.3.5 최적 토픽 수(K) 탐색

STM 모델에서 가장 중요한 하이퍼파라미터 중 하나는 토픽의 수 K 입니다. 최적의 K 를 찾기 위해 `searchK()` 함수를 사용하여 여러 K 값에 대해 모델을 실행하고, 다양한 평가 지표를 비교합니다.

`searchK()` 함수 실행 `searchK()`는 지정된 범위의 K 값에 대해 STM 모델을 훈련하고, 각 K 에 대한 평가 지표를 계산합니다.

```
1 2개부터 10개까지의 토픽 수에 대해 모델 실행k_results <- searchK(documents =
  docs_final, vocab = vocab_final, K = c(2 : 10), formula =
2
3
4
5
6
7 date, data = meta_final, init.type = "Spectral", max.em.iter = 50 ) plot(k_results)
```

Listing 7.22: 최적 토픽 수 탐색 (searchK)

```
plot(k_results)
```

평가 지표 시각화 및 최적 K 선택

는 held-out likelihood, 잔차(residuals) 등을 보여주지만, 토픽 모델링에서는 의미론적 일관성(semantic coherence)

는 held-out likelihood, 잔차(residuals) 등을 보여주지만, 토픽 모델링에서는 의미론적 일관성(semantic coherence)과 배타성(exclusivity)이 더 중요한 지표로 간주됩니다.

평가 지표 시각화 및 최적

searchK() 결과에서 의미론적 일관성과 배타성 값을 추출하여 시각화하고, 이 두 지표를 동시에 최대화하는 K 를 선택합니다.

```
1 각 K에 대한 의미론적 일관성(coherence)과 배타성(exclusivity) 추출
coherence_values <- unlist(k_results$results$semcoh)
exclusivity_values <- unlist(k_results$results$exclus)
k_values <- k_results$results$K
metrics_df <- data.frame(K = k_values, Coherence = coherence_values, Exclusivity = exclusivity_values)
ggplot2::ggplot(metrics_df, aes(x = Exclusivity, y = Coherence, label = K)) +
  geom_point(size = 3) + geom_text(vjust = -1, hjust = 0.5) +
  labs(title = "Coherence vs. Exclusivity for Different K", x = "Exclusivity", y = "Coherence") +
  theme_minimal()
```

Listing 7.23: 일관성 및 배타성 시각화

이 그래프에서 우측 상단(높은 일관성과 높은 배타성)에 위치하는 K 값을 선택하는 것이 일반적입니다. 두 지표의 스케일이 다를 수 있으므로, 각 지표를 0-1 사이로 재조정(rescale)한 후 평균을 내어 최적 K 를 선택하는 방법도 있습니다.

최적의 K 를 선택하는 것은 완전히 객관적이지 않을 수 있습니다. 통계적 지표 외에도, 추출된 토픽들이 인간이 이해하기에 얼마나 의미 있고 해석 가능한지(interpretability)도 중요한 고려 사항입니다. findThoughts() 함수를 사용하여 각 K 에서 추출된 토픽들의 대표 문서를 확인하며 의미를 평가하는 것이 좋습니다.

최적 모델 선택 및 재현성

searchK()는 각 K 값에 대해 여러 번 실행될 수 있으며, 각 실행마다 결과가 다를 수 있습니다. 따라서 특정 K 에 대해 가장 좋은 성능을 보인 모델을 명시적으로 선택하는 것이 중요합니다.

```
1 예를 들어, K=5가 최적이라고 판단했을 때, 해당 K에 대한 최적 모델을 선택
k_results_runout[[K_index]][[run_index]] K = 5 selected_k_model <-
-k_results_runout[[which(k_results$results$K == 5)]] summary(selected_k_model)
```

Listing 7.24: 최적 K 에 대한 모델 선택

이렇게 선택된 모델을 사용하여 이후의 분석을 진행합니다.

7.3.6 토픽 내용 이해

토픽의 의미를 명확히 이해하기 위해서는 findThoughts() 함수를 사용하여 각 토픽과 가장 연관성이 높은 문서들을 살펴보는 것이 가장 효과적입니다.

```
1 Topic 1과 가장 연관된 문서 3개 확인
findThoughts(model = stm_model, texts = docs, n = 3, topics = 1)
Topic3 findThoughts(model = stm_model, texts = docs, n = 3, topics = 3)
```

Listing 7.25: 토픽별 대표 문서 확인 (findThoughts)

findThoughts()는 특정 토픽에 대한 theta 값이 가장 높은 문서들을 보여줍니다. 이 문서들을 읽어보면 해당 토픽이 어떤 주제를 다루는지 직관적으로 파악할 수 있습니다. 예를 들어, Topic 1의 대표 문서들이 '정부 정책', '선거'와 같은 단어를 포함한다면, Topic 1을 '정치' 토픽으로 명명할 수 있습니다.

theta 행렬의 해석

STM 모델의 결과 객체에는 문서-토픽 분포를 나타내는 *theta* 행렬이 포함됩니다. 이 행렬은 각 문서가 각 토픽에 얼마나 기여하는지를 확률 값으로 보여줍니다.

- 행: 각 문서
- 열: 각 토픽
- 값: 문서가 해당 토픽에 기여하는 확률 (모든 토픽에 대한 확률의 합은 1)

예를 들어, $\theta[1, 2] = 0.7$ 이라면 첫 번째 문서가 두 번째 토픽에 70% 기여한다는 의미입니다.

7.4 과제 및 제출 가이드라인

강의에서 언급된 과제 제출 관련 지침을 정리합니다.

과제 5: 텍스트 오토인코더

- 목표: 텍스트 오토인코더의 구현 및 한계 이해. 완벽한 재현보다는 모델의 어려움을 경험하는 것이 중요합니다.
- 제출물:
 - 보고서 (PDF): Jupyter Notebook으로 생성하거나 수동으로 작성할 수 있습니다. 최종적으로는 하나의 PDF 파일로 통합하여 제출하는 것이 권장됩니다. 보고서에는 모델 구현, 결과, 그리고 결과가 기대만큼 좋지 않은 이유에 대한 논의가 포함되어야 합니다.
 - 코드 파일: Jupyter Notebook(.ipynb) 파일 또는 기타 코드 파일. 여러 문제를 해결하기 위해 여러 Jupyter Notebook 파일을 제출하는 것이 허용됩니다.
 - 압축 파일 (ZIP): 여러 코드 파일을 제출할 경우, 모든 파일을 하나의 ZIP 파일로 압축하여 제출하는 것이 좋습니다. 2 3개의 파일은 개별 제출도 허용되지만, 20개와 같이 파일 수가 많으면 ZIP 파일로 압축해야 합니다.
- 핵심: 보고서와 원본 코드를 모두 제출하여, 보고서 내용을 검증할 수 있도록 해야 합니다.

과제 6: 구조적 토픽 모델링 (STM)

- 목표: R 언어를 사용하여 STM 모델을 구축하고, 메타데이터의 영향을 분석하며, 최적의 토픽 수를 탐색하는 실습.
- 데이터: 선택한 뉴스 기사 데이터셋 (강의에서 사용된 Kaggle 데이터셋 포함).
- 제출물: 과제 5와 동일한 제출 가이드라인을 따릅니다. R 코드와 보고서를 포함해야 합니다.

가장 중요한 것은 보고서와 원본 코드를 모두 제출하는 것입니다. 파일 압축 여부는 부차적인 문제이지만, 여러 파일일 경우 ZIP으로 묶는 것이 채점자에게 편리합니다. 보고서가 없거나 코드가 없으면 채점 자체가 불가능합니다.

7.5 체크리스트

학습 내용 복습, 과제 작성 및 제출 시 활용할 수 있는 체크리스트입니다.

학습 내용 점검

- 텍스트 오토인코더의 어려움과 이미지 오토인코더와의 차이점을 이해했는가?
- LDA의 기본 가정(문서-토픽 혼합)과 동작 원리를 이해했는가?
- LDA가 비지도 학습이며, 레이블 데이터가 필요 없음을 이해했는가?
- LDA에서 최적 토픽 수 K 를 결정하는 어려움과 평가 지표(일관성, 배타성)를 이해했는가?
- LDA가 확률적 모델이며, 실행마다 결과가 달라질 수 있음을 이해했는가?
- NMF가 행렬 분해 기반의 토픽 모델임을 이해했는가?
- LDA와 NMF의 주요 차이점을 설명할 수 있는가?
- 최대 우도 추정(MLE)의 개념과 잠재 변수가 있을 때 EM 알고리즘이 사용되는 이유를 이해했는가?
- STM이 LDA의 한계를 어떻게 극복하고 메타데이터를 활용하는지 이해했는가?
- STM에서 토픽 비율(θ_d)이 로지스틱 정규 분포에서 생성되는 과정을 이해했는가?

- R 언어에서 *stm* 패키지를 설치하고 로드하는 방법을 아는가?
- *textProcessor*와 *prepareDocuments* 함수를 사용하여 텍스트를 전처리하는 방법을 아는가?
- *stm()* 함수로 모델을 훈련하고 *summary()*, *plot()*으로 결과를 해석하는 방법을 아는가?
- *estimateEffect()* 함수로 공변량의 영향을 추정하고 시각화하는 방법을 아는가?
- *searchK()* 함수로 최적 토픽 수 K 를 탐색하고 평가 지표를 해석하는 방법을 아는가?
- *findThoughts()* 함수로 토픽의 의미를 파악하는 방법을 아는가?

과제 제출 점검

- 과제 보고서(PDF)를 작성했는가?
- 보고서에 모든 문제에 대한 해결 과정, 결과, 분석 및 논의가 포함되어 있는가?
- 보고서가 단일 PDF 파일로 통합되어 있는가?
- 모든 원본 코드 파일(Jupyter Notebook 등)을 포함했는가?
- 여러 코드 파일이 있다면 하나의 ZIP 파일로 압축했는가?
- 제출 파일에 출처, 파일명, 페이지 언급 등 금지된 내용이 없는가?
- 모든 파일이 Canvas에 올바르게 업로드되었는가?

7.6 FAQ (자주 묻는 질문)

Q: 텍스트 오토인코더가 이미지 오토인코더보다 어려운 이유는 무엇인가요?

A: 텍스트는 이산적인 단어들의 시퀀스로 구성되며, 단어의 순서와 문맥이 의미를 결정하는 데 매우 중요합니다. 이미지는 연속적인 픽셀 값으로 공간적 인접성이 중요하죠. 텍스트를 단일 벡터로 압축하는 과정에서 이러한 시퀀스 정보(시간적 요소)가 크게 손실되는 '병목 현상'이 발생하여, 원본 문장을 완벽하게 복원하기가 매우 어렵습니다.

Q: LDA는 왜 실행할 때마다 결과가 달라질 수 있나요?

A: LDA는 확률론적 모델이며, EM 알고리즘과 같은 반복적인 최적화 방법을 사용합니다. 이 과정에서 우도 함수가 여러 지역 최대값(local maxima)을 가질 수 있어, 알고리즘의 초기값이나 내부적인 확률적 샘플링에 따라 다른 결과에 수렴할 수 있습니다. 따라서 여러 번 실행하여 가장 좋은 결과를 선택하는 것이 일반적입니다.

Q: STM에서 메타데이터는 어떻게 토픽 모델링에 활용되나요?

A: STM은 LDA의 토픽 생성 과정 중, 문서별 토픽 분포(θ_d)를 생성하는 부분에 메타데이터를 통합합니다. 구체적으로, θ_d 가 로지스틱 정규 분포에서 생성되는데, 이 분포의 중심(평균)이 메타데이터(X_d)와 계수(γ)의 선형 결합($X_d\gamma$)으로 정의됩니다. 이를 통해 메타데이터 변수가 토픽의 출현 빈도(prevalence)에 미치는 영향을 모델링할 수 있습니다.

Q: R Markdown(RMD)은 Jupyter Notebook과 어떻게 다른가요?

A: 둘 다 코드, 텍스트, 실행 결과를 통합하는 도구이지만, RMD는 주로 R 언어에 최적화되어 있으며, LaTeX, HTML, Word 등 다양한 형식의 보고서 생성이 용이합니다. 코드 블록은 백틱(`)으로 구분하며, RMD 파일 자체는 텍스트 기반이어서 버전 관리에 유리합니다. Jupyter Notebook은 Python을 포함한 다양한 언어를 지원하며, 웹 기반 인터페이스가 특징입니다.

Q: `searchK()`로 최적 토픽 수 K 를 찾을 때 어떤 지표를 봐야 하나요?

A: 주로 의미론적 일관성(Semantic Coherence)과 배타성(Exclusivity)을 함께 고려합니다. 일관성은 토픽 내 단어들의 의미적 관련성을, 배타성은 토픽들이 서로 겹치지 않는 정도를 나타냅니다. 이 두 지표를 동시에 최대화하는 K 를 선택하는 것이 이상적입니다. 시각화된 그래프에서 우측 상단에 위치하는 K 를 선택하는 경향이 있습니다.

7.7 빠르게 훑어보기 (1페이지 요약)

CSCI89 Lecture 8 핵심 요약

- 텍스트 오토인코더의 어려움: 이미지와 달리 텍스트는 시퀀스 정보 손실이 커 완벽한 재현이 어렵습니다. 병목 현상, 관측치 부족이 주원인입니다. 데이터 증강이나 `Softmax` 온도 함수 활용, 또는 병목 없는 시퀀스-투-시퀀스 모델로 임베딩을 훈련하는 방법을 고려할 수 있습니다.
- LDA (잠재 디리클레 할당): 문서가 여러 토픽의 혼합으로 구성된다는 비지도 학습 모델입니다. 최적 토픽 수 K 결정이 어렵고, 확률적 특성으로 인해 실행마다 결과가 달라질 수 있습니다.
- NMF (음수 행렬 분해): 텍스트 행렬을 두 개의 비음수 행렬로 분해하여 잠재 토픽 구조를 찾는 행렬 대수 기반 모델입니다. LDA와 달리 결정론적이며, 차원 축소에 활용됩니다.
- EM 알고리즘: 잠재 변수(예: LDA의 토픽 할당)가 있는 모델의 파라미터를 추정하는 반복적인 최적화 방법입니다. 기대(E) 단계와 최대화(M) 단계로 구성됩니다.
- STM (구조적 토픽 모델): LDA를 확장하여 문서의 메타데이터(예: 저자, 출판일, 성별)를 토픽 모델링에 통합합니다. 메타데이터가 토픽 분포에 미치는 영향을 분석하고 가설을 검정할 수 있어 더 정확하고 해석 가능한 토픽을 제공합니다.
- R 실습 (`stm` 패키지):
 - 데이터 준비: 뉴스 헤드라인 데이터셋을 로드하고, `publish_date`를 연속적인 숫자형 공변량으로 전처리합니다.
 - 텍스트 전처리: `textProcessor`와 `prepareDocuments`를 사용하여 불용어 제거, 어간 추출, 빈 문서 및 희귀 단어 제거 등을 수행합니다.
 - 모델 훈련 및 해석: `stm()` 함수로 모델을 훈련하고, `summary()`, `plot()`으로 토픽별 단어 및 `prevalence`를 확인합니다.
 - 효과 분석: `estimateEffect()`로 공변량(예: 시간)이 토픽 `prevalence`에 미치는 영향을 추정하고 시각화합니다.
 - 최적 K 탐색: `searchK()`로 다양한 토픽 수 K 에 대해 모델을 실행하고, 의미론적 일관성 및 배타성 지표를 비교하여 최적 K 를 선택합니다.
 - 토픽 이해: `findThoughts()`로 각 토픽과 가장 연관된 문서를 확인하여 토픽의 의미를 파악합니다.
- 과제 제출: 보고서(PDF)와 모든 코드 파일(ZIP 압축 권장)을 함께 제출해야 합니다.

